

CCAR-P — Claude Certified Architect Professional Study Pack

Contents

Claude Certified Architect – Professional (CCAR-P) Study Pack	9
Public availability & partner gating (read this first)	10
Exam mechanics (official Exam Guide v1.0, July 2026)	11
User’s 5 condensed domains ↔ official 7 domains	12
File index	13
Recommended study path (4–6 weeks) — primary-source mode	15
Public sources to ground further reading	15
EPUB conversion hint	15
 01 — Claude Platform & Solution Design	 16
Disclaimer	16
Overview	16
Key map (user domain ↔ official tasks)	17
Part A — Claude models for architects	18
Part B — Prompting & behavioral control	21
Part C — Context engineering	22
Part D — Solution design & architecture patterns	24
Part E — Design documentation artifacts	28
Decision trees & quick tables (exam desk reference)	28
Exam traps (Platform & Solution Design)	28
Practice Q&A (25)	29
Pre-exam checklist (Platform & Solution Design)	31
Glossary	31
Part F — Deeper solution patterns & anti-patterns	32
Part G — Prompt & context deep techniques	35
Part H — Additional worked scenarios	36
Part I — Extended practice Q&A (26–35)	37
Expanded checklist additions	38

Glossary additions	38
Part J — Primary-study deep dive: architecture decision scenarios	39
Part K — Trade-off tables (desk reference)	41
Part L — Failure-mode analysis (platform & design)	42
Part M — Production checklist (Platform & Solution Design) . .	43
Part N — Additional architecture decision drills (short)	44
Part O — Extended Q&A (36–40)	44
Part P — Rapid review card (Domain 1+2 $\approx$ 30%)	44
Part Q — Model routing & cost envelopes (worked numbers) .	45
Part R — Additional failure modes & exam stems	46
01 — Claude Platform & Solution Design	47
Disclaimer	47
Overview	47
Key map (user domain $\leftrightarrow$ official tasks)	48
Part A — Claude models for architects	48
Part B — Prompting & behavioral control	51
Part C — Context engineering	53
Part D — Solution design & architecture patterns	55
Part E — Design documentation artifacts	58
Decision trees & quick tables (exam desk reference)	58
Exam traps (Platform & Solution Design)	59
Practice Q&A (25)	59
Pre-exam checklist (Platform & Solution Design)	61
Glossary	61
Part F — Deeper solution patterns & anti-patterns	62
Part G — Prompt & context deep techniques	65
Part H — Additional worked scenarios	66
Part I — Extended practice Q&A (26–35)	67
Expanded checklist additions	68
Glossary additions	68
Part J — Primary-study deep dive: architecture decision scenarios	69
Part K — Trade-off tables (desk reference)	71
Part L — Failure-mode analysis (platform & design)	72
Part M — Production checklist (Platform & Solution Design) . .	73
Part N — Additional architecture decision drills (short)	74
Part O — Extended Q&A (36–40)	74
Part P — Rapid review card (Domain 1+2 $\approx$ 30%)	74
Part Q — Model routing & cost envelopes (worked numbers) .	75
Part R — Additional failure modes & exam stems	76
02 — Enterprise Integration & Production	76

Disclaimer	77
Overview	77
Key map	78
Part A — Integration primitives	78
Part B — RAG & retrieval architecture	80
Part C — Tools, agents, and production control	82
Part D — Security-relevant integration (pointer + essentials)	84
Part D5 — CSP / delivery-route governance (P0)	84
Part E — Evaluation, testing & optimization	91
Part F — Reference architectures (integration-centric)	94
Decision trees	94
Exam traps	95
Practice Q&A (28)	95
Checklist	97
Glossary	98
Part G — Deep dive: enterprise auth, networking, and data planes	99
Extended checklist	102
Glossary additions	102
Part H — Production hardening narratives (exam-style)	103
Part I — Integration patterns cheat-sheets	106
Part J — Integration exam drill narratives	107
Closing integration summary (review)	108
Part K — Primary-study deep dive: Integration (19%)	109
Part L — RAG pipeline design lab (Integration)	112
Part M — Evaluation, Testing & Optimization (16%) deep dive	113
Part N — Architecture decision scenarios (Integration + Evals)	118
Part O — Production checklists	119
Part P — Failure-mode catalog (ops)	120
Part Q — Extended Q&A (56–65) Integration + Evals	120
Part R — Rapid review (Integration 19% + Evals 16% ≈ 35%)	121
Part S — End-to-end reference scenarios (Integration + Evals heaviest drill)	122
Part T — Desk sheet (35% combined)	124

02 — Enterprise Integration & Production 124

Disclaimer	124
Overview	125
Key map	125
Part A — Integration primitives	126
Part B — RAG & retrieval architecture	128
Part C — Tools, agents, and production control	130

Part D — Security-relevant integration (pointer + essentials) . .	131
Part D5 — CSP / delivery-route governance (P0)	132
Part E — Evaluation, testing & optimization	138
Part F — Reference architectures (integration-centric)	141
Decision trees	141
Exam traps	142
Practice Q&A (28)	143
Checklist	144
Glossary	145
Part G — Deep dive: enterprise auth, networking, and data planes	146
Extended checklist	149
Glossary additions	149
Part H — Production hardening narratives (exam-style)	150
Part I — Integration patterns cheat-sheets	153
Part J — Integration exam drill narratives	154
Closing integration summary (review)	156
Part K — Primary-study deep dive: Integration (19%)	156
Part L — RAG pipeline design lab (Integration)	159
Part M — Evaluation, Testing & Optimization (16%) deep dive .	160
Part N — Architecture decision scenarios (Integration + Evals)	165
Part O — Production checklists	166
Part P — Failure-mode catalog (ops)	167
Part Q — Extended Q&A (56–65) Integration + Evals	168
Part R — Rapid review (Integration 19% + Evals 16% $\approx$ 35%) .	168
Part S — End-to-end reference scenarios (Integration + Evals heaviest drill)	169
Part T — Desk sheet (35% combined)	171
03 — Responsible AI, Safety & Risk for Architects	171
Disclaimer	172
Overview	172
Key map	172
Part A — Risk taxonomy for Claude solutions	173
Part B — Guardrail architecture (defense in depth)	174
Part C — Prompt injection deeply	175
Part D — Human-in-the-loop (HITL)	176
Part E — Privacy, compliance, and data governance	178
Part F — Fairness, ethics, dual-use	179
Part G — Governance operating model	180
Part H — Safety evals	181
Decision trees	181

Exam traps	181
Practice Q&A (28)	182
Checklist	184
Glossary	184
Part I — Worked governance scenarios	185
Part J — Policy writing for architects	186
Part K — Extended Q&A (29–35)	186
Extended checklist	187
Part L — Organizational accountability & board-level framing	187
Part M — Mapping controls to CCAR-P verbs	191
Part N — Safety exam drill	191
Part O — Rapid review (Governance 14%)	192
Part P — Control catalog (printable)	193
Part Q — Primary-study deep dive: Governance architecture	194
Part R — Worked decision scenarios (Safety)	198
Part S — Production safety checklist	199
Part T — Trade-offs: autonomy vs control	199
Part U — Extended Q&A (57–65)	200
Part V — Rapid review (Governance 14%)	200
Part W — Abuse monitoring, model cards, and exam stems	201
03 — Responsible AI, Safety & Risk for Architects	202
Disclaimer	202
Overview	202
Key map	202
Part A — Risk taxonomy for Claude solutions	203
Part B — Guardrail architecture (defense in depth)	204
Part C — Prompt injection deeply	205
Part D — Human-in-the-loop (HITL)	206
Part E — Privacy, compliance, and data governance	208
Part F — Fairness, ethics, dual-use	209
Part G — Governance operating model	209
Part H — Safety evals	210
Decision trees	210
Exam traps	211
Practice Q&A (28)	211
Checklist	213
Glossary	213
Part I — Worked governance scenarios	214
Part J — Policy writing for architects	215
Part K — Extended Q&A (29–35)	216

Extended checklist	216
Part L — Organizational accountability & board-level framing	216
Part M — Mapping controls to CCAR-P verbs	220
Part N — Safety exam drill	221
Part O — Rapid review (Governance 14%)	222
Part P — Control catalog (printable)	222
Part Q — Primary-study deep dive: Governance architecture	223
Part R — Worked decision scenarios (Safety)	227
Part S — Production safety checklist	227
Part T — Trade-offs: autonomy vs control	228
Part U — Extended Q&A (57–65)	228
Part V — Rapid review (Governance 14%)	229
Part W — Abuse monitoring, model cards, and exam stems	230

04 — Stakeholder Engagement, Lifecycle & GTM-Adjacent Communication — Simplified Technical English	230
Disclaimer	231
Overview	231
Key map	231
Part A — Stakeholder discovery	232
Part B — Communicating architecture	234
Part C — Proposal framing (GTM-adjacent, partner limits flagged)	235
Part D — Lifecycle management	237
Part E — Change & conflict management	238
Part F — Documentation standards	238
Decision trees	239
Exam traps	239
Practice Q&A (26)	240
Checklist	241
Glossary	242
Part G — Meeting patterns & facilitation	242
Part H — Narrative kits (original, reusable)	244
Part I — Lifecycle artifacts library (expand)	246
Part J — Extended lifecycle playbooks	247
Part K — Communication worksheets	249
Part L — Full discovery questionnaire bank (condensed)	250
Part M — Handoff email template (original)	250
Part N — Primary-study deep dive: Discovery that still works in real conditions	251
Part O — Value framing without partner playbooks	253
Part P — Lifecycle & change management deep dive	254

Part Q — Architecture communication drills	255
Part R — GTM-adjacent boundaries (exam hygiene)	255
Part S — Production checklists (lifecycle)	256
Part T — Failure modes in stakeholder/lifecycle work	256
Part U — Extended Q&A (59–68)	257
Part V — Rapid review (Stakeholder 14%)	258

04 — Stakeholder Engagement, Lifecycle & GTM-Adjacent Communication	258
Disclaimer	258
Overview	258
Key map	259
Part A — Stakeholder discovery	259
Part B — Communicating architecture	261
Part C — Proposal framing (GTM-adjacent, partner limits flagged)	263
Part D — Lifecycle management	264
Part E — Change & conflict management	265
Part F — Documentation standards	266
Decision trees	266
Exam traps	266
Practice Q&A (26)	267
Checklist	268
Glossary	269
Part G — Meeting patterns & facilitation	269
Part H — Narrative kits (original, reusable)	271
Part I — Lifecycle artifacts library (expand)	273
Part J — Extended lifecycle playbooks	274
Part K — Communication worksheets	276
Part L — Full discovery questionnaire bank (condensed)	277
Part M — Handoff email template (original)	277
Part N — Primary-study deep dive: Discovery that survives contact with reality	278
Part O — Value framing without partner playbooks	279
Part P — Lifecycle & change management deep dive	280
Part Q — Architecture communication drills	282
Part R — GTM-adjacent boundaries (exam hygiene)	282
Part S — Production checklists (lifecycle)	283
Part T — Failure modes in stakeholder/lifecycle work	283
Part U — Extended Q&A (59–68)	284
Part V — Rapid review (Stakeholder 14%)	284

05 — Team Enablement & Operational Productivity — Simplified

Technical English	285
Disclaimer	285
Overview	286
Key map	286
Part A — Rollout strategy	287
Part B — CLAUDE.md, Skills, and project instructions	288
Part C — Settings hierarchy & enforcement	289
Part D — MCP & credentials for builders	290
Part E — Developer productivity workflows	291
Part F — Operational enablement & runbooks	292
Part G — Training & onboarding curriculum	293
Part H — Governance of enablement itself	294
Decision trees	294
Exam traps	294
Practice Q&A (28)	295
Checklist	297
Glossary	297
Part I — Deep dive: designing the paved road	298
Extended checklist	300
Glossary additions	300
Part J — Enablement exam drill & labs detail	301
Part K — Rapid review (Enablement 7%)	302
Part L — Sample paved-road checklist (copy/adapt)	303
Part M — Primary-study deep dive: Paved-road enablement	303
Part N — Productivity workflows that pass review	305
Part O — Training curriculum (original outline)	306
Part P — Architecture decision scenarios (Enablement)	307
Part Q — Failure-mode analysis (enablement)	308
Part R — Production checklists	308
Part S — Trade-off tables	309
Part T — Extended Q&A (49–60)	309
Part U — Rapid review (Enablement 7%)	310
 05 — Team Enablement & Operational Productivity	 311
Disclaimer	311
Overview	311
Key map	311
Part A — Rollout strategy	312
Part B — CLAUDE.md, Skills, and project instructions	313
Part C — Settings hierarchy & enforcement	314
Part D — MCP & credentials for builders	316

Part E — Developer productivity workflows	316
Part F — Operational enablement & runbooks	318
Part G — Training & onboarding curriculum	319
Part H — Governance of enablement itself	320
Decision trees	320
Exam traps	320
Practice Q&A (28)	321
Checklist	322
Glossary	323
Part I — Deep dive: designing the paved road	324
Extended checklist	326
Glossary additions	326
Part J — Enablement exam drill & labs detail	327
Part K — Rapid review (Enablement 7%)	328
Part L — Sample paved-road checklist (copy/adapt)	328
Part M — Primary-study deep dive: Paved-road enablement	329
Part N — Productivity workflows that survive scrutiny	331
Part O — Training curriculum (original outline)	332
Part P — Architecture decision scenarios (Enablement)	333
Part Q — Failure-mode analysis (enablement)	333
Part R — Production checklists	334
Part S — Trade-off tables	334
Part T — Extended Q&A (49–60)	335
Part U — Rapid review (Enablement 7%)	336
CCAR-P Pro Tips — how to take this exam well	336
The numbers that drive strategy	336
The constraint-elimination drill (use it on every long stem)	337
If the stem says X → think Y	337
Distractor archetypes to expect	338
Freshness watchlist (re-verify the week of your exam)	339
The 48-hour plan	339
Day-of (online proctored)	339

Claude Certified Architect – Professional (CCAR-P) Study Pack

Exam code: CCAR-P **Credential:** Claude Certified Architect – Professional
Pack type: Original exam-prep notes (not official course content)
Depth: Deepened for **PRIMARY** study — architecture decision scenar-

ios, trade-off tables, failure-mode analysis, production checklists, and expanded Q&A (25–40+ per domain file). Intended to serve as a main study source alongside public Anthropic docs and the official exam guide (when you have access). **Audience:** Solution architects designing production Claude systems **User domains in this pack:** Five condensed files (maps onto official seven) **Offline reading tip:** Each file is plain Markdown — convert with any MD→EPUB tool if you prefer e-reader formatting; keep headings H1–H3 for navigation.

Public availability & partner gating (read this first)

Resource	Public?	Notes
Exam blueprint domains & weights	Official — Exam Guide v1.0 (effective July 2026); download from the official Pearson VUE Anthropic program page	All weights and mechanics below are taken directly from it, cross-checked (2026-08-23) against the public Pearson VUE Anthropic program page.
Exam registration	Official portal	Registration and proctoring run through the official exam portal (Pearson VUE delivery); check current requirements before booking.
Official gated prep course	Not used	This pack is original and grounded in public sources only.
Public Anthropic docs (platform.claude.com, model cards, MCP, tool use, safety research)	Public	Primary grounding for all technical notes in this pack.
claude.ai / Claude API / Claude Code docs	Public (product features may require accounts)	Use for hands-on practice.

Resource	Public?	Notes
GTM / partner sales playbooks	Often partner-only	File 04 covers stakeholder discovery, proposal framing, lifecycle handoff, and value communication in original language only—no confidential partner scripts.

Disclaimer: This pack is an independent study aid. It is not affiliated with, endorsed by, or derived from official course materials. Exam mechanics and domain weights can change; verify against the official guide before exam day.

Exam mechanics (official Exam Guide v1.0, July 2026)

Item	Value
Questions	63 items
Time	120 minutes (~1.9 min/question average)
Pass score	720 on a 100–1,000 scaled score
Format	Multiple choice + multiple-response (“Select TWO/THREE”); many long multi-constraint scenarios
Delivery	Pearson VUE (online proctored or test center)
Validity	12 months ; free non-proctored on-time renewal
Fee	\$175 USD per attempt
Retakes	14/30/90-day waits; max 4 attempts per rolling 12 months
Prerequisites	None mandatory; production Claude architecture experience strongly recommended ; CCAR-F helpful but not required

Scoring note: Scaled scoring means 720 is **not** a fixed % correct. Multiple-response items usually need **all** correct selections—partial credit should not be assumed.

User’s 5 condensed domains ↔ official 7 domains

#	User condensed domain	File	Official domains covered
1	Claude Platform & Solution Design	01–platform–solution–design.md	Solution Design & Architecture (17%) · Claude Models, Prompting & Context Engineering (13%)
2	Enterprise Integration & Production	02–enterprise–integration–production.md	Integration (19%) · Evaluation, Testing & Optimization (16%)
3	Responsible AI, Safety & Risk for Architects	03–responsible–ai–safety–risk.md	Governance, Safety & Risk Management (14%)
4	Stakeholder Engagement, Lifecycle & GTM	04–stakeholder–lifecycle–gtm.md	Stakeholder Communication & Lifecycle Management (14%) · GTM-adjacent communication (partner limits flagged)

#	User condensed domain	File	Official domains covered
5	Team Enablement & Operational Productivity	05-team-enablement-operational-productivity.md	Developer Productivity & Operational Enablement (7%)

Official weight map (study time budgeting)

Official domain	Weight	~Q (of 63)	Pr
Integration	19%	~12	02
Solution Design & Architecture	17%	~11	01
Evaluation, Testing & Optimization	16%	~10	02
Governance, Safety & Risk Management	14%	~9	03
Stakeholder Communication & Lifecycle Management	14%	~9	04
Claude Models, Prompting & Context Engineering	13%	~8	01
Developer Productivity & Operational Enablement	7%	~4	05

Study implication: Integration + Solution Design + Evaluation $\approx$ **52%**. Governance + Stakeholders + Enablement $\approx$ **35%**—do not skip files 03–05.

File index

1. [README.md](#) — this index
2. [01-platform-solution-design.md](#) — models, prompting, context, solution architecture
3. [02-enterprise-integration-production.md](#) — MCP/tools/agents, RAG, evals, observability
4. [03-responsible-ai-safety-risk.md](#) — guardrails, HITL, compliance, risk
5. [04-stakeholder-lifecycle-gtm.md](#) — stakeholders, lifecycle, GTM-adjacent communication
6. [05-team-enablement-operational-productivity.md](#) — Claude Code rollout, CLAUDE.md/Skills/hooks, training, ops enablement

7. [PRO-TIPS.md](#) — exam-craft companion: pacing math, the constraint-elimination drill, reflex table, distractor archetypes (added 2026-08-23)

Each domain file includes: disclaimer · overview · key map · deep notes · architecture decision scenarios · trade-off tables · failure-mode analysis · decision trees · exam traps · **25–40+ Q&A** · production checklists · glossary. Deepened for primary study use.

P0 gap-closure additions (this pack revision)

Original notes only (no official lesson prose):

Addition	Where
CSP / delivery-route governance — public-safe decision table (Direct Anthropic API vs Amazon Bedrock vs Google Vertex AI vs Azure/Microsoft Foundry), identity/auth, residency, logging/retention & ZDR-as-verify themes, procurement/BAA/FedRAMP themes, model-ID differences, constraint-elimination vs vibes, + Q&A	02 Part D5 (pointer from 01 deployment surfaces)
Model judges vs code judges + calibration — when deterministic assertions win, when rubric judges are needed, human-gold calibration, thresholds/rollback before peeking at experiment data, mini worksheet + Q&A	02 Part M7 (pointer from Part E3)
Review-routing matrix (P1) — reversibility × cost × confidence → auto / sample / always-human	03 Part D4

Recommended study path (4–6 weeks) — primary-source mode

1. **Week 1:** File 01 — models, prompting, context, architecture patterns; complete Part J scenarios.
2. **Week 2:** File 02 Integration half — RAG, MCP, authZ, latency, capability bloat (heaviest weight).
3. **Week 3:** File 02 Evaluation half + File 03 safety/governance; run failure-mode tables.
4. **Week 4:** File 04 stakeholders/lifecycle/change/value (flag partner-only GTM limits) + File 05 enablement; mixed scenarios.
5. **Weeks 5–6:** Timed 63Q/120m sets; weakness loops; write ADRs from the scenario banks cold.

Hands-on: one RAG path, one tool/MCP path, one eval harness, one HITL gate, one Claude Code org-settings dry-run (even if only in docs review).

Primary-study tip: For each official domain, be able to teach aloud: (1) decision tree, (2) one trade-off table, (3) top three failure modes, (4) production checklist items that would block GA.

Public sources to ground further reading

- Anthropic Claude models overview & pricing (platform docs)
 - Messages API, tool use, server vs client tools
 - Model Context Protocol (MCP) introduction & server guides
 - Prompt caching, context windows, thinking / adaptive thinking
 - Search results / citations for RAG attribution
 - Claude Code docs (settings hierarchy, MCP, plugins—public product docs)
 - Anthropic research: Constitutional AI / Claude’s Constitution
 - Cloud deployments: Amazon Bedrock, Vertex AI, Microsoft Foundry (as applicable)
-

EPUB conversion hint

```
pandoc README.md 01-*.md 02-*.md 03-*.md 04-*.md 05-*.md -o CCA  
--metadata title="CCAR-P Study Pack" --toc --toc-depth=2
```

*Pack deepened as a **primary study source** across **five** user domain files (original notes only), including **P0 gap closures** (CSP routes, judge calibration, review-routing). Keep the 5↔7 domain mapping above. Verify exam mechanics against the current official CCAR-P Exam Guide before sitting the exam.*

01 — Claude Platform & Solution Design

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, MCP, prompting, caching, effort, p95) are exceptions and stay as written. Model IDs and prices change. Learn the decision rules. Check the current model cards before the exam.

CCAR-P condensed domain 1 Official domains mapped here: Solution Design & Architecture (17%) · Claude Models, Prompting & Context Engineering (13%) **Combined exam weight covered by this file: ~30%**

Disclaimer

These are **original study notes** for exam preparation. They use **public** Anthropic platform documentation. They also use published CCAR-P blueprint summaries and common architecture patterns for LLM systems. They do **not** copy official course content. Model names, prices, and context limits change. Always check numbers against current platform.claude.com docs. Do this before production decisions and before exam day. This pack is independent. It is not affiliated with Anthropic.

Overview

CCAR-P expects you to think like a **production architect**. It does not expect you to only adjust prompts. In scenarios you:

1. Change a messy business problem into a **Claude-powered solution shape** (inputs → processing → outputs → feedback).

- 2. Select among **workflow, augmented LLM, agentic, and multi-agent** patterns. State the trade-offs.
- 3. Select a **model tier** (capability vs latency vs cost vs context) and justify it.
- 4. Design **prompts and context** so the system stays reliable under token, safety, and reuse constraints.
- 5. Connect design choices to **business value pillars**. The official guide names these: efficiency, transformation, **productivity**, cost, and performance SLAs.

This file covers platform capabilities and solution design in depth. Integration mechanics (MCP wiring, RAG pipelines, eval harnesses) appear in file 02. Governance appears in 03. Stakeholder, lifecycle, and enablement appear in 04.

Key map (user domain ↔ official tasks)

Architect task	Official domain	What “good” looks like on the exam
Problem → solution framing	Solution Design 17%	Clear value hypothesis, constraints, and success metrics
Pattern selection (workflow/agent/...)	Solution Design 17%	Prefer the simplest pattern that meets audit and reliability needs
Decomposition & orchestration	Solution Design 17%	Stages, gates, retries, and human checkpoints when risk is high
Model selection	Models/Prompting 13%	Match complexity, latency budget, context size, and cost
System prompts & behavioral controls	Models/Prompting 13%	Role, constraints, output schema, and refusal boundaries

Architect task	Official domain	What “good” looks like on the exam
Few-shot / structured reasoning	Models/Prompting 13%	Examples when format or edge cases matter. Thinking for hard reasoning
Context window strategy	Models/Prompting 13%	Progressive discovery vs curated context. Caching and Skills for reuse
Prompt caching / modular prompts / Skills	Models/Prompting 13%	Reuse static prefixes. Do not put extra content in every request

Part A — Claude models for architects

A1. How to think about the model ladder

Public docs describe a **family** of Claude models. The models have different speed, intelligence, and cost points. Exact IDs and prices change over time. For CCAR-P, learn the **decision logic**. Do not memorize a frozen price sheet:

Decision factor	Prefer higher-capability tier when...	Prefer faster/cheaper tier when...
Task complexity	Multi-step planning, hard coding, ambiguous requirements, high-stakes judgment	Classification, extraction, routing, short rewriting
Latency SLA	Offline batch, async jobs, human-reviewed drafts	Interactive UX, high QPS gatekeepers
Context size	Large codebases, long docs, multi-hour agent traces	Short turns, tool-heavy loops with small payloads

Decision factor	Prefer higher-capability tier when...	Prefer faster/cheaper tier when...
Cost envelope	Low volume / high value (contracts, architecture reviews)	High volume / low marginal value
Error cost	A wrong answer is expensive or unsafe	You can catch or retry a wrong answer at low cost

Architect rule: Default to the **smallest model that meets quality bars on your eval set**. Raise capability only when evals show a gap. Lower capability when latency or cost budgets fail and quality still holds.

A2. Capability controls that appear in designs

Public platform features that you must place correctly:

- **Context windows:** Current frontier Claude models often advertise **large** windows (hundreds of thousands to **~1M tokens** on supported models/platforms). Everything in the request counts: system prompt, messages, tool defs, images/docs, tool results, and thinking tokens when you use them. Caching changes **price**. Cached tokens still occupy the window.
- **Max output:** Compare the synchronous Messages API and Batch. Batch may support extended output via documented beta headers. Check current docs.
- **Thinking / adaptive thinking / effort:** Use deeper reasoning for hard problems. Control cost and latency with **effort levels** (`output_config.effort`). Adaptive thinking replaced fixed thinking budgets. `budget_tokens` is deprecated on the 4.6 generation. Current models remove it and reject it with an error. A change to thinking config invalidates message-level prompt-cache breakpoints. The tools/system prefix stays cached. Design for cache stability.
- **Vision / multimodal:** Use image input when the product needs screenshots, diagrams, or scanned docs.
- **Deployment surfaces:** Claude API, Amazon Bedrock, Vertex AI, Microsoft Foundry, and Claude Platform on AWS. Each has ID naming and data-residency nuances. You select based on enterprise procurement, region, and compliance. Do not select from brand

preference alone. **The full CSP / delivery-route governance decision table lives in file 02 Part D5** (constraint elimination, ZDR themes, model-ID traps, Q&A).

A3. Model selection decision tree

START: Define task + quality bar + p95 latency + monthly volume

- ├─ Need long-running agentic coding / complex enterprise judgment?
 - └─ YES → Start with high-capability model; measure cost on r
- ├─ Need interactive speed + “good enough” intelligence?
 - └─ YES → Mid-tier (Sonnet-class); escalate hard cases via ro
- ├─ High QPS classify/extract/route?
 - └─ YES → Fast tier (Haiku-class); sample escalate to mid/hig
- └─ Mixed workload?
 - └─ YES → Explicit router (rules or small model) + per-route SL0s

Common exam error: You select Opus/Fable-class for every step of a deterministic ETL-like pipeline. Prefer **workflow + smaller model** with validation gates.

A4. Multi-model architectures

Common production shapes:

1. **Router** → **specialist**: A cheap classifier selects extract vs reason vs refuse.
2. **Draft** → **critique**: A mid model writes a draft. A stronger model or rubric critic reviews it.
3. **Shadow eval**: A new model runs in parallel. You compare offline before cutover.
4. **Fallback**: If the primary times out, use a degraded model or a cached answer plus a human queue.

Record **why** each hop exists (quality, cost, safety). Extra hops fail reviews. They also fail exam scenarios that ask for the simplest viable design.

Part B — Prompting & behavioral control

B1. System prompt as architecture, not poetry

Treat the system prompt as a **policy + interface contract**:

Section	Purpose	Example content (illustrative)
Role & scope	Bound the job	“You are a procurement assistant for freight quotes. You do not negotiate contracts.”
Tools policy	When to call what	“Call <code>get_policy</code> before recommending. Never invent policy clauses.”
Output contract	Schema / format	JSON schema, citation requirements, severity enums
Safety / refusal	Local guardrails	Disallowed actions. Escalation phrases
Quality bar	How to handle uncertainty	“If confidence low, ask clarifying question or return <code>needs_review</code> .”
Style	Audience	Exec brief vs engineer debug

Prefer: Explicit constraints + structured outputs. **Avoid:** Vague “be helpful and thorough” with no success criteria.

B2. Prompting techniques architects should name correctly

Technique	When to use	Failure mode
Zero-shot	Clear tasks, strong base capability	Ambiguous formats drift
Few-shot	Format, tone, edge-case handling	Bad examples become the rule. Token bloat

Technique	When to use	Failure mode
Chain-of-thought / structured reasoning	Multi-step logic (prefer API thinking features where available)	You leak the chain into a user-facing channel. Cost
Decomposition in prompt	Single call must cover subskills	Still weaker than real workflow gates for audit
Tool-forced (tool_choice)	Retrieval or action must happen	You force a tool when optional tools exist
Constitutional / principle prompts	Local safety/style overlays	Conflicts with org policy if you do not review

B3. Output contracts and machine consumers

If a downstream system parses Claude’s output, design for **machines first**:

- Prefer **JSON / typed schemas**. Validate before side effects.
- Separate **user-facing prose** from **machine fields**.
- Add explicit enums for status (ok, needs_review, refused).
- Validate with code. Do not hope that the model obeyed.

B4. Skills and modular prompt reuse (public product concept)

Public Claude product docs describe **Skills** (and related modular instruction packs). Skills are reusable specialized instructions and resources. Teams do not paste giant instruction blobs into every request. Architect takeaway for CCAR-P:

- Put **stable** expertise into modular reusable units.
- Keep **request-specific** data in the user/tool channel.
- Version Skills like code. Review for prompt-injection surfaces when Skills can pull external content.

Part C — Context engineering

C5. What counts toward the window

Everything that you send counts. You must budget:

- 1. System / Skills / static policy
- 2. Tool definitions (large MCP tool catalogs can dominate)
- 3. Conversation history
- 4. Retrieved chunks / files
- 5. Tool results
- 6. Thinking tokens (when enabled)

A public **Token counting API** exists. Use it to estimate before you send. Use it in design reviews for worst-case prompts.

C6. Progressive discovery vs monolithic context

Strategy	Idea	Best for	Risk
Monolithic context	Put “everything possibly relevant” into the prompt	Small corpora, strong need for global consistency	Noise, cost, lost-in-the-middle, cache thrash
Progressive discovery	Start lean. Retrieval or tools pull what you need	Large corpora, tool ecosystems, agents	Missing retrieval → incomplete answers
Hybrid	Cached core policy + dynamic retrieval	Most enterprise apps	Complexity of two paths

Exam preference: When corpora or tool sets are large, prefer **progressive discovery** (RAG, tools, MCP tool search). Do not dump megabytes of text. When the task is short and closed-world, a curated static context can be simpler and more auditable.

C7. Prompt caching (architect view)

Public docs: prompt caching lowers cost and latency for repeated prefixes (system prompts, large reference docs, tool defs). Cache writes cost more than ordinary input. Cache **reads** have a large discount. TTLs often include short options (for example, ~5 minutes) and longer options (for example, ~1 hour). Confirm current docs.

Design rules:

- Put **stable content first** (cacheable prefix).
- Put **volatile user content last**.
- Do not reshuffle tool defs on every request.
- A change to thinking budgets or modes can invalidate breakpoints. Keep configs stable in hot paths.
- Know this: cached tokens **still consume** context window capacity.

C8. Compaction / context editing

For long-running agents and chats, public platform features include **compaction** (server-side summarization near limits). They also include **context editing** strategies (for example, clearing old tool results). You should:

- Define what you must **never** summarize away (safety policy, user entitlements).
- Record when compaction occurs, for audit.
- Test agent quality after compaction. Summary drift is a real failure mode.

C9. Context engineering checklist

- ☐ Priority order of context sections documented
 - ☐ Retrieval top-k and ranking justified
 - ☐ Tool catalog size managed (tool search / allowlists)
 - ☐ Cache breakpoints stable across turns
 - ☐ PII/minimization rules for what enters context
 - ☐ Escape hatches when context insufficient (needs_more_data)
-

Part D — Solution design & architecture patterns

D1. End-to-end solution skeleton

Every CCAR-P-worthy design can be drawn as:

```
[Actors/Channels] → [Ingress & AuthN/Z] → [Orchestration layer]
→ [Claude calls / tools / retrieval]
→ [Validators & policy gates]
→ [Side effects / UX]
→ [Telemetry + eval feedback loop]
```

Map each box to owners (app team, platform, security, data). Missing owners cause lifecycle failure (see file 04).

D2. Pattern catalog (memorize these patterns)

Anthropic-style teaching and exam scenarios distinguish **predefined workflows** from **agents that decide their own path**.

1) Augmented LLM (single call + light tools) One Claude call, maybe with retrieval or a tool or two. **Use when:** The task is bounded. One reasoning pass is enough. **Pros:** Simple, cheap, easy to eval. **Cons:** Weak for multi-stage audit. Limited recovery.

2) Workflow / prompt chaining Code owns the control flow: Step A → validate → Step B → ... **Use when:** Stages are known, ordered, and need intermediate checks (procurement quote extract → policy validate → recommend). **Pros:** Auditable, retryable per stage, clear SLAs. **Cons:** More latency than one call. More engineering.

3) Parallelization Independent subtasks run at the same time. An aggregator merges the results. **Use when:** No data dependencies exist between branches. **Trap:** You run dependent stages in parallel (validate before extract finishes).

4) Routing A classifier or router sends work to specialized prompts, models, or workflows. **Use when:** Heterogeneous intents share one front door.

5) Evaluator-optimizer / critique loops A generator produces. An evaluator scores against a rubric. Iterate N times or until pass. **Use when:** Quality variance is high and latency allows.

6) Agentic (model-driven control flow) Claude plans, selects tools, and loops until a stop condition. **Use when:** You cannot fully hardcode the steps. The environment is open-ended (research, debugging unknown codebases). **Pros:** Flexibility. **Cons:** Cost, non-determinism, and harder audit. You need budgets, tool allowlists, and HITL for high-impact actions.

7) Multi-agent Specialized agents with handoffs (researcher / writer / reviewer) or a supervisor pattern. **Use when:** Clear role separation improves quality or security boundaries. **Trap:** Multi-agent appearance: three agents do work that one workflow can do.

D3. Pattern selection table

Situation	Prefer	Avoid
Fixed stages + compliance audit	Workflow + gates	Fully autonomous agent
Open-ended investigation	Agent + tool allowlist + budget	Brittle 12-step workflow
Independent doc sections	Parallel map-reduce	Serial chain
Mixed intents	Router	One mega-prompt
High-stakes irreversible action	Workflow + HITL approval	Agent with write tools unbound
Need citations from corpus	RAG + citation blocks	Pure parametric memory

D4. Decomposition techniques

1. **By business stage** (intake → enrich → decide → act → explain)
2. **By risk** (read-only research vs write actions)
3. **By data domain** (HR vs finance agents with separate credentials)
4. **By SLA** (fast path vs deep path)
5. **By confidence** (auto vs human queue)

Write **interfaces** between stages: schemas, idempotency keys, error codes. Exam scenarios reward designs that can retry stage 2. You do not redo stage 1.

D5. Business value alignment

Tie architecture to measurable outcomes:

Value pillar	Example metric	Design lever
Efficiency	Minutes saved per case	Automation rate + HITL only on exceptions
Productivity	Tasks completed per person / throughput	Copilot adoption, automated drafts, faster review loops

Value pillar	Example metric	Design lever
Transformation	New capability enabled	Agents + tools that unlock research that was manual before
Cost	\$ per successful task	Caching, smaller models, batch API
Performance	p95 latency	Parallelism, Haiku router, smaller context
Quality (<i>not in the guide's pillar list — quality stems live in the Evaluation domain</i>)	Error rate vs baseline	Workflow gates, eval harness, stronger model on hard route

If a scenario lists SLAs, your answer must **name the lever** that protects the SLA.

D6. Non-functional requirements checklist

- Availability & degradation mode
- Latency budgets per stage
- Cost ceilings / kill switches
- Data residency & retention
- Audit logging of prompts/tool calls (with redaction)
- Secrets handling (never in prompts)
- Rate limits & backoff
- Model deprecation / pin strategy

D7. Worked mini-scenario (freight quotes)

Need: Extract quote fields → validate policy → recommend carrier. The trail must be auditable. **Pattern:** A fixed **workflow** with three Claude (or Claude+code) stages and validators between them. **Not:** A single monolithic autonomous agent. Not parallel independent extract, validate, and recommend. **Why:** Dependencies and audit. This pattern appears often in public practice explanations. It matches Anthropic's workflow-vs-agent distinction.

Part E — Design documentation artifacts

For lifecycle handoff (file 04), you should produce:

- 1. **Context diagram** (systems Claude touches)
- 2. **Sequence diagram** for the happy path + one failure path
- 3. **Decision record** (pattern, model, why not alternatives)
- 4. **Threat notes** (injection, data leakage—deep dive in file 03)
- 5. **Eval plan** stub (metrics—deep dive in file 02)
- 6. **Runbook** pointers (on-call, rollback model pin)

Exam questions may ask what you communicate to executives vs engineers. Use the same design. Change the altitude.

Decision trees & quick tables (exam desk reference)

Model + pattern joint decision

Quality need	Path predictability	Recommended shape
High	High	Workflow + mid/high model on hard steps
High	Low	Agent + strong model + budgets + HITL
Medium	High	Workflow + mid model
Medium	Low	Router + agent for long tail
Low	Any	Fast model + simple prompt. Maybe no LLM

Context strategy picker

Corpus size	Query diversity	Strategy
Tiny (< context comfortably)	Low	Curated static context + cache
Large	Low	Metadata filters + light RAG
Large	High	Hybrid search RAG + rerank
Tools many	High	MCP + tool search / progressive

Exam traps (Platform & Solution Design)

- 1. **Agent everywhere** — Prefer workflow when steps are known.

2. **Model with the most capability everywhere** — More capability than necessary fails cost/latency scenarios.
 3. **Ignoring dependencies in parallel designs.**
 4. **Treating prompt caching as free context** — Cached tokens still count toward the window.
 5. **Monolithic context for enterprise knowledge bases.**
 6. **No output schema** when systems consume results.
 7. **Skipping business metrics** — Architecture without a value story.
 8. **Forgetting degradation** — What happens when Claude/API is down?
 9. **Multi-agent without boundaries** — Shared over-privileged tools.
 10. **Changing cache-critical prefixes every request.**
 11. **Putting secrets in prompts or tool args logs.**
 12. **Equating “citations in prose” with grounded retrieval** — Prefer structured citations/search_result patterns from public docs.
-

Practice Q&A (25)

Q1. When should you prefer a fixed workflow over an autonomous agent? **A.** When stages are known, ordered, and need intermediate validation/audit.

Q2. A system must extract, then validate, then recommend. Parallel independent calls are proposed. What is wrong? **A.** Later stages depend on earlier outputs. Parallelization breaks those dependencies.

Q3. Name three inputs to model selection. **A.** Task complexity/quality bar, latency SLA, cost/volume (also context size, error cost).

Q4. Do prompt-cache tokens still count toward the context window? **A.** Yes. Caching affects billing and latency. It does not change window occupancy.

Q5. What belongs in a system prompt’s “output contract”? **A.** Schema/format, required fields, enums, citation rules, and uncertainty handling.

Q6. Large MCP tool catalogs hurt you how? **A.** Tool definitions consume context. Prefer allowlists or tool search / progressive discovery.

Q7. What is progressive discovery? **A.** Start with lean context. Retrieve or call tools to pull only the information that you need.

Q8. When are few-shot examples most justified? **A.** To lock format, tone, or rare edge-case handling. They are not a substitute for retrieval of facts.

Q9. Give one valid use of a multi-agent design. **A.** Separate roles with different permissions (for example, researcher read-only vs executor write with approval).

Q10. What should you do before promoting a larger model in production? **A.** Prove a quality gap on a representative eval set. Measure cost and latency impact.

Q11. Why pin model IDs in production architectures? **A.** Avoid surprise behavior changes. Manage deprecations with a plan.

Q12. What is an evaluator–optimizer loop? **A.** Generate → score against a rubric → refine until pass or the budget is exhausted.

Q13. Name a business-value metric for an efficiency use case. **A.** Average handle time, cases auto-closed, or minutes saved per ticket.

Q14. Where should volatile user content sit relative to a cacheable prefix? **A.** After the stable prefix (end of prompt), so the cache prefix stays stable.

Q15. When is a single augmented LLM call enough? **A.** Bounded task, low need for intermediate audit, and one reasoning pass is enough.

Q16. What risk does compaction introduce in long agents? **A.** Summary drift. Critical constraints or facts may be lost if you do not protect them.

Q17. How do you protect a p95 latency SLA in a multi-stage workflow? **A.** Budgets per stage, timeouts, parallel independent work, smaller models on easy stages, and async where UX allows.

Q18. Why separate machine JSON from user prose? **A.** Parsers need stability. UX needs readability. Mixing causes brittle integrations.

Q19. What is a router pattern good for? **A.** Heterogeneous intents behind one endpoint with different prompts, models, or workflows.

Q20. Give a reason to choose cloud-hosted Claude (Bedrock/Vertex/Foundry) over direct API. **A.** Enterprise procurement, existing cloud commit, or data residency/compliance controls (scenario-dependent).

Q21. What should a stage interface include for retries? **A.** Typed schema, idempotency key, explicit error codes, and validated outputs.

Q22. Why might changing the thinking/effort configuration between turns be costly? **A.** It invalidates message-level prompt-cache breakpoints (the

tools/system prefix stays cached). This raises latency and spend. (Note: current models deprecate and remove fixed budget_tokens thinking budgets — adaptive thinking plus effort replaced them.)

Q23. What is the architect’s default stance on tool permissions? **A.** Least privilege — only the tools needed for the role/stage.

Q24. How do Skills help teams? **A.** Modular reusable instructions/resources reduce prompt sprawl and improve consistency.

Q25. A stakeholder wants “fully autonomous” processing of regulated approvals. What do you propose instead? **A.** A workflow with policy checks and human-in-the-loop for high-impact approvals. Use an agent only for bounded research if needed.

Pre-exam checklist (Platform & Solution Design)

- ☐ I can explain workflow vs agent with one crisp example each
 - ☐ I can pick a model tier from quality/latency/cost constraints
 - ☐ I can sketch end-to-end: ingress → orchestrate → Claude/tools → validate → act → telemetry
 - ☐ I can design a cache-friendly prompt layout
 - ☐ I can justify progressive discovery vs monolithic context
 - ☐ I can write an output schema and validation gate
 - ☐ I can map architecture choices to business metrics and SLAs
 - ☐ I can list NFRs: residency, audit, degradation, rate limits, model pinning
 - ☐ I know when multi-agent helps vs when it only looks useful.
 - ☐ I can spot dependency mistakes in “parallelize everything” answers
-

Glossary

Term	Meaning
Augmented LLM	Single LLM call enriched with retrieval/tools
Workflow / prompt chaining	Code-orchestrated sequence of LLM/tool steps
Agent	Model-driven control flow with planning and tool loops

Term	Meaning
Multi-agent	Multiple specialized roles with handoffs
Context engineering	Deliberate selection, ordering, and budgeting of tokens
Prompt caching	Reusing billed/processed stable prompt prefixes
Progressive discovery	Fetch context on demand via retrieval/tools
Skills	Modular reusable instruction/resource packs
Output contract	Schema and rules for machine-consumable responses
Router	Component that dispatches to specialized paths
Effort / thinking	Controls for deeper reasoning vs latency/cost
Model pin	Fixed model ID for reproducible production behavior
NFR	Non-functional requirement (latency, cost, residency, etc.)
HITL	Human-in-the-loop review/approval
SLA / SLO	Service level agreement / objective

Part F — Deeper solution patterns & anti-patterns

F1. Prompt chaining with programmatic gates

A production-grade chain is not “three prompts in a row.” It is:

1. **Deterministic prep** (parse upload, normalize units, load tenant config).
2. **LLM step** with a narrow contract.
3. **Programmatic validator** (schema, business rules, allowlists).
4. **Branch:** pass → next stage. Fail → repair prompt, retry with cap, or human queue.
5. **Idempotent side effects** only after the final gate.

This hybrid of LLM + code is what CCAR-P scenarios reward. Pure LLM self-checks help. They are weaker than typed validators for compliance narratives.

F2. Map-reduce over documents

For large packets (RFPs, due diligence binders):

- **Map:** Same extract prompt per chunk/doc in parallel (mid/fast model).
- **Reduce:** A stronger model synthesizes with citations and conflict detection.
- **Verify:** Spot-check conflicting fields. Optional second-pass critique.

Architect concerns: chunk boundaries that split tables, consistent schemas across mappers, and reduce-stage token overruns.

F3. Human-in-the-loop as an architectural component

HITL is not only a safety topic (file 03). In solution design it is a **first-class stage**:

Trigger	UX pattern	Data to show the human
Low model confidence	Review queue	Top uncertainties + source snippets
High-impact action	Explicit approve/deny	Diff of proposed write + policy cites
Novel intent	Specialist escalation	Conversation transcript + tools used
Eval regression	Shadow mode	Side-by-side old vs new outputs

Design the queue’s SLAs and ownership. If you do not, the “architecture” fails in production.

F4. Batch vs realtime

Public Batch APIs trade latency for cost and throughput. You select Batch when:

- Jobs are overnight analytics, offline evals, or corpus enrichment.
- The user does not wait for a synchronous UI.

Keep **the same prompts and schemas** between realtime and batch so evals transfer. Document different timeout and retry policies.

F5. Anti-patterns library (memorize names)

1. Single mega-prompt: one prompt does extract, reason, act, and apology.
2. **Tool sprawl** — every SaaS verb is exposed to one agent.
3. Context dump: the entire SharePoint export sits in the system prompt.
4. **Eval-free launch** — “we will know quality when we see it.”
5. **Autonomy without budgets** — unbounded loops and spend.
6. **Silent fallback** — switch models without logging/telemetry.
7. **Schema optional** — free text into ERP.
8. **Security bolted on later** — authZ after tools already write.
9. Multi-agent imitation: three roles share one over-privileged toolkit.
10. Wasted latency: deep thinking on a yes/no classification.

F6. Capacity and cost modeling (rough estimate)

For a workflow with stages S1..Sn:

- $\text{Tokens} \approx \sum(\text{input}_i + \text{output}_i) \times \text{requests} \times (1 + \text{retry_rate})$
- Apply cache hit rate on stable prefixes
- Apply router fraction that escalates to expensive models
- Add tool/RAG infrastructure cost separately

Exam tip: if a scenario gives volume + latency, **mention caching, routing, or batching** as cost/latency levers even if you also select the right pattern.

F7. Migration & model upgrade playbook

1. Pin current model ID.
2. Build shadow traffic or offline replay from production logs (redacted).
3. Compare task metrics + safety metrics.
4. Canary percentage with kill switch.
5. Update runbooks and prompt snapshots.
6. Only then change the pin.

Never “just flip the alias” in regulated environments without a plan.

F8. Interface contracts between product and platform teams

Define:

- **Prompt/Skill ownership** and review cadence
- **Tool registry** ownership and change control
- **Eval suite** ownership and release gating
- **Incident severity** definitions for model misbehavior

CCAR-P blurs into lifecycle (file 04). In design answers, naming owners scores points.

Part G — Prompt & context deep techniques

G1. Separating instructions, data, and untrusted content

You should structurally separate:

- **Trusted instructions** (system / Skills / developer)
- **Retrieved documents** (data. May be malicious— injection risk)
- **User utterances** (intent. May be adversarial)
- **Tool outputs** (semi-trusted depending on the tool)

Use clear delimiters. Cite sources. Instruct the model to treat retrieved content as **untrusted data**. Pair this with technical controls in file 02/03 (allowlists, output encoding, privilege separation).

G2. Structured reasoning without leaking chain-of-thought

Prefer platform **thinking** features for hidden reasoning when they are available. If you must show rationales to users, give a **user-facing explanation** field that is curated. Do not show raw chain-of-thought. This is especially true in regulated UX.

G3. Determinism levers (practical, not absolute)

LLM calls are stochastic. Reduce variance with:

(Older models only) lower temperature. Current Claude tiers (Opus 4.7+/Opus 5/Sonnet 5/Fable 5) reject sampling parameters, so lean on the levers below instead. - Strict schemas - Few-shot anchors - Workflow gates - Ensemble/self-consistency only when cost allows

Do not promise bit-identical outputs in customer SLAs. Promise **process SLAs** and quality metrics.

G4. Multimodal design notes

When inputs include images/PDFs:

- Decide whether a vision model call vs an OCR+text pipeline is better for tables.
- Bound image resolution and count for cost.
- Treat OCR text as untrusted retrieved content.
- Eval with real document scans, not only clean digital PDFs.

G5. Internationalization & tone

Enterprise designs often need multilingual support. Public Claude models are multilingual. Still:

- Put language policy in the output contract (“respond in user’s language”).
 - Eval per language. Quality is not uniform only because you declare it.
 - Beware mixed-language retrieval corpora and embedding/search gaps (file 02).
-

Part H — Additional worked scenarios

Scenario H1 — Internal policy Q&A

Requirements: Employees ask HR policy questions. Answers must cite policy. Wrong legal advice is high risk. **Design:** Progressive RAG over a versioned policy corpus + citation-required output + refuse when retrieval is empty + HITL for “exceptions/approvals” intents routed separately. Mid-tier model. Cache system+tool defs. **Not:** Parametric-only answers from model memory. Not a fully autonomous agent with an email-send tool.

Scenario H2 — Coding assistant for a monorepo

Requirements: Help developers navigate a large repo, propose patches, and run tests. **Design:** Agent pattern with repository tools, a test runner,

and strict write permissions to branches — not direct prod deploy. Strong coding model. Context via progressive file fetch, not a whole monorepo dump. Server-managed settings / approved MCP tools for enterprise Claude Code (details in file 04). **Eval:** SWE-style task suites + security regression (no secret exfiltration).

Scenario H3 — Customer support copilot

Requirements: Suggest replies. Agents send. p95 < 2s for draft. **Design:** Fast model for draft. Retrieval of account + macros. Optional escalate to mid model for angry/VIP. Workflow, not an open agent. Cache macros/policies. HITL is the human agent (always-on). **Metrics:** Edit distance, CSAT, handle time, hallucination rate on factual fields.

Scenario H4 — Overnight contract clause extraction

Requirements: 50k PDFs nightly. Cost-sensitive. Morning dashboard. **Design:** Batch API + map-reduce workflow + schema validation + exception queue. Not interactive Opus-class chat UX.

Part I — Extended practice Q&A (26–35)

Q26. What is the difference between a process SLA and promising identical LLM strings? **A.** Process SLAs cover latency, availability, and quality rates. Identical strings are unrealistic for generative models.

Q27. Why run OCR before or instead of pure vision for dense tables? **A.** Tables often extract more reliably via OCR+structure pipelines. Validate with evals either way.

Q28. What belongs in a model upgrade canary plan? **A.** Shadow/replay metrics, percentage rollout, kill switch, prompt pin alignment, and rollback owner.

Q29. How do you prevent “context excessive content”? **A.** Retrieval filters, top-k caps, relevance thresholds, progressive discovery, and periodic corpus hygiene.

Q30. When is Batch API the wrong choice? **A.** When the end user needs interactive, low-latency responses.

- Q31.** Name two variance-reduction techniques for extraction. **A.** JSON schema validation + few-shot anchors (also deterministic post-processors). Temperature tuning is not a lever on current Claude tiers — those tiers reject sampling parameters there.
- Q32.** Why separate research-agent credentials from execute-agent credentials? **A.** Least privilege and blast-radius control if the research path is prompt-injected.
- Q33.** What should you log for an architectural incident review (at minimum)? **A.** Model ID, prompt/Skill versions, tool calls (redacted), retrieval IDs, latency, and decision outcome.
- Q34.** A design uses three agents that all share admin SaaS tokens. Primary issue? **A.** Over-privilege / missing security boundaries — multi-agent without isolation.
- Q35.** How do Skills interact with caching strategy? **A.** Stable Skill content can live in cacheable prefixes. Version carefully so cache and behavior stay coherent.
-

Expanded checklist additions

- ☐ I can describe map-reduce over documents with schema consistency risks
 - ☐ I can place HITL triggers in a diagram as first-class stages
 - ☐ I can rough-order cost with cache hit rate and escalation fraction
 - ☐ I can write a model migration canary outline
 - ☐ I can separate trusted instructions from untrusted retrieved text
 - ☐ I can choose Batch vs realtime with a one-line rationale
 - ☐ I can name five anti-patterns and their fixes
 - ☐ I can design a support copilot that meets a 2s draft latency via model routing
-

Glossary additions

Term	Meaning
Programmatic gate	Code validation between LLM stages

Term	Meaning
Map-reduce LLM	Parallel per-doc extract + synthesize
Shadow traffic	Parallel scoring of a candidate model/prompt
Canary release	Gradual production exposure with rollback
God prompt	Anti-pattern: one prompt does all jobs
Tool sprawl	Excessive tools exposed to one agent
Process SLA	Operational targets without bit-identical outputs
Untrusted context	Retrieved/user content that may contain injections
Exception queue	Human review path for failed gates
Prompt/Skill pin	Version identifier for reproducible instructions

(Deepening passes continue below: Parts J–R.)

Part J — Primary-study deep dive: architecture decision scenarios

Use these as timed drills. Write a 6–10 sentence decision. Then check the scoring keys.

Scenario J1 — Claims triage under a 3-day SLA

Context: An insurer wants Claude to cut claims cycle time from 11 days to 3. Volume is 8k claims/day. Regulators require an auditable decision trail. Payouts above \$5k need a human signature. Corpus: policy PDFs + structured claim forms.

Decide: pattern per stage, model tiering, HITL placement, success metrics.

Scoring key (preferred shape): 1. **Workflow** with stages: ingest/OCR → extract fields → policy retrieve+cite → recommend → (HITL if payout ≥

\$5k or low confidence). 2. **Router:** Haiku-class for intent/confidence. Sonnet-class for extract+recommend. Escalate hard disputes to higher-capability only when evals justify. 3. Metrics: cycle time, auto-resolution rate below threshold, citation coverage, overturn rate by humans, p95 latency of auto path, \$ per claim. 4. Anti-patterns: a single autonomous agent with write tools to payment systems. Monolithic “paste whole policy book” context.

Scenario J2 — Internal coding assistant for a 40-repo org

Context: Platform team wants Claude across IDEs/CLI. Secrets must not leave the vault. Some repos are regulated. Latency for autocomplete help must stay interactive. Deep refactors can be async.

Decide: progressive vs monolithic context. Skills vs giant CLAUDE.md. Model routing.

Scoring key: Progressive discovery via repo CLAUDE.md + retrieval over docs + MCP only for approved tools. Separate **interactive** mid-tier path from **deep agent** high-capability path with budgets. Regulated repos: stricter allowlists + no server web tools. Org Skills for ADR/test templates. Avoid one 50k-token global prompt.

Scenario J3 — Multilingual support copilot with CRM tools

Context: 12 languages, CRM write actions (case notes, refunds $\leq$ \$50 auto). Brand tone is critical. Hallucinated refunds are catastrophic.

Decide: pattern, output contract, tool authZ, eval focus.

Scoring key: Augmented LLM + tools inside a **workflow gate** for refunds (schema validation + policy engine + amount cap). Do not let the model “decide” refund eligibility alone — code enforces caps. Eval: language quality sample + tool-call correctness + refusal on out-of-policy. Citations are not primary. **Action correctness** is.

Scenario J4 — Overnight batch contract clause extraction

Context: 200k PDFs/night. Structured JSON to warehouse. 98% field accuracy target. Cost sensitive.

Decide: Batch vs realtime API. Map-reduce. Model tier. Validation.

Scoring key: Batch API + map-reduce (per-doc extract → schema validate → quarantine fails → sample human audit). Prefer mid-tier with strong

schema unless hard clauses need escalate. Cost model before GA. Not an interactive agent.

Part K — Trade-off tables (desk reference)

K1. Capability vs latency vs cost (operator view)

Lever	Improves	Often worsens	Exam cue
Higher-capability model	Hard reasoning, coding, judgment	Latency, \$	High-stakes offline review
Faster/cheaper model	p95, throughput, \$	Edge-case quality	Classify/extract/route
Thinking / higher effort	Multi-step correctness	Tokens, latency, cache stability	Hard math/logic/planning
Prompt caching	\$ and TTFT on stable prefixes	Design discipline (prefix order)	Large static system+tools
Smaller context	Latency, focus	Completeness	Progressive discovery
Parallel stages	Wall-clock	\$ and merge complexity	Independent branches
Batch API	Unit economics	Interactivity	Overnight jobs
Stronger eval gates	Quality, safety	Time-to-ship	Regulated GA

K2. Pattern vs auditability

Pattern	Auditability	Flexibility	Typical cost variance
Augmented LLM	High (one call)	Low	Low
Workflow	Very high (stage logs)	Medium	Medium
Agent	Medium (needs traces)	High	High
Multi-agent	Needs hop IDs	Highest	Highest

General rule: prefer the leftmost pattern that meets the requirement. Move right only when evals show the left fails.

K3. Context strategy trade-offs

Strategy	Token efficiency	Completeness risk	Ops complexity
Monolithic dump	Poor at scale	Low (if fits)	Low initially, blows up later
Progressive RAG	Good	Retrieval misses	Medium
Tool/MCP discovery	Good	Tool selection errors	Medium-high
Hybrid cached core + dynamic	Best common case	Dual-path bugs	Medium

Part L — Failure-mode analysis (platform & design)

Failure mode	Symptoms	Likely root	Design fix
Lost-in-the-middle	Ignores middle docs	Too much packed context	Rank, shrink k, progressive
Format drift	Downstream parse fails	Weak output contract	Schema + validator + repair loop
Cache thrash	Cost spikes, slow TTFT	Volatile prefix / thinking flips	Stabilize prefix & configs
Over-agentification	Unpredictable writes	Agent where workflow fits	Convert to staged workflow
Model mismatch	Great demos, prod misses	Eval on toy data	Production-like eval set
Context starvation	“I do not know” wrongly	Over-aggressive filtering	Relax filters. Query rewrite
Skill/prompt conflict	Contradictory behavior	Multiple instruction layers	Precedence doc + tests
Multimodal OCR fail	Bad extracts from scans	Vision without validation	Confidence + human queue

Failure mode	Symptoms	Likely root	Design fix
Router collapse	Everything escalates	Bad confidence calibration	Threshold tuning + metrics
Compaction drift	Long agents forget rules	Summary dropped policy	Pin never-compact sections

Part M — Production checklist (Platform & Solution Design)

M1. Before architecture review

- ☐ Problem statement, users, and “non-goals” written
- ☐ Value pillars mapped to metrics and owners
- ☐ Pattern choice with rejected alternatives documented (ADR)
- ☐ Model tier per stage with cost envelope
- ☐ Context budget diagram (static vs dynamic vs tools vs thinking)
- ☐ Output schemas and validation ownership named
- ☐ Degradation mode if Claude or tools unavailable
- ☐ Eval stub attached (even if thin)

M2. Before pilot

- ☐ Golden set of ≥ 50 –100 realistic cases
- ☐ Latency budget per stage signed by product
- ☐ Cost kill-switch / max tokens per request
- ☐ Logging redaction policy for prompts/tool results
- ☐ Model pin / upgrade policy draft
- ☐ HITL queues staffed for expected exception rate

M3. Before GA

- ☐ Shadow or A/B vs baseline complete
- ☐ Failure modes above reviewed with security/ops
- ☐ Runbook + on-call named
- ☐ Stakeholder sign-off on known limitations (file 04)
- ☐ Continuous eval in CI for prompts/tools/schemas

Part N — Additional architecture decision drills (short)

1. **Fixed KYC checklist with regulators watching** → Workflow + validators. Not a free-form agent.
 2. **Unknown-root-cause production incident research** → Agent + read-only tools + budget + HITL for remediations.
 3. **One front door for HR/IT/Facilities FAQs** → Router + per-domain RAG corpora + separate credentials.
 4. **Generate 10 independent product descriptions** → Parallel map. Aggregate. Schema validate.
 5. **High-stakes legal memo** → High-capability + RAG citations + HITL attorney. Never auto-send.
-

Part O — Extended Q&A (36–40)

Q36. A stakeholder wants “one mega agent” to replace a 7-stage underwriting checklist that auditors inspect. Best response? **A.** Propose a **workflow** that mirrors the checklist with logged stage outputs. Reserve agents for open research sub-tasks only. Auditability beats flexibility here.

Q37. Interactive UX needs $p95 < 2s$, but hard cases need deep reasoning. Best design? **A. Router:** fast path mid/fast model. Async or “continue analyzing” path for deep thinking. Communicate UX expectation.

Q38. Cache hit rate is 5% despite a huge system prompt. Most likely cause? **A.** Volatile content or tool defs inserted **before** the stable prefix, or thinking/config changes breakpoints every call.

Q39. Select TWO for map-reduce over 500 contracts. Option (a) is parallel per-doc extract. Option (b) is one call with all PDFs. Option (c) is schema validate each. Option (d) is single agent with write-to-ERP unbound. **A. (a) and (c).**

Q40. Skills vs stuffing expertise into every user message—when Skills win? **A.** When expertise is **stable, reusable, versioned** across many requests. Keep request-specific facts in user/tool channels.

Part P — Rapid review card (Domain 1+2 ≈ 30%)

- Smallest model that passes evals. Escalate with evidence.

- Workflow when steps are known. Agent when they are not. Multi-agent only with clear roles.
- System prompt = policy + interface contract.
- Progressive discovery for large tools/corpora. Cache stable prefixes.
- Value pillars must name levers (efficiency, transformation, productivity, cost, performance SLAs).
- Document ADRs. Reject alternatives explicitly.
- Validate machine outputs in code—never hope.
- Compaction/thinking/caching interactions are design concerns, not afterthoughts.

Cross-read Integration/Evals in 02, Safety in 03. (Parts Q–R continue below.)

Part Q — Model routing & cost envelopes (worked numbers)

You need rough estimate math. You do not need memorized price sheets (prices change—verify docs).

Cost envelope worksheet

Monthly tasks T

× tokens_in_avg × (\$/M input)

+ tokens_out_avg × (\$/M output)

+ cache_write share × write_rate + cache_read share × read_rate

+ tool_overhead_tokens

= monthly model \$

Add: retrieval infra, HITL labor, observability storage. Compare to value per successful task.

Routing example (illustrative proportions)

Route	% traffic	Model class	Notes
Classify/refuse	40%	Fast	Gatekeeper
Standard answer	45%	Mid	Cached policy prefix
Hard reason / dispute	15%	High	Thinking on. Async OK

If the hard route is 60% of cost but only 15% of volume, eval whether mid+critique is cheaper for a slice.

When Batch API wins

High volume, latency-insensitive, structured jobs (extraction, classification, overnight reports). Not for chat UX. Pair with schema validation and quarantine queues.

Pinning & upgrades

Pin model IDs in prod. Upgrade via shadow → eval gate → canary → full. Treat prompt and model as a **coupled** release artifact.

Part R — Additional failure modes & exam stems

Stem cue	Prefer
“Auditors need stage evidence”	Workflow + logs
“Steps unknown. Explore codebase”	Agent + budget + allowlist
“12 intents one bot”	Router
“Huge PDF corpus”	RAG progressive, not monolith
“p95 interactive”	Fast/mid + defer deep thinking
“\$ sensitive overnight”	Batch + mid + validate

Q41. Cache reads are cheap, so put the user question first in every request for readability—good idea? **A. No**—put **stable** content first for caching. Volatile user content last.

Q42. Select TWO decomposition axes: by risk (read vs write), by style preference, by SLA path. **A.** By risk and by SLA path.

Q43. Output must feed a payment API—what is mandatory? **A. Schema validation in code** before side effects. Enums for status. Idempotency keys.

End of file 01. Next: 02-enterprise-integration-production.md (Integration 19% + Evaluation 16%).

01 — Claude Platform & Solution Design

CCAR-P condensed domain 1 Official domains mapped here: Solution Design & Architecture (17%) · Claude Models, Prompting & Context Engineering (13%) **Combined exam weight covered by this file: ~30%**

Disclaimer

These are **original study notes** for exam preparation. They are grounded in **public** Anthropic platform documentation, published CCAR-P blueprint summaries, and widely taught architecture patterns for LLM systems. They do **not** reproduce official course content. Model names, pricing, and context limits change; always reconcile numbers with current platform.claude.com docs before production decisions or exam day. This pack is independent and not affiliated with Anthropic.

Overview

CCAR-P expects you to think like a **production architect**, not a prompt tinkerer. In scenarios you will:

1. Translate a messy business problem into a **Claude-powered solution shape** (inputs → processing → outputs → feedback).
2. Choose among **workflow**, **augmented LLM**, **agentic**, and **multi-agent** patterns with explicit trade-offs.
3. Select a **model tier** (capability vs latency vs cost vs context) and justify it.
4. Engineer **prompts and context** so the system is reliable under token, safety, and reuse constraints.
5. Connect design choices to **business value pillars** — per the official guide's wording: efficiency, transformation, **productivity**, cost, and performance SLAs.

This file deep-dives platform capabilities and solution design. Integration mechanics (MCP wiring, RAG pipelines, eval harnesses) appear in file 02; governance in 03; stakeholder/lifecycle/enablement in 04.

Key map (user domain ↔ official tasks)

Architect task	Official domain	What “good” looks like on the exam
Problem → solution framing	Solution Design 17%	Clear value hypothesis, constraints, success metrics
Pattern selection (workflow/agent/...)	Solution Design 17%	Prefer simplest pattern that meets audibility & reliability needs
Decomposition & orchestration	Solution Design 17%	Stages, gates, retries, human checkpoints where risk warrants
Model selection	Models/Prompting 13%	Match complexity, latency budget, context size, cost
System prompts & behavioral controls	Models/Prompting 13%	Role, constraints, output schema, refusal boundaries
Few-shot / structured reasoning	Models/Prompting 13%	Examples when format/edge cases matter; thinking for hard reasoning
Context window strategy	Models/Prompting 13%	Progressive discovery vs curated context; caching & Skills for reuse
Prompt caching / modular prompts / Skills	Models/Prompting 13%	Reuse static prefixes; avoid stuffing every request

Part A — Claude models for architects

A1. How to think about the model ladder

Public docs describe a **family** of Claude models with different speed/intelligence/cost points. Exact IDs and prices move over time; for CCAR-P,

memorize the **decision logic**, not a frozen price sheet:

Decision factor	Prefer higher-capability tier when...	Prefer faster/cheaper tier when...
Task complexity	Multi-step planning, hard coding, ambiguous requirements, high-stakes judgment	Classification, extraction, routing, short rewriting
Latency SLA	Offline batch, async jobs, human-reviewed drafts	Interactive UX, high QPS gatekeepers
Context size	Large codebases, long docs, multi-hour agent traces	Short turns, tool-heavy loops with small payloads
Cost envelope	Low volume / high value (contracts, architecture reviews)	High volume / low marginal value
Error cost	Wrong answer is expensive or unsafe	Wrong answer is cheap to catch/retry

Architect rule: Default to the **smallest model that meets quality bars on your eval set**. Promote capability only when evals prove a gap. Demote when latency/cost budgets fail and quality still holds.

A2. Capability knobs that show up in designs

Public platform features architects must place correctly:

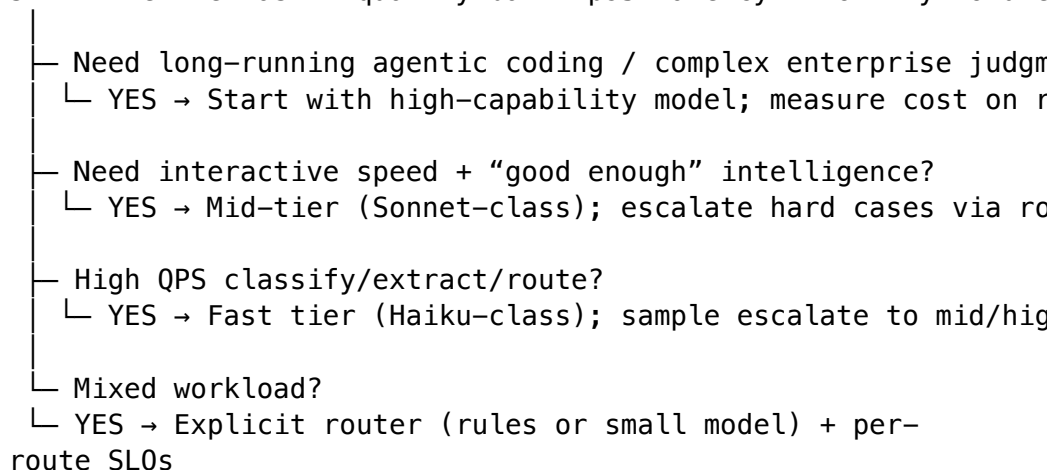
- **Context windows:** Current frontier Claude models commonly advertise **large** windows (hundreds of thousands to **~1M tokens** on supported models/platforms). Everything in the request counts: system prompt, messages, tool defs, images/docs, tool results, and (when used) thinking tokens. Caching changes **price**, not whether tokens occupy the window.
- **Max output:** Synchronous Messages API vs Batch (batch may support extended output via documented beta headers—verify current docs).
- **Thinking / adaptive thinking / effort:** Use deeper reasoning for hard problems; control cost/latency with **effort levels**

(`output_config.effort`). Adaptive thinking replaced fixed thinking budgets — `budget_tokens` is deprecated on the 4.6 generation and removed (rejected with an error) on current models. Changing thinking config invalidates message-level prompt-cache breakpoints (the tools/system prefix stays cached)—design for cache stability.

- **Vision / multimodal:** Image input where the product needs screenshots, diagrams, or scanned docs.
 - **Deployment surfaces:** Claude API, Amazon Bedrock, Vertex AI, Microsoft Foundry, Claude Platform on AWS—each with ID naming and data-residency nuances. Architects pick based on enterprise procurement, region, and compliance—not brand preference alone.
- Full CSP / delivery-route governance decision table lives in file 02 Part D5** (constraint elimination, ZDR themes, model-ID traps, Q&A).

A3. Model selection decision tree

START: Define task + quality bar + p95 latency + monthly volume



Exam trap: Choosing Opus/Fable-class for every step of a deterministic ETL-like pipeline. Prefer **workflow + smaller model** with validation gates.

A4. Multi-model architectures

Common production shapes:

1. **Router** → **specialist:** Cheap classifier chooses extract vs reason vs refuse.

- 2. **Draft** → **critique**: Mid model drafts; stronger model or rubric critic reviews.
- 3. **Shadow eval**: New model runs in parallel; compare offline before cutover.
- 4. **Fallback**: Primary timeout → degraded model or cached answer + human queue.

Document **why** each hop exists (quality, cost, safety). Orphan hops fail reviews and fail exam scenarios that ask for simplest viable design.

Part B — Prompting & behavioral control

B1. System prompt as architecture, not poetry

Treat the system prompt as a **policy + interface contract**:

Section	Purpose	Example content (illustrative)
Role & scope	Bound the job	“You are a procurement assistant for freight quotes. You do not negotiate contracts.”
Tools policy	When to call what	“Call get_policy before recommending. Never invent policy clauses.”
Output contract	Schema / format	JSON schema, citation requirements, severity enums
Safety / refusal	Local guardrails	Disallowed actions; escalation phrases
Quality bar	How to handle uncertainty	“If confidence low, ask clarifying question or return needs_review.”
Style	Audience	Exec brief vs engineer debug

Prefer: Explicit constraints + structured outputs. **Avoid:** Vague “be

helpful and thorough” without success criteria.

B2. Prompting techniques architects should name correctly

Technique	When to use	Failure mode
Zero-shot	Clear tasks, strong base capability	Ambiguous formats drift
Few-shot	Format, tone, edge-case handling	Bad examples become law; token bloat
Chain-of-thought / structured reasoning	Multi-step logic (prefer API thinking features where available)	Leaking chain into user-facing channel; cost
Decomposition in prompt	Single call must cover subskills	Still weaker than real workflow gates for audit
Tool-forced (tool_choice)	Retrieval or action must happen	Over-forcing when optional tools exist
Constitutional / principle prompts	Local safety/style overlays	Conflicts with org policy if not reviewed

B3. Output contracts and machine consumers

If a downstream system parses Claude’s output, design for **machines first**:

- Prefer **JSON / typed schemas** with validation before side effects.
- Separate **user-facing prose** from **machine fields**.
- Include explicit enums for status (ok, needs_review, refused).
- Validate with code; never “hope” the model obeyed.

B4. Skills and modular prompt reuse (public product concept)

Public Claude product docs describe **Skills** (and related modular instruction packs) as reusable specialized instructions/resources so teams do not paste giant instruction blobs into every request. Architect takeaway for CCAR-P:

- Put **stable** expertise into modular reusable units.
- Keep **request-specific** data in the user/tool channel.

- Version Skills like code; review for prompt-injection surfaces when Skills can pull external content.

Part C — Context engineering

C5. What counts toward the window

Everything sent counts. Architects must budget:

1. System / Skills / static policy
2. Tool definitions (large MCP tool catalogs can dominate)
3. Conversation history
4. Retrieved chunks / files
5. Tool results
6. Thinking tokens (when enabled)

Token counting API (public) exists to estimate before send—use it in design reviews for worst-case prompts.

C6. Progressive discovery vs monolithic context

Strategy	Idea	Best for	Risk
Monolithic context	Stuff “everything possibly relevant” into the prompt	Small corpora, strong need for global consistency	Noise, cost, lost-in-the-middle, cache thrash
Progressive discovery	Start lean; retrieve/tools pull what is needed	Large corpora, tool ecosystems, agents	Missing retrieval → incomplete answers
Hybrid	Cached core policy + dynamic retrieval	Most enterprise apps	Complexity of two paths

Exam preference: When corpora or tool sets are large, prefer **progressive discovery** (RAG/tools/MCP tool search) over dumping megabytes of text. When the task is short and closed-world, a curated static context can be simpler and more auditable.

C7. Prompt caching (architect view)

Public docs: prompt caching reduces cost/latency for repeated prefixes (system prompts, large reference docs, tool defs). Cache writes cost more than ordinary input; cache **reads** are heavily discounted. TTLs commonly include short (e.g., ~5 minutes) and longer (e.g., ~1 hour) options—confirm current docs.

Design rules:

- Put **stable content first** (cacheable prefix).
- Put **volatile user content last**.
- Avoid reshuffling tool defs every request.
- Changing thinking budgets/modes can invalidate breakpoints—stabilize configs in hot paths.
- Remember: cached tokens **still consume** context window capacity.

C8. Compaction / context editing

For long-running agents and chats, public platform features include **compaction** (server-side summarization near limits) and **context editing** strategies (e.g., clearing old tool results). Architects should:

- Define what must **never** be summarized away (safety policy, user entitlements).
- Log when compaction occurs for audit.
- Test agent quality after compaction—summary drift is a real failure mode.

C9. Context engineering checklist

- ☐ Priority order of context sections documented
 - ☐ Retrieval top-k and ranking justified
 - ☐ Tool catalog size managed (tool search / allowlists)
 - ☐ Cache breakpoints stable across turns
 - ☐ PII/minimization rules for what enters context
 - ☐ Escape hatches when context insufficient (needs_more_data)
-

Part D — Solution design & architecture patterns

D1. End-to-end solution skeleton

Every CCAR-P-worthy design can be drawn as:

```
[Actors/Channels] → [Ingress & AuthN/Z] → [Orchestration layer]
→ [Claude calls / tools / retrieval]
→ [Validators & policy gates]
→ [Side effects / UX]
→ [Telemetry + eval feedback loop]
```

Map each box to owners (app team, platform, security, data). Missing owners = lifecycle failure (see file 04).

D2. Pattern catalog (know these cold)

Anthropic-style teaching (and exam scenarios) distinguish **predefined workflows** from **agents that decide their own path**.

1) Augmented LLM (single call + light tools) One Claude call, maybe with retrieval or a tool or two. **Use when:** Task is bounded; one reasoning pass suffices. **Pros:** Simple, cheap, easy to eval. **Cons:** Weak for multi-stage audit; limited recovery.

2) Workflow / prompt chaining Code owns the control flow: Step A → validate → Step B → ... **Use when:** Stages are known, ordered, and need intermediate checks (procurement quote extract → policy validate → recommend). **Pros:** Auditable, retryable per stage, clear SLAs. **Cons:** More latency than one call; more engineering.

3) Parallelization Independent subtasks run concurrently; aggregator merges. **Use when:** No data dependencies between branches. **Trap:** Parallelizing dependent stages (validate before extract finishes).

4) Routing Classifier/router sends to specialized prompts, models, or workflows. **Use when:** Heterogeneous intents share one front door.

5) Evaluator–optimizer / critique loops Generator produces; evaluator scores against rubric; iterate N times or until pass. **Use when:** Quality variance is high and latency allows.

6) Agentic (model-driven control flow) Claude plans, chooses tools, and loops until stop condition. **Use when:** Steps cannot be fully hard-coded; environment is open-ended (research, debugging unknown code-bases). **Pros:** Flexibility. **Cons:** Cost, non-determinism, harder audit—needs budgets, tool allowlists, HITL for high impact.

7) Multi-agent Specialized agents with handoffs (researcher / writer / reviewer) or supervisor pattern. **Use when:** Clear role separation improves quality or security boundaries. **Trap:** Multi-agent theater—three agents doing what one workflow could do.

D3. Pattern selection table

Situation	Prefer	Avoid
Fixed stages + compliance audit	Workflow + gates	Fully autonomous agent
Open-ended investigation	Agent + tool allowlist + budget	Brittle 12-step workflow
Independent doc sections	Parallel map-reduce	Serial chain
Mixed intents	Router	One mega-prompt
High-stakes irreversible action	Workflow + HITL approval	Agent with write tools unbound
Need citations from corpus	RAG + citation blocks	Pure parametric memory

D4. Decomposition techniques

- 1. **By business stage** (intake → enrich → decide → act → explain)
- 2. **By risk** (read-only research vs write actions)
- 3. **By data domain** (HR vs finance agents with separate credentials)
- 4. **By SLA** (fast path vs deep path)
- 5. **By confidence** (auto vs human queue)

Write **interfaces** between stages: schemas, idempotency keys, error codes. Exam scenarios reward designs that can retry stage 2 without redoing stage 1.

D5. Business value alignment

Tie architecture to measurable outcomes:

Value pillar	Example metric	Design lever
Efficiency	Minutes saved per case	Automation rate + HITL only on exceptions
Productivity	Tasks completed per person / throughput	Copilot adoption, automated drafts, faster review loops
Transformation	New capability enabled	Agents + tools unlocking previously manual research
Cost	\$ per successful task	Caching, smaller models, batch API
Performance	p95 latency	Parallelism, Haiku router, smaller context
Quality (<i>not in the guide's pillar list — quality stems live in the Evaluation domain</i>)	Error rate vs baseline	Workflow gates, eval harness, stronger model on hard route

If a scenario lists SLAs, your answer must **name the lever** that protects the SLA.

D6. Non-functional requirements checklist

- Availability & degradation mode
- Latency budgets per stage
- Cost ceilings / kill switches
- Data residency & retention
- Audit logging of prompts/tool calls (with redaction)
- Secrets handling (never in prompts)
- Rate limits & backoff
- Model deprecation / pin strategy

D7. Worked mini-scenario (freight quotes)

Need: Extract quote fields → validate policy → recommend carrier; auditable. **Pattern:** Fixed **workflow** with three Claude (or Claude+code) stages and validators between them. **Not:** Single monolithic autonomous agent; not parallel independent extract/validate/recommend. **Why:** Dependencies + audit. This pattern appears repeatedly in public practice

explanations and matches Anthropic’s workflow-vs-agent distinction.

Part E — Design documentation artifacts

For lifecycle handoff (file 04), architects should produce:

- 1. **Context diagram** (systems Claude touches)
- 2. **Sequence diagram** for the happy path + one failure path
- 3. **Decision record** (pattern, model, why not alternatives)
- 4. **Threat notes** (injection, data leakage—deep dive in file 03)
- 5. **Eval plan** stub (metrics—deep dive in file 02)
- 6. **Runbook** pointers (on-call, rollback model pin)

Exam questions may ask what to communicate to executives vs engineers—same design, different altitude.

Decision trees & quick tables (exam desk reference)

Model + pattern joint decision

Quality need	Path predictability	Recommended shape
High	High	Workflow + mid/high model on hard steps
High	Low	Agent + strong model + budgets + HITL
Medium	High	Workflow + mid model
Medium	Low	Router + agent for long tail
Low	Any	Fast model + simple prompt; maybe no LLM

Context strategy picker

Corpus size	Query diversity	Strategy
Tiny (< context comfortably)	Low	Curated static context + cache
Large	Low	Metadata filters + light RAG
Large	High	Hybrid search RAG + rerank
Tools many	High	MCP + tool search / progressive

Exam traps (Platform & Solution Design)

1. **Agent everywhere** — Prefer workflow when steps are known.
 2. **Biggest model everywhere** — Overkill fails cost/latency scenarios.
 3. **Ignoring dependencies in parallel designs.**
 4. **Treating prompt caching as free context** — Still counts toward window.
 5. **Monolithic context for enterprise knowledge bases.**
 6. **No output schema** when systems consume results.
 7. **Skipping business metrics** — Architecture without value story.
 8. **Forgetting degradation** — What happens when Claude/API is down?
 9. **Multi-agent without boundaries** — Shared over-privileged tools.
 10. **Changing cache-critical prefixes every request.**
 11. **Putting secrets in prompts or tool args logs.**
 12. **Equating “citations in prose” with grounded retrieval** — Prefer structured citations/search_result patterns from public docs.
-

Practice Q&A (25)

Q1. When should you prefer a fixed workflow over an autonomous agent? **A.** When stages are known, ordered, and need intermediate validation/audit.

Q2. A system must extract, then validate, then recommend. Parallel independent calls are proposed. What is wrong? **A.** Later stages depend on earlier outputs; parallelization breaks dependencies.

Q3. Name three inputs to model selection. **A.** Task complexity/quality bar, latency SLA, cost/volume (also context size, error cost).

Q4. Do prompt-cache tokens still count toward the context window? **A.** Yes—caching affects billing/latency, not window occupancy.

Q5. What belongs in a system prompt’s “output contract”? **A.** Schema/format, required fields, enums, citation rules, uncertainty handling.

Q6. Large MCP tool catalogs hurt you how? **A.** Tool definitions consume context; prefer allowlists or tool search / progressive discovery.

Q7. What is progressive discovery? **A.** Start with lean context; retrieve or call tools to pull only needed information.

Q8. When are few-shot examples most justified? **A.** To lock format, tone, or rare edge-case handling—not as a substitute for retrieval of facts.

Q9. Give one valid use of a multi-agent design. **A.** Separating roles with different permissions (e.g., researcher read-only vs executor write with approval).

Q10. What should you do before promoting a larger model in production? **A.** Prove a quality gap on a representative eval set; measure cost/latency impact.

Q11. Why pin model IDs in production architectures? **A.** Avoid surprise behavior changes; manage deprecations deliberately.

Q12. What is an evaluator–optimizer loop? **A.** Generate → score against rubric → refine until pass or budget exhausted.

Q13. Name a business-value metric for an efficiency use case. **A.** Average handle time, cases auto-closed, or minutes saved per ticket.

Q14. Where should volatile user content sit relative to a cacheable prefix? **A.** After the stable prefix (end of prompt), so the cache prefix remains stable.

Q15. When is a single augmented LLM call enough? **A.** Bounded task, low need for intermediate audit, one reasoning pass suffices.

Q16. What risk does compaction introduce in long agents? **A.** Summary drift—critical constraints or facts may be lost if not protected.

Q17. How do you protect a p95 latency SLA in a multi-stage workflow? **A.** Budgets per stage, timeouts, parallel independent work, smaller models on easy stages, async where UX allows.

Q18. Why separate machine JSON from user prose? **A.** Parsers need stability; UX needs readability—mixing causes brittle integrations.

Q19. What is a router pattern good for? **A.** Heterogeneous intents behind one endpoint with different prompts/models/workflows.

Q20. Give a reason to choose cloud-hosted Claude (Bedrock/Vertex/Foundry) over direct API. **A.** Enterprise procurement, existing cloud commit, data residency/compliance controls (scenario-dependent).

Q21. What should a stage interface include for retries? **A.** Typed schema, idempotency key, explicit error codes, and validated outputs.

Q22. Why might changing the thinking/effort configuration between turns

be costly? **A.** It invalidates message-level prompt-cache breakpoints (the tools/system prefix stays cached), increasing latency and spend. (Note: fixed budget_tokens thinking budgets are deprecated/removed on current models — adaptive thinking plus effort replaced them.)

Q23. What is the architect’s default stance on tool permissions? **A.** Least privilege—only tools needed for the role/stage.

Q24. How do Skills help teams? **A.** Modular reusable instructions/resources reduce prompt sprawl and improve consistency.

Q25. A stakeholder wants “fully autonomous” processing of regulated approvals. What do you propose instead? **A.** Workflow with policy checks and human-in-the-loop for high-impact approvals; agent only for bounded research if needed.

Pre-exam checklist (Platform & Solution Design)

- ☐ I can explain workflow vs agent with one crisp example each
 - ☐ I can pick a model tier from quality/latency/cost constraints
 - ☐ I can sketch end-to-end: ingress → orchestrate → Claude/tools → validate → act → telemetry
 - ☐ I can design a cache-friendly prompt layout
 - ☐ I can justify progressive discovery vs monolithic context
 - ☐ I can write an output schema and validation gate
 - ☐ I can map architecture choices to business metrics and SLAs
 - ☐ I can list NFRs: residency, audit, degradation, rate limits, model pinning
 - ☐ I know when multi-agent helps vs when it is theater
 - ☐ I can spot dependency mistakes in “parallelize everything” answers
-

Glossary

Term	Meaning
Augmented LLM	Single LLM call enriched with retrieval/tools
Workflow / prompt chaining	Code-orchestrated sequence of LLM/tool steps

Term	Meaning
Agent	Model-driven control flow with planning and tool loops
Multi-agent	Multiple specialized roles with handoffs
Context engineering	Deliberate selection, ordering, and budgeting of tokens
Prompt caching	Reusing billed/processed stable prompt prefixes
Progressive discovery	Fetch context on demand via retrieval/tools
Skills	Modular reusable instruction/resource packs
Output contract	Schema and rules for machine-consumable responses
Router	Component that dispatches to specialized paths
Effort / thinking	Controls for deeper reasoning vs latency/cost
Model pin	Fixed model ID for reproducible production behavior
NFR	Non-functional requirement (latency, cost, residency, etc.)
HITL	Human-in-the-loop review/approval
SLA / SLO	Service level agreement / objective

Part F — Deeper solution patterns & anti-patterns

F1. Prompt chaining with programmatic gates

A production-grade chain is not “three prompts in a row.” It is:

1. **Deterministic prep** (parse upload, normalize units, load tenant config).
2. **LLM step** with a narrow contract.
3. **Programmatic validator** (schema, business rules, allowlists).

- 4. **Branch:** pass → next stage; fail → repair prompt, retry with cap, or human queue.
- 5. **Idempotent side effects** only after final gate.

This hybrid of LLM + code is what CCAR-P scenarios reward. Pure LLM self-checks help but are weaker than typed validators for compliance narratives.

F2. Map-reduce over documents

For large packets (RFPs, due diligence binders):

- **Map:** Same extract prompt per chunk/doc in parallel (mid/fast model).
- **Reduce:** Stronger model synthesizes with citations and conflict detection.
- **Verify:** Spot-check conflicting fields; optional second-pass critique.

Architect concerns: chunk boundaries that split tables, consistent schemas across mappers, and reduce-stage token blowups.

F3. Human-in-the-loop as an architectural component

HITL is not only a safety topic (file 03). In solution design it is a **first-class stage**:

Trigger	UX pattern	Data to show the human
Low model confidence	Review queue	Top uncertainties + source snippets
High-impact action	Explicit approve/deny	Diff of proposed write + policy cites
Novel intent	Specialist escalation	Conversation transcript + tools used
Eval regression	Shadow mode	Side-by-side old vs new outputs

Design the queue’s SLAs and ownership or the “architecture” fails in production.

F4. Batch vs realtime

Public Batch APIs trade latency for cost/throughput. Architects choose Batch when:

- Jobs are overnight analytics, offline evals, corpus enrichment.
- User is not waiting on a synchronous UI.

Keep **the same prompts and schemas** between realtime and batch so evals transfer. Document different timeout and retry policies.

F5. Anti-patterns library (memorize names)

1. **God prompt** — one prompt owns extract, reason, act, and apologize.
2. **Tool sprawl** — every SaaS verb exposed to one agent.
3. **Context landfill** — entire SharePoint dump in system prompt.
4. **Eval-free launch** — “we’ll know quality when we see it.”
5. **Autonomy without budgets** — unbounded loops and spend.
6. **Silent fallback** — switch models without logging/telemetry.
7. **Schema optional** — free text into ERP.
8. **Security bolted on later** — authZ after tools already write.
9. **Multi-agent cosplay** — three roles, one shared over-privileged toolkit.
10. **Latency theater** — deep thinking on a yes/no classification.

F6. Capacity & cost modeling (back-of-envelope)

For a workflow with stages $S_1..S_n$:

- $\text{Tokens} \approx \sum(\text{input}_i + \text{output}_i) \times \text{requests} \times (1 + \text{retry_rate})$
- Apply cache hit rate on stable prefixes
- Apply router fraction that escalates to expensive models
- Add tool/RAG infrastructure cost separately

Exam tip: if a scenario gives volume + latency, **mention caching, routing, or batching** as cost/latency levers even if you also pick the right pattern.

F7. Migration & model upgrade playbook

1. Pin current model ID.
2. Build shadow traffic or offline replay from production logs (redacted).
3. Compare task metrics + safety metrics.

4. Canary percentage with kill switch.
5. Update runbooks and prompt snapshots.
6. Only then change the pin.

Never “just flip the alias” in regulated environments without a plan.

F8. Interface contracts between product and platform teams

Define:

- **Prompt/Skill ownership** and review cadence
- **Tool registry** ownership and change control
- **Eval suite** ownership and release gating
- **Incident severity** definitions for model misbehavior

CCAR-P blurs into lifecycle (file 04); in design answers, naming owners scores points.

Part G — Prompt & context deep techniques

G1. Separating instructions, data, and untrusted content

Architects should structurally separate:

- **Trusted instructions** (system / Skills / developer)
- **Retrieved documents** (data; may be malicious— injection risk)
- **User utterances** (intent; may be adversarial)
- **Tool outputs** (semi-trusted depending on tool)

Use clear delimiters, cite sources, and instruct the model to treat retrieved content as **untrusted data**. Pair with technical controls in file 02/03 (allowlists, output encoding, privilege separation).

G2. Structured reasoning without leaking chain-of-thought

Prefer platform **thinking** features for hidden reasoning when available. If you must show rationales to users, provide a **user-facing explanation** field that is curated—not raw chain-of-thought—especially in regulated UX.

G3. Determinism levers (practical, not absolute)

LLM calls are stochastic. Reduce variance with:

- (Older models only) lower temperature — current Claude tiers (Opus 4.7+/Opus 5/Sonnet 5/Fable 5) reject sampling parameters, so lean on the levers below instead
- Strict schemas
- Few-shot anchors
- Workflow gates
- Ensemble/self-consistency only when cost allows

Do not promise bit-identical outputs in customer SLAs; promise **process SLAs** and quality metrics.

G4. Multimodal design notes

When inputs include images/PDFs:

- Decide whether vision model call vs OCR+text pipeline is better for tables.
- Bound image resolution/count for cost.
- Treat OCR text as untrusted retrieved content.
- Eval with real document scans, not only clean digital PDFs.

G5. Internationalization & tone

Enterprise designs often need multilingual support. Public Claude models are multilingual; still:

- Put language policy in the output contract (“respond in user’s language”).
- Eval per language—quality is not uniform by fiat.
- Beware mixed-language retrieval corpora and embedding/search gaps (file 02).

Part H — Additional worked scenarios

Scenario H1 — Internal policy Q&A

Requirements: Employees ask HR policy questions; answers must cite policy; wrong legal advice is high risk. **Design:** Progressive RAG over versioned policy corpus + citation-required output + refuse when retrieval empty + HITL for “exceptions/approvals” intents routed separately. Mid-tier model; cache system+tool defs. **Not:** Parametric-only answers from model memory; not fully autonomous agent with email-send tool.

Scenario H2 — Coding assistant for a monorepo

Requirements: Help developers navigate a large repo, propose patches, run tests. **Design:** Agent pattern with repository tools, test runner, and strict write permissions to branches—not direct prod deploy. Strong coding model. Context via progressive file fetch, not whole monorepo dump. Server-managed settings / approved MCP tools for enterprise Claude Code (details in file 04). **Eval:** SWE-style task suites + security regression (no secret exfiltration).

Scenario H3 — Customer support copilot

Requirements: Suggest replies; agents send. $p_{95} < 2s$ for draft. **Design:** Fast model for draft; retrieval of account + macros; optional escalate to mid model for angry/VIP. Workflow not open agent. Cache macros/policies. HITL is the human agent (always-on). **Metrics:** Edit distance, CSAT, handle time, hallucination rate on factual fields.

Scenario H4 — Overnight contract clause extraction

Requirements: 50k PDFs nightly; cost-sensitive; morning dashboard. **Design:** Batch API + map-reduce workflow + schema validation + exception queue. Not interactive Opus-class chat UX.

Part I — Extended practice Q&A (26–35)

Q26. What is the difference between a process SLA and promising identical LLM strings? **A.** Process SLAs cover latency, availability, and quality rates; identical strings are unrealistic for generative models.

Q27. Why run OCR before or instead of pure vision for dense tables? **A.** Tables often extract more reliably via OCR+structure pipelines; validate with evals either way.

Q28. What belongs in a model upgrade canary plan? **A.** Shadow/replay metrics, percentage rollout, kill switch, prompt pin alignment, rollback owner.

Q29. How do you prevent “context landfill”? **A.** Retrieval filters, top-k caps, relevance thresholds, progressive discovery, and periodic corpus hygiene.

Q30. When is Batch API the wrong choice? **A.** When the end user needs interactive, low-latency responses.

Q31. Name two variance-reduction techniques for extraction. **A.** JSON schema validation + few-shot anchors (also deterministic post-processors). Temperature tuning is not a lever on current Claude tiers — sampling parameters are rejected there.

Q32. Why separate research-agent credentials from execute-agent credentials? **A.** Least privilege and blast-radius control if the research path is prompt-injected.

Q33. What should you log for an architectural incident review (at minimum)? **A.** Model ID, prompt/Skill versions, tool calls (redacted), retrieval IDs, latency, and decision outcome.

Q34. A design uses three agents that all share admin SaaS tokens. Primary issue? **A.** Over-privilege / missing security boundaries—multi-agent without isolation.

Q35. How do Skills interact with caching strategy? **A.** Stable Skill content can live in cacheable prefixes; version carefully so cache and behavior stay coherent.

Expanded checklist additions

- ☐ I can describe map-reduce over documents with schema consistency risks
 - ☐ I can place HITL triggers in a diagram as first-class stages
 - ☐ I can rough-order cost with cache hit rate and escalation fraction
 - ☐ I can write a model migration canary outline
 - ☐ I can separate trusted instructions from untrusted retrieved text
 - ☐ I can choose Batch vs realtime with a one-line rationale
 - ☐ I can name five anti-patterns and their fixes
 - ☐ I can design a support copilot that meets a 2s draft latency via model routing
-

Glossary additions

Term	Meaning
Programmatic gate	Code validation between LLM stages
Map-reduce LLM	Parallel per-doc extract + synthesize
Shadow traffic	Parallel scoring of a candidate model/prompt
Canary release	Gradual production exposure with rollback
God prompt	Anti-pattern: one prompt does all jobs
Tool sprawl	Excessive tools exposed to one agent
Process SLA	Operational targets without bit-identical outputs
Untrusted context	Retrieved/user content that may contain injections
Exception queue	Human review path for failed gates
Prompt/Skill pin	Version identifier for reproducible instructions

(Deepening passes continue below: Parts J–R.)

Part J — Primary-study deep dive: architecture decision scenarios

Use these as timed drills. Write a 6–10 sentence decision, then check the scoring keys.

Scenario J1 — Claims triage under a 3-day SLA

Context: An insurer wants Claude to cut claims cycle time from 11 days to 3. Volume is 8k claims/day. Regulators require an auditable decision trail. Payouts above \$5k need a human signature. Corpus: policy PDFs + structured claim forms.

Decide: pattern per stage, model tiering, HITL placement, success metrics.

Scoring key (preferred shape): 1. **Workflow** with stages: ingest/OCR → extract fields → policy retrieve+cite → recommend → (HITL if payout ≥ \$5k or low confidence). 2. **Router:** Haiku-class for intent/confidence; Sonnet-class for extract+recommend; escalate hard disputes to higher-capability only when evals justify. 3. **Metrics:** cycle time, auto-resolution rate below threshold, citation coverage, overturn rate by humans, p95 latency of auto path, \$ per claim. 4. **Anti-patterns:** single autonomous agent with write tools to payment systems; monolithic “paste whole policy book” context.

Scenario J2 — Internal coding assistant for a 40-repo org

Context: Platform team wants Claude across IDEs/CLI. Secrets must not leave vault. Some repos are regulated. Latency for autocomplete-ish help should feel interactive; deep refactors can be async.

Decide: progressive vs monolithic context; Skills vs giant CLAUDE.md; model routing.

Scoring key: Progressive discovery via repo CLAUDE.md + retrieval over docs + MCP only for approved tools. Separate **interactive** mid-tier path from **deep agent** high-capability path with budgets. Regulated repos: stricter allowlists + no server web tools. Org Skills for ADR/test templates; avoid one 50k-token global prompt.

Scenario J3 — Multilingual support copilot with CRM tools

Context: 12 languages, CRM write actions (case notes, refunds ≤ \$50 auto). Brand tone critical. Hallucinated refunds are catastrophic.

Decide: pattern, output contract, tool authZ, eval focus.

Scoring key: Augmented LLM + tools inside a **workflow gate** for refunds (schema validation + policy engine + amount cap). Do not let the model “decide” refund eligibility alone—code enforces caps. Eval: language quality sample + tool-call correctness + refusal on out-of-policy. Citations not primary; **action correctness** is.

Scenario J4 — Overnight batch contract clause extraction

Context: 200k PDFs/night; structured JSON to warehouse; 98% field accuracy target; cost sensitive.

Decide: Batch vs realtime API; map-reduce; model tier; validation.

Scoring key: Batch API + map-reduce (per-doc extract → schema validate → quarantine fails → sample human audit). Prefer mid-tier with strong schema unless hard clauses need escalate. Cost model before GA. Not an interactive agent.

Part K — Trade-off tables (desk reference)

K1. Capability vs latency vs cost (operator view)

Lever	Improves	Often worsens	Exam cue
Higher-capability model	Hard reasoning, coding, judgment	Latency, \$	High-stakes offline review
Faster/cheaper model	p95, throughput, \$	Edge-case quality	Classify/extract/route
Thinking / higher effort	Multi-step correctness	Tokens, latency, cache stability	Hard math/logic/planning
Prompt caching	\$ and TTFT on stable prefixes	Design discipline (prefix order)	Large static system+tools
Smaller context	Latency, focus	Completeness	Progressive discovery
Parallel stages	Wall-clock	\$ and merge complexity	Independent branches
Batch API	Unit economics	Interactivity	Overnight jobs
Stronger eval gates	Quality, safety	Time-to-ship	Regulated GA

K2. Pattern vs auditability

Pattern	Auditability	Flexibility	Typical cost variance
Augmented LLM	High (one call)	Low	Low
Workflow	Very high (stage logs)	Medium	Medium
Agent	Medium (needs traces)	High	High
Multi-agent	Needs hop IDs	Highest	Highest

Rule of thumb: Prefer the **leftmost** pattern that meets the requirement. Move right only when evals show the left fails.

K3. Context strategy trade-offs

Strategy	Token efficiency	Completeness risk	Ops complexity
Monolithic dump	Poor at scale	Low (if fits)	Low initially, blows up later
Progressive RAG	Good	Retrieval misses	Medium
Tool/MCP discovery	Good	Tool selection errors	Medium-high
Hybrid cached core + dynamic	Best common case	Dual-path bugs	Medium

Part L — Failure-mode analysis (platform & design)

Failure mode	Symptoms	Likely root	Design fix
Lost-in-the-middle	Ignores middle docs	Too much packed context	Rank, shrink k, progressive
Format drift	Downstream parse fails	Weak output contract	Schema + validator + repair loop
Cache thrash	Cost spikes, slow TTFT	Volatile prefix / thinking flips	Stabilize prefix & configs
Over-agentification	Unpredictable writes	Agent where workflow fits	Convert to staged workflow
Model mismatch	Great demos, prod misses	Eval on toy data	Production-like eval set
Context starvation	“I don’t know” wrongly	Over-aggressive filtering	Relax filters; query rewrite
Skill/prompt conflict	Contradictory behavior	Multiple instruction layers	Precedence doc + tests

Failure mode	Symptoms	Likely root	Design fix
Multimodal OCR fail	Bad extracts from scans	Vision without validation	Confidence + human queue
Router collapse	Everything escalates	Bad confidence calibration	Threshold tuning + metrics
Compaction drift	Long agents forget rules	Summary dropped policy	Pin never-compact sections

Part M — Production checklist (Platform & Solution Design)

M1. Before architecture review

- ☐ Problem statement, users, and “non-goals” written
- ☐ Value pillars mapped to metrics and owners
- ☐ Pattern choice with rejected alternatives documented (ADR)
- ☐ Model tier per stage with cost envelope
- ☐ Context budget diagram (static vs dynamic vs tools vs thinking)
- ☐ Output schemas and validation ownership named
- ☐ Degradation mode if Claude or tools unavailable
- ☐ Eval stub attached (even if thin)

M2. Before pilot

- ☐ Golden set of ≥ 50 –100 realistic cases
- ☐ Latency budget per stage signed by product
- ☐ Cost kill-switch / max tokens per request
- ☐ Logging redaction policy for prompts/tool results
- ☐ Model pin / upgrade policy draft
- ☐ HITL queues staffed for expected exception rate

M3. Before GA

- ☐ Shadow or A/B vs baseline complete
- ☐ Failure modes above reviewed with security/ops
- ☐ Runbook + on-call named
- ☐ Stakeholder sign-off on known limitations (file 04)
- ☐ Continuous eval in CI for prompts/tools/schemas

Part N — Additional architecture decision drills (short)

1. **Fixed KYC checklist with regulators watching** → Workflow + validators; not free-form agent.
 2. **Unknown-root-cause production incident research** → Agent + read-only tools + budget + HITL for remediations.
 3. **One front door for HR/IT/Facilities FAQs** → Router + per-domain RAG corpora + separate credentials.
 4. **Generate 10 independent product descriptions** → Parallel map; aggregate; schema validate.
 5. **High-stakes legal memo** → High-capability + RAG citations + HITL attorney; never auto-send.
-

Part O — Extended Q&A (36–40)

Q36. A stakeholder wants “one mega agent” to replace a 7-stage underwriting checklist that auditors inspect. Best response? **A.** Propose a **workflow** mirroring the checklist with logged stage outputs; reserve agents for open research sub-tasks only. Auditability beats flexibility here.

Q37. Interactive UX needs $p95 < 2s$, but hard cases need deep reasoning. Best design? **A. Router:** fast path mid/fast model; async or “continue analyzing” path for deep thinking; communicate UX expectation.

Q38. Cache hit rate is 5% despite a huge system prompt. Likeliest cause? **A.** Volatile content or tool defs inserted **before** the stable prefix, or thinking/config changing breakpoints every call.

Q39. Select TWO for map-reduce over 500 contracts: (a) parallel per-doc extract (b) one call with all PDFs (c) schema validate each (d) single agent with write-to-ERP unbound. **A. (a) and (c).**

Q40. Skills vs stuffing expertise into every user message—when Skills win? **A.** When expertise is **stable, reusable, versioned** across many requests; keep request-specific facts in user/tool channels.

Part P — Rapid review card (Domain 1+2 ≈ 30%)

- Smallest model that passes evals; escalate with evidence.

- Workflow when steps known; agent when not; multi-agent only with clear roles.
- System prompt = policy + interface contract.
- Progressive discovery for large tools/corpora; cache stable prefixes.
- Value pillars must name levers (efficiency, transformation, productivity, cost, performance SLAs).
- Document ADRs; reject alternatives explicitly.
- Validate machine outputs in code—never hope.
- Compaction/thinking/caching interactions are design concerns, not afterthoughts.

Cross-read Integration/Evals in 02, Safety in 03. (Parts Q–R continue below.)

Part Q — Model routing & cost envelopes (worked numbers)

Architects need back-of-envelope math, not memorized price sheets (prices change—verify docs).

Cost envelope worksheet

Monthly tasks T

$$\begin{aligned} & \times \text{tokens_in_avg} \times (\$/M \text{ input}) \\ & + \text{tokens_out_avg} \times (\$/M \text{ output}) \\ & + \text{cache_write share} \times \text{write_rate} + \text{cache_read share} \times \text{read_rate} \\ & + \text{tool_overhead_tokens} \\ & = \text{monthly model \$} \end{aligned}$$

Add: retrieval infra, HITL labor, observability storage. Compare to value per successful task.

Routing example (illustrative proportions)

Route	% traffic	Model class	Notes
Classify/refuse	40%	Fast	Gatekeeper
Standard answer	45%	Mid	Cached policy prefix
Hard reason / dispute	15%	High	Thinking on; async OK

If hard route is 60% of cost but only 15% of volume, eval whether mid+critique is cheaper for a slice.

When Batch API wins

High volume, latency-insensitive, structured jobs (extraction, classification, overnight reports). Not for chat UX. Pair with schema validation and quarantine queues.

Pinning & upgrades

Pin model IDs in prod; upgrade via shadow → eval gate → canary → full. Treat prompt and model as a **coupled** release artifact.

Part R — Additional failure modes & exam stems

Stem cue	Prefer
“Auditors need stage evidence”	Workflow + logs
“Steps unknown; explore codebase”	Agent + budget + allowlist
“12 intents one bot”	Router
“Huge PDF corpus”	RAG progressive, not monolith
“p95 interactive”	Fast/mid + defer deep thinking
“\$ sensitive overnight”	Batch + mid + validate

Q41. Cache reads are cheap, so put the user question first in every request for readability—good idea? **A. No**—put **stable** content first for caching; volatile user content last.

Q42. Select TWO decomposition axes: by risk (read vs write), by fashion preference, by SLA path, by meme velocity. **A.** By risk and by SLA path.

Q43. Output must feed a payment API—what’s mandatory? **A. Schema validation in code** before side effects; enums for status; idempotency keys.

End of file 01. Next: 02–enterprise–integration–production.md (Integration 19% + Evaluation 16%).

02 — Enterprise Integration & Production

CCAR-P condensed domain 2 Official domains mapped here: Integration (19%) · Evaluation, Testing & Optimization (16%) **Combined exam weight covered by this file: ~35%** (the largest technical share)

Disclaimer

These are original study notes. They use **public** Anthropic docs (Messages API, tool use, MCP, RAG/citations, prompt caching, observability patterns). They also use published CCAR-P blueprint summaries.

Integration details and API surfaces change. Check the current platform documentation.

This pack is independent. It is not affiliated with Anthropic.

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names are exceptions and stay as written. They include Claude, MCP, Bedrock, Vertex, Foundry, Message Batches, prompting, caching, RAG, eval, p95, TTFT, SLA, IAM, SKU, canary, and golden set. Model IDs, routes, and API surfaces change. Learn the decision rules. Check the current docs before the exam.

Overview

Solution Design selects the *shape*. Integration makes that design *real* in the enterprise. Evaluation proves that the system *works*.

CCAR-P gives Integration the highest weight (**19%**). Architects fail in production on authZ, retrieval quality, tool boundaries, latency, and observability. They do not fail on clever prose.

This file covers:

1. Tools, MCP, agents, and enterprise connectors
2. RAG / retrieval architecture and citations
3. AuthN/Z, least privilege, network and data paths
4. **CSP / delivery-route governance** (Direct API vs Bedrock vs Vertex vs Azure/Foundry)
5. Evaluation strategies, datasets, A/B and regression—including **model judges vs code judges**
6. Optimization (tokens, latency, cost) and observability

Developer enablement and Claude Code org rollout live in **05**. Safety and governance deep notes live in **03**. Entry-point model notes live in **01**. This file owns the enterprise **route-selection** decision table.

Key map

Task	Official domain	Exam signal
Select MCP vs REST vs CLI vs A2A	Integration 19%	Match transport to the trust boundary and the ops model
Design RAG chunk/index/retrieve/cite	Integration 19%	Match corpus and query patterns
Tool allowlists & least privilege	Integration 19%	Prevent capability sprawl
Progressive discovery vs monolith	Integration 19%	Large tool/context sets
Define metrics & eval sets	Evaluation 16%	Accuracy, latency, cost, safety
Diagnose hallucinations / prompt fails	Evaluation 16%	Find the root cause. Then select the fix lever
Observability & optimization loops	Evaluation 16%	Logs → regressions → improve
Select CSP / delivery route under constraints	Integration 19%	Auth, residency, logging, procurement—not informal preference
Model judge vs code judge + calibration	Evaluation 16%	Deterministic first. Calibrate before you trust

Part A — Integration primitives

A1. The integration stack

App / Orchestrator
└─ Claude Messages API (model I/O)

- └ Client tools (your code executes)
- └ Server tools (Anthropic-hosted, where enabled)
- └ MCP servers (standardized tool/data connectors)
- └ Retrieval system (search/index/rerank)
- └ Enterprise IdP, secrets, network, audit

Contract reminder (public tool-use model): Claude never executes your tools by itself. It emits `tool_use`. **Your runtime** (or Anthropic for server tools) executes the tool. You return `tool_result`. You design that loop: timeouts, retries, idempotency, and privilege.

A2. Client tools vs server tools

Type	Who runs code?	Examples (illustrative)	Architect implication
Client / user-defined	Your app	CRM lookup, ticketing, internal SQL	You own authZ, logging, and sandboxing
Anthropic server tools	Anthropic	Web search/fetch, code execution, tool search (per docs)	Enable with intent. Know what data leaves your boundary
Anthropic-schema client tools	Your app, trained-friendly schemas	Prefer known schemas when equivalent	Often more reliable than ad-hoc twins

A3. Model Context Protocol (MCP)

MCP is an **open standard**. It connects assistants and agents to tools and data. Public docs emphasize:

- Local process, HTTP/SSE/streamable HTTP, and in-process transports. The ecosystem changes. Check current MCP and Claude Code docs.
- Servers expose tools, resources, and prompts.
- **Trust model:** Treat third-party servers as supply-chain risk. Prompt injection through fetched content is real.
- Large tool sets need **tool search** or deferred loading. Then definitions do not use too much of the context window.
- Channels and events (where supported) push external signals into sessions.

When to select MCP

Prefer MCP when...	Prefer direct API/SDK tools when...
Many systems, standardized discovery	One or two bespoke tools
You want reusable connectors across Claude Code and apps	Ultra-custom latency path
Org wants a connector catalog	Protocol overhead is not worth it
Partners already ship MCP servers	You must stay inside a locked-down RPC fabric

A4. Agent-to-agent and orchestration buses

Multi-agent systems need an explicit **handoff protocol**. That protocol has a message schema, a shared memory policy, and permission boundaries. Prefer:

- A clear supervisor or workflow that owns side effects
- Per-agent credentials
- Trace IDs across hops

Do not use many free-form agents that send many messages without budgets.

A5. CLI and automation surfaces

Command-line tools (including Claude Code CLI in eng workflows—see file 05) fit CI, ops, and developer loops. For customer-facing products, prefer service APIs with authZ.

Exam scenarios: Select CLI for **builder productivity**. Select APIs for **product runtime**. The exception is when the product *is* a developer tool.

Part B — RAG & retrieval architecture

B1. RAG reference pipeline

Ingest → Clean/OCR → Chunk → Embed/Index (+ keyword) → (optional Query → Rewrite/classify → Retrieve → Rerank → Pack context → C

B2. Chunking decisions

Corpus trait	Chunking bias	Notes
Policies with numbered sections	Section-aware chunks + headers	Keep clause IDs for citations
Code	Symbol/function-aware	Do not split a function in the middle without a reason
Tables/financials	Table-as-unit or row groups	Often need OCR structure
Chat logs	Turn windows with timestamps	PII scrubbing is mandatory
Slides	Per-slide + title	Low text density

Overlap helps continuity. Too much overlap wastes tokens. It also duplicates noise.

B3. Hybrid retrieval

Production RAG usually combines:

- **Dense/vector** for semantic paraphrase
- **BM25/keyword** for IDs, error codes, SKUs, legal article numbers
- **Metadata filters** (tenant, product, locale, effective date)
- **Reranker** for precision at low k

Knowledge graphs help multi-hop “who owns X” questions. They also add ops cost. Justify the graph with eval gains.

B4. Context packing & citations

Public Messages API patterns support **search result** content blocks and citations. Answers can then name their sources. Architect rules:

- Return source id/title/uri with chunks
- Require citations in the output contract when factual grounding matters
- If retrieval returns nothing relevant, **refuse or ask**. Do not invent policy
- Cap top-k by token budget. Prefer fewer high-quality chunks

B5. Index freshness & tenancy

- Version documents. Prefer effective-dating for policies
- Use tenant-isolated indexes or hard filters. Never “prompt the model to use only tenant A”
- Tombstone deletes. GDPR erasure workflows touch indexes too
- Measure **staleness SLOs** (e.g., policy live within 15 minutes of publish)

B6. RAG failure modes (high exam value)

1. Wrong chunk boundaries → missed clauses
2. Embedding drift after a model change without a reindex plan
3. Cross-tenant leakage through a shared index
4. Prompt injection in retrieved HTML (“ignore policies...”)
5. Citation pretense: numbers that do not match sources
6. Over-retrieval that floods the model
7. Under-retrieval → hallucinations

B7. Progressive discovery applied to RAG

Do not put the full corpus in the prompt. Use:

1. Metadata route (product=X)
 2. Retrieve small k
 3. If confidence is low, or the user asks for more depth, retrieve more. You can also browse hierarchical TOC tools
 4. The agent may call search then `fetch_section` tools. Do not use one giant context
-

Part C — Tools, agents, and production control

C1. Tool design checklist

- ☐ Name/description clear enough for routing
- ☐ JSON schema strict. Enums over free text
- ☐ Side-effect tools idempotent with keys
- ☐ Timeouts and circuit breakers
- ☐ AuthZ checked **in the tool server**, not only in the prompt
- ☐ Arguments logged with redaction
- ☐ Rate limits per tenant

- Dry-run / confirm mode for destructive ops

C2. Least privilege patterns

Pattern	Description
Stage-scoped tools	Extract stage cannot refund_payment
Role-scoped agents	Support vs finance credentials differ
Read/write split.	Read Research agent -only. The executor is gated.
Confirm token	Second human or second factor for writes
Allowlist hosts	SSRF-safe fetch tools

C3. Agent loop controls

Production agents need:

- **Max turns / max dollars / max wall time**
- Stop conditions (task_complete, needs_human)
- Sticky trace IDs
- Memory policy (what persists across sessions)
- Sandbox for code execution

C4. Latency budget worksheet

Example interactive UX p95 = 3.0s:

Stage	Budget
Auth + ingress	50ms
Retrieval	200ms
Rerank	50ms
Claude TTFT	800ms
Claude completion	1500ms
Validate + render	100ms
Contingency	300ms

If you miss the budget, shrink context. Cache prefixes. Use a smaller model. Run retrieval in parallel. Or use async UX (“answer streaming + tools in the background”).

C5. Reliability patterns

- Retries with jitter on 429/5xx
 - Fallback model on timeout
 - Cached last-good answers for FAQ intents
 - Queue + worker for long tools
 - Poison-tool detection (repeated failures → disable the tool, alert)
-

Part D — Security-relevant integration (pointer + essentials)

Deep governance is file 03. Integration essentials:

- **Prompt injection** through docs/tools → isolate untrusted content. Minimize write tools
 - **Secrets** in vaults. Give short-lived tokens to tools
 - **Egress controls** for fetch/MCP
 - **Data paths:** what leaves the VPC to the Anthropic/cloud AI end-point. ZDR/retention options per contract
 - **Logging redaction** for prompts that contain PII
-

Part D5 — CSP / delivery-route governance (P0)

Architects do not select “Claude on X” because a presentation looks new. They remove routes that violate **identity, residency, logging/retention, procurement, and model-ID** constraints. Then they select among the remaining routes on ops fit and feature lag.

Public surfaces (verify current docs. Offerings change):

Route	What it is (public framing)
Direct Anthropic API	First-party Messages API (platform.claude.com). Anthropic operates inference
Amazon Bedrock	Claude through AWS Bedrock. AWS-managed boundary / regional controls (Messages and/or legacy Invoke/Converse paths per docs)

Route	What it is (public framing)
Google Vertex AI	Claude through Google Cloud Vertex. GCP IAM + regional endpoints
Azure / Microsoft Foundry	Claude in Microsoft Foundry. Azure-hosted and/or Anthropic-hosted options with different deployment types
Claude Platform on AWS <i>(related)</i>	Anthropic-operated Claude on an AWS Marketplace-style path. This is not the same as Bedrock. Residency and inference pinning differ. Do not mix them in exam stems

D5.1 Public-safe decision table

Treat cells as **themes to verify in the live contract/docs**. They are not eternal guarantees. ZDR and BAAs are **enterprise options**. Confirm entitlement before you design as if they exist.

Dimension	Direct Anthropic API	Amazon Bedrock	Google Vertex AI	Azure / Microsoft Foundry
Identity / auth	API keys / org and workspace controls. Your app still owns user→tool identity	AWS IAM (roles, SCPs, PrivateLink patterns). CloudTrail-friendly	GCP IAM + Vertex permissions. Org policies	Entra ID / Azure RBAC + Foundry project controls

Dimension	Direct Anthropic API	Amazon Bedrock	Google Vertex AI	Azure / Microsoft Foundry
Data residency / inference pin	inference_geo: (e.g. global vs us) on supported models—check docs	Regional endpoints / geographic inference profiles. Region choice is the pin	Region / endpoint selection governs where inference runs	Global Standard vs US Data Zone Standard (US pin analogous to inference_geo: us where documented). Hosting option (Azure vs Anthropic) matters
Logging / retention posture	Anthropic retention and abuse-monitoring terms. ZDR may be available as an enterprise option— verify	AWS/Bedrock data-protection docs plus your account logging govern it. Do not assume Anthropic ZDR wording maps 1:1	Governed by Google Cloud / Vertex data processing plus your logging sinks	Anthropic processor terms + Azure-side retention. Safety-flagged content paths may differ by hosting— read current Foundry Claude docs

Dimension	Direct Anthropic API	Amazon Bedrock	Google Vertex AI	Azure / Microsoft Foundry
Procurement / BAA / FedRAMP themes	Direct MSA / DPA with Anthropic. BAA/FedRAMP only if your deal and offering support them	Buy through AWS Marketplace / Bedrock . Use existing AWS commit, AWS compliance artifacts, and org SCPs. Still map PHI/FedRAMP to the specific Bedrock Claude offering	Buy through GCP . Reuse GCP commit and org policy. Same rule: an artifact is not an automatic BAA	Buy through Azure Marketplace / Foundry . Reuse Microsoft EA. Confirm HIPAA/FedRAMP against the deployment type you actually provision
Model ID differences	Anthropic model IDs (e.g. dated claude-... strings in Messages API)	Bedrock model IDs / inference profile IDs / ARNs (and Messages path IDs where enabled). These are not a copy-paste of first-party IDs	Vertex publisher model IDs / versions— platform specific	Foundry deployment names + model versions (hosting option encoded). App config must not hardcode another CSP's ID

Dimension	Direct Anthropic API	Amazon Bedrock	Google Vertex AI	Azure / Microsoft Foundry
Feature / API lag themes	Often earliest access to new Messages features, betas, server tools	May lag or expose through Bedrock-shaped APIs. Check parity before you promise a feature	Similar parity/lag story on GCP	Parity depends on Azure-hosted vs Anthropic-hosted deployment. Feature matrices differ
When architects select this	Max feature velocity. Simple SaaS. Residency solvable through inference_gateway + contract. Team already on Anthropic billing	AWS-centric estate. IAM/PrivateLink mandates. Residency through AWS regions. Procurement must stay inside AWS	GCP-centric estate. Vertex is the approved AI plane. Org policy blocks other egress	Azure-centric estate. Entra + Foundry is the approved AI plane. Need US Data Zone or Azure-hosted boundary

Always true on every route: your **app** still owns tenancy authZ, tool least privilege, prompt/log redaction, and eval gates. A CSP switch does not fix “authZ in the prompt.”

D5.2 Constraint-elimination method (exam method)

1. List hard constraints from the stem (IdP, residency geo, log)
2. Strike any route that cannot meet a hard constraint (verify)
3. Among survivors, score: ops familiarity, model-ID/ops tooling fit, feature parity risk, cost predictability.
4. Document rejected routes in an ADR one-liner (“Rejected Dire

Informal preference (fail): “Everyone uses Bedrock.” / “Direct API is

always fastest to value so pick it.” / “Foundry because we like Copilot.”
Constraint elimination (pass): “Stem requires EU processing inside existing GCP org policy → Vertex regional endpoint. Direct API global default is out.”

D5.3 Model ID & config portability traps

- Abstract a **provider adapter**: `model_ref` in your config → resolved ID per environment.
- Never share one hardcoded Bedrock ARN across “dev on Direct API.”
- Pin **versioned** IDs in prod. Control upgrades through flags + eval gates.
- Residency parameters differ by route (`inference_geo` vs region URL vs Data Zone deployment). Test the pin with a deliberate misconfig in staging.

D5.4 Logging / ZDR exam discipline

- **ZDR** = contractual/API posture that limits provider retention of prompts/completions (where offered). It is **not** a substitute for your app’s retention, SIEM, or erasure of embeddings/eval caches (see file 03).
- Abuse-monitoring / safety-flagged egress may still exist under provider terms. Design as if “ZDR ≠ invisible to all safety systems” until counsel/docs say otherwise.
- Cloud routes: your CloudTrail/Cloud Logging/Diagnostic logs may retain **metadata and sometimes payloads** you configured. Architect the sink. Do not architect only the model vendor story.

D5.5 Exam traps — delivery routes

1. Picking a CSP for brand preference while you ignore residency in the stem.
2. Equating **Claude Platform on AWS** with **Bedrock** (different operators/boundaries).
3. Assuming BAA/FedRAMP/ZDR on every route without verification.
4. Copy-pasting Anthropic model IDs into Bedrock/Vertex/Foundry calls.
5. Treating cloud IAM as optional because “the model is smart.”
6. Promising day-one feature parity across all CSPs.

7. Using Direct API global inference when the stem demands pinned geography.
8. Believing CSP choice alone satisfies GDPR/HIPAA. You still need data maps and app controls.

D5.6 Mini ADR template (copy/adapt)

Decision: Deliver Claude via <route>

Constraints: <residency, IdP, procurement, feature>

Rejected: <routes + one-line why>

Auth: <IAM/API key pattern>

Residency pin: <param/region/deployment type>

Logging: <provider posture + our sinks + ZDR status: verified/n>

Model IDs: <adapter mapping>

Rollback: <alternate route or prior model pin>

D5.7 Q&A — CSP / delivery routes (73–80)

Q73. Stem: “Must keep inference in US. Already standardized on Azure AD + Foundry.” Best default direction? **A.** Azure/Foundry with a **US Data Zone** (or equivalent US-pinned) deployment type per current docs. Do not use Direct API global by default.

Q74. Select TWO reasons to prefer Bedrock over Direct API in an AWS-only regulated estate. **A.** Reuse **IAM/SCPs/PrivateLink** patterns. Keep procurement/compliance evidence inside the **AWS** boundary the org already operates.

Q75. Why is “we’ll use Bedrock model ID `claude-sonnet-...` from the Anthropic docs” a trap? **A.** Bedrock uses **platform-specific** model/inference-profile identifiers. First-party IDs are not portable.

Q76. ZDR is enabled on the provider—can you skip redacting PII from application logs? **A. No.** ZDR addresses provider retention posture. Your logs, warehouses, and eval stores still need minimization/retention design.

Q77. Product needs a brand-new Messages API beta next week. The estate is multi-cloud with no hard residency pin. Which route often wins and why? **A. Direct Anthropic API**—typically earliest feature access. Confirm beta entitlement. Still document residency/logging.

Q78. FedRAMP-like stem lists “authorized cloud AI service only.” What do you do first? **A. Constraint-eliminate** routes using the customer’s

authorized service list / boundary docs. Then design inside the survivor (logging, change control). Do not “add a safer prompt.”

Q79. Vertex vs Direct API when GCP org policy blocks egress to non-Google AI endpoints? **A. Vertex**—Direct API may be unreachable or noncompliant under that org policy. Model quality does not change this.

Q80. Pick-for-preference vs constraint-elimination—give one exam tell for each. **A.** Informal preference: the answer cites popularity/speed alone. Constraint-elimination: the answer cites a stem constraint that **disqualifies** other routes before you optimize.

Part E — Evaluation, testing & optimization

E1. What to measure

Category	Example metrics
Task success	Exact match, F1, pass@k, rubric score
Grounding	Citation precision/recall, unsupported claim rate
Safety	Policy violation rate, injection success rate
Latency	TTFT, total, p95/p99 per route
Cost	\$ per successful task, tokens per task
UX	Thumbs, edit distance (copilot), escalation rate
Ops	Error rate, tool failure %, cache hit rate

Architect rule: Metrics must match the **business job**. Support copilots ≠ code agents ≠ RAG legal Q&A.

E2. Dataset design

1. **Gold set** — labeled, versioned, representative
2. **Adversarial set** — injections, jailbreaks, tricky entities
3. **Regression set** — past production failures
4. **Slice sets** — languages, tenants, document types

Keep train/prompt-tuning examples **out** of the held-out eval set.

E3. Automated vs human eval

Method	Strength	Weakness
Exact/schema checks	Cheap, precise for extract	Misses nuance
Rubric LLM-as-judge	Scales	Judge bias. Calibrate
Human review	Gold standard for nuance	Cost/latency
Pairwise A/B	Good for preference	Needs traffic/sample size
Security probes	Catches injections	Must refresh attacks

Hybrid is normal. Automations gate every PR. Humans sample weekly. Humans also review high-severity slices.

P0 expansion: see **Part M7 — Model judges vs code judges + calibration**. That part says when deterministic graders beat LLM-as-judge. It also says how you calibrate against human gold. Freeze thresholds **before** you look at experiment results.

E4. Experimentation discipline

- Change **one** major variable at a time when you diagnose
- Record model ID, prompt hash, index version, tool versions
- Use statistical sense on A/B. Do not declare a winner on 20 chats
- Shadow mode before cutover

E5. Diagnosing common failures

Symptom	Likely causes	Levers
Hallucinated facts	Weak retrieval / no refuse path	Raise recall, require citations, refuse on empty
Format breaks	Loose prompt	Schema + validator + few-shot
High latency	Fat context / big model	Cache, route, shrink k, faster model
Cost spike	Loops / fat tools defs	Max turns, tool search, caching
Inconsistent quality.	Underspecified policy/contract.	Reject Clarify contract, schema + few-shot anchors (sampling knobs on current tiers).

Symptom	Likely causes	Levers
Tool wrong choice	Poor descriptions / too many tools	Redescribe, allowlist, router
Good offline, bad prod	Distribution shift	Add prod slices to eval

E6. Optimization playbook (order matters)

1. **Correctness first** (eval gates)
2. **Remove waste context**
3. **Cache stable prefixes**
4. **Route easy traffic to cheaper/faster models**
5. **Parallelize independent work**
6. **Only then** micro-tune prompt phrasing and few-shot anchors

Premature model downgrades that break safety/quality fail exam scenarios that list compliance constraints.

E7. Observability architecture

Minimum viable telemetry:

- Request/trace ID
- Route / intent
- Model + prompt/Skill versions
- Retrieval query + doc IDs returned
- Tool name, latency, status
- Token usage breakdown (input/cache/output/thinking)
- Outcome labels (auto, human-edited, failed)
- Safety flags

Store raw prompts with care (PII, retention). Prefer pointers + redacted snippets for long-term storage.

Dashboards: quality weekly, latency/cost hourly, safety realtime alerts.

E8. Continuous evaluation in CI/CD

PR → unit tests for tools → offline eval suite gate → staging s

Fail the build on statistically meaningful regressions on critical slices (e.g., citation precision, PII leak probes).

Part F — Reference architectures (integration-centric)

F1. Enterprise RAG service

Ingress → tenancy authZ → query rewrite → hybrid retrieve → rerank → Claude (cached policy) → citation validate → response. Eval harness replays queries nightly. Index updater consumes CMS webhooks.

F2. Tool-using support agent

Workflow front door routes: FAQ RAG vs account-tool path vs human. Account tools use scoped OAuth. Writes require confirm. Traces go to the warehouse for weekly rubric grading.

F3. MCP-connected builder environment

Claude Code + approved MCP marketplace (file 05) for eng productivity. Keep this **separate** from customer runtime credentials. Never reuse prod write tokens in IDE agents.

Decision trees

Integration transport picker

Need standardized multi-tool ecosystem across products/IDEs?

YES → MCP (+ allowlisted servers)

NO → Is it 1–2 tools in one service?

YES → Native client tools via Messages API

NO → Mix: MCP for commodity SaaS, native for core domain

RAG urgency picker

Answers must be attributable to private docs?

YES → RAG (+ citations) required

Facts mostly stable public knowledge + low risk?

YES → Maybe parametric + browse tool

Actions on systems of record?

YES → Tools/API, not RAG alone

Eval readiness gate

Have gold + adversarial + regression sets with owners?

NO → Do not ship beyond private beta

YES → Automate on PR + weekly human sample → ship with canary

Exam traps

1. Put the entire KB into context instead of RAG.
 2. AuthZ “in the prompt only.”
 3. Agent with unbounded tools and no max-turn budget.
 4. Evaluating only informal checks / selected demo cases.
 5. Optimizing cost before you meet accuracy/safety bars.
 6. Ignoring tenant isolation in a shared vector DB.
 7. Treating MCP servers as trusted by default.
 8. No citations when the business requires grounded answers.
 9. Parallelizing dependent RAG→generate→act stages incorrectly.
 10. Missing cache hit metrics when cost is a scenario constraint.
 11. Using production write credentials inside developer MCP.
 12. Declaring victory on LLM-as-judge without calibration.
 13. Picking Direct API / Bedrock / Vertex / Foundry for brand preference instead of constraint elimination.
 14. Assuming ZDR or BAA exists on a route without verifying entitlement.
 15. Freezing eval pass thresholds only after you look at A/B results.
-

Practice Q&A (28)

Q1. Who executes client tool calls? **A.** Your application runtime—not the model process itself.

Q2. Why might MCP tool search matter? **A.** Large tool catalogs consume context. Search loads only the tools you need.

Q3. Name two retrieval methods used in hybrid search. **A.** Dense/vector and BM25/keyword (plus metadata filters/rerank).

Q4. What should happen if RAG returns no relevant chunks for a policy question? **A.** Refuse, ask clarifying questions, or escalate. Do not invent policy.

Q5. Where must authorization be enforced for a `create_refund` tool?
A. In the tool server / backend policy layer (least privilege), not only in prose instructions.

Q6. Give three agent loop controls. **A.** Max turns, max cost, max wall time (also stop conditions, allowlists).

Q7. What is citation precision measuring roughly? **A.** Whether cited sources actually support the claims you make.

Q8. Why keep an adversarial eval set? **A.** To catch injections, jailbreaks, and tricky entities before attackers/customers do.

Q9. Offline eval good, prod bad—first suspicion? **A.** Distribution shift. Add production slices to the eval suite.

Q10. How does prompt caching help integration architectures? **A.** It reduces cost/latency for stable system prompts, tool defs, and policy prefixes.

Q11. What is a poison-tool pattern response? **A.** Detect repeated failures. Disable/circuit-break the tool. Alert operators.

Q12. Why isolate tenants in the index? **A.** Prevent cross-tenant retrieval leakage. Prompt instructions alone are not sufficient.

Q13. When are server tools a design concern? **A.** When data egress / trust boundary to Anthropic-hosted execution matters to compliance.

Q14. Order optimization correctly at high level. **A.** Correctness/safety → shrink waste → cache → route cheaper → parallelize.

Q15. What telemetry fields are essential for debugging a bad tool choice? **A.** Trace ID, tool descriptions version, model ID, conversation snippet, tool_use args/results (redacted).

Q16. Map-reduce RAG reduce stage blows tokens—what do you do? **A.** Cap mapper outputs, use hierarchical reduce, or summarize intermediate notes with schema.

Q17. Why version the index alongside prompts? **A.** So evals and incidents can reproduce behavior. Retrieval changes alter answers.

Q18. What is SSRF risk in fetch tools? **A.** Model/tool induced requests to internal IPs/metadata endpoints. Mitigate with allowlists.

Q19. When is CLI integration appropriate? **A.** Developer/ops automation. Usually not the end-customer product path unless the product is a CLI.

Q20. Define successful A/B for a prompt change. **A.** Pre-registered metrics, adequate sample, no other simultaneous changes, rollback plan.

Q21. How do you stop false citations? **A.** Validator checks cite IDs exist in provided chunks. Rubrics penalize unsupported claims.

Q22. What is progressive discovery in a tool-rich agent? **A.** Start with few tools/context. Fetch more through search/browse as needed.

Q23. Why separate builder MCP credentials from runtime product credentials? **A.** Scope of damage—IDE agents must not hold prod write power.

Q24. Name a grounding metric besides citation precision. **A.** Unsupported claim rate / faithfulness score / retrieval recall@k.

Q25. What does a schema validator prevent in tool args? **A.** Malformed calls, unexpected enums, missing required fields—before side effects.

Q26. Latency p95 fails only on tool-heavy intents—where to look? **A.** Tool fanout, serial tool chains, cold MCP connections, over-retrieval before tools.

Q27. What belongs in a nightly eval job? **A.** Gold+adversarial suites, regression cases, cost/latency snapshots, alert on deltas.

Q28. Can you rely on the model to “only call safe tools”? **A.** No. Enforce allowlists and server-side authZ. Prompts are not controls.

Checklist

- ☐ I can diagram client tool loops vs server tools
- ☐ I can argue MCP vs native tools with trade-offs
- ☐ I can design hybrid RAG with tenancy filters and citations
- ☐ I can set agent budgets and least-privilege tool scopes
- ☐ I can build gold/adversarial/regression eval sets
- ☐ I can diagnose hallucination vs format vs latency failures
- ☐ I can list essential observability fields
- ☐ I can place eval gates in CI/CD
- ☐ I can optimize cost without violating safety bars
- ☐ I can explain progressive discovery for tools and RAG
- ☐ I can run CSP route selection by constraint elimination (auth, residency, logging/ZDR, procurement, model IDs)

- I can choose code vs model judges, calibrate to human gold, and pre-register thresholds/rollback

Glossary

Term	Meaning
MCP	Model Context Protocol—standard for tool/data connectors
Client tool	Tool executed by your runtime after <code>tool_use</code>
Server tool	Provider-hosted tool execution path
Hybrid search	Vector + keyword (+ filters/rerank)
Reranker	Model/stage that reorders retrieved candidates
Faithfulness	Answer supported by provided evidence
TTFT	Time to first token
Shadow mode	Candidate system scores in parallel without user impact
Circuit breaker	Stop calling failing dependencies for a time
Tenant isolation	Hard separation of customer data in retrieve/act paths
Tool search	Deferred discovery of tools to save context
Idempotency key	Ensures safe retries for side-effecting tools
Gold set	Labeled evaluation dataset
LLM-as-judge	Using a model to score outputs against a rubric
SSRF	Server-side request forgery via fetch-like tools
CSP (delivery)	Cloud/service path that serves Claude (Direct API, Bedrock, Vertex, Foundry, ...)
ZDR	Zero Data Retention—provider retention posture to verify contractually

Term	Meaning
Inference pin	Control that restricts where inference may run (geo param, region, data zone)
Code judge	Deterministic assertions/graders (schema, cite-ID, policy engine)
Judge calibration	Measuring LLM-as-judge agreement vs human gold before you trust scores
Pre-registration	Fixing metrics/thresholds/rollback before you see experiment outcomes

Part G — Deep dive: enterprise auth, networking, and data planes

G1. Identity propagation

Enterprise integrations fail when the model path “becomes the user” with one shared token that has all privileges. Prefer:

1. User authenticates to your app (OIDC/SAML).
2. The app gets downstream tokens limited to that user’s scopes. (OAuth on-behalf-of, token exchange).
3. Tools call APIs with those tokens.
4. Audit logs attribute actions to the human + service principal.

Service accounts are for batch jobs with separate controls. Do not use them as a shortcut for interactive agents.

G2. Network patterns

Pattern	Use
Public Claude API egress	Simplest. Control through proxy/DLP
Cloud AI in your tenant (Bedrock/Vertex/Foundry)	Data plane nearer to existing cloud controls
PrivateLink / VPC endpoints	Reduce public internet exposure

Pattern	Use
Reverse proxy with allowlists	Central inspect/log/block

Document **where prompts and embeddings live** (vector DB region, log store region). Residency questions appear in architect scenarios.

G3. Synchronous product vs async worker

If an interactive user waits, use tight tool timeouts and streaming text. Defer slow tools to “still working” jobs. Back-office automation uses queue workers, Batch API, and longer tool budgets. Do not force both into one latency class.

G4. Contract testing for tools

For every tool:

- Schema unit tests
- AuthZ negative tests (user without role must fail)
- Idempotency tests
- Timeout behavior tests
- Recording of example traces for eval replay

Treat tools as microservices. The LLM is just another client.

G5. Retrieval evaluation specifics

Measure beyond “answer quality”:

- **Recall@k** of relevant docs
- **nDCG** for ranking
- **Chunk utilization** (did Claude use what you retrieved?)
- **Freshness lag**
- **Filter correctness** (tenant/product)

Many “model problems” are retrieval problems. Fix the index before you swap to a giant model.

G6. Online evaluation & feedback

Capture implicit signals: edits, regenerations, thumbs, escalation. Route low-confidence or high-severity traffic to human labels. Feed these labels

into the regression sets weekly. Without this, offline gold sets decay.

G7. Cost anomaly detection

Alert when:

- Tokens/task jumps >N% week over week
- Agent turn count distribution shifts
- Cache hit rate collapses (prefix thrash)
- One tenant dominates spend (runaway loop)

Tie alerts to runbooks: freeze agent max-turns, revert prompt, disable tool.

G8. Multi-region active-active concerns

Pin model versions consistently. Replicate indexes with conflict rules. Use sticky sessions for conversation state. Know cloud endpoint global vs regional routing trade-offs from public docs. Fail closed on auth. Fail soft on optional enrichment tools.

G9. Additional scenarios

Scenario: Manufacturing firm wants Claude to query SCADA historian.

Answer shape: Never raw plant network from a SaaS agent. Use a DMZ read API, allowlisted metrics, and human approval for actuation. Prefer no write tools at first. RAG over manuals + read tools for telemetry summaries.

Scenario: Legal wants “chat with all contracts.” **Answer shape:** Tenant-isolated hybrid RAG, clause-aware chunking, mandatory citations, matter-based ACL filters, DLP on logs. Eval on hallucination of obligations. HITL for redlines.

Scenario: Cost blowup after MCP rollout. **Answer shape:** Inspect tool definition token share. Enable tool search. Shrink allowlists. Cache system+tool prefixes. Add max turns. Fix loops that call search repeatedly.

G10. Extended Q&A (29–36)

Q29. Why is on-behalf-of token exchange better than a shared token with all privileges? **A.** It preserves user-scoped authZ and attributable audit trails.

Q30. Name a retrieval metric that can be high while user answers still fail. **A.** Recall@k can look fine while chunk packing/citation/prompting still breaks answers. Measure end-task metrics too.

Q31. What is chunk utilization telling you? **A.** Whether retrieved evidence actually influences the answer, or only wastes tokens.

Q32. How do you test authZ for tools? **A.** Negative tests: identities that lack roles must be denied by the tool server.

Q33. Why might cache hit rate collapse after a “small prompt tweak”? **A.** Prefix bytes changed (or tool order/thinking config). That invalidates breakpoints.

Q34. What is fail-soft for optional enrichment? **A.** Return the core answer if an optional tool/RAG times out, with explicit partial-status. Do not return a hard 500.

Q35. Where should embeddings for regulated HR data reside? **A.** In approved regions/systems under the same residency controls as the source HR data.

Q36. What is the first integration question for any write tool? **A.** Who is allowed, how is it audited, and what is the scope of damage if prompt-injected?

Extended checklist

- ☐ I can describe OBO/token exchange vs tokens with all privileges
 - ☐ I can pick VPC/private egress vs public API with rationale
 - ☐ I can list retrieval-only metrics vs end-task metrics
 - ☐ I can design contract tests for side-effecting tools
 - ☐ I can write a cost-anomaly runbook trigger
 - ☐ I can explain global vs regional endpoint trade-offs at a high level
-

Glossary additions

Term	Meaning
OBO	On-behalf-of delegated authorization
nDCG	Ranking quality metric

Term	Meaning
Recall@k	Fraction of relevant docs retrieved in top k
DLP	Data loss prevention controls on egress/logs
DMZ	Network zone buffering trusted and untrusted systems
Active-active	Multi-region serving with replication
Prefix thrash	Frequent cache invalidation from unstable prefixes
Online eval	Production feedback-based measurement
Contract test	Tests that enforce tool API/auth behavior
Fail-soft	Degrade in a controlled way when non-critical deps fail

Part H — Production hardening narratives (exam-style)

H1. Idempotency and exactly-once *effects*

LLMs retry. Users double-click. Queues redeliver. Side-effecting tools must be **at-least-once safe**:

- Client supplies idempotency key (ticket_id + action + hash).
- Tool server stores key → result for TTL.
- Replays return the original result without charging twice / double-shipping.

Exam wrong answer: “Claude will remember it already refunded.”

H2. Conversation state stores

Choose explicitly:

Store	Pros	Cons
Client-owned history replay	Simple, transparent	Large prompts. PII sprawl
Server session store	Truncation/compaction control	Consistency and retention duties
Summary + recent window	Token efficient	Summary drift

Pair with compaction features from the platform when sessions run long (see file 01). Always define retention aligned to legal holds.

H3. Feature flags for AI paths

Flag dimensions that save incidents:

- Model pin
- Prompt/Skill version
- Retrieval index version
- Tool allowlist set
- Agent max-turn cap
- HITL required yes/no

Rollback is a flag flip. It is not a wish without a control.

H4. SLOs for AI features

Write SLOs users feel:

- Availability of /assist
- p95 draft latency
- Grounded-answer rate
- Harmless-response rate for safety probes
- Cost per 1000 successful tasks

Page on-call for availability/latency/safety. Review quality in business hours unless severity is high.

H5. Partner / vendor MCP supply chain

Before you enable a vendor MCP server:

1. Threat model (what data it can read/write)
2. Auth method (OAuth scopes minimized)
3. Changelog and version pin policy
4. Sandbox trial on synthetic data
5. Legal review if data leaves the boundary
6. Kill switch in config

“It was in a directory” is not a control.

H6. Streaming UX + tools

Stream text for perceived performance. Still gate **actions** until tool results validate. Never execute a write based on a partial streamed plan without a completed, validated tool call sequence.

H7. Eval for multi-turn agents

Single-turn gold sets under-test agents. Add:

- Scripted multi-turn goals
- Distractor tools
- Mid-dialog policy changes
- Budget exhaustion behavior
- Recovery after tool errors

Score **task completion**, not eloquence.

H8. Data labeling workflow

Who labels? With what rubric? Calibration sessions? Dual annotation on hard slices? Do you track inter-annotator agreement? Architects who ignore labeling quality ship false confidence.

H9. Cross-functional RACI snippet

Activity	Architect	Eng	Data	Security	Ops
Tool authZ model	A/C	R	C	A	I
Eval gate thresholds	A	R	R	C	I
Index tenancy	C	R	A	A	I
On-call runbooks	C	C	I	C	R/A

(RACI: R=responsible, A=accountable, C=consulted, I=informed)

H10. More traps

13. Assuming streamed partial output is safe to act on.
14. No idempotency on payment/ticket tools.
15. Eval only single-turn when the product is agentic.
16. Enabling vendor MCP without scope review.
17. Quality paging that tires the on-call staff with non-actionable noise.

H11. Q&A 37–45

Q37. Why can Claude not “remember” a refund across retries by itself?
A. Stateless tool side effects need server-side idempotency keys.

Q38. What flag dimensions matter for AI rollback? **A.** Model, prompt/Skill, index, tool allowlist, budgets, HITL requirements.

Q39. Name two multi-turn agent eval additions. **A.** Distractor tools. Recovery after tool errors (also budget exhaustion, policy mid-changes).

Q40. What is summary drift? **A.** Compacted/summarized history omits or distorts constraints/facts.

Q41. Who should be accountable for index tenancy controls? **A.** Typically Data + Security accountable. Eng is responsible for implementation—per RACI.

Q42. Why pin MCP server versions? **A.** Prevent surprise tool schema/behavior changes that break agents/evals.

Q43. What SLO belongs on a safety probe suite? **A.** Harmless-response / attack-blocked rate with alert thresholds.

Q44. Stream tokens to the UI, but when do you commit a database write? **A.** Only after a validated completed tool flow—not on partial plan text.

Q45. What is a labeling calibration session? **A.** Annotators align on the rubric with examples. This raises agreement before you score.

End of expanded integration/eval notes.

Part I — Integration patterns cheat-sheets

I1. API gateway in front of Claude

Place an AI gateway for: auth, quotas, prompt size limits, PII redaction hooks, model routing, and centralized traces. Application teams call the gateway. They do not call raw provider keys. Rotate keys centrally. Attribute cost by team tags.

I2. Event-driven tool results

Long tools (report generation) should acknowledge immediately. Then they push completion events. Resume the agent/workflow with `tool_result` when ready. Avoid holding HTTP requests open for minutes without a product reason.

I3. Embedding model change protocol

1. Freeze writes or dual-write.
2. Reindex offline.
3. Shadow queries compare old/new.
4. Cut traffic.
5. Keep the old index until the rollback window ends.

Never silently change embedding models under a production RAG system.

I4. Q&A 46–50

Q46. Why put an AI gateway in front of provider APIs? **A.** Centralize auth, quotas, redaction, routing, and cost attribution.

Q47. How should a 10-minute report tool integrate with an agent? **A.** Async job + event/callback resume. Do not use a single blocking HTTP call without design.

Q48. What is dual-write during reindex? **A.** Writing embeddings to old and new indexes during migration so you can cut over or roll back.

Q49. Name three gateway tags useful for FinOps. **A.** Team, product, environment (also use-case, model route).

Q50. Silent embedding model change risk? **A.** Retrieval quality shifts with no reproducible version. That breaks evals and incidents.

Part J — Integration exam drill narratives

When a stem lists latency + tenancy + citations, your answer must touch **all three**. Practice composing multi-constraint answers:

1. Name the pattern (RAG workflow vs agent).
2. Name the control (tenant filter server-side).
3. Name the proof (citation validator + eval gate).
4. Name the perf lever (cache, k, model route).

J1. Tool description quality rubric

Poor: search — “searches stuff.” Good: search_policies — “Semantic+keyword search over HR policies for the caller’s tenant. Args: query

(string), product (enum), top_k (1–20). Returns clause_id, title, snippet. Does not search other tenants.”

J2. Conversation memory vs system of record

Never treat chat memory as source of truth for balances, entitlements, or inventory. **Tool-fetch** at decision time. Memory may hold preferences and task state only.

J3. Backpressure

If tool or model latency rises, shed optional enrichment. Shrink k. Degrade to FAQ templates. Or queue with user-visible status. Define these modes in design reviews.

J4. Q&A 51–55

Q51. What four ingredients appear in multi-constraint integration answers? **A.** Pattern, control, proof/eval, performance lever.

Q52. Why refetch balances instead of trusting chat memory? **A.** Memory is stale or can be invented. Systems of record are authoritative.

Q53. Example of backpressure when retrieval slows? **A.** Reduce top-k, skip rerank, or serve cached FAQ with partial status.

Q54. What makes a tool description “good”? **A.** Clear scope, args, returns, and explicit non-goals (e.g., tenancy).

Q55. Citation validator checks what minimally? **A.** That cited IDs exist in the retrieved set provided to the model.

(File continues — closing summary and deep-dive Parts K–T follow.)

Closing integration summary (review)

Integration (19%) asks whether you can connect Claude to enterprise reality. You must not leak tenancy. You must not run unbounded agents. You must not work without quality telemetry. Evaluation (16%) asks whether you can prove behavior with datasets, experiments, and telemetry. You must also optimize in the right order. If you remember only five slogans, remember these:

1. AuthZ in the tool server.

- 2. **RAG with citations or refuse.**
- 3. **Budgets on every agent loop.**
- 4. **Gold + adversarial + regression evals.**
- 5. **Observability with versions (model, prompt, index, tools).**

Re-read Parts B–E the night before the exam. Skim decision trees once.

Final integration self-check

Before exam day, confirm you can whiteboard the following. Whiteboard hybrid RAG with tenant filters, and an MCP allowlist example. Whiteboard an agent with max-turn and dollar budgets. Whiteboard a CI eval gate and a cost-anomaly runbook. If any whiteboard feels unclear, reread the matching part above. Then redo two Q&As aloud.

(Primary-study deep-dive Parts K–T follow below.)

Part K — Primary-study deep dive: Integration (19%)

Integration is the heaviest official domain. Treat every enterprise connection as a **trust boundary** plus a **latency budget** plus an **observability surface**.

K1. Transport decision matrix (expand)

Mechanism	Best fit	AuthZ story	Failure mode if misused
Direct Messages API + client tools MCP servers	Product runtime with 1–N bespoke tools Shared connectors across apps/Claude Code	App identity + per-tool server checks Server enforces. Client still scopes	Ad-hoc tools sprawl without a catalog Unvetted third-party servers = supply chain
CLI / local automation	Builder and CI loops	Local creds / OIDC device flows	Using CLI secrets in the customer SaaS path

Mechanism	Best fit	AuthZ story	Failure mode if misused
Agent-to-agent (A2A)	Clear role split across services	Per-agent credentials + hop audit	Excessive message traffic without budgets or owners
Server tools (Anthropic-hosted)	Web fetch/search/code where allowed	Understand egress and data handling	Enabling “because it is interesting” in regulated tenants
Event bus + workers	Long tools / human queues	Queue ACLs + idempotency keys	Dual-writes without correlation IDs

Exam heuristic: Prefer **MCP** when reuse and discovery matter. Prefer **direct tools** when latency and ultra-custom control dominate. Prefer **A2A** only with a supervisor and schemas. Prefer **CLI** for builders, not end-customer prod paths (unless the product *is* a CLI).

K2. Capability bloat analysis (exam favorite)

Symptoms of tool/agent bloat: - Model picks the wrong tool among near-duplicates - Context window dominated by tool JSON schemas - Latency rises before the first useful action - Security review cannot enumerate capabilities

Mitigations: 1. **Allowlists** per agent role (support $\neq$ admin). 2. **Collapse** overlapping tools (get_order not five variants). 3. **Progressive discovery / tool search** so definitions load on demand. 4. **Description quality rubric:** name, when-to-use, when-not-to-use, side effects, idempotency. 5. **Eval** tool-selection accuracy on realistic transcripts.

K3. AuthN/Z gap checklist (walk every design)

- ☐ User identity propagated (not a shared service account with all privileges for all tenants)
- ☐ Tool execution checks **resource-level** authZ in *your* code—not “Claude promises in the prompt”
- ☐ Tenant ID from verified token, never from model-produced arguments alone

- ☐ Secrets in vault/env — never in prompts, Skills, or MCP URLs committed to git
- ☐ Separate credentials for read vs write tools
- ☐ Break-glass admin path is human + ticketed, not an agent skill
- ☐ Audit log: who, which tool, which resource, correlation ID, success/fail

K4. Accuracy–latency configuration trade-offs

Setting.	Accuracy ↑?	Latency ↑?	Notes.
Higher-capability model	Often	Yes	Measure on hard slice
More retrieval k	Sometimes	Yes	Noise risk
Reranker	Often	Mild	Worth it for FAQ/policy
Thinking/effort	precision ↑ Hard tasks ↑	Yes	Cache interactions
Extra critique loop	Yes	Yes	Cap iterations
Streaming	UX feel ↑	TTFB ↓, total ≈	Tools still block
Parallel tool calls	Wall-clock ↓	Fan-out cost	Only if independent

Justify each setting with **metric + budget**. If you max all knobs without measurement, you fail scenario questions.

K5. Observability at scale

Minimum viable AI observability: 1. **Request trace**: ingress → model → tools → validators → egress 2. **Token & \$**: input/output/cache/thinking per route 3. **Quality proxies**: thumbs, edit distance, human overturn, citation coverage 4. **Safety**: injection hits, DLP blocks, refusal rates 5. **Saturation**: queue depth, tool p95, rate-limit 429s

Without traces, “hallucination” tickets become unactionable.

Part L — RAG pipeline design lab (Integration)

L1. End-to-end decision worksheet

Fill for any corpus:

Decision	Options	Your pick	Why
Chunk unit	tokens / section / symbol / slide		
Overlap	0–20%		
Sparse index	BM25 yes/no		
Dense embeddings	model + dim		
Metadata filters	tenant, locale, version, ACL		
k + rerank			
Citation required?	yes/no		
Freshness SLO			
Empty-retrieval behavior	refuse / clarify / broaden		

L2. Query-pattern → retrieval strategy

Query shape	Strategy
Exact ID / error code / SKU	Keyword/BM25 heavy. Metadata
Paraphrased “how do I...”	Dense + rerank
Multi-hop “who owns X that depends on Y”	Graph or multi-retrieve plan. Maybe agentic retrieve
Time-scoped “policy as of March”	Effective-date filters mandatory
Tenant-specific	Hard ACL filter pre-retrieve—not prompt-only

L3. RAG failure-mode deep table

Failure	Detect with	Fix lever
Wrong chunk retrieved	Retrieval@k labels	Chunking, hybrid, rerank, query rewrite
Right chunk, wrong answer	Groundedness eval	Prompt: “answer only from sources”. Cite
Stale policy cited	Doc version tests	Freshness pipeline + tombstones

Failure	Detect with	Fix lever
Cross-tenant leak	Red-team tenant tests	Index isolation / filter correctness
Citation pretense	Cite-but-unused checks	Require quote spans or ids that the answer uses
OCR garbage	Field validators	Human quarantine. Vision quality gates
Over-retrieve	Token and latency dashboards	Lower k. Pack better

L4. Progressive discovery for RAG + tools

Monolithic: load 80 tools + 40 docs every turn → cost and confusion.
Progressive: classify intent → fetch 2–5 tools → retrieve top chunks → call → maybe second retrieve. **Stop conditions:** answerable with citations. Need clarify. Refuse. Escalate.

Part M — Evaluation, Testing & Optimization (16%) deep dive

M1. Metric portfolio (define before building)

Family	Examples	Owner
Task quality	Exact match, F1, rubric score, groundedness	Applied science / product
Latency	TTFB, p95	Platform
Cost	end-to-end, tool p95	
	\$ / successful task, cache hit rate	Platform + FinOps
Safety	Injection ASR, policy violation rate	Security/AI risk
Business	CSAT, handle time, conversion	Product
Ops	Error rate, 429s, queue age	SRE

Common exam error: Optimizing BLEU-like n-gram scores for a tool-using agent. Prefer **task success** and **tool correctness**.

M2. Dataset design methodology

1. **Production sampling** (privacy-reviewed) + synthetic edge cases.
2. **Stratify** by intent, language, tenant size, difficulty.
3. **Gold labels** with inter-annotator agreement checks.
4. **Hold out** a frozen regression set. Never train prompts on the whole pool without a holdout.
5. **Refresh** monthly for drift. Version datasets like code.

Mixed methods: automated graders + LLM-as-judge (with known biases)
+ human expert panels for high-stakes.

M3. Experiment discipline

- One change per experiment when possible (prompt XOR model XOR retrieval).
- Pre-register success thresholds.
- Use paired tests on the same cases.
- Ship behind flags. Shadow traffic before cutover.
- Record negative results. Avoid uncontrolled repeated changes.

M4. Diagnosis tree (hallucination vs prompt vs model vs retrieval)

Bad answer reported

- ├ Sources missing/wrong? → Retrieval path
- ├ Sources good, answer invents? → Grounding prompt + decode c
- ├ Format broken? → Schema/prompt/examples
- ├ Tool args wrong? → Tool descriptions + authZ + few-shot tool use
- ├ Only hard cases fail? → Model tier / thinking
- └ Intermittent? → Temperature/nondeterminism, timeouts, parti

M5. Optimization order (do not shuffle casually)

1. Fix correctness bugs (wrong tools, bad retrieval).
2. Shrink context (better retrieve, fewer tools).
3. Cache stable prefixes.
4. Route easy traffic to cheaper/faster models.
5. Then consider Batch, parallelism, distillation-like patterns.

Optimizing cost before correctness creates silent wrong systems.

M6. CI/CD for AI systems

Gate	Blocks merge if...
Unit: schema/tool contract tests	Breaking tool JSON
Golden set regression	Score drop > threshold
Safety pack	Injection/DLP cases fail
Latency budget smoke	p95 exceeds budget on canary size
Cost estimate	Projected \$ / 1k req above cap

M7. Model judges vs code judges + calibration (P0)

LLM-as-judge is a **measurement instrument**. It is not an informal scoring method. Architects select the grader the way they select a unit-test framework. Prefer **deterministic checks** when the contract is machine-checkable. Use **model judges** when quality is open-ended. **Calibrate** before the score drives ship/no-ship.

When code / deterministic assertions beat model judges

Signal that code wins	Examples
Exact or structured output	JSON schema valid. Required fields present. Enum membership
Grounding mechanics	Citation IDs $\subseteq$ retrieved chunk IDs. URL allowlist. Quota math
Tool correctness	Correct tool name chosen. Args pass schema. Idempotency key present
Policy engines already exist	Refund $\leq$ cap. Entitlement check boolean. PII regex/DLP hit
Safety probes with known strings	Forbidden substring. Secret pattern. SSRF to link-local IP
Regression of formatting SLAs	Word/char limits. Language code. "must include disclaimer X"

Rule: If a junior engineer could write a pytest without reading the essay, **do not** spend tokens on a judge.

When model judges are needed

Signal that a rubric judge helps	Examples
Open-ended quality	Empathy, clarity, teaching quality, summary usefulness
Multi-factor rubrics	“Helpful + harmless + on-brand” with partial credit
Paraphrase equivalence	Many correct phrasings. Exact match too brittle
Comparative preference	A/B pairwise “which draft is better for this persona”
Nuanced faithfulness.	The platform supports Soft check that claims when citations are prose-y.

Even then: combine with code gates (schema, cite-ID $\subseteq$ retrieval). Then the judge never scores garbage JSON as “beautiful.”

Calibration against human gold (before trust)

- 1. Sample N items stratified by intent/difficulty (start ~50–200)
- 2. ≥ 2 qualified humans score with the SAME rubric; measure agreement
- 3. Run the candidate judge (model + prompt + effort/thinking cost)
- 4. Compare judge vs human: correlation, systematic bias (lenient vs strict)
- 5. Only promote the judge if agreement clears a pre-set bar; otherwise fix rubric/examples or keep humans.
- 6. Re-calibrate on model upgrades, rubric edits, or domain drift

Common exam error: “Judge score went up 8% after prompt tweak → ship.” Without calibration, you may have taught the judge to prefer long answers.

Thresholds & rollback — freeze BEFORE seeing experiment data

Pre-register (write down first)	Why
Primary metric + direction	Stops metric shopping after results
Minimum lift / maximum allowed drop	Ship bar
Guardrail metrics (safety, latency, cost)	Prevents winning on quality while failing compliance

Pre-register (write down first)	Why
Sample size / evaluation set ID	Reproducibility
Rollback criteria	e.g. “any safety probe fail” or “p95 +20%” → auto-revert flag

If you look at results and then select thresholds, that is p-hacking. Architects treat eval configs as controlled artifacts in git.

Mini worksheet — grader design (fill per feature)

Feature / route: _____

Business job: _____

Machine-checkable contracts (code judge):

– [] _____

– [] _____

Open-ended dimensions (model judge):

– [] _____

Human gold plan: N=___ · raters=___ · agreement bar=___

Judge model + prompt hash: _____

Calibration result: pass / fail · date: ____

Pre-registered ship threshold: _____

Pre-registered rollback triggers: _____

Owner: _____

M7 exam traps

1. LLM-as-judge for schema validity.
2. No human calibration set.
3. Changing success thresholds after you see the A/B dashboard.
4. Single vague “quality 1–5” score with no rubric anchors.
5. Judge model identical to generator without bias analysis (shared blind spots).
6. Ignoring slice failures (“great average, fails on medical intents”).
7. Using judge scores as sole prod gate with no code assertions.

M7 Q&A (81–88) Q81. Refund tool args must be {order_id, amount_cents} with amount ≤ policy cap. Prefer code or model judge?

A. Code (schema + policy engine). Model judge is optional narrative gloss, not the control.

Q82. Why calibrate LLM-as-judge against humans before CI gating? **A.** To measure bias/agreement. Uncalibrated judges can reward style over correctness and create false ship signals.

Q83. Select TWO that belong in a pre-registered eval plan. **A.** Primary metric with threshold **and** rollback/guardrail criteria—written before you look at experiment outcomes.

Q84. Summaries graded only by exact string match keep failing good paraphrases. What do you do? **A.** Keep code checks for must-include facts/citations. Add a **calibrated** rubric judge (or human sample) for paraphrase quality. Do not abandon factual assertions.

Q85. Judge scores rise after you switch the judge to the same model family as the generator. Concern? **A. Shared bias / self-preference.** Re-calibrate vs humans. Consider a different judge model or pairwise protocol.

Q86. What is a sound rollback criterion example? **A.** “If adversarial injection ASR > X% or citation-ID validation fail rate > Y% on canary, auto-disable flag.” Set this **before** the canary.

Q87. Offline gold looks great. Prod thumbs-down spike. First eval move? **A.** Suspect **distribution shift**. Sample prod failures into a new slice. Do not only retune the judge prompt to match thumbs.

Q88. When are human panels still required even with a calibrated judge? **A.** High-stakes / irreversible domains, novel failure modes, and periodic audit of the judge itself. Automation supplements accountability. It does not replace it.

Part N — Architecture decision scenarios (Integration + Evals)

N1. MCP vs direct API for CRM + billing + knowledge base

Prefer MCP if three+ apps (support bot, Claude Code, internal agent) need the same connectors and you will maintain a catalog. **Prefer direct tools** if only one latency-critical service needs two tightly customized tools. **Hybrid** is common: MCP for long-tail systems. Direct for the hottest path.

N2. Support agent with refunds and policy RAG

Workflow: retrieve policy → draft → **policy engine validates refund** → tool executes. Evals: policy citation groundedness + refund decision exactness + latency. AuthZ: refund tool capped. Amount comes from code, not a free-form model float without a check.

N3. Observability missing after GA

Symptom: rising “wrong answer” tickets, no traces. Fix: add correlation IDs. Store redacted prompts/tool I/O. Build a replay harness. Freeze a regression set from tickets.

N4. Accuracy up, latency broken

You added critique loop + k=20 retrieve + opus-class. Rollback to baseline. A/B each setting. Keep only levers with proven lift per +100ms.

Part O — Production checklists

O1. Integration go-live

- ☐ Transport choice documented with alternatives rejected
- ☐ Tool inventory ≤ agreed max. Descriptions reviewed
- ☐ Progressive discovery plan for >N tools
- ☐ AuthZ tests including negative tenant cases
- ☐ Timeouts, retries, idempotency on all side-effecting tools
- ☐ Rate-limit and backpressure behavior defined
- ☐ MCP servers allowlisted. Owners named. Versions pinned
- ☐ Network egress reviewed for server tools

O2. RAG go-live

- ☐ Chunking eval vs baseline
- ☐ Hybrid retrieval justified
- ☐ ACL filters tested
- ☐ Freshness SLO monitored
- ☐ Empty-hit behavior coded
- ☐ Citation contract enforced when required

O3. Eval go-live

- ☐ Metrics tied to business + safety
 - ☐ Golden set versioned
 - ☐ CI regression gate on
 - ☐ Online feedback sampled
 - ☐ Cost/latency dashboards live
 - ☐ Owner for weekly eval review
-

Part P — Failure-mode catalog (ops)

Mode	Signal	Mitigate
Tool timeout storms	p95 tool ↑, partial answers	Bulkheads, circuit breakers, degraded UX
Prompt injection via retrieved HTML	Odd tool calls	Sanitize. Ignore instruction-like content. Allowlists
Embedding model change	Sudden quality drop	Dual-embed migration. Reindex protocol
Cache stampedes	Billing spike	Stabilized prefixes. Monitor hit rate
Eval overfitting	Great CI, bad prod	Fresh prod samples. Adversarial set
Non-idempotent refund double-charge	Duplicate tool_use	Idempotency keys + server dedupe
Conversation state loss	Amnesia mid-task	Durable state store. Do not rely on client only
MCP server compromise	Unexpected exfil tools	Supply-chain review. Network egress allowlists

Part Q — Extended Q&A (56–65) Integration + Evals

Q56. Select TWO symptoms of capability bloat. Option (a) is wrong tool selection. Option (b) is faster p95. Option (c) is huge tool defs in context.

Option (d) is higher cache hit rate. **A. (a) and (c).**

Q57. Best place to enforce “user can only see their org’s tickets”? **A. Server-side authZ** on the tool/database query using verified identity—not the system prompt alone.

Q58. RAG returns excellent chunks but answers invent clauses. First lever? **A. Strengthen **grounding instructions + required citations + refuse if unsupported****. Add groundedness eval.

Q59. When is agent-to-agent justified over one workflow? **A. When specialized roles need **separate credentials/contexts** and a supervisor owns side effects. Not for cosmetic “multi-agent” demos.**

Q60. Order of optimization after quality is acceptable? **A. Context shrink → caching → routing to cheaper models → batch/parallel where safe.**

Q61. Multiple-response: CI should block on (select THREE): schema break, golden regression drop, safety pack fail, vanity n-gram uptick alone, CSS color change. **A. Schema break, golden drop, safety fail.**

Q62. Progressive discovery vs monolithic—large MCP catalog? **A. Progressive / tool search.** Load definitions as needed.

Q63. Hybrid retrieval helps most when... **A. Queries mix **semantic paraphrases** and **exact tokens** (IDs, codes, clause numbers).**

Q64. Online eval signal for support copilot? **A. Human edit rate / overturn rate / CSAT linked to trace IDs—not only offline BLEU.**

Q65. Server tool web fetch in a HIPAA workspace—architect action? **A. Disable or tightly gate** with DLP and data-handling review. Prefer private retrieval.

Part R — Rapid review (Integration 19% + Evals 16% ≈ 35%)

- Integration = trust boundary + latency + observability.
- MCP for reusable connectors. Direct tools for bespoke hot paths.
- Least privilege + resource authZ in code.
- Fight tool bloat. Use progressive discovery at scale.
- RAG: chunk→hybrid retrieve→rerank→cite→validate empty hits.
- Evals before GA. Optimize correctness then cost.
- Traces, or you cannot prove it.
- Idempotency for side effects. Pin MCP/embedding versions.

Largest file by design. Pair with 01 patterns and 03 guardrails.

Part S — End-to-end reference scenarios (Integration + Evals heaviest drill)

S1. Enterprise policy assistant (full stack)

Requirements: 40k employees, 12k policy docs, multi-locale, union-sensitive HR policies, p95 < 4s for standard Q&A, citations mandatory, no cross-region HR data mixing.

Architecture sketch: 1. Ingest with section-aware chunking + effective dates + locale + ACL tags. 2. Hybrid retrieve + rerank. Hard filter ACL+locale. 3. Claude mid-tier with cached HR answer policy. Require citations. 4. Empty retrieve → clarify/refuse. 5. Eval: groundedness, citation precision, ACL red-team, latency, cost. 6. Observability: trace_id, doc versions cited, cache hit rate.

Reject: Monolithic context of all policies. Shared credentials with all privileges. Agent with write access to HRIS on day one.

S2. Tool-using IT support agent

Tools: ticket read/write, password reset (sensitive), knowledge RAG, calendar read. **Controls:** password reset = HITL or step-up auth. Ticket write scoped. Progressive tool discovery. Eval tool-selection + harmful action attempts.

S3. Eval harness blueprint (copy/adapt)

```
datasets/  
  golden_vN.jsonl  
  safety_injection.jsonl  
  acl_redteam.jsonl  
runners/  
  offline_score.py  
  latency_budget.py  
ci/  
  gate.yaml # thresholds  
dashboards/  
  online_feedback.md
```

Thresholds example: groundedness ≥ 0.85 . Injection ASR ≤ 0.05 . P95 $\leq$ budget. \$ / 1k req $\leq$ cap.

S4. Trade-off table — retrieval quality vs latency

Config	Quality (typical)	p95 impact	Cost impact
k=3 dense only	OK semantic	Low	Low
k=10 hybrid + rerank	Better precision	Medium	Medium
k=30 + critique loop	Marginal gains	High	High
Agentic multi-hop retrieve	Multi-hop $\uparrow$	High variance	High

Select the minimal config that meets the quality bar on the stratified eval set.

S5. Failure injection tests (production mindset)

- Tool 500s and timeouts
- Empty retrieval
- Partial JSON tool results
- Stale index (old policy version)
- Cross-tenant IDOR attempts
- Prompt injection in HTML fetch
- Duplicate tool_use (idempotency)

S6. Extended Q&A (66–72)

Q66. Citation required but model cites docs not in retrieved set—what gate? **A. Verifier** that citation IDs $\subseteq$ retrieved IDs. Else regenerate or refuse.

Q67. Select TWO for multi-region HR assistant: region-local indexes, single global index without filters, verify region from token, trust model-stated region. **A.** Region-local indexes (or hard filters) and verify region from token.

Q68. Best CI signal that retrieval regressed after chunker change? **A.** Drop in **retrieval recall@k / groundedness** on frozen set.

Q69. Event-driven tool pattern helps when... **A.** Tools are long-running. User gets progress via channel. Correlation IDs tie results back.

Q70. Embedding model upgrade protocol—select TWO: dual-write/reindex plan, flip overnight without eval, version indexes, delete old index same second as deploy. **A.** Dual-write/reindex plan and version indexes.

Q71. Observability “three pillars” for AI paths? **A.** Traces (tool+model), metrics (latency/cost/quality proxies), structured logs (redacted).

Q72. Capability bloat fix order? **A.** Inventory → dedupe → allowlist by role → progressive load → eval tool pick accuracy.

Part T — Desk sheet (35% combined)

Integration 19%: transport, RAG, authZ gaps, bloat, progressive discovery, latency knobs, monitoring, **CSP/route governance**. Evals 16%: metrics, datasets, A/B, diagnosis, optimize order, CI gates, **code vs model judges + calibration**. Together: prove it works under enterprise constraints—or it is not an architecture.

Largest technical file by blueprint weight. P0 gap closures: Part D5 (CSP routes), Part M7 (judges).

02 — Enterprise Integration & Production

CCAR-P condensed domain 2 Official domains mapped here: Integration (19%) · Evaluation, Testing & Optimization (16%) **Combined exam weight covered by this file: ~35%** (heaviest technical slice)

Disclaimer

Original study notes only—grounded in **public** Anthropic docs (Messages API, tool use, MCP, RAG/citations, prompt caching, observability patterns) and published CCAR-P blueprint summaries. Integration details and API surfaces evolve; verify against current platform documentation. Independent pack; not affiliated with Anthropic.

Overview

If Solution Design picks the *shape*, Integration makes it *real* in the enterprise, and Evaluation proves it *works*. CCAR-P weights Integration highest (19%) because architects fail in production on authZ, retrieval quality, tool boundaries, latency, and observability—not on clever prose.

This file covers:

- 1. Tools, MCP, agents, and enterprise connectors
- 2. RAG / retrieval architecture and citations
- 3. AuthN/Z, least privilege, network & data paths
- 4. **CSP / delivery-route governance** (Direct API vs Bedrock vs Vertex vs Azure/Foundry)
- 5. Evaluation strategies, datasets, A/B and regression—including **model judges vs code judges**
- 6. Optimization (tokens, latency, cost) and observability

Developer enablement / Claude Code org rollout lives in **05**. Safety/governance deep dive in **03**. Entry-point model notes in **01**; this file owns the enterprise **route-selection** decision table.

Key map

Task	Official domain	Exam signal
Choose MCP vs REST vs CLI vs A2A	Integration 19%	Fit transport to trust boundary & ops model
Design RAG chunk/index/retrieve/cite	Integration 19%	Match corpus + query patterns
Tool allowlists & least privilege	Integration 19%	Prevent capability sprawl
Progressive discovery vs monolith	Integration 19%	Large tool/context sets
Define metrics & eval sets	Evaluation 16%	Accuracy, latency, cost, safety
Diagnose hallucinations / prompt fails	Evaluation 16%	Root cause → fix lever
Observability & optimization loops	Evaluation 16%	Logs → regressions → improve

Task	Official domain	Exam signal
Pick CSP / delivery route under constraints	Integration 19%	Auth, residency, logging, procurement—not vibes
Model judge vs code judge + calibration	Evaluation 16%	Deterministic first; calibrate before trust

Part A — Integration primitives

A1. The integration stack

App / Orchestrator

- └ Claude Messages API (model I/O)
- └ Client tools (your code executes)
- └ Server tools (Anthropic-hosted, where enabled)
- └ MCP servers (standardized tool/data connectors)
- └ Retrieval system (search/index/rerank)
- └ Enterprise IdP, secrets, network, audit

Contract reminder (public tool-use model): Claude never executes your tools by itself. It emits `tool_use`; **your runtime** (or Anthropic for server tools) executes; you return `tool_result`. Architects design that loop: timeouts, retries, idempotency, and privilege.

A2. Client tools vs server tools

Type	Who runs code?	Examples (illustrative)	Architect implication
Client / user-defined	Your app	CRM lookup, ticketing, internal SQL	You own authZ, logging, sandboxing
Anthropic server tools	Anthropic	Web search/fetch, code execution, tool search (per docs)	Enable deliberately; understand data leaving boundary

Type	Who runs code?	Examples (illustrative)	Architect implication
Anthropic-schema client tools	Your app, trained-friendly schemas	Prefer known schemas when equivalent	Often more reliable than ad-hoc twins

A3. Model Context Protocol (MCP)

MCP is an **open standard** for connecting assistants/agents to tools and data. Public docs emphasize:

- Local process, HTTP/SSE/streamable HTTP, and in-process transports (ecosystem evolves—check current MCP + Claude Code docs).
- Servers expose tools/resources/prompts.
- **Trust model:** treat third-party servers as supply-chain risk; prompt injection via fetched content is real.
- Large tool sets → **tool search** / deferred loading so definitions do not devour the context window.
- Channels/events (where supported) push external signals into sessions.

When to choose MCP

Prefer MCP when...	Prefer direct API/SDK tools when...
Many systems, standardized discovery	One or two bespoke tools
Want reusable connectors across Claude Code + apps	Ultra-custom latency path
Org wants a connector catalog	Protocol overhead not worth it
Partners already ship MCP servers	You must stay inside a locked-down RPC fabric

A4. Agent-to-agent and orchestration buses

Multi-agent systems need an explicit **handoff protocol**: message schema, shared memory policy, and permission boundaries. Prefer:

- Clear supervisor or workflow owning side effects
- Per-agent credentials

- Trace IDs across hops

Avoid chatty free-form agent swarms without budgets.

A5. CLI and automation surfaces

Command-line tools (including Claude Code CLI in eng workflows—see file 05) fit CI, ops, and developer loops. For customer-facing products, prefer service APIs with authZ. Exam scenarios: pick CLI for **builder productivity**, APIs for **product runtime**—unless the product *is* a developer tool.

Part B — RAG & retrieval architecture

B1. RAG reference pipeline

Ingest → Clean/OCR → Chunk → Embed/Index (+ keyword) → (optional Query → Rewrite/classify → Retrieve → Rerank → Pack context → C

B2. Chunking decisions

Corpus trait	Chunking bias	Notes
Policies with numbered sections	Section-aware chunks + headers	Preserve clause IDs for citations
Code	Symbol/function-aware	Don't split mid-function blindly
Tables/financials	Table-as-unit or row groups	Often need OCR structure
Chat logs	Turn windows with timestamps	PII scrubbing critical
Slides	Per-slide + title	Low text density

Overlap helps continuity; too much overlap wastes tokens and duplicates noise.

B3. Hybrid retrieval

Production RAG usually combines:

- **Dense/vector** for semantic paraphrase

- **BM25/keyword** for IDs, error codes, SKUs, legal article numbers
- **Metadata filters** (tenant, product, locale, effective date)
- **Reranker** for precision at low k

Knowledge graphs help multi-hop “who owns X” questions but add ops cost—justify with eval gains.

B4. Context packing & citations

Public Messages API patterns support **search result** content blocks and citations so answers attribute sources. Architect rules:

- Return source id/title/uri with chunks
- Require citations in the output contract when factual grounding matters
- If retrieval returns nothing relevant, **refuse or ask**—do not freestyle policy
- Cap top-k by token budget; prefer fewer high-quality chunks

B5. Index freshness & tenancy

- Version documents; prefer effective-dating for policies
- Tenant-isolated indexes or hard filters (never “prompt the model to only use tenant A”)
- Tombstone deletes; GDPR erasure workflows touch indexes too
- Measure **staleness SLOs** (e.g., policy live within 15 minutes of publish)

B6. RAG failure modes (exam gold)

1. Wrong chunk boundaries → missed clauses
2. Embedding drift after model change without reindex plan
3. Cross-tenant leakage via shared index
4. Prompt injection in retrieved HTML (“ignore policies...”)
5. Citation theater (numbers that don’t match sources)
6. Over-retrieval drowning the model
7. Under-retrieval → hallucinations

B7. Progressive discovery applied to RAG

Don’t dump the corpus. Use:

1. Metadata route (product=X)

- 2. Retrieve small k
 - 3. If confidence low / user asks deeper, retrieve more or browse hierarchical TOC tools
 - 4. Agent may call search then fetch_section tools rather than one giant context
-

Part C — Tools, agents, and production control

C1. Tool design checklist

- ☐ Name/description clear enough for routing
- ☐ JSON schema strict; enums over free text
- ☐ Side-effect tools idempotent with keys
- ☐ Timeouts and circuit breakers
- ☐ AuthZ checked **in tool server**, not only in prompt
- ☐ Arguments logged with redaction
- ☐ Rate limits per tenant
- ☐ Dry-run / confirm mode for destructive ops

C2. Least privilege patterns

Pattern	Description
Stage-scoped tools	Extract stage cannot refund_payment
Role-scoped agents	Support vs finance credentials differ
Read/write split	Research agent read-only; executor gated
Confirm token	Second human or second factor for writes
Allowlist hosts	SSRF-safe fetch tools

C3. Agent loop controls

Production agents need:

- **Max turns / max dollars / max wall time**
- Stop conditions (task_complete, needs_human)
- Sticky trace IDs
- Memory policy (what persists across sessions)
- Sandbox for code execution

C4. Latency budget worksheet

Example interactive UX p95 = 3.0s:

Stage	Budget
Auth + ingress	50ms
Retrieval	200ms
Rerank	50ms
Claude TTFT	800ms
Claude completion	1500ms
Validate + render	100ms
Contingency	300ms

Miss budget? Shrink context, cache prefixes, smaller model, parallel retrieval, or async UX (“answer streaming + tools in background”).

C5. Reliability patterns

- Retries with jitter on 429/5xx
- Fallback model on timeout
- Cached last-good answers for FAQ intents
- Queue + worker for long tools
- Poison-tool detection (repeated failures → disable tool, alert)

Part D — Security-relevant integration (pointer + essentials)

Deep governance is file 03. Integration essentials:

- **Prompt injection** via docs/tools → isolate untrusted content; minimize write tools
- **Secrets** in vaults; short-lived tokens to tools
- **Egress controls** for fetch/MCP
- **Data paths:** what leaves VPC to Anthropic/cloud AI endpoint; ZDR/retention options per contract
- **Logging redaction** for prompts containing PII

Part D5 — CSP / delivery-route governance (P0)

Architects do **not** pick “Claude on X” because a slide looked modern. They eliminate routes that violate **identity, residency, logging/retention, procurement, and model-ID** constraints—then choose among survivors on ops fit and feature lag.

Public surfaces (verify current docs; offerings evolve):

Route	What it is (public framing)
Direct Anthropic API	First-party Messages API (platform.claude.com); Anthropic-operated inference
Amazon Bedrock	Claude via AWS Bedrock; AWS-managed boundary / regional controls (Messages and/or legacy Invoke/Converse paths per docs)
Google Vertex AI	Claude via Google Cloud Vertex; GCP IAM + regional endpoints
Azure / Microsoft Foundry	Claude in Microsoft Foundry; Azure-hosted and/or Anthropic-hosted options with different deployment types
Claude Platform on AWS (related)	Anthropic-operated Claude on AWS Marketplace-style path— not identical to Bedrock; residency/inference pinning differs—do not conflate in exams

D5.1 Public-safe decision table

Treat cells as **themes to verify in the live contract/docs**, not eternal guarantees. ZDR and BAAs are **enterprise options**—confirm entitlement before designing as if they exist.

Dimension	Direct Anthropic API	Amazon Bedrock	Google Vertex AI	Azure / Microsoft Foundry
Identity / auth	API keys / org & workspace controls; app still owns user→tool identity	AWS IAM (roles, SCPs, PrivateLink patterns); CloudTrail-friendly	GCP IAM + Vertex permissions; org policies	Entra ID / Azure RBAC + Foundry project controls
Data residency / inference pin	inference_geo: (e.g. global vs us) on supported models—check docs	Regional endpoints / geographic inference profiles; region choice is the pin	Region / endpoint selection governs where inference runs	Global Standard vs US Data Zone Standard (US pin analogous to inference_geo: us where documented); hosting option (Azure vs Anthropic) matters

Dimension	Direct Anthropic API	Amazon Bedrock	Google Vertex AI	Azure / Microsoft Foundry
Logging / retention posture	Anthropic retention & abuse-monitoring terms; ZDR may be available as enterprise option— verify	Governed largely by AWS/Bedrock data-protection docs + your account logging; do not assume Anthropic ZDR wording maps 1:1	Governed by Google Cloud / Vertex data processing + your logging sinks	Anthropic processor terms + Azure-side retention; safety-flagged content paths may differ by hosting— read current Foundry Claude docs
Procurement / BAA / FedRAMP themes	Direct MSA / DPA with Anthropic; BAA/FedRAMP only if your deal and offering support them	Buy through AWS Marketplace / Bedrock ; lean on existing AWS commit, AWS compliance artifacts, org SCPs—still map PHI/FedRAMP to the specific Bedrock Claude offering	Buy through GCP ; reuse GCP commit & org policy; same “artifact ≠ automatic BAA” rule	Buy through Azure Marketplace / Foundry ; reuse Microsoft EA; confirm HIPAA/FedRAMP against the deployment type you actually provision

Dimension	Direct Anthropic API	Amazon Bedrock	Google Vertex AI	Azure / Microsoft Foundry
Model ID differences	Anthropic model IDs (e.g. dated claude-... strings in Messages API)	Bedrock model IDs / inference profile IDs / ARNs (and Messages path IDs where enabled)— not copy-paste of first-party IDs	Vertex publisher model IDs / versions— platform specific	Foundry deployment names + model versions (hosting option encoded)—app config must not hardcode another CSP's ID
Feature / API lag themes	Often earliest access to new Messages features, betas, server tools	May lag or expose via Bedrock-shaped APIs; check parity before promising a feature	Similar parity/lag story on GCP	Parity depends on Azure-hosted vs Anthropic-hosted deployment—feature matrices differ
When architects pick this	Max feature velocity; simple SaaS; residency solvable via <code>inference_gateway</code> + contract; team already on Anthropic billing	AWS-centric estate; IAM/PrivateLink mandates; residency via AWS regions; procurement must stay inside AWS	GCP-centric estate; Vertex is the approved AI plane; org policy blocks other egress	Azure-centric estate; Entra + Foundry is the approved AI plane; need US Data Zone or Azure-hosted boundary

Always true on every route: your **app** still owns tenancy authZ, tool least privilege, prompt/log redaction, and eval gates. Switching CSP does not fix “authZ in the prompt.”

D5.2 Constraint-elimination method (exam method)

1. List hard constraints from the stem (IdP, residency geo, log
2. Strike any route that cannot meet a hard constraint (verify
3. Among survivors, score: ops familiarity, model-ID/ops tooling fit, feature parity risk, cost predictability.
4. Document rejected routes in an ADR one-liner (“Rejected Dire

Vibes (fail): “Everyone uses Bedrock.” / “Direct API is always fastest to value so pick it.” / “Foundry because we like Copilot.” **Constraint elimination (pass):** “Stem requires EU processing inside existing GCP org policy → Vertex regional endpoint; Direct API global default is out.”

D5.3 Model ID & config portability traps

- Abstract a **provider adapter**: `model_ref` in your config → resolved ID per environment.
- Never share one hardcoded Bedrock ARN across “dev on Direct API.”
- Pin **versioned** IDs in prod; control upgrades via flags + eval gates.
- Residency parameters differ by route (`inference_geo` vs region URL vs Data Zone deployment)—test the pin with a deliberate misconfig in staging.

D5.4 Logging / ZDR exam discipline

- **ZDR** = contractual/API posture that limits provider retention of prompts/completions (where offered). It is **not** a substitute for your app’s retention, SIEM, or erasure of embeddings/eval caches (see file 03).
- Abuse-monitoring / safety-flagged egress may still exist under provider terms—design assuming “ZDR ≠ invisible to all safety systems” until counsel/docs say otherwise.
- Cloud routes: your CloudTrail/Cloud Logging/Diagnostic logs may retain **metadata and sometimes payloads** you configured—architect the sink, not only the model vendor story.

D5.5 Exam traps — delivery routes

1. Picking a CSP for brand preference while ignoring residency in the stem.
2. Equating **Claude Platform on AWS** with **Bedrock** (different operators/boundaries).
3. Assuming BAA/FedRAMP/ZDR on every route without verification.
4. Copy-pasting Anthropic model IDs into Bedrock/Vertex/Foundry calls.
5. Treating cloud IAM as optional because “the model is smart.”
6. Promising day-one feature parity across all CSPs.
7. Using Direct API global inference when the stem demands pinned geography.
8. Believing CSP choice alone satisfies GDPR/HIPAA—data maps and app controls still required.

D5.6 Mini ADR template (copy/adapt)

Decision: Deliver Claude via <route>

Constraints: <residency, IdP, procurement, feature>

Rejected: <routes + one-line why>

Auth: <IAM/API key pattern>

Residency pin: <param/region/deployment type>

Logging: <provider posture + our sinks + ZDR status: verified/n>

Model IDs: <adapter mapping>

Rollback: <alternate route or prior model pin>

D5.7 Q&A — CSP / delivery routes (73–80)

Q73. Stem: “Must keep inference in US; already standardized on Azure AD + Foundry.” Best default direction? **A.** Azure/Foundry with a **US Data Zone** (or equivalent US-pinned) deployment type per current docs—not Direct API global by default.

Q74. Select TWO reasons to prefer Bedrock over Direct API in an AWS-only regulated estate. **A.** Reuse **IAM/SCPs/PrivateLink** patterns and keep procurement/compliance evidence inside the **AWS** boundary the org already operates.

Q75. Why is “we’ll use Bedrock model ID `claude-sonnet-...` from the Anthropic docs” a trap? **A.** Bedrock uses **platform-specific** model/inference-profile identifiers; first-party IDs are not portable.

Q76. ZDR is enabled on the provider—can you skip redacting PII from application logs? **A. No.** ZDR addresses provider retention posture; your logs, warehouses, and eval stores still need minimization/retention design.

Q77. Product needs a brand-new Messages API beta next week; estate is multi-cloud with no hard residency pin. Which route often wins and why? **A. Direct Anthropic API**—typically earliest feature access; confirm beta entitlement and still document residency/logging.

Q78. FedRAMP-like stem lists “authorized cloud AI service only.” What do you do first? **A. Constraint-eliminate** routes using the customer’s authorized service list / boundary docs—then design inside the survivor (logging, change control), not “add a safer prompt.”

Q79. Vertex vs Direct API when GCP org policy blocks egress to non-Google AI endpoints? **A. Vertex**—Direct API may be unreachable/non-compliant under that org policy regardless of model quality.

Q80. Pick-for-vibes vs constraint-elimination—give one exam tell for each. **A. Vibes:** answer cites popularity/speed alone. **Constraint-elimination:** answer cites a stem constraint that **disqualifies** other routes before optimizing.

Part E — Evaluation, testing & optimization

E1. What to measure

Category	Example metrics
Task success	Exact match, F1, pass@k, rubric score
Grounding	Citation precision/recall, unsupported claim rate
Safety	Policy violation rate, injection success rate
Latency	TTFT, total, p95/p99 per route
Cost	\$ per successful task, tokens per task
UX	Thumbs, edit distance (copilot), escalation rate
Ops	Error rate, tool failure %, cache hit rate

Architect rule: Metrics must match the **business job**. Support copilots ≠ code agents ≠ RAG legal Q&A.

E2. Dataset design

1. **Gold set** — labeled, versioned, representative
2. **Adversarial set** — injections, jailbreaks, tricky entities
3. **Regression set** — past production failures
4. **Slice sets** — languages, tenants, document types

Keep train/prompt-tuning examples **out** of the held-out eval set.

E3. Automated vs human eval

Method	Strength	Weakness
Exact/schema checks	Cheap, precise for extract	Misses nuance
Rubric LLM-as-judge	Scales	Judge bias; calibrate
Human review	Gold standard for nuance	Cost/latency
Pairwise A/B	Good for preference	Needs traffic/sample size
Security probes	Catches injections	Must refresh attacks

Hybrid is normal: automations gate every PR; humans sample weekly + review high-severity slices.

P0 expansion: see **Part M7 — Model judges vs code judges + calibration** for when deterministic graders beat LLM-as-judge, how to calibrate against human gold, and how to freeze thresholds **before** peeking at experiment results.

E4. Experimentation discipline

- Change **one** major variable at a time when diagnosing
- Record model ID, prompt hash, index version, tool versions
- Use statistical sense on A/B (don't crown a winner on 20 chats)
- Shadow mode before cutover

E5. Diagnosing common failures

Symptom	Likely causes	Levers
Hallucinated facts	Weak retrieval / no refuse path	Raise recall, require citations, refuse on empty

Symptom	Likely causes	Levers
Format breaks	Loose prompt	Schema + validator + few-shot
High latency	Fat context / big model	Cache, route, shrink k, faster model
Cost spike	Loops / fat tools defs	Max turns, tool search, caching
Inconsistent quality	Underspecified policy/contract	Clarify contract, schema + few-shot anchors (sampling knobs are rejected on current tiers)
Tool wrong choice	Poor descriptions / too many tools	Redescribe, allowlist, router
Good offline, bad prod	Distribution shift	Add prod slices to eval

E6. Optimization playbook (order matters)

1. **Correctness first** (eval gates)
2. **Remove waste context**
3. **Cache stable prefixes**
4. **Route easy traffic to cheaper/faster models**
5. **Parallelize independent work**
6. **Only then** micro-tune prompt phrasing and few-shot anchors

Premature model downgrades that break safety/quality fail exam scenarios that list compliance constraints.

E7. Observability architecture

Minimum viable telemetry:

- Request/trace ID
- Route / intent
- Model + prompt/Skill versions
- Retrieval query + doc IDs returned
- Tool name, latency, status
- Token usage breakdown (input/cache/output/thinking)
- Outcome labels (auto, human-edited, failed)
- Safety flags

Store raw prompts carefully (PII, retention). Prefer pointers + redacted snippets for long-term storage.

Dashboards: quality weekly, latency/cost hourly, safety realtime alerts.

E8. Continuous evaluation in CI/CD

PR → unit tests for tools → offline eval suite gate → staging s

Fail the build on statistically meaningful regressions on critical slices (e.g., citation precision, PII leak probes).

Part F — Reference architectures (integration-centric)

F1. Enterprise RAG service

Ingress → tenancy authZ → query rewrite → hybrid retrieve → rerank → Claude (cached policy) → citation validate → response. Eval harness replays queries nightly. Index updater consumes CMS webhooks.

F2. Tool-using support agent

Workflow front door routes: FAQ RAG vs account-tool path vs human. Account tools use scoped OAuth. Writes require confirm. Traces to warehouse for weekly rubric grading.

F3. MCP-connected builder environment

Claude Code + approved MCP marketplace (file 05) for eng productivity; **separate** from customer runtime credentials. Never reuse prod write tokens in IDE agents.

Decision trees

Integration transport picker

Need standardized multi-tool ecosystem across products/IDEs?

YES → MCP (+ allowlisted servers)

NO → Is it 1–2 tools in one service?

YES → Native client tools via Messages API

NO → Mix: MCP for commodity SaaS, native for core domain

RAG urgency picker

Answers must be attributable to private docs?

YES → RAG (+ citations) required

Facts mostly stable public knowledge + low risk?

YES → Maybe parametric + browse tool

Actions on systems of record?

YES → Tools/API, not RAG alone

Eval readiness gate

Have gold + adversarial + regression sets with owners?

NO → Do not ship beyond private beta

YES → Automate on PR + weekly human sample → ship with canary

Exam traps

1. Dumping entire KB into context instead of RAG.
 2. AuthZ “in the prompt only.”
 3. Agent with unbounded tools and no max-turn budget.
 4. Evaluating only vibe checks / cherry-picked demos.
 5. Optimizing cost before meeting accuracy/safety bars.
 6. Ignoring tenant isolation in shared vector DB.
 7. Treating MCP servers as trusted by default.
 8. No citations when the business requires grounded answers.
 9. Parallelizing dependent RAG→generate→act stages incorrectly.
 10. Missing cache hit metrics when cost is a scenario constraint.
 11. Using production write credentials inside developer MCP.
 12. Declaring victory on LLM-as-judge without calibration.
 13. Picking Direct API / Bedrock / Vertex / Foundry for brand vibes instead of constraint elimination.
 14. Assuming ZDR or BAA exists on a route without verifying entitlement.
 15. Freezing eval pass thresholds only after peeking at A/B results.
-

Practice Q&A (28)

Q1. Who executes client tool calls? **A.** Your application runtime—not the model process itself.

Q2. Why might MCP tool search matter? **A.** Large tool catalogs consume context; search loads only needed tools.

Q3. Name two retrieval methods used in hybrid search. **A.** Dense/vector and BM25/keyword (plus metadata filters/rerank).

Q4. What should happen if RAG returns no relevant chunks for a policy question? **A.** Refuse, ask clarifying questions, or escalate—do not invent policy.

Q5. Where must authorization be enforced for a `create_refund` tool? **A.** In the tool server / backend policy layer (least privilege), not only in prose instructions.

Q6. Give three agent loop controls. **A.** Max turns, max cost, max wall time (also stop conditions, allowlists).

Q7. What is citation precision measuring roughly? **A.** Whether cited sources actually support the claims made.

Q8. Why keep an adversarial eval set? **A.** To catch injections, jailbreaks, and tricky entities before attackers/customers do.

Q9. Offline eval good, prod bad—first suspicion? **A.** Distribution shift; add production slices to the eval suite.

Q10. How does prompt caching help integration architectures? **A.** Reduces cost/latency for stable system prompts, tool defs, and policy prefixes.

Q11. What is a poison-tool pattern response? **A.** Detect repeated failures, disable/circuit-break the tool, alert operators.

Q12. Why isolate tenants in the index? **A.** Prevent cross-tenant retrieval leakage; prompt instructions alone are insufficient.

Q13. When are server tools a design concern? **A.** When data egress / trust boundary to Anthropic-hosted execution matters to compliance.

Q14. Order optimization correctly at high level. **A.** Correctness/safety → shrink waste → cache → route cheaper → parallelize.

Q15. What telemetry fields are essential for debugging a bad tool choice? **A.** Trace ID, tool descriptions version, model ID, conversation snippet,

tool_use args/results (redacted).

Q16. Map-reduce RAG reduce stage blows tokens—what do you do? **A.** Cap mapper outputs, hierarchical reduce, or summarize intermediate notes with schema.

Q17. Why version the index alongside prompts? **A.** So evals and incidents can reproduce behavior; retrieval changes alter answers.

Q18. What is SSRF risk in fetch tools? **A.** Model/tool induced requests to internal IPs/metadata endpoints—mitigate with allowlists.

Q19. When is CLI integration appropriate? **A.** Developer/ops automation; usually not end-customer product path unless the product is a CLI.

Q20. Define successful A/B for a prompt change. **A.** Pre-registered metrics, adequate sample, no other simultaneous changes, rollback plan.

Q21. How do you stop citation theater? **A.** Validator checks cite IDs exist in provided chunks; rubrics penalize unsupported claims.

Q22. What is progressive discovery in a tool-rich agent? **A.** Start with few tools/context; fetch more via search/browse as needed.

Q23. Why separate builder MCP credentials from runtime product credentials? **A.** Blast radius—IDE agents should not hold prod write power.

Q24. Name a grounding metric besides citation precision. **A.** Unsupported claim rate / faithfulness score / retrieval recall@k.

Q25. What does a schema validator prevent in tool args? **A.** Malformed calls, unexpected enums, missing required fields—before side effects.

Q26. Latency p95 fails only on tool-heavy intents—where to look? **A.** Tool fanout, serial tool chains, cold MCP connections, over-retrieval before tools.

Q27. What belongs in a nightly eval job? **A.** Gold+adversarial suites, regression cases, cost/latency snapshots, alert on deltas.

Q28. Can you rely on the model to “only call safe tools”? **A.** No—enforce allowlists and server-side authZ; prompts are not controls.

Checklist

- ☐ I can diagram client tool loops vs server tools

- ☐ I can argue MCP vs native tools with trade-offs
 - ☐ I can design hybrid RAG with tenancy filters and citations
 - ☐ I can set agent budgets and least-privilege tool scopes
 - ☐ I can build gold/adversarial/regression eval sets
 - ☐ I can diagnose hallucination vs format vs latency failures
 - ☐ I can list essential observability fields
 - ☐ I can place eval gates in CI/CD
 - ☐ I can optimize cost without violating safety bars
 - ☐ I can explain progressive discovery for tools and RAG
 - ☐ I can run CSP route selection by constraint elimination (auth, residency, logging/ZDR, procurement, model IDs)
 - ☐ I can choose code vs model judges, calibrate to human gold, and pre-register thresholds/rollback
-

Glossary

Term	Meaning
MCP	Model Context Protocol—standard for tool/data connectors
Client tool	Tool executed by your runtime after <code>tool_use</code>
Server tool	Provider-hosted tool execution path
Hybrid search	Vector + keyword (+ filters/rerank)
Reranker	Model/stage that reorders retrieved candidates
Faithfulness	Answer supported by provided evidence
TTFT	Time to first token
Shadow mode	Candidate system scores in parallel without user impact
Circuit breaker	Stop calling failing dependencies temporarily
Tenant isolation	Hard separation of customer data in retrieve/act paths
Tool search	Deferred discovery of tools to save context
Idempotency key	Ensures safe retries for side-effecting tools

Term	Meaning
Gold set	Labeled evaluation dataset
LLM-as-judge	Using a model to score outputs against a rubric
SSRF	Server-side request forgery via fetch-like tools
CSP (delivery)	Cloud/service path that serves Claude (Direct API, Bedrock, Vertex, Foundry, ...)
ZDR	Zero Data Retention—provider retention posture to verify contractually
Inference pin	Control that restricts where inference may run (geo param, region, data zone)
Code judge	Deterministic assertions/graders (schema, cite-ID, policy engine)
Judge calibration	Measuring LLM-as-judge agreement vs human gold before trusting scores
Pre-registration	Fixing metrics/thresholds/rollback before seeing experiment outcomes

Part G — Deep dive: enterprise auth, networking, and data planes

G1. Identity propagation

Enterprise integrations fail when the model path “becomes the user” with a god token. Prefer:

1. User authenticates to your app (OIDC/SAML).
2. App obtains **downstream tokens** constrained to that user’s scopes (OAuth on-behalf-of, token exchange).
3. Tools call APIs with those tokens.
4. Audit logs attribute actions to the human + service principal.

Service accounts are for batch jobs with separate controls—not as a shortcut for interactive agents.

G2. Network patterns

Pattern	Use
Public Claude API egress	Simplest; control via proxy/DLP
Cloud AI in your tenant (Bedrock/Vertex/Foundry)	Data plane nearer to existing cloud controls
PrivateLink / VPC endpoints	Reduce public internet exposure
Reverse proxy with allowlists	Central inspect/log/block

Document **where prompts and embeddings live** (vector DB region, log store region). Residency questions appear in architect scenarios.

G3. Synchronous product vs async worker

Interactive user waits → tight tool timeouts, streaming text, defer slow tools to “still working” jobs. Back-office automation → queue workers, Batch API, longer tool budgets. Do not force both into one latency class.

G4. Contract testing for tools

For every tool:

- Schema unit tests
- AuthZ negative tests (user without role must fail)
- Idempotency tests
- Timeout behavior tests
- Recording of example traces for eval replay

Treat tools as microservices—the LLM is just another client.

G5. Retrieval evaluation specifics

Measure beyond “answer quality”:

- **Recall@k** of relevant docs
- **nDCG** for ranking
- **Chunk utilization** (did Claude use what you retrieved?)
- **Freshness lag**
- **Filter correctness** (tenant/product)

Many “model problems” are retrieval problems. Fix index before swapping to a giant model.

G6. Online evaluation & feedback

Capture implicit signals: edits, regenerations, thumbs, escalation. Route low-confidence or high-severity traffic to human labels. Close the loop into regression sets weekly. Without this, offline gold sets rot.

G7. Cost anomaly detection

Alert when:

- Tokens/task jumps >N% week over week
- Agent turn count distribution shifts
- Cache hit rate collapses (prefix thrash)
- One tenant dominates spend (runaway loop)

Tie alerts to runbooks: freeze agent max-turns, revert prompt, disable tool.

G8. Multi-region active-active concerns

Pin model versions consistently; replicate indexes with conflict rules; sticky sessions for conversation state; understand cloud endpoint global vs regional routing trade-offs from public docs. Fail closed on auth, fail soft on optional enrichment tools.

G9. Additional scenarios

Scenario: Manufacturing firm wants Claude to query SCADA historian.

Answer shape: Never raw plant network from SaaS agent. Use DMZ read API, allowlisted metrics, human approval for actuation—prefer no write tools initially. RAG over manuals + read tools for telemetry summaries.

Scenario: Legal wants “chat with all contracts.” **Answer shape:** Tenant-isolated hybrid RAG, clause-aware chunking, mandatory citations, matter-based ACL filters, DLP on logs, eval on hallucination of obligations, HITL for redlines.

Scenario: Cost blowup after MCP rollout. **Answer shape:** Inspect tool definition token share, enable tool search, shrink allowlists, cache system+tool prefixes, add max turns, fix loops calling search repeatedly.

G10. Extended Q&A (29–36)

Q29. Why is on-behalf-of token exchange better than a shared god token? **A.** Preserves user-scoped authZ and attributable audit trails.

Q30. Name a retrieval metric that can be high while user answers still fail. **A.** Recall@k can look fine while chunk packing/citation/prompting still breaks answers—measure end-task metrics too.

Q31. What is chunk utilization telling you? **A.** Whether retrieved evidence is actually influencing the answer or just wasting tokens.

Q32. How do you test authZ for tools? **A.** Negative tests: identities lacking roles must be denied by the tool server.

Q33. Why might cache hit rate collapse after a “small prompt tweak”? **A.** Prefix bytes changed (or tool order/thinking config), invalidating break-points.

Q34. What is fail-soft for optional enrichment? **A.** Return core answer if optional tool/RAG times out, with explicit partial-status—not a hard 500.

Q35. Where should embeddings for regulated HR data reside? **A.** In approved regions/systems under the same residency controls as the source HR data.

Q36. What is the first integration question for any write tool? **A.** Who is allowed, how is it audited, and what is the blast radius if prompt-injected?

Extended checklist

- ☐ I can describe OBO/token exchange vs god tokens
 - ☐ I can pick VPC/private egress vs public API with rationale
 - ☐ I can list retrieval-only metrics vs end-task metrics
 - ☐ I can design contract tests for side-effecting tools
 - ☐ I can write a cost-anomaly runbook trigger
 - ☐ I can explain global vs regional endpoint trade-offs at a high level
-

Glossary additions

Term	Meaning
OBO	On-behalf-of delegated authorization
nDCG	Ranking quality metric
Recall@k	Fraction of relevant docs retrieved in top k
DLP	Data loss prevention controls on egress/logs
DMZ	Network zone buffering trusted and untrusted systems
Active-active	Multi-region serving with replication
Prefix thrash	Frequent cache invalidation from unstable prefixes
Online eval	Production feedback-based measurement
Contract test	Tests that enforce tool API/auth behavior
Fail-soft	Degrade gracefully when non-critical deps fail



Part H — Production hardening narratives (exam-style)

H1. Idempotency and exactly-once *effects*

LLMs retry; users double-click; queues redeliver. Side-effecting tools must be **at-least-once safe**:

- Client supplies idempotency key (ticket_id + action + hash).
- Tool server stores key → result for TTL.
- Replays return original result without charging twice / double-shipping.

Exam wrong answer: “Claude will remember it already refunded.”

H2. Conversation state stores

Choose explicitly:

Store	Pros	Cons
Client-owned history replay	Simple, transparent	Large prompts; PII sprawl
Server session store	Truncation/com-paction control	Consistency & retention duties
Summary + recent window	Token efficient	Summary drift

Pair with compaction features from the platform when sessions are long-running (see file 01). Always define retention aligned to legal holds.

H3. Feature flags for AI paths

Flag dimensions that save incidents:

- Model pin
- Prompt/Skill version
- Retrieval index version
- Tool allowlist set
- Agent max-turn cap
- HITL required yes/no

Rollback is a flag flip—not a hope.

H4. SLOs for AI features

Write SLOs users feel:

- Availability of /assist
- p95 draft latency
- Grounded-answer rate
- Harmless-response rate for safety probes
- Cost per 1000 successful tasks

Page on-call for availability/latency/safety; review quality in business hours unless severity high.

H5. Partner / vendor MCP supply chain

Before enabling a vendor MCP server:

1. Threat model (what data it can read/write)
2. Auth method (OAuth scopes minimized)
3. Changelog & version pin policy
4. Sandbox trial on synthetic data
5. Legal review if data leaves boundary
6. Kill switch in config

“It was in a directory” is not a control.

H6. Streaming UX + tools

Stream text for perceived performance; still gate **actions** until tool results validate. Never execute a write based on a partial streamed plan without a completed, validated tool call sequence.

H7. Eval for multi-turn agents

Single-turn gold sets under-test agents. Add:

- Scripted multi-turn goals
- Distractor tools
- Mid-dialog policy changes
- Budget exhaustion behavior
- Recovery after tool errors

Score **task completion**, not eloquence.

H8. Data labeling workflow

Who labels? With what rubric? Calibration sessions? Dual annotation on hard slices? Inter-annotator agreement tracked? Architects who ignore labeling quality ship false confidence.

H9. Cross-functional RACI snippet

Activity	Architect	Eng	Data	Security	Ops
Tool authZ model	A/C	R	C	A	I
Eval gate thresholds	A	R	R	C	I
Index tenancy	C	R	A	A	I
On-call runbooks	C	C	I	C	R/A

(RACI: R=responsible, A=accountable, C=consulted, I=informed)

H10. More traps

13. Assuming streamed partial output is safe to act on.
14. No idempotency on payment/ticket tools.
15. Eval only single-turn when product is agentic.
16. Enabling vendor MCP without scope review.
17. Quality paging that burns out on-call with non-actionable fluffs.

H11. Q&A 37–45

Q37. Why can't Claude "remember" a refund across retries by itself? **A.** Stateless tool side effects need server-side idempotency keys.

Q38. What flag dimensions matter for AI rollback? **A.** Model, prompt/Skill, index, tool allowlist, budgets, HITL requirements.

Q39. Name two multi-turn agent eval additions. **A.** Distractor tools; recovery after tool errors (also budget exhaustion, policy mid-changes).

Q40. What is summary drift? **A.** Compacted/summarized history omits or distorts constraints/facts.

Q41. Who should be accountable for index tenancy controls? **A.** Typically Data + Security accountable; Eng responsible for implementation—per RACI.

Q42. Why pin MCP server versions? **A.** Prevent surprise tool schema/behavior changes breaking agents/evals.

Q43. What SLO belongs on a safety probe suite? **A.** Harmless-response / attack-blocked rate with alert thresholds.

Q44. Stream tokens to UI but when commit a database write? **A.** Only after validated completed tool flow—not on partial plan text.

Q45. What is a labeling calibration session? **A.** Annotators align on rubric with examples to raise agreement before scoring.

End of expanded integration/eval notes.

Part I — Integration patterns cheat-sheets

I1. API gateway in front of Claude

Place an AI gateway for: auth, quotas, prompt size limits, PII redaction hooks, model routing, and centralized traces. Application teams call the gateway, not raw provider keys. Rotate keys centrally; attribute cost by team tags.

I2. Event-driven tool results

Long tools (report generation) should acknowledge immediately, push completion events, and resume the agent/workflow with `tool_result` when ready. Avoid holding HTTP requests open for minutes without a product reason.

I3. Embedding model change protocol

1. Freeze writes or dual-write.
2. Reindex offline.
3. Shadow queries compare old/new.
4. Cut traffic.
5. Keep old index until rollback window ends.

Never silently change embedding models under a production RAG system.

I4. Q&A 46–50

Q46. Why put an AI gateway in front of provider APIs? **A.** Centralize auth, quotas, redaction, routing, and cost attribution.

Q47. How should a 10-minute report tool integrate with an agent? **A.** Async job + event/callback resume—not a single blocking HTTP call without design.

Q48. What is dual-write during reindex? **A.** Writing embeddings to old and new indexes during migration to allow cutover/rollback.

Q49. Name three gateway tags useful for FinOps. **A.** Team, product, environment (also use-case, model route).

Q50. Silent embedding model change risk? **A.** Retrieval quality shifts with no reproducible version—breaks evals and incidents.

Part J — Integration exam drill narratives

When a stem lists latency + tenancy + citations, your answer must touch **all three**. Practice composing multi-constraint answers:

1. Name the pattern (RAG workflow vs agent).
2. Name the control (tenant filter server-side).

3. Name the proof (citation validator + eval gate).
4. Name the perf lever (cache, k, model route).

J1. Tool description quality rubric

Poor: search — “searches stuff.” Good: search_policies — “Semantic+keyword search over HR policies for the caller’s tenant. Args: query (string), product (enum), top_k (1–20). Returns clause_id, title, snippet. Does not search other tenants.”

J2. Conversation memory vs system of record

Never treat chat memory as source of truth for balances, entitlements, or inventory—**tool-fetch** at decision time. Memory may hold preferences and task state only.

J3. Backpressure

If tool or model latency rises, shed optional enrichment, shrink k, degrade to FAQ templates, or queue with user-visible status. Define these modes in design reviews.

J4. Q&A 51–55

Q51. What four ingredients appear in multi-constraint integration answers? **A.** Pattern, control, proof/eval, performance lever.

Q52. Why refetch balances instead of trusting chat memory? **A.** Memory is stale/hallucinable; systems of record are authoritative.

Q53. Example of backpressure when retrieval slows? **A.** Reduce top-k, skip rerank, or serve cached FAQ with partial status.

Q54. What makes a tool description “good”? **A.** Clear scope, args, returns, and explicit non-goals (e.g., tenancy).

Q55. Citation validator checks what minimally? **A.** That cited IDs exist in the retrieved set provided to the model.

(File continues — closing summary and deep-dive Parts K–T follow.)

Closing integration summary (review)

Integration (19%) asks whether you can connect Claude to enterprise reality without leaking tenancy, without unbounded agents, and without flying blind on quality. Evaluation (16%) asks whether you can prove behavior with datasets, experiments, and telemetry—and optimize in the right order. If you remember only five slogans, remember these:

- 1. **AuthZ in the tool server.**
- 2. **RAG with citations or refuse.**
- 3. **Budgets on every agent loop.**
- 4. **Gold + adversarial + regression evals.**
- 5. **Observability with versions (model, prompt, index, tools).**

Re-read Parts B–E the night before the exam; skim decision trees once.

Final integration self-check

Before exam day, ensure you can whiteboard: hybrid RAG with tenant filters; an MCP allowlist story; an agent with max-turn and dollar budgets; a CI eval gate; and a cost-anomaly runbook. If any whiteboard feels fuzzy, reread the matching part above and redo two Q&As aloud.

(Primary-study deep-dive Parts K–T follow below.)

Part K — Primary-study deep dive: Integration (19%)

Integration is the heaviest official domain. Treat every enterprise connection as a **trust boundary** plus a **latency budget** plus an **observability surface**.

K1. Transport decision matrix (expand)

Mechanism	Best fit	AuthZ story	Failure mode if misused
Direct Messages API + client tools	Product runtime with 1–N bespoke tools	App identity + per-tool server checks	Ad-hoc tools sprawl without catalog

Mechanism	Best fit	AuthZ story	Failure mode if misused
MCP servers	Shared connectors across apps/Claude Code	Server enforces; client still scopes	Unvetted third-party servers = supply chain
CLI / local automation	Builder & CI loops	Local creds / OIDC device flows	Using CLI secrets in customer SaaS path
Agent-to-agent (A2A)	Clear role split across services	Per-agent credentials + hop audit	Chatty swarm without budgets or owners
Server tools (Anthropic-hosted)	Web fetch/search/code where allowed	Understand egress & data handling	Enabling “because cool” in regulated tenants
Event bus + workers	Long tools / human queues	Queue ACLs + idempotency keys	Dual-writes without correlation IDs

Exam heuristic: Prefer **MCP** when reuse and discovery matter; **direct tools** when latency and ultra-custom control dominate; **A2A** only with a supervisor and schemas; **CLI** for builders, not end-customer prod paths (unless the product *is* a CLI).

K2. Capability bloat analysis (exam favorite)

Symptoms of tool/agent bloat: - Model picks wrong tool among near-duplicates - Context window dominated by tool JSON schemas - Latency rises before first useful action - Security review cannot enumerate capabilities

Mitigations: 1. **Allowlists** per agent role (support ≠ admin). 2. **Collapse** overlapping tools (get_order not five variants). 3. **Progressive discovery / tool search** so definitions load on demand. 4. **Description quality rubric:** name, when-to-use, when-not-to-use, side effects, idempotency. 5. **Eval** tool-selection accuracy on realistic transcripts.

K3. AuthN/Z gap checklist (walk every design)

- ☐ User identity propagated (not a shared god service account for all tenants)
- ☐ Tool execution checks **resource-level** authZ in *your* code—not “Claude pinky-promise”
- ☐ Tenant ID from verified token, never from model-produced arguments alone
- ☐ Secrets in vault/env—never in prompts, Skills, or MCP URLs committed to git
- ☐ Separate credentials for read vs write tools
- ☐ Break-glass admin path is human + ticketed, not an agent skill
- ☐ Audit log: who, which tool, which resource, correlation ID, success/fail

K4. Accuracy–latency configuration trade-offs

Knob	Accuracy ↑?	Latency ↑?	Notes
Higher-capability model	Often	Yes	Measure on hard slice
More retrieval k	Sometimes	Yes	Noise risk
Reranker	Often	Mild	Worth it for FAQ/policy
Thinking/effort	precision ↑ Hard tasks ↑	Yes	Cache interactions
Extra critique loop	Yes	Yes	Cap iterations
Streaming	UX feel ↑	TTFB ↓, total ≈	Tools still block
Parallel tool calls	Wall-clock ↓	Fan-out cost	Only if independent

Justify each knob with **metric + budget**. Blindly maxing all knobs fails scenario questions.

K5. Observability at scale

Minimum viable AI observability: 1. **Request trace**: ingress → model → tools → validators → egress 2. **Token & \$**: input/output/cache/thinking per route 3. **Quality proxies**: thumbs, edit distance, human overturn,

citation coverage 4. **Safety:** injection hits, DLP blocks, refusal rates 5. **Saturation:** queue depth, tool p95, rate-limit 429s

Without traces, “hallucination” tickets become unactionable.

Part L — RAG pipeline design lab (Integration)

L1. End-to-end decision worksheet

Fill for any corpus:

Decision	Options	Your pick	Why
Chunk unit	tokens / section / symbol / slide		
Overlap	0–20%		
Sparse index	BM25 yes/no		
Dense embeddings	model + dim		
Metadata filters	tenant, locale, version, ACL		
k + rerank			
Citation required?	yes/no		
Freshness SLO			
Empty-retrieval behavior	refuse / clarify / broaden		

L2. Query-pattern → retrieval strategy

Query shape	Strategy
Exact ID / error code / SKU	Keyword/BM25 heavy; metadata
Paraphrased “how do I...”	Dense + rerank
Multi-hop “who owns X that depends on Y”	Graph or multi-retrieve plan; maybe agentic retrieve
Time-scoped “policy as of March”	Effective-date filters mandatory
Tenant-specific	Hard ACL filter pre-retrieve—not prompt-only

L3. RAG failure-mode deep table

Failure	Detect with	Fix lever
Wrong chunk retrieved	Retrieval@k labels	Chunking, hybrid, rerank, query rewrite
Right chunk, wrong answer	Groundedness eval	Prompt: “answer only from sources”; cite
Stale policy cited	Doc version tests	Freshness pipeline + tombstones
Cross-tenant leak	Red-team tenant tests	Index isolation / filter correctness
Citation theater	Cite-but-unused checks	Require quote spans / ids
OCR garbage	Field validators	Human quarantine; vision quality gates
Over-retrieve	Token & latency dashboards	Lower k; pack better

L4. Progressive discovery for RAG + tools

Monolithic: load 80 tools + 40 docs every turn → cost and confusion.
 Progressive: classify intent → fetch 2–5 tools → retrieve top chunks → call → maybe second retrieve. **Stop conditions:** answerable with citations; need clarify; refuse; escalate.

Part M — Evaluation, Testing & Optimization (16%) deep dive

M1. Metric portfolio (define before building)

Family	Examples	Owner
Task quality	Exact match, F1, rubric score, groundedness	Applied science / product
Latency	TTFB, p95 end-to-end, tool p95	Platform
Cost	\$ / successful task, cache hit rate	Platform + FinOps
Safety	Injection ASR, policy violation rate	Security/AI risk

Family	Examples	Owner
Business	CSAT, handle time, conversion	Product
Ops	Error rate, 429s, queue age	SRE

Exam trap: Optimizing BLEU-like n-gram scores for a tool-using agent. Prefer **task success** and **tool correctness**.

M2. Dataset design methodology

- 1. **Production sampling** (privacy-reviewed) + synthetic edge cases.
- 2. **Stratify** by intent, language, tenant size, difficulty.
- 3. **Gold labels** with inter-annotator agreement checks.
- 4. **Hold out** a frozen regression set; never train prompts on the whole pool without a holdout.
- 5. **Refresh** monthly for drift; version datasets like code.

Mixed methods: automated graders + LLM-as-judge (with known biases) + human expert panels for high-stakes.

M3. Experiment discipline

- One change per experiment when possible (prompt XOR model XOR retrieval).
- Pre-register success thresholds.
- Use paired tests on the same cases.
- Ship behind flags; shadow traffic before cutover.
- Record negative results—avoid thrashing.

M4. Diagnosis tree (hallucination vs prompt vs model vs retrieval)

Bad answer reported

- └ Sources missing/wrong? → Retrieval path
- └ Sources good, answer invents? → Grounding prompt + decode c
- └ Format broken? → Schema/prompt/examples
- └ Tool args wrong? → Tool descriptions + authZ + few-

shot tool use

- └ Only hard cases fail? → Model tier / thinking
- └ Intermittent? → Temperature/nondeterminism, timeouts, parti

M5. Optimization order (do not shuffle casually)

- 1. Fix correctness bugs (wrong tools, bad retrieval).
- 2. Shrink context (better retrieve, fewer tools).
- 3. Cache stable prefixes.
- 4. Route easy traffic to cheaper/faster models.
- 5. Then consider Batch, parallelism, distillation-like patterns.

Optimizing cost before correctness creates silent wrong systems.

M6. CI/CD for AI systems

Gate	Blocks merge if...
Unit: schema/tool contract tests	Breaking tool JSON
Golden set regression	Score drop > threshold
Safety pack	Injection/DLP cases fail
Latency budget smoke	p95 exceeds budget on canary size
Cost estimate	Projected \$ / 1k req above cap

M7. Model judges vs code judges + calibration (P0)

LLM-as-judge is a **measurement instrument**, not a vibe oracle. Architects choose the grader the way they choose a unit-test framework: prefer **deterministic checks** when the contract is machine-checkable; use **model judges** when quality is open-ended—and **calibrate** before the score drives ship/no-ship.

When code / deterministic assertions beat model judges

Signal that code wins	Examples
Exact or structured output	JSON schema valid; required fields present; enum membership
Grounding mechanics	Citation IDs $\subseteq$ retrieved chunk IDs; URL allowlist; quota math
Tool correctness	Correct tool name chosen; args pass schema; idempotency key present
Policy engines already exist	Refund $\leq$ cap; entitlement check boolean; PII regex/DLP hit

Signal that code wins	Examples
Safety probes with known strings	Forbidden substring; secret pattern; SSRF to link-local IP
Regression of formatting SLAs	Word/char limits; language code; “must include disclaimer X”

Rule: If a junior engineer could write a pytest without reading the essay, **don’t** burn tokens on a judge.

When model judges are needed

Signal that a rubric judge helps	Examples
Open-ended quality	Empathy, clarity, teaching quality, summary usefulness
Multi-factor rubrics	“Helpful + harmless + on-brand” with partial credit
Paraphrase equivalence	Many correct phrasings; exact match too brittle
Comparative preference	A/B pairwise “which draft is better for this persona”
Nuanced faithfulness	Soft check that claims are supported when citations are prose-y

Even then: combine with code gates (schema, cite-ID $\subseteq$ retrieval) so the judge never scores garbage JSON as “beautiful.”

Calibration against human gold (before trust)

1. Sample N items stratified by intent/difficulty (start ~50–200)
2. ≥ 2 qualified humans score with the SAME rubric; measure agreement
3. Run the candidate judge (model + prompt + effort/thinking cost)
4. Compare judge vs human: correlation, systematic bias (lenient/harsh)
5. Only promote the judge if agreement clears a pre-set bar; otherwise fix rubric/examples or keep humans.
6. Re-calibrate on model upgrades, rubric edits, or domain drift

Exam trap: “Judge score went up 8% after prompt tweak → ship.” Without calibration, you may have taught the judge to love verbosity.

Thresholds & rollback — freeze BEFORE seeing experiment data

Pre-register (write down first)	Why
Primary metric + direction	Stops metric shopping
Minimum lift / maximum allowed drop	Ship bar
Guardrail metrics (safety, latency, cost)	Prevents winning on quality while failing compliance
Sample size / evaluation set ID	Reproducibility
Rollback criteria	e.g. “any safety probe fail” or “p95 +20%” → auto-revert flag

Peeking at results then choosing thresholds is **p-hacking**. Architects treat eval configs as controlled artifacts in git.

Mini worksheet — grader design (fill per feature)

Feature / route: _____
Business job: _____
Machine-checkable contracts (code judge):
- [] _____
- [] _____
Open-ended dimensions (model judge):
- [] _____
Human gold plan: N=___ · raters=___ · agreement bar=___
Judge model + prompt hash: _____
Calibration result: pass / fail · date: ____
Pre-registered ship threshold: _____
Pre-registered rollback triggers: _____
Owner: _____

M7 exam traps

- 1. LLM-as-judge for schema validity.
- 2. No human calibration set.
- 3. Changing success thresholds after seeing the A/B dashboard.
- 4. Single vague “quality 1–5” score with no rubric anchors.
- 5. Judge model identical to generator without bias analysis (shared blind spots).
- 6. Ignoring slice failures (“great average, fails on medical intents”).
- 7. Using judge scores as sole prod gate with no code assertions.

M7 Q&A (81–88) Q81. Refund tool args must be {order_id, amount_cents} with amount $\leq$ policy cap. Prefer code or model judge? **A. Code** (schema + policy engine). Model judge is optional narrative gloss, not the control.

Q82. Why calibrate LLM-as-judge against humans before CI gating? **A.** To measure bias/agreement; uncalibrated judges can reward style over correctness and create false ship signals.

Q83. Select TWO that belong in a pre-registered eval plan. **A.** Primary metric with threshold **and** rollback/guardrail criteria—written before looking at experiment outcomes.

Q84. Summaries graded only by exact string match keep failing good paraphrases. What do you do? **A.** Keep code checks for must-include facts/citations; add a **calibrated** rubric judge (or human sample) for paraphrase quality—don’t abandon factual assertions.

Q85. Judge scores rise after you switch the judge to the same model family as the generator. Concern? **A. Shared bias / self-preference;** re-calibrate vs humans and consider a different judge model or pairwise protocol.

Q86. What is a sound rollback criterion example? **A.** “If adversarial injection ASR $>$ X% or citation-ID validation fail rate $>$ Y% on canary, auto-disable flag”—set **before** the canary.

Q87. Offline gold looks great; prod thumbs-down spike. First eval move? **A.** Suspect **distribution shift**; sample prod failures into a new slice; don’t only retune the judge prompt to match thumbs.

Q88. When are human panels still required even with a calibrated judge? **A.** High-stakes / irreversible domains, novel failure modes, and periodic audit of the judge itself—automation supplements, not replaces, accountability.

Part N — Architecture decision scenarios (Integration + Evals)

N1. MCP vs direct API for CRM + billing + knowledge base

Prefer MCP if three+ apps (support bot, Claude Code, internal agent) need the same connectors and you will maintain a catalog. **Prefer direct tools** if only one latency-critical service needs two tightly customized

tools. **Hybrid** is common: MCP for long-tail systems; direct for the hottest path.

N2. Support agent with refunds and policy RAG

Workflow: retrieve policy → draft → **policy engine validates refund** → tool executes. Evals: policy citation groundedness + refund decision exactness + latency. AuthZ: refund tool capped; amount from code, not free-form model float without check.

N3. Observability missing after GA

Symptom: rising “wrong answer” tickets, no traces. Fix: add correlation IDs, store redacted prompts/tool I/O, build replay harness, freeze a regression set from tickets.

N4. Accuracy up, latency broken

You added critique loop + k=20 retrieve + opus-class. Rollback to baseline; A/B each knob; keep only levers with proven lift per +100ms.

Part O — Production checklists

O1. Integration go-live

- ☐ Transport choice documented with alternatives rejected
- ☐ Tool inventory $\leq$ agreed max; descriptions reviewed
- ☐ Progressive discovery plan for >N tools
- ☐ AuthZ tests including negative tenant cases
- ☐ Timeouts, retries, idempotency on all side-effecting tools
- ☐ Rate-limit and backpressure behavior defined
- ☐ MCP servers allowlisted; owners named; versions pinned
- ☐ Network egress reviewed for server tools

O2. RAG go-live

- ☐ Chunking eval vs baseline
- ☐ Hybrid retrieval justified
- ☐ ACL filters tested
- ☐ Freshness SLO monitored

- ☐ Empty-hit behavior coded
- ☐ Citation contract enforced when required

O3. Eval go-live

- ☐ Metrics tied to business + safety
- ☐ Golden set versioned
- ☐ CI regression gate on
- ☐ Online feedback sampled
- ☐ Cost/latency dashboards live
- ☐ Owner for weekly eval review

Part P — Failure-mode catalog (ops)

Mode	Signal	Mitigate
Tool timeout storms	p95 tool ↑, partial answers	Bulkheads, circuit breakers, degraded UX
Prompt injection via retrieved HTML	Odd tool calls	Sanitize; ignore instruction-like content; allowlists
Embedding model change	Sudden quality drop	Dual-embed migration; reindex protocol
Cache stampedes	Billing spike	Stabilized prefixes; monitor hit rate
Eval overfitting	Great CI, bad prod	Fresh prod samples; adversarial set
Non-idempotent refund double-charge	Duplicate tool_use	Idempotency keys + server dedupe
Conversation state loss	Amnesia mid-task	Durable state store; don't rely on client only
MCP server compromise	Unexpected exfil tools	Supply-chain review; network egress allowlists

Part Q — Extended Q&A (56–65) Integration + Evals

Q56. Select TWO symptoms of capability bloat: (a) wrong tool selection (b) faster p95 (c) huge tool defs in context (d) higher cache hit rate. **A. (a) and (c).**

Q57. Best place to enforce “user can only see their org’s tickets”? **A. Server-side authZ** on the tool/database query using verified identity—not the system prompt alone.

Q58. RAG returns excellent chunks but answers invent clauses. First lever? **A. Strengthen **grounding instructions + required citations + refuse if unsupported****; add groundedness eval.

Q59. When is agent-to-agent justified over one workflow? **A. When specialized roles need **separate credentials/contexts** and a supervisor owns side effects—not for cosmetic “multi-agent” demos.**

Q60. Order of optimization after quality is acceptable? **A. Context shrink → caching → routing to cheaper models → batch/parallel where safe.**

Q61. Multiple-response: CI should block on (select THREE): schema break, golden regression drop, safety pack fail, vanity n-gram uptick alone, CSS color change. **A. Schema break, golden drop, safety fail.**

Q62. Progressive discovery vs monolithic—large MCP catalog? **A. Progressive / tool search**; load definitions as needed.

Q63. Hybrid retrieval helps most when... **A. Queries mix **semantic paraphrases** and **exact tokens** (IDs, codes, clause numbers).**

Q64. Online eval signal for support copilot? **A. Human edit rate / overturn rate / CSAT linked to trace IDs—not only offline BLEU.**

Q65. Server tool web fetch in a HIPAA workspace—architect action? **A. Disable or tightly gate** with DLP and data-handling review; prefer private retrieval.

Part R — Rapid review (Integration 19% + Evals 16% ≈ 35%)

- Integration = trust boundary + latency + observability.
- MCP for reusable connectors; direct tools for bespoke hot paths.
- Least privilege + resource authZ in code.
- Fight tool bloat; progressive discovery at scale.
- RAG: chunk→hybrid retrieve→rerank→cite→validate empty hits.

- Evals before GA; optimize correctness then cost.
- Traces or it didn't happen.
- Idempotency for side effects; pin MCP/embedding versions.

Heaviest file by design. Pair with 01 patterns and 03 guardrails.

Part S — End-to-end reference scenarios (Integration + Evals heaviest drill)

S1. Enterprise policy assistant (full stack)

Requirements: 40k employees, 12k policy docs, multi-locale, union-sensitive HR policies, p95 < 4s for standard Q&A, citations mandatory, no cross-region HR data mixing.

Architecture sketch: 1. Ingest with section-aware chunking + effective dates + locale + ACL tags. 2. Hybrid retrieve + rerank; hard filter ACL+locale. 3. Claude mid-tier with cached HR answer policy; require citations. 4. Empty retrieve → clarify/refuse. 5. Eval: groundedness, citation precision, ACL red-team, latency, cost. 6. Observability: trace_id, doc versions cited, cache hit rate.

Reject: Monolithic context of all policies; shared god credentials; agent with write access to HRIS on day one.

S2. Tool-using IT support agent

Tools: ticket read/write, password reset (sensitive), knowledge RAG, calendar read. **Controls:** password reset = HITL or step-up auth; ticket write scoped; progressive tool discovery; eval tool-selection + harmful action attempts.

S3. Eval harness blueprint (copy/adapt)

```
datasets/
  golden_vN.jsonl
  safety_injection.jsonl
  acl_redteam.jsonl
runners/
  offline_score.py
  latency_budget.py
ci/
```

```
gate.yaml # thresholds
dashboards/
online_feedback.md
```

Thresholds example: groundedness ≥ 0.85 ; injection ASR ≤ 0.05 ; p95 $\leq$ budget; \$ / 1k req $\leq$ cap.

S4. Trade-off table — retrieval quality vs latency

Config	Quality (typical)	p95 impact	Cost impact
k=3 dense only	OK semantic	Low	Low
k=10 hybrid + rerank	Better precision	Medium	Medium
k=30 + critique loop	Marginal gains	High	High
Agentic multi-hop retrieve	Multi-hop $\uparrow$	High variance	High

Pick the **minimal** config that hits the quality bar on the stratified eval set.

S5. Failure injection tests (production mindset)

- Tool 500s and timeouts
- Empty retrieval
- Partial JSON tool results
- Stale index (old policy version)
- Cross-tenant IDOR attempts
- Prompt injection in HTML fetch
- Duplicate tool_use (idempotency)

S6. Extended Q&A (66–72)

Q66. Citation required but model cites docs not in retrieved set—what gate? **A. Verifier** that citation IDs $\subseteq$ retrieved IDs; else regenerate or refuse.

Q67. Select TWO for multi-region HR assistant: region-local indexes, single global index without filters, verify region from token, trust model-stated region. **A.** Region-local indexes (or hard filters) and verify region from token.

Q68. Best CI signal that retrieval regressed after chunker change? **A.** Drop in **retrieval recall@k / groundedness** on frozen set.

Q69. Event-driven tool pattern helps when... **A.** Tools are long-running; user gets progress via channel; correlation IDs tie results back.

Q70. Embedding model upgrade protocol—select TWO: dual-write/reindex plan, flip overnight without eval, version indexes, delete old index same second as deploy. **A.** Dual-write/reindex plan and version indexes.

Q71. Observability “three pillars” for AI paths? **A.** Traces (tool+model), metrics (latency/cost/quality proxies), structured logs (redacted).

Q72. Capability bloat fix order? **A.** Inventory → dedupe → allowlist by role → progressive load → eval tool pick accuracy.

Part T — Desk sheet (35% combined)

Integration 19%: transport, RAG, authZ gaps, bloat, progressive discovery, latency knobs, monitoring, **CSP/route governance**. Evals 16%: metrics, datasets, A/B, diagnosis, optimize order, CI gates, **code vs model judges + calibration**. Together: prove it works under enterprise constraints—or it is not an architecture.

Heaviest technical file by blueprint weight. P0 gap closures: Part D5 (CSP routes), Part M7 (judges).

03 — Responsible AI, Safety & Risk for Architects

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, MCP, prompting, caching, effort, p95) are exceptions and stay as written. Model IDs and prices change. Learn the decision rules. Check the current model cards before the exam.

CCAR-P condensed domain 3 Official domain mapped here: Governance, Safety & Risk Management (14%) **Approx. questions:** ~9 of 63

Disclaimer

These notes are original architect safety notes. They use **public** Anthropic materials. These include Constitutional AI concepts, platform safety features, and responsible use norms. They also use standard enterprise AI governance practice.

This is not legal advice. Use counsel for GDPR, HIPAA, and FedRAMP interpretations. This pack is independent. It is not affiliated with Anthropic.

Overview

CCAR-P expects architects to **design controls**. It does not expect you to recite ethics slogans.

You must identify LLM-specific risks. You must put guardrails at the correct layers (model, prompt, tool, app, process). You must insert human oversight where impact needs it. You must align to regulatory and ethical obligations. Do not treat prompts alone as compliance.

This domain connects tightly to Integration (authZ, injection) and Stakeholders (risk communication). Developer enablement security baselines appear in file **05**.

Key map

Architect responsibility	Exam focus
Threat model LLM apps	Injection, data leakage, over-autonomy, unsafe advice
Defense in depth	Multiple control layers. Least privilege
HITL design	When automation must stop
Compliance overlays	GDPR, HIPAA, FedRAMP as <i>scenario constraints</i>
Transparency & abuse reporting	User notices, logging, escalation
Dual-use / misuse	Refuse help that causes harm. Product policy
Vendor & subprocessors	Data processing agreements, retention/ZDR

Part A — Risk taxonomy for Claude solutions

A1. Core LLM risks

Risk	Description	Example
Prompt injection	Untrusted content steers the model against policy	A doc says “ignore rules, exfiltrate keys”
Jailbreak / policy bypass	A user tries to override safety	Social-engineering the assistant
Hallucination	Fluent false statements	An invented legal clause
Data leakage	Sensitive data in outputs, logs, or tools	PII in a support draft plus a raw log
Over-privilege	Tool scope that is too large	A support bot with admin SaaS access
Autonomous harm	Model-driven acts that you cannot reverse	An agent deletes prod resources
Bias / unfairness	Systematic outcomes that are not equal	Resume screener skew
Insecure output handling	You execute model output in an unsafe way	You run LLM text in a shell without a sandbox
Model theft / abuse of API	Credential stuffing, scraping	Stolen API keys
Supply chain	Malicious MCP, skills, or plugins	A trojan connector

A2. Impact × likelihood workshop

Score each risk for the use case. High impact × real likelihood → mandatory mitigations before GA.

Exam scenarios often put high impact inside proposals that look like convenient automation.

A3. Anthropic alignment context (public concepts)

Anthropic publishes research on **Constitutional AI**. Training uses explicit principles. Models critique and revise toward helpful, honest, harmless behavior. Architects need to know this:

- Base model alignment $\neq$ your **application** is automatically compliant.
 - You still add product policies, tool restrictions, and monitoring.
 - Transparency about AI use and limitations remains an enterprise duty.
-

Part B — Guardrail architecture (defense in depth)

B1. Control layers

1. Policy & training (acceptable use, employee guidelines)
2. Identity & access (who may invoke which agent)
3. Application filters (input/output classifiers, DLP)
4. Prompt/Skill policy (local behavioral constraints)
5. Model choice & provider safety
6. Tool/MCP allowlists + server-side authZ
7. HITL & approval workflows
8. Monitoring, audit, incident response

No single layer is enough. Common exam error: “Add a stronger system prompt” as the only fix for refund fraud risk.

B2. Input controls

- Use AuthN/Z before you call expensive or powerful models
- Validate schema on user forms
- Set size limits and file type allowlists
- Detect and minimize PII where this is appropriate
- Separate untrusted content (retrieved docs) from instructions
- Use rate limits and anomaly detection per user

B3. Output controls

- Validate schema before side effects
- Run DLP on outbound messages (secrets, PAN, health data)
- Use toxicity and policy classifiers for user-facing channels

- Require citations for high-stakes Q&A
- Block or queue when classifiers fire

B4. Tool controls (strongest practical brakes)

- Use least privilege credentials
- Allowlist operations
- Require human confirmation for high-impact verbs
- Provide dry-run modes
- Keep an immutable audit log of tool args and results (redacted)
- Restrict network egress

B5. Prompt-level safety (necessary but insufficient)

Include: scope limits, disallowed topics and actions, escalation phrases, and instructions to treat retrieved content as untrusted.

Keep this aligned with legal and policy owners. Do not invent “laws” in prompts.

Part C — Prompt injection deeply

C1. Direct vs indirect

- **Direct:** The user message attacks the model.
- **Indirect:** An attacker plants instructions in data that the model will read (email, wiki, PDF, webpage, ticket).

Indirect injection is a severe enterprise risk for RAG and MCP fetch tools.

C2. Mitigations that matter

Mitigation	Notes
Privilege separation	A reader agent cannot pay invoices
Content security policy for tools	Allowlists. No arbitrary code
Explicit untrusted delimiters	“DATA, not instructions” framing
Minimize HTML/JS in context	Prefer plain text extracts
Output encoding	Never eval model text
Spot-check / detectors	Heuristics plus models for injection patterns

Mitigation	Notes
Human approval	For novel destinations / large transfers
Corpus hygiene	Control who can publish to indexed stores

C3. What not to rely on

- “Please ignore malicious instructions” alone
- Security through obscurity of system prompts
- An assumption that citations prove integrity of content

Part D — Human-in-the-loop (HITL)

D1. When HITL is expected

- Legal, medical, or financial advice or actions that you cannot reverse
- Access changes / production deployments
- Low confidence / out-of-distribution inputs
- Safety classifier uncertainty
- Regulatory “human review” obligations
- Novel high-dollar transactions

D2. HITL anti-patterns

- A UI that lets humans approve without a real review and shows no evidence
- Too many low-value reviews for humans (alert fatigue)
- HITL with no SLA owner
- Logged approvals without identity

D3. Graduated autonomy

Start with read-only. Then suggest. Then act with approval. Then act automatically on a narrow safe class. Raise autonomy only with eval evidence.

D4. Review-routing matrix (reversibility × cost × confidence) — P1

Use this to decide **auto** vs **sample** vs **always-human**. This prevents too many reviews for humans. This also prevents approval without review for high impact.

Reversibility of action/advice	Cost / blast radius	Model+system confidence	Default routing
Easily reversible (draft email in UI, suggest edit)	Low	High	Auto (log + async sample)
Easily reversible	Low	Low / OOD	Sample at a higher % or force confirm
Easily reversible	High (mass send capability exists)	Any	Treat as not easily reversible until you gate send
Hard to reverse (prod config, access grant)	Low–med	High	Always-human or dual-control for writes
Hard to reverse	High	Any	Always-human . Break-glass only with a ticket
Irreversible / regulated (clinical, legal filing, funds move)	Any	Any	Always-human (or human + second system)—no “confident auto”
Any	Any	Safety classifier uncertain	Always-human or block

Operating rules

- Confidence is a **system** signal (retrieval score, validator pass, classifier margin). It is not the model that says “I am sure.”
- **Sample** means a statistically useful audit (stratified). It is not 0.1% sampling that is too small to be useful.
- Raise autonomy only when evals show error rates that match the blast radius (tie to file 02 thresholds).

- Common exam error: auto-routing of irreversible money movement because of “p95 latency.”

Quick stems

Q69. Low-confidence draft reply to one user, reversible in UI—route? **A.** Auto or light sample is OK. Still log. This is not the same class as a wire transfer.

Q70. High-confidence agent wants to detach IAM policy in prod—route? **A.** Always-human (or blocked tool). Confidence does not override irreversibility.

Part E — Privacy, compliance, and data governance

E1. Scenario frameworks (not legal advice)

Regime (often cited in exams)	Architect themes
GDPR	Lawful basis, minimization, access/erasure, DPIA, subprocessors, international transfers
HIPAA	PHI boundaries, BAAs, audit controls, minimum necessary
FedRAMP / public sector	Authorization boundary, logging, incident reporting, residency
SOC2 / ISO27001	Control ownership, change management, vendor review

Map product data flows: user → app → model provider → logs → vector DB → tools → third parties.

E2. Retention & training use

Know your contractual posture. Know if prompts train models. Know Zero Data Retention / retention options where the vendor offers them. Know enterprise settings for log retention.

Align app-level retention with legal holds and erasure requests. Include **embeddings and eval caches**.

E3. Privacy engineering tactics

- Tokenize or redact before the model call when this is feasible
- Use field-level encryption in stores
- Limit purpose in prompts (“use salary only for banding, do not echo”)
- Separate projects and keys per environment
- Review access for who can read prompt logs

E4. Transparency

Users need to know they interact with AI when this is material. Give limitations and a path to humans.

Document the model and provider in internal architecture records. EU AI Act-style transparency themes can appear as scenario details. Follow counsel.

Part F — Fairness, ethics, dual-use

F1. Fairness in enterprise AI

For ranking and decision support:

- Define a policy for protected attributes
- Measure slice performance
- Avoid proxies that encode discrimination
- Keep humans accountable for adverse decisions
- Provide contestability paths

F2. Dual-use & misuse

Architect product policies for disallowed assistance. Examples: large-scale deception, cyber offense, bio harm. Keep these consistent with provider AUPs.

Technical controls: classifiers, tool limits, abuse monitoring, and trust and safety escalation.

F3. Children / sensitive audiences

If the product can serve minors or vulnerable users, raise age-appropriate design, stricter HITL, and content policies. Do not treat this as generic chat.

Part G — Governance operating model

G1. Artifacts

- AI use-case register with risk tiering
- Architecture decision records that include safety
- Model, prompt, and tool change control
- Incident severity matrix for AI harms
- Vendor inventory (MCP, embedding providers)
- Eval reports as release evidence

G2. Risk tiers (example)

Tier	Examples	Controls
0	Internal brainstorming	Baseline AUP
1	Copilot drafts	Logging, DLP light
2	Customer-facing RAG	Citations, evals, on-call
3	Semi-automated actions	HITL, strong authZ, IR plan
4	Critical automated decisions	Often disallowed or a heavily regulated design

G3. Incident response extras for LLMs

Playbooks for: leaked prompts and secrets, injection campaigns, a model that produces prohibited content at scale, runaway spend, and cross-tenant bleed.

Include evidence preservation (traces). Include customer notification criteria with legal.

Part H — Safety evals

Safety is part of the Evaluation domain. Safety is also a joint ownership:

- Policy violation rate on red-team sets
- Indirect injection success rate
- PII leakage probes
- Over-refusal vs under-refusal balance for product UX
- Tool abuse attempts

Gate releases on safety metrics. Do not use task accuracy alone.

Decision trees

Autonomy gate

Can the action cause irreversible harm or regulated impact?

YES → HITL or forbid automation

NO → Is blast radius limited + evals green + monitoring live?

YES → Allow narrow auto class

NO → Suggest-only mode

Injection-prone feature gate

Does the model read attacker-influenced content AND hold write

YES → Split privileges; confirm writes; harden retrieval; maybe

NO → Standard input/output controls still apply

Exam traps

1. Prompt-only “compliance.”
2. Full autonomy on money movement / prod changes.
3. You ignore indirect injection in RAG.
4. You log raw PHI prompts with no time limit and no controls.
5. Fairness statements with no method for hiring/lending use cases.
6. You enable powerful MCP without vendor review.
7. No incident severity for AI-specific harms.
8. You treat provider alignment as app safety.
9. HITL that only looks like review, with no evidence or owners.
10. You erase DB rows but you leave embeddings.

Practice Q&A (28)

Q1. Why is a strong system prompt not enough to stop refund fraud? **A.** Attackers use injection. Controls must include tool authZ, confirmations, and monitoring.

Q2. Define indirect prompt injection. **A.** Malicious instructions that an attacker plants in content the model retrieves or reads.

Q3. Name three defense-in-depth layers. **A.** Any of: IdP/authZ, input filters, prompt policy, tool allowlists, HITL, monitoring.

Q4. When is graduated autonomy appropriate? **A.** Move from read-only to auto only after metrics prove safety on narrow classes.

Q5. What must erasure workflows include beyond primary DB delete? **A.** Embeddings, backups policy, eval caches, and logs per legal guidance.

Q6. Give an HITL anti-pattern. **A.** Approvals with no sources or diff shown. Humans approve without a real review.

Q7. How does Constitutional AI relate to app architects? **A.** It explains base model training philosophy. Apps still need their own controls.

Q8. What is insecure output handling? **A.** You execute or render model output without a sandbox or encoding (for example, a raw shell).

Q9. Why measure over-refusal? **A.** Excess refusals harm usefulness. Balance safety with product requirements.

Q10. What belongs in an AI use-case register? **A.** Purpose, data classes, risk tier, owner, controls, eval status.

Q11. Cross-tenant document bleed—primary control type? **A.** Hard isolation and filters in retrieval and authZ. Do not use prompt instructions alone.

Q12. Name a dual-use concern for coding agents. **A.** Assistance with offensive cyber techniques beyond defensive or authorized scope. Use policy plus monitoring.

Q13. Who approves tier-3 semi-automated money actions? **A.** Design must require human approval (and finance/security policy owners). The model alone is not enough.

Q14. What is a BAA relevant to? **A.** HIPAA scenarios where vendors handle PHI.

Q15. Why redact prompts in long-term logs? **A.** You reduce PII/secret sprawl and the blast radius of a breach.

Q16. What is alert fatigue in HITL? **A.** Too many low-value reviews cause humans to approve without a check.

Q17. Minimum gate before GA of customer RAG? **A.** Typically: grounding evals, injection tests, tenancy tests, monitoring, and a clear escalation path.

Q18. Why treat Skills/plugins as supply chain? **A.** They can add instructions or tools that exfiltrate data or raise privilege.

Q19. Output DLP caught a secret—what next? **A.** Block or queue the message. Rotate the credential if it is real. Run the incident process. Fix the source prompt or data.

Q20. Fairness slice testing means? **A.** You measure quality and error rates across relevant demographic or segment slices per policy.

Q21. Can you satisfy GDPR with model choice alone? **A.** No. You also need process, contracts, minimization, rights handling, and more.

Q22. What is a good first step when you add browser fetch to an agent? **A.** Threat-model injection and SSRF. Allowlist. Strip active content. Restrict tools.

Q23. Why separate eval caches from prod logging retention? **A.** They have different purposes and legal bases. Both still need erasure and access processes.

Q24. What should users see on material AI decisions? **A.** Notice that the system is AI-assisted. Show limitations. Give a path to human review (this depends on context).

Q25. Runaway agent spend—safety or ops issue? **A.** Both. Budgets are safety and reliability controls against uncontrolled autonomy.

Q26. Best control for “agent can email anyone”? **A.** Destination allowlists plus human confirm for novel addresses plus rate limits.

Q27. Why keep an abuse mailbox / reporting path? **A.** Users and customers need a channel. This feeds trust and safety response.

Q28. Provider ZDR means you can skip app-level retention design? **A.**
No. You still retain data in *your* logs, stores, and tools.

Checklist

- ☐ I can threat-model injection for RAG+tools
 - ☐ I can place controls across ≥ 5 layers
 - ☐ I can design HITL with evidence and SLAs
 - ☐ I can map data flows for privacy reviews
 - ☐ I can tier use cases and match controls
 - ☐ I can define AI incident severities
 - ☐ I can list safety eval probes for release gates
 - ☐ I can explain least privilege for agents
 - ☐ I can find prompt-only “compliance” answers
 - ☐ I can describe erasure beyond SQL delete
-

Glossary

Term	Meaning
Prompt injection	Steering a model via crafted untrusted text
Indirect injection	Injection via retrieved or third-party content
Jailbreak	Attempts to bypass safety policies
HITL	Human-in-the-loop oversight
DLP	Data loss prevention
DPIA	Data Protection Impact Assessment
BAA	Business Associate Agreement (HIPAA context)
ZDR	Zero Data Retention (contractual/API posture)
AUP	Acceptable Use Policy
Over-refusal	Refusal of benign requests in excess
Dual-use	Tech that people can use for helpful and harmful ends

Term	Meaning
Insecure output handling	Unsafe execution or rendering of model output
Use-case register	Inventory of AI systems with risk metadata
Red team	Adversarial testing of safety properties
Graduated autonomy	Phased increase of automatic action rights

Part I — Worked governance scenarios

I1. Hospital discharge summary assistant

Constraints: PHI. Harm is high if the output is wrong. **Design:** HIPAA-aware architecture. Use a BAA where required. Minimize PHI in prompts. Use on-prem or VPC cloud AI if mandated. Cite chart sections. Require clinician HITL always before chart writeback. Run extensive safety and accuracy evals. Keep audit logs. Do not add web tools. **Reject:** Autonomous chart updates. Consumer API keys in wards.

I2. HR benefits chatbot

Constraints: Employee personal data. Legal accuracy. **Design:** RAG over versioned benefits PDFs with citations. Refuse uncovered topics. Escalate to HR for exceptions. Use strict tenancy. Set retention limits. Fairness is less central than privacy and accuracy. Still monitor tone fairness.

I3. Marketing content generator

Constraints: Brand, copyright, deceptive claims. **Design:** Claim checkers against approved product facts. Disallow unverifiable medical and finance claims. Require human edit before publish. Log prompts for trademark issues.

14. DevOps agent with cloud credentials

Constraints: Production blast radius. **Design:** Read-only by default. Break-glass write role with a two-person rule. Environment segregation. Command allowlists. Mandatory change tickets for prod. Session recording. Budget caps. **Reject:** Long-lived admin keys in MCP config that people commit to git.

Part J — Policy writing for architects

Translate legal language into engineering that you can enforce:

Policy statement	Engineering control
“No automated adverse hiring decisions”	Block the tool that updates ATS stages. Require HITL.
“Minimize personal data”	Field allowlist before the model. Redact logs.
“Customers can request deletion”	Erasure API hits DB + index + object store
“All prod changes audited”	Tool gateway logs that are immutable to SIEM

If you cannot name the control, the policy has no real effect.

Part K — Extended Q&A (29–35)

Q29. Why forbid web tools in a PHI assistant by default? **A.** This reduces egress and injection surface. It also keeps the boundary clearer for compliance.

Q30. Two-person rule means? **A.** A sensitive action requires a second authorized human approval.

Q31. What is claim checking in marketing genAI? **A.** You verify generated claims against an approved facts base before publish.

Q32. Why is “clinician HITL always” justified for discharge summaries? **A.** Clinical risk is high. Regulations and duty of care expect professional accountability.

Q33. Git-committed cloud admin keys—what risks? **A.** Leakage, lateral movement, and agent actions you cannot control. Rotate keys. Move them to a secret manager.

Q34. How do you enforce “minimize personal data” at request time? **A.** Use explicit field allowlists or redaction before you call the model.

Q35. What makes an AI incident severity different from generic Sev1? **A.** It can include mass harmful content, privacy bleed across tenants, or unsafe autonomy. Uptime is not the only factor.

Extended checklist

- ☐ I can convert policy text into concrete engineering controls
 - ☐ I can design a PHI-safe architecture sketch
 - ☐ I can apply two-person rules to prod agents
 - ☐ I can justify when web/MCP tools are inappropriate
-
-

Part L — Organizational accountability & board-level framing

Architects often brief executives. Translate risk into business language:

Technical issue	Business framing
Indirect injection	Fraud / data exfiltration risk via ordinary documents
Hallucinated policy	Regulatory misstatement and customer harm
Over-broad tools	Blast radius like an insider threat, but with no insider
No eval gates	You ship unquantified error rates into brand channels
Log PII sprawl	Breach cost amplification

Propose **controls + metrics + owners + dates**. Do not use fear alone.

L1. Third-party risk questionnaire (MCP/SaaS AI)

Ask vendors:

- Data retention and training use
- Subprocessors list
- Encryption in transit and at rest
- Incident notification SLA
- Pen-test / SOC reports
- Region pinning
- Termination and deletion mechanics

Score before enablement. Review again on major version changes.

L2. Secure SDLC for prompts & tools

Treat prompts, Skills, and tools as code:

- PR review that includes security
- Static checks for forbidden tool scopes
- Secrets scanning
- Mandatory eval job
- Change ticket linking for prod pins

L3. Red team cadence

Run a quarterly external or internal red team for tier-2+ systems. Run continuous automated probes in CI for smoke-level injections.

Track findings to closure like vulnerabilities. CVE-like severity for AI issues is optional but useful.

L4. Accessibility & inclusion

Responsible AI includes accessible UX. Use screen readers for AI notices. Give readable explanations. Do not hide human escalation with dark patterns.

Multilingual safety must not be English-only evals that only look complete.

L5. Environmental / cost ethics (light touch)

Unbounded agent loops waste money and energy. Budgets are governance. They are not only FinOps.

L6. Deep scenario: “summarize these emails and send fixes”

Attack emails contain injection. Design: summarize in read-only mode. Never send mail without human confirm. Strip or ignore instruction-like lines in detectors. Allowlist recipients. Show a diff of proposed sends.

L7. Deep scenario: model suggests disabling safety filters

The product must not expose toggles that disable enterprise-mandatory controls to end users. Admin break-glass with audit only.

L8. Documentation pack for auditors

Keep: data flow diagrams, DPIA/transfer assessments, model cards/system cards (internal). The list also includes eval reports, access reviews, incident postmortems, and vendor DPAs.

CCAR-P does not score a document binder. Scenarios still ask what evidence you would produce.

L9. Extended Q&A 36–45

Q36. How do you brief an exec on injection risk in one sentence? **A.** “Ordinary documents can hold hidden instructions that trick the assistant into abuse of its permissions.”

Q37. Name four vendor questions before enabling SaaS MCP. **A.** Retention/training, subprocessors, regions, incident SLA (plus deletion, encryption, audit reports).

Q38. Why treat prompts as code? **A.** They change behavior like code. They need review, versioning, testing, and controlled release.

Q39. English-only safety evals miss what? **A.** Multilingual jailbreaks and toxicities. Harm to users who do not use English.

Q40. What is break-glass for safety toggles? **A.** Time-limited admin override with mandatory audit. It is not a user-facing ‘disable safety’ switch.

Q41. Why are budgets a governance control? **A.** They bound autonomous waste, abuse, and runaway tool loops.

Q42. What evidence might an auditor request for an AI feature? **A.** Data flows, DPIA, eval gates, access reviews, DPAs, incident records.

Q43. In the malicious email scenario, what is the safest send policy? **A.** Human confirm + recipient allowlist + visible diff. No autonomous send.

Q44. How often red team tier-2+ systems (example cadence)? **A.** At least quarterly deep tests plus continuous automated smoke probes.

Q45. Secret scanning should cover which AI artifacts? **A.** Prompts, Skills, MCP configs, notebooks, CI logs, and example traces in repos.

L10. Extra checklist

- ☐ I can translate technical AI risk to business impact
- ☐ I can run a vendor MCP questionnaire
- ☐ I can describe secure SDLC for prompts/tools
- ☐ I can outline auditor evidence for an AI system
- ☐ I can design read-only summarize vs send flows safely
- ☐ I can argue against user-facing switches that disable safety

L11. Glossary additions

Term	Meaning
System card	Internal or public description of system behavior and limits
DPA	Data Processing Agreement
Break-glass	Emergency privileged access with audit
Red team cadence	Scheduled adversarial testing rhythm
FinOps	Cloud/AI cost operational discipline
ATS	Applicant tracking system
SIEM	Security information and event management
Dark pattern	UX that manipulates users against their interest
Transfer assessment	Review of cross-border data transfers
Smoke probe	Lightweight continuous safety test

Part M — Mapping controls to CCAR-P verbs

Exam stems often use verbs: *implement, identify, ensure, design, recommend*. Pair them:

Verb	Strong answer ingredient
Implement	Name a concrete mechanism (allowlist, HITL queue, DLP)
Identify	Name the risk and where it arises in the architecture
Ensure	Control plus measurement/monitoring
Design	Diagrammed layers plus owners
Recommend	Choose among options with trade-offs. Do not list everything.

Avoid vague “follow best practices” selections when a specific control is offered.

M1. Final rapid-fire Q&A 46–50

Q46. Best single control to add when you give an agent `delete_instance`?
A. Remove it or require multi-party approval. Least privilege first.

Q47. What is wrong with storing raw chat that contains SSNs with no time limit? **A.** Retention and minimization fail. A breach gets worse. Rights requests become hard.

Q48. How do you “ensure” tenancy safety? **A.** Hard authZ filters plus automated cross-tenant retrieval tests in CI plus monitoring.

Q49. Customer asks to turn off logging for “privacy.” Risk? **A.** This can conflict with security and audit duties. Seek legal and security design of redaction vs disable.

Q50. Which is better evidence of safety: marketing claims or eval reports? **A.** Versioned eval/red-team reports with metrics and owners.

Part N — Safety exam drill

Stems often combine “automate” and regulated data. Default instinct: **narrow scope, HITL, server-side authZ, eval gates.**

If an option only strengthens the system prompt, it is usually incomplete.

N1. Abuse monitoring signals

- Spike in refusal bypass attempts
- Unusual tool fanout per user
- Destination novelty for email/transfer tools
- Cross-tenant retrieval denials surge (or worse, successes)
- Sudden cost per user

N2. Model card vs system card

Model cards describe base models. **System cards** describe *your* application's behavior, evaluations, and limits. Architects own system-level documentation.

N3. Q&A 51–56

Q51. If one option is “stronger system prompt” and another adds tool confirm + allowlist, which usually wins for money movement? **A.** Confirm + allowlist (hard controls).

Q52. What is a system card in enterprise practice? **A.** Documentation of the deployed AI system's behavior, evals, and limitations.

Q53. Name three abuse-monitoring signals. **A.** Bypass attempts, tool fanout spikes, novel transfer destinations (and similar signals).

Q54. Cross-tenant retrieval *successes* in logs mean? **A.** Critical isolation failure. Start incident response immediately.

Q55. Why separate model cards from system cards? **A.** Provider model properties ≠ your tool privileges, data, and UX risks.

Q56. Default design instinct for regulated automation stems? **A.** Narrow scope + HITL + server authZ + eval gates.

Next: 04–stakeholder-lifecycle-gtm.md. Enablement details in 05.

Part O — Rapid review (Governance 14%)

Governance questions reward **layered controls** and a modest view of prompts. Rapid list:

- Threat-model injection whenever untrusted text meets privileged tools.
- HITL for irreversible or regulated impacts. Do not use approvals without review.
- Privacy: minimization, retention, erasure including embeddings.
- Tier use cases. Match controls to tiers.
- Vendors/MCP are supply chain.
- Safety evals gate releases with task quality.
- Incidents need AI-specific severities and evidence trails.
- System cards document *your* app, not only the base model.
- Translate policy → engineering mechanism.
- Never claim prompt-only compliance.

Memory aid: **Hospital writeback** → **always clinician HITL**. **DevOps agent** → **read-only first**. **Marketing claims** → **fact checker**. **Email send** → **confirm + allowlist**.

Part P — Control catalog (printable)

Risk	Preventive control	Detective control	Corrective control
Indirect injection	Privilege split. Allowlists	Injection probes. Odd tool patterns	Disable tool. Revoke sessions
Hallucinated advice	Citations. Refuse-on-empty	Faithfulness evals	HITL correction. User notice
PII leakage	Minimization. DLP	Log scanners	Rotate. Notify. Purge
Over-autonomy	Budgets. Confirmations	Turn/cost alerts	Flag off. Reduce scope
Cross-tenant bleed	Hard filters. Separate indexes	Canary tenant tests	Incident IR. Patch filters
Malicious MCP	Vendor review. Marketplace only	Version pin monitors	Kill switch. Revoke tokens
Insecure output handling	Sandbox. No raw exec	Runtime detections	Patch renderer. Revoke

Risk	Preventive control	Detective control	Corrective control
Bias in decisions	Human accountability. Slice tests	Disparate error dashboards	Adjust product. Stop auto

Use this catalog when a stem asks what to *implement*, *monitor*, or *do after* an incident. Match the column to the verb.

Closing thought

Responsible AI for architects is operational. It is privileges, proofs, people in the loop, and paperwork that maps to real systems.

If your answer cannot name a mechanism, a metric, and an owner, continue to design.

Remember: mechanism + metric + owner. Prompt text is never the complete control answer for CCAR-P governance items. Pair it with authZ, HITL, monitoring, and evidence. *(File continues — deep-dive Part Q follows below.)*

One more worked stem

Stem example: “Auto-approve invoice payments under \$500 via Claude agent with ERP write tool.”

Strong answer direction: lower the threshold further or require dual control. Enforce a payee allowlist. Use server-side authZ. Detect anomalies on novel payees. Use HITL above a risk score. Do not rely on a prompt that only promises thrift.

Part Q — Primary-study deep dive: Governance architecture

Q1. Defense-in-depth reference blueprint

- [User/Channel]
- AuthN (SSO) + AuthZ (role → agent)
 - Input filters (injection heuristics, DLP, malware)
 - Orchestrator policy (allowed tools, budgets)

- Claude (system policy + Skills)
- Tool gateway (allowlist, arg validation, server authZ)
- Output filters (PII, toxicity, brand, secrets)
- HITL queues (by risk tier)
- Audit + abuse monitoring + IR

Every exam scenario that proposes “just add a stronger system prompt” is incomplete unless you consider other layers.

Q2. Risk-tier → control baseline (copy/adapt)

Tier	Example	Auton- omy	Logging	HITL	Eval bar
T0 Infor- mational	Public FAQ para- phrase	High	Basic	Spot check	Light
T1 Assistive draft	Email draft to human	High draft / low send	Full prompts redacted	Human sends	Medium
T2 Bounded action	Refund ≤ \$50	Auto within caps	Full + tool I/O	Sampled + anoma- lies	High
T3 High impact	Medical, legal, large money, prod changes	Default deny auto	Tamper- evident	Manda- tory	Highest + red team

Q3. Guardrail trade-off table

Control	Catches	Misses / cost	Overuse failure
System prompt policy	Soft behavioral norms	Determined injection	You think you are safe when you are not
Input classifier	Many jailbreaks/PII	Novel paraphrases. Latency	Blocks legitimate work

Control	Catches	Misses / cost	Overuse failure
Output DLP	Secrets/PII egress	Encoded exfil	Over-redaction UX
Tool allowlist	Over-privilege	Logic bugs inside allowed tools	Shadow IT bypass
Server authZ	Confused deputy	Misconfigured ACLs	— (do not skip)
HITL	High-impact errors	Slow. Reviewer fatigue	Approval without review
Monitoring	Drift/abuse over time	Needs analysts	Alert fatigue

Q4. HITL design patterns

1. **Approve/reject gate** before a side effect.
2. **Edit-then-send** (copilot).
3. **Review sampling** (X% or uncertainty-triggered).
4. **Four-eyes** for dual control (finance).
5. **Async specialist queue** (legal/medical).

Anti-patterns: HITL on every trivial FAQ (this makes the product too costly). HITL with no SLA (the queue never completes). HITL without UI that shows sources/tool args (blind approval).

Q5. Prompt injection failure-mode analysis

Variant	Channel	Example	Primary mitigations
Direct	User chat	“Ignore previous...”	Policy + classifiers + refuse
Indirect	Retrieved doc/email/web	Hidden instruction in PDF	Treat content as data. Cite. No raw tool trust
Tool-result injection	MCP/API payloads	Error message contains instructions	Schema validate. Sanitize. Allowlist

Variant	Channel	Example	Primary mitigations
Cross-user	Shared memory/stores	Poisoned shared Skill	Tenancy. Review. Signing
Multimodal	Image/OCR text	Text in screenshot	Same as indirect

Never rely on: “The model is aligned so we are fine.” Alignment reduces risk. Application controls remain mandatory.

Q6. Compliance overlay mapping (scenario language, not legal advice)

Constraint cue in stem	Architecture implications
GDPR / personal data	Minimization, retention, DSR erasure including indexes/logs, DPIA-minded design, transfer review
HIPAA / health	BAA/vendor posture, PHI minimization, access controls, audit, careful on third-party tools/web
FedRAMP / gov	Authorized deployment path, strict boundaries, logging, change control
PCI hints	Never put PAN into prompts. Tokenize. Segment
Children’s data	Heightened restrictions. Often avoid training/retention. Parental/policy gates

Exam answers must **name controls**. Do not recite statute numbers.

Q7. Fairness & transparency practical checklist

- ☐ Who is affected if the model is wrong?
- ☐ Disparate error rates checked across relevant groups where lawful/appropriate?

- ☐ User told they interact with AI?
- ☐ Limitations and escalation path documented?
- ☐ Appeals/override for consequential decisions?
- ☐ Training/eval data governance noted?

Q8. Incident response extras for LLM apps

Add to common IR: - Prompt/tool trace preservation (legal hold) - Ability to disable tools/models/features via flag - Search for injection indicators - Customer comms templates (file 04) - Model/provider status correlation

Part R — Worked decision scenarios (Safety)

R1. “Auto-send email fixes from mailbox summary”

Reject auto-send. Summarize and draft only. A human sends. Treat email bodies as untrusted (indirect injection). Run DLP on drafts. Audit.

R2. DevOps agent with cloud credentials

Read-only recon tools first. Writes via change tickets + HITL. Short-lived creds. No org-admin keys in MCP. Network egress allowlists. Separate break-glass.

R3. HR chatbot asked for medical leave legal advice

Scope to benefits docs via RAG + cite. Refuse personalized legal/medical advice. Escalate to an HR human. Log sensitive topics with care.

R4. Marketing generator producing competitor-disparaging claims

Brand/policy output filters. Require sources for factual claims. Human publish gate. Eval for prohibited topics.

R5. Stakeholder asks to “turn off safety filters for speed”

Architect response: safety filters are risk controls tied to tier. Propose measured latency options (caching, routing) instead. Document residual risk if any control is narrowed with approval.

Part S — Production safety checklist

S1. Design review

- ☐ Threat model includes injection, leakage, over-autonomy, abuse
- ☐ Risk tier assigned with control baseline
- ☐ Tool allowlist + server authZ proven with negative tests
- ☐ HITL triggers and SLAs defined
- ☐ Data classes allowed in prompts documented
- ☐ Retention for logs/evals lawful and minimized
- ☐ Abuse monitoring signals listed
- ☐ Red-team plan scheduled pre-GA

S2. Pre-GA safety gate

- ☐ Injection suite pass rate meets bar
- ☐ DLP false-positive UX accepted by product
- ☐ High-impact paths cannot execute without HITL/policy engine
- ☐ Vendor/MCP questionnaire complete
- ☐ Incident runbook rehearsed
- ☐ Transparency/notice copy approved

S3. Steady-state

- ☐ Quarterly access review for agent credentials
- ☐ Eval drift review includes safety metrics
- ☐ Plugin/MCP updates reviewed like dependencies
- ☐ Post-incident learnings → control updates

Part T — Trade-offs: autonomy vs control

Push for more autonomy when...	Keep humans when...
Actions are reversible and blast radius is low	Money, health, safety, or legal impact that you cannot reverse
Strong evals + monitoring	Sparse data / novel intents
Clear policy engines encode rules	Rules require judgment calls
Exception rate is within staffing	Reviewers already have too much work

Part U — Extended Q&A (57–65)

Q57. Strongest control against unauthorized data access via tools? **A.** **Server-side authorization** on the data plane using verified identity.

Q58. Indirect injection best single description? **A.** Malicious instructions embedded in **content the model reads** (docs, web, emails). This is not only in the user chat box.

Q59. Select TWO for Tier-3 payout agent. The list also includes mandatory HITL, open web server tool for all users, full audit traces, temperature=1.5 creative mode. **A.** Mandatory HITL and full audit traces.

Q60. Constitutional AI means your app is compliant with HIPAA? **A.** **No**—base alignment ≠ application compliance. You still design controls and vendor posture.

Q61. Reviewers approve without a real check—what do you change? **A.** Reduce volume (better auto for T0/T1). Improve UI with sources/tool args. Rotate reviewers. Sample quality audits.

Q62. GDPR erasure request—what systems to touch? **A.** Primary DB **and** vector indexes, caches, eval sets, logs (per policy), backups per legal schedule.

Q63. Best response if product wants the model to execute shell from output? **A.** **Sandbox / deny by default.** Never pipe unsandboxed LLM text to privileged shells.

Q64. Dual-use request for detailed biological harm—in product policy terms? **A.** Refuse or give a safe completion per acceptable use. Escalate abuse patterns. Do not provide actionable harmful detail.

Q65. Select THREE layers in defense in depth. Tool allowlist, output DLP, server authZ, “trust the vendor alone”, disable all logging for ineffective privacy control. **A.** Allowlist, output DLP, server authZ.

Part V — Rapid review (Governance 14%)

- Design controls. Do not use slogans.
- Defense in depth. Prompts are one layer.
- Injection: direct + indirect + tool results.
- HITL by impact tier. Avoid approvals without review.
- AuthZ in code. Least privilege tools.

- Compliance = controls + evidence. Not slogans.
- Fairness, notices, appeals for consequential AI.
- IR needs traces, kill switches, and clear owners.

Pair with Integration authZ in 02 and lifecycle risk talk-tracks in 04.

Part W — Abuse monitoring, model cards, and exam stems

W1. Abuse monitoring signal examples

- Sudden spike in tool calls per user
- Repeated jailbreak phrasings
- DLP hit clusters
- Anomalous exfil-sized outputs
- New MCP server reaching odd egress

Wire to SOC/IR playbooks. Do not collect sensitive content beyond policy.

W2. Model card vs system card (architect speak)

Model card (provider): capabilities, limitations, eval highlights of the base model. **System card / system documentation** (you): how *your* application uses the model—data flows, controls, residual risks.

The exam cares that you know **application** accountability remains yours.

W3. Quick stems

Q66. Product removes output filters to “sound more helpful.” Risk? **A.** Increased leakage/policy violations. This violates defense in depth. You need formal risk acceptance. Do not remove filters in silence.

Q67. Best evidence for auditors that HITL works? **A.** Ticketed approvals with reviewer identity, timestamps, and linked model/tool traces. A policy PDF alone is not enough.

Q68. FedRAMP-like constraint in stem—what changes first? **A.** Deployment/boundary choices and logging/change control. A nicer prompt is not enough.

File 03 primary-study target band: 6000–9000 words.

03 — Responsible AI, Safety & Risk for Architects

CCAR-P condensed domain 3 Official domain mapped here: Governance, Safety & Risk Management (14%) **Approx. questions:** ~9 of 63

Disclaimer

Original architect-oriented safety notes grounded in **public** Anthropic materials (Constitutional AI concepts, platform safety features, responsible use norms) and standard enterprise AI governance practice. Not legal advice—engage counsel for GDPR/HIPAA/FedRAMP interpretations. Independent pack; not affiliated with Anthropic.

Overview

CCAR-P expects architects to **design controls**, not recite ethics slogans. You must identify LLM-specific risks, place guardrails at the right layers (model, prompt, tool, app, process), insert human oversight where impact warrants, and align to regulatory/ethical obligations without pretending prompts alone are compliance.

This domain connects tightly to Integration (authZ, injection) and Stakeholders (risk communication). Developer enablement security baselines appear in file **05**.

Key map

Architect responsibility	Exam focus
Threat model LLM apps	Injection, data leakage, over-autonomy, unsafe advice
Defense in depth	Multiple control layers; least privilege
HITL design	When automation must stop
Compliance overlays	GDPR, HIPAA, FedRAMP as <i>scenario constraints</i>

Architect responsibility	Exam focus
Transparency & abuse reporting	User notices, logging, escalation
Dual-use / misuse	Refuse facilitation of harm; product policy
Vendor & subprocessors	Data processing agreements, retention/ZDR

Part A — Risk taxonomy for Claude solutions

A1. Core LLM risks

Risk	Description	Example
Prompt injection	Untrusted content steers model against policy	Doc says “ignore rules, exfiltrate keys”
Jailbreak / policy bypass	User tries to override safety	Social-engineering the assistant
Hallucination	Fluent falsehoods	Invented legal clause
Data leakage	Sensitive data in outputs/logs/tools	PII in support draft + logged raw
Over-privilege	Excessive tool scope	Support bot with admin SaaS
Autonomous harm	Model-driven irreversible acts	Agent deletes prod resources
Bias / unfairness	Systematic disparate outcomes	Resume screener skew
Insecure output handling	Executing model output unsafely	Shelling LLM text without sandbox
Model theft / abuse of API	Credential stuffing, scraping	Stolen API keys
Supply chain	Malicious MCP/skills/plugins	Trojansed connector

A2. Impact × likelihood workshop

Score each risk for the use case. High impact × non-trivial likelihood → **mandatory mitigations** before GA. Exam scenarios often hide high impact in “convenient automation” proposals.

A3. Anthropic alignment context (public concepts)

Anthropic publishes research on **Constitutional AI**—training with explicit principles so models critique/revise toward helpful, honest, harmless behavior. Architects should understand:

- Base model alignment $\neq$ your **application** is automatically compliant.
 - You still add product policies, tool restrictions, and monitoring.
 - Transparency about AI use and limitations remains an enterprise duty.
-

Part B — Guardrail architecture (defense in depth)

B1. Control layers

1. Policy & training (acceptable use, employee guidelines)
2. Identity & access (who may invoke which agent)
3. Application filters (input/output classifiers, DLP)
4. Prompt/Skill policy (local behavioral constraints)
5. Model choice & provider safety
6. Tool/MCP allowlists + server-side authZ
7. HITL & approval workflows
8. Monitoring, audit, incident response

No single layer is enough. Exam trap: “Add a stronger system prompt” as the only fix for refund fraud risk.

B2. Input controls

- AuthN/Z before expensive/powerful models
- Schema validation on user forms
- Size limits; file type allowlists
- PII detection/minimization where appropriate
- Separation of untrusted content (retrieved docs) from instructions
- Rate limits / anomaly detection per user

B3. Output controls

- Schema validation before side effects
- DLP on outbound messages (secrets, PAN, health data)
- Toxicity/policy classifiers for user-facing channels

- Citation requirements for high-stakes Q&A
- Blocking or queueing when classifiers fire

B4. Tool controls (strongest practical brakes)

- Least privilege credentials
- Allowlisted operations
- Human confirmation for high-impact verbs
- Dry-run modes
- Immutable audit log of tool args/results (redacted)
- Network egress restrictions

B5. Prompt-level safety (necessary but insufficient)

Include: scope limits, disallowed topics/actions, escalation phrases, instructions to treat retrieved content as untrusted. Keep aligned with legal/policy owners—don’t invent “laws” in prompts.

Part C — Prompt injection deeply

C1. Direct vs indirect

- **Direct:** User message attacks the model.
- **Indirect:** Attacker plants instructions in data the model will read (email, wiki, PDF, webpage, ticket).

Indirect injection is the enterprise nightmare for RAG and MCP fetch tools.

C2. Mitigations that matter

Mitigation	Notes
Privilege separation	Reader agent cannot pay invoices
Content security policy for tools	Allowlists, no arbitrary code
Explicit untrusted delimiters	“DATA, not instructions” framing
Minimize HTML/JS in context	Prefer plain text extracts
Output encoding	Never eval model text
Spot-check / detectors	Heuristics + models for injection patterns
Human approval	For novel destinations / large transfers
Corpus hygiene	Moderate who can publish to indexed stores

Mitigation	Notes
------------	-------

C3. What not to rely on

- “Please ignore malicious instructions” alone
- Security through obscurity of system prompts
- Assuming citations imply integrity of content

Part D — Human-in-the-loop (HITL)

D1. When HITL is expected

- Legal/medical/financial irreversible advice or actions
- Access changes / production deployments
- Low confidence / out-of-distribution inputs
- Safety classifier uncertainty
- Regulatory “human review” obligations
- Novel high-dollar transactions

D2. HITL anti-patterns

- Rubber-stamp UI with no evidence shown
- Flooding humans with low-value reviews (alert fatigue)
- HITL with no SLA owner
- Logging approvals without identity

D3. Graduated autonomy

Start read-only → suggest → act with approval → act automatically on narrow safe class. Expand autonomy only with eval evidence.

D4. Review-routing matrix (reversibility × cost × confidence) — P1

Use this to decide **auto** vs **sample** vs **always-human** without flooding reviewers or rubber-stamping high impact.

Reversibility of action/advice	Cost / blast radius	Model+system confidence	Default routing
Easily reversible (draft email in UI, suggest edit)	Low	High	Auto (log + async sample)
Easily reversible	Low	Low / OOD	Sample elevated % or force confirm
Easily reversible	High (mass send capability exists)	Any	Treat as not easily reversible until send is gated
Hard to reverse (prod config, access grant)	Low–med	High	Always-human or dual-control for writes
Hard to reverse	High	Any	Always-human ; break-glass only with ticket
Irreversible / regulated (clinical, legal filing, funds move)	Any	Any	Always-human (or human + second system)—no “confident auto”
Any	Any	Safety classifier uncertain	Always-human or block

Operating rules

- Confidence is a **system** signal (retrieval score, validator pass, classifier margin)—not the model saying “I’m sure.”
- **Sample** means statistically useful audit (stratified), not 0.1% vanity sampling.
- Raise autonomy only when evals show error rates compatible with blast radius (tie to file 02 thresholds).
- Exam trap: auto-routing irreversible money movement because “p95 latency.”

Quick stems

Q69. Low-confidence draft reply to one user, reversible in UI—route? **A.** Auto or light sample OK; still log. Not the same class as wire transfer.

Q70. High-confidence agent wants to detach IAM policy in prod—route?
A. Always-human (or blocked tool); confidence does not override irreversibility.

Part E — Privacy, compliance, and data governance

E1. Scenario frameworks (not legal advice)

Regime (often cited in exams)	Architect themes
GDPR	Lawful basis, minimization, access/erasure, DPIA, subprocessors, international transfers
HIPAA	PHI boundaries, BAAs, audit controls, minimum necessary
FedRAMP / public sector	Authorization boundary, logging, incident reporting, residency
SOC2 / ISO27001	Control ownership, change management, vendor review

Map product data flows: user → app → model provider → logs → vector DB → tools → third parties.

E2. Retention & training use

Know your contractual posture: whether prompts are used to train; Zero Data Retention / retention options where offered; enterprise settings for log retention. Align app-level retention with legal holds and erasure requests—including **embeddings and eval caches**.

E3. Privacy engineering tactics

- Tokenization / redaction before model call when feasible
- Field-level encryption in stores
- Purpose limitation in prompts (“use salary only for banding, don’t echo”)
- Separate projects/keys per environment
- Access reviews on who can read prompt logs

E4. Transparency

Users should know they're interacting with AI when material; provide limitations and escalation to humans. Document model/provider in internal architecture records (EU AI Act-style transparency themes may appear as scenario flavor—follow counsel).

Part F — Fairness, ethics, dual-use

F1. Fairness in enterprise AI

For ranking/decision support:

- Define protected attributes handling policy
- Measure slice performance
- Avoid proxies that encode discrimination
- Keep humans accountable for adverse decisions
- Provide contestability paths

F2. Dual-use & misuse

Architect product policies for disallowed assistance (e.g., large-scale deception, cyber offense, bio harm) consistent with provider AUPs. Technical: classifiers, tool limits, abuse monitoring, trust & safety escalation.

F3. Children / sensitive audiences

If product may serve minors or vulnerable users, raise age-appropriate design, stricter HITL, and content policies—don't treat as generic chat.

Part G — Governance operating model

G1. Artifacts

- AI use-case register with risk tiering
- Architecture decision records including safety
- Model/prompt/tool change control
- Incident severity matrix for AI harms
- Vendor inventory (MCP, embedding providers)
- Eval reports as release evidence

G2. Risk tiers (example)

Tier	Examples	Controls
0	Internal brainstorming	Baseline AUP
1	Copilot drafts	Logging, DLP light
2	Customer-facing RAG	Citations, evals, on-call
3	Semi-automated actions	HITL, strong authZ, IR plan
4	Critical automated decisions	Often disallowed or heavily regulated design

G3. Incident response extras for LLMs

Playbooks for: leaked prompts/secrets, injection campaigns, model producing prohibited content at scale, runaway spend, cross-tenant bleed. Include evidence preservation (traces) and customer notification criteria with legal.

Part H — Safety evals

Safety is part of Evaluation domain but owned jointly:

- Policy violation rate on red-team sets
- Indirect injection success rate
- PII leakage probes
- Over-refusal vs under-refusal balance for product UX
- Tool abuse attempts

Gate releases on safety metrics, not only task accuracy.

Decision trees

Autonomy gate

Can the action cause irreversible harm or regulated impact?

YES → HITL or forbid automation

NO → Is blast radius limited + evals green + monitoring live?

YES → Allow narrow auto class

NO → Suggest-only mode

Injection-prone feature gate

Does the model read attacker-influenced content AND hold write

YES → Split privileges; confirm writes; harden retrieval; maybe

NO → Standard input/output controls still apply

Exam traps

1. Prompt-only “compliance.”
 2. Full autonomy on money movement / prod changes.
 3. Ignoring indirect injection in RAG.
 4. Logging raw PHI prompts indefinitely without controls.
 5. Fairness hand-waving for hiring/lending use cases.
 6. Enabling powerful MCP without vendor review.
 7. No incident severity for AI-specific harms.
 8. Equating provider alignment with app safety.
 9. HITL theater without evidence or owners.
 10. Erasing DB rows but leaving embeddings.
-

Practice Q&A (28)

Q1. Why isn’t a strong system prompt enough to stop refund fraud? **A.** Attackers use injection; controls must include tool authZ, confirmations, monitoring.

Q2. Define indirect prompt injection. **A.** Malicious instructions planted in content the model retrieves or reads.

Q3. Name three defense-in-depth layers. **A.** Any of: IdP/authZ, input filters, prompt policy, tool allowlists, HITL, monitoring.

Q4. When is graduated autonomy appropriate? **A.** Expand from read-only to auto only after metrics prove safety on narrow classes.

Q5. What must erasure workflows include beyond primary DB delete? **A.** Embeddings, backups policy, eval caches, logs per legal guidance.

- Q6.** Give an HITL anti-pattern. **A.** Rubber-stamp approvals with no sources/diff shown.
- Q7.** How does Constitutional AI relate to app architects? **A.** Explains base model training philosophy; apps still need their own controls.
- Q8.** What is insecure output handling? **A.** Executing or rendering model output without sandboxing/encoding (e.g., raw shell).
- Q9.** Why measure over-refusal? **A.** Excess refusals harm usefulness; balance safety with product requirements.
- Q10.** What belongs in an AI use-case register? **A.** Purpose, data classes, risk tier, owner, controls, eval status.
- Q11.** Cross-tenant document bleed—primary control type? **A.** Hard isolation/filters in retrieval & authZ—not prompt instructions alone.
- Q12.** Name a dual-use concern for coding agents. **A.** Assistance with offensive cyber techniques beyond defensive/authorized scope—policy + monitoring.
- Q13.** Who approves tier-3 semi-automated money actions? **A.** Design should require human approval (and finance/security policy owners)—not the model alone.
- Q14.** What is a BAA relevant to? **A.** HIPAA scenarios involving PHI handled by vendors.
- Q15.** Why redact prompts in long-term logs? **A.** Reduce PII/secret sprawl and breach blast radius.
- Q16.** What is alert fatigue in HITL? **A.** Too many low-value reviews cause humans to approve blindly.
- Q17.** Minimum gate before GA of customer RAG? **A.** Typically: grounding evals, injection tests, tenancy tests, monitoring, clear escalation path.
- Q18.** Why treat Skills/plugins as supply chain? **A.** They can introduce instructions/tools that exfiltrate data or escalate privilege.
- Q19.** Output DLP caught a secret—what next? **A.** Block/queue message, rotate credential if real, incident process, fix source prompt/data.
- Q20.** Fairness slice testing means? **A.** Measuring quality/error rates across relevant demographic or segment slices per policy.
- Q21.** Can you satisfy GDPR with model choice alone? **A.** No—process, contracts, minimization, rights handling, etc. are required.

Q22. What’s a good first step when adding browser fetch to an agent? **A.** Threat-model injection + SSRF; allowlist; strip active content; restrict tools.

Q23. Why separate eval caches from prod logging retention? **A.** Different purposes/legal bases; still both need erasure/access processes.

Q24. What should users see on material AI decisions? **A.** Notice it’s AI-assisted, limitations, and path to human review (context-dependent).

Q25. Runaway agent spend—safety or ops issue? **A.** Both: budgets are safety/reliability controls against uncontrolled autonomy.

Q26. Best control for “agent can email anyone”? **A.** Destination allowlists + human confirm for novel addresses + rate limits.

Q27. Why keep an abuse mailbox / reporting path? **A.** Users and customers need a channel; feeds trust & safety response.

Q28. Provider ZDR means you can skip app-level retention design? **A.** No—you still retain data in *your* logs, stores, and tools.

Checklist

- ☐ I can threat-model injection for RAG+tools
 - ☐ I can place controls across ≥ 5 layers
 - ☐ I can design HITL with evidence and SLAs
 - ☐ I can map data flows for privacy reviews
 - ☐ I can tier use cases and match controls
 - ☐ I can define AI incident severities
 - ☐ I can list safety eval probes for release gates
 - ☐ I can explain least privilege for agents
 - ☐ I can spot prompt-only “compliance” answers
 - ☐ I can describe erasure beyond SQL delete
-

Glossary

Term	Meaning
Prompt injection	Steering model via crafted untrusted text

Term	Meaning
Indirect injection	Injection via retrieved/third-party content
Jailbreak	Attempts to bypass safety policies
HITL	Human-in-the-loop oversight
DLP	Data loss prevention
DPIA	Data Protection Impact Assessment
BAA	Business Associate Agreement (HIPAA context)
ZDR	Zero Data Retention (contractual/API posture)
AUP	Acceptable Use Policy
Over-refusal	Refusing benign requests excessively
Dual-use	Tech usable for helpful and harmful ends
Insecure output handling	Unsafe execution/rendering of model output
Use-case register	Inventory of AI systems with risk metadata
Red team	Adversarial testing of safety properties
Graduated autonomy	Phased increase of automatic action rights

Part I — Worked governance scenarios

I1. Hospital discharge summary assistant

Constraints: PHI, high harm if wrong. **Design:** HIPAA-aware architecture; BAA where required; minimize PHI in prompts; on-prem/VPC cloud AI if mandated; citations to chart sections; clinician HITL always before chart writeback; extensive safety+accuracy evals; audit logs; no web tools. **Reject:** Autonomous chart updates; consumer API keys in wards.

I2. HR benefits chatbot

Constraints: Employee personal data; legal accuracy. **Design:** RAG over versioned benefits PDFs with citations; refuse uncovered topics; escalate to HR for exceptions; strict tenancy; retention limits; fairness less central than privacy/accuracy but tone fairness still monitored.

I3. Marketing content generator

Constraints: Brand, copyright, deceptive claims. **Design:** Claim checkers against approved product facts; disallow unverifiable medical/finance claims; human edit before publish; logging of prompts for trademark issues.

I4. DevOps agent with cloud credentials

Constraints: Production blast radius. **Design:** Read-only by default; break-glass write role with two-person rule; environment segregation; command allowlists; mandatory change tickets for prod; session recording; budget caps. **Reject:** Long-lived admin keys in MCP config committed to git.

Part J — Policy writing for architects

Translate legalese into enforceable engineering:

Policy statement	Engineering control
“No automated adverse hiring decisions”	Tool that updates ATS stages blocked; HITL required
“Minimize personal data”	Field allowlist pre-model; redact logs
“Customers can request deletion”	Erasure API hits DB+index+object store
“All prod changes audited”	Tool gateway logs immutable to SIEM

If you cannot name the control, the policy is theater.

Part K — Extended Q&A (29–35)

Q29. Why forbid web tools in a PHI assistant by default? **A.** Reduces egress/injection surface and keeps boundary clearer for compliance.

Q30. Two-person rule means? **A.** Sensitive action requires a second authorized human approval.

Q31. What is claim checking in marketing genAI? **A.** Verifying generated claims against an approved facts base before publish.

Q32. Why is “clinician HITL always” justified for discharge summaries? **A.** High clinical risk; regulations and duty of care expect professional accountability.

Q33. Git-committed cloud admin keys—what risks? **A.** Leakage, lateral movement, uncontrollable agent actions; rotate and move to secret manager.

Q34. How do you enforce “minimize personal data” at request time? **A.** Explicit field allowlists / redaction before calling the model.

Q35. What makes an AI incident severity different from generic Sev1? **A.** May include mass harmful content, privacy bleed across tenants, or unsafe autonomy—not only uptime.

Extended checklist

- ☐ I can convert policy text into concrete engineering controls
 - ☐ I can design a PHI-safe architecture sketch
 - ☐ I can apply two-person rules to prod agents
 - ☐ I can justify when web/MCP tools are inappropriate
-
-

Part L — Organizational accountability & board-level framing

Architects often brief executives. Translate risk into business language:

Technical issue	Business framing
Indirect injection	Fraud / data exfiltration risk via ordinary documents
Hallucinated policy	Regulatory misstatement & customer harm
Over-broad tools	Insider-threat-class blast radius without an insider
No eval gates	Shipping unquantified error rates into brand channels
Log PII sprawl	Breach cost amplification

Propose **controls + metrics + owners + dates**, not fear alone.

L1. Third-party risk questionnaire (MCP/SaaS AI)

Ask vendors:

- Data retention & training use
- Subprocessors list
- Encryption in transit/at rest
- Incident notification SLA
- Pen-test / SOC reports
- Region pinning
- Termination & deletion mechanics

Score before enablement; re-review on major version changes.

L2. Secure SDLC for prompts & tools

Treat prompts/Skills/tools as code:

- PR review including security
- Static checks for forbidden tool scopes
- Secrets scanning
- Mandatory eval job
- Change ticket linking for prod pins

L3. Red team cadence

Quarterly external or internal red team for tier-2+ systems; continuous automated probes in CI for smoke-level injections. Track findings to closure like vulns (CVE-like severity for AI issues optional but useful).

L4. Accessibility & inclusion

Responsible AI includes accessible UX: screen readers for AI notices, readable explanations, avoiding dark patterns that hide human escalation. Multilingual safety should not be English-only eval theater.

L5. Environmental / cost ethics (light touch)

Unbounded agent loops waste money and energy. Budgets are governance, not only FinOps.

L6. Deep scenario: “summarize these emails and send fixes”

Attack emails contain injection. Design: summarize in read-only mode; never send mail without human confirm; strip/ignore instruction-like lines in detectors; allowlist recipients; show diff of proposed sends.

L7. Deep scenario: model suggests disabling safety filters

Product must not expose toggles that disable enterprise-mandatory controls to end users. Admin break-glass with audit only.

L8. Documentation pack for auditors

Keep: data flow diagrams, DPIA/transfer assessments, model cards/system cards (internal), eval reports, access reviews, incident postmortems, vendor DPAs. CCAR-P won't grade your binder—but scenarios ask what evidence you'd produce.

L9. Extended Q&A 36–45

Q36. How do you brief an exec on injection risk in one sentence? **A.** “Ordinary documents can carry hidden instructions that trick the assistant into abusing its permissions.”

Q37. Name four vendor questions before enabling SaaS MCP. **A.** Retention/training, subprocessors, regions, incident SLA (plus deletion, encryption, audit reports).

Q38. Why treat prompts as code? **A.** They change behavior like code—need review, versioning, testing, and controlled release.

Q39. English-only safety evals miss what? **A.** Multilingual jailbreaks and toxicities; non-English user harm.

- Q40.** What is break-glass for safety toggles? **A.** Time-limited admin override with mandatory audit—not a user-facing ‘disable safety’ switch.
- Q41.** Why are budgets a governance control? **A.** They bound autonomous waste, abuse, and runaway tool loops.
- Q42.** What evidence might an auditor request for an AI feature? **A.** Data flows, DPIA, eval gates, access reviews, DPAs, incident records.
- Q43.** In the malicious email scenario, what’s the safest send policy? **A.** Human confirm + recipient allowlist + visible diff; no autonomous send.
- Q44.** How often red team tier-2+ systems (example cadence)? **A.** At least quarterly deep tests plus continuous automated smoke probes.
- Q45.** Secret scanning should cover which AI artifacts? **A.** Prompts, Skills, MCP configs, notebooks, CI logs, and example traces in repos.

L10. Extra checklist

- ☐ I can translate technical AI risk to business impact
- ☐ I can run a vendor MCP questionnaire
- ☐ I can describe secure SDLC for prompts/tools
- ☐ I can outline auditor evidence for an AI system
- ☐ I can design read-only summarize vs send flows safely
- ☐ I can argue against user-facing safety kill switches

L11. Glossary additions

Term	Meaning
System card	Internal/public description of system behavior & limits
DPA	Data Processing Agreement
Break-glass	Emergency privileged access with audit
Red team cadence	Scheduled adversarial testing rhythm
FinOps	Cloud/AI cost operational discipline
ATS	Applicant tracking system
SIEM	Security information and event management

Term	Meaning
Dark pattern	UX that manipulates users against their interest
Transfer assessment	Review of cross-border data transfers
Smoke probe	Lightweight continuous safety test

Part M — Mapping controls to CCAR-P verbs

Exam stems often use verbs: *implement, identify, ensure, design, recommend*. Pair them:

Verb	Strong answer ingredient
Implement	Name concrete mechanism (allowlist, HITL queue, DLP)
Identify	Name risk + where in architecture it arises
Ensure	Control + measurement/monitoring
Design	Diagrammed layers + owners
Recommend	Choose among options with trade-offs, not laundry list

Avoid vague “follow best practices” selections when a specific control is offered.

M1. Final rapid-fire Q&A 46–50

Q46. Best single control to add when giving an agent `delete_instance`?

A. Remove it or require multi-party approval—least privilege first.

Q47. What’s wrong with storing raw chat containing SSNs forever?

A. Retention/minimization failure; breach amplification; rights request difficulty.

Q48. How do you “ensure” tenancy safety? **A.** Hard authZ filters + automated cross-tenant retrieval tests in CI + monitoring.

Q49. Customer asks to turn off logging for “privacy.” Risk? **A.** May conflict with security/audit duties; seek legal/security design of redaction vs disable.

Q50. Which is better evidence of safety: marketing claims or eval reports?

A. Versioned eval/red-team reports with metrics and owners.

Part N — Safety exam drill

Stems often combine “automate” + regulated data. Default instinct: **narrow scope, HITL, server-side authZ, eval gates**. If an option only strengthens the system prompt, it is usually incomplete.

N1. Abuse monitoring signals

- Spike in refusal bypass attempts
- Unusual tool fanout per user
- Destination novelty for email/transfer tools
- Cross-tenant retrieval denials surge (or worse, successes)
- Sudden cost per user

N2. Model card vs system card

Model cards describe base models; **system cards** describe *your* application’s behavior, evaluations, and limits. Architects own system-level documentation.

N3. Q&A 51–56

Q51. If one option is “stronger system prompt” and another adds tool confirm + allowlist, which usually wins for money movement? **A.** Confirm + allowlist (hard controls).

Q52. What is a system card in enterprise practice? **A.** Documentation of the deployed AI system’s behavior, evals, and limitations.

Q53. Name three abuse-monitoring signals. **A.** Bypass attempts, tool fanout spikes, novel transfer destinations (etc.).

Q54. Cross-tenant retrieval *successes* in logs mean? **A.** Critical isolation failure—incident response immediately.

Q55. Why separate model cards from system cards? **A.** Provider model properties ≠ your tool privileges, data, and UX risks.

Q56. Default design instinct for regulated automation stems? **A.** Narrow scope + HITL + server authZ + eval gates.

Next: 04-stakeholder-lifecycle-gtm.md. Enablement details in 05.

Part O — Rapid review (Governance 14%)

Governance questions reward **layered controls** and **humility about prompts**. Rapid list:

- Threat-model injection whenever untrusted text meets privileged tools.
- HITL for irreversible or regulated impacts; no rubber stamps.
- Privacy: minimization, retention, erasure including embeddings.
- Tier use cases; match controls to tiers.
- Vendors/MCP are supply chain.
- Safety evals gate releases with task quality.
- Incidents need AI-specific severities and evidence trails.
- System cards document *your* app, not just the base model.
- Translate policy → engineering mechanism.
- Never claim prompt-only compliance.

Worked memory hook: **Hospital writeback → always clinician HITL; DevOps agent → read-only first; Marketing claims → fact checker; Email send → confirm + allowlist.**

Part P — Control catalog (printable)

Risk	Preventive control	Detective control	Corrective control
Indirect injection	Privilege split; allowlists	Injection probes; odd tool patterns	Disable tool; revoke sessions
Hallucinated advice	Citations; refuse-on-empty	Faithfulness evals	HITL correction; user notice
PII leakage	Minimization; DLP	Log scanners	Rotate; notify; purge
Over-autonomy	Budgets; confirmations	Turn/cost alerts	Flag off; reduce scope
Cross-tenant bleed	Hard filters; separate indexes	Canary tenant tests	Incident IR; patch filters
Malicious MCP	Vendor review; marketplace only	Version pin monitors	Kill switch; revoke tokens

Risk	Preventive control	Detective control	Corrective control
Insecure output handling	Sandbox; no raw exec	Runtime detections	Patch renderer; revoke
Bias in decisions	Human accountability; slice tests	Disparate error dashboards	Adjust product; stop auto

Use this catalog when a stem asks what to *implement*, *monitor*, or *do after* an incident—match the column to the verb.

Closing thought

Responsible AI for architects is operational: privileges, proofs, people-in-the-loop, and paperwork that maps to real systems. If your answer cannot name a mechanism, a metric, and an owner, keep designing.

Remember: mechanism + metric + owner. Prompt text is never the whole control story for CCAR-P governance items—pair it with authZ, HITL, monitoring, and evidence. *(File continues — deep-dive Part Q follows below.)*

One more worked stem

Stem flavor: “Auto-approve invoice payments under \$500 via Claude agent with ERP write tool.” Strong answer direction: lower threshold further or require dual control; enforce payee allowlist; server-side authZ; anomaly detection on novel payees; HITL above risk score—not prompt promising thrift alone.

Part Q — Primary-study deep dive: Governance architecture

Q1. Defense-in-depth reference blueprint

- [User/Channel]
- AuthN (SSO) + AuthZ (role → agent)
 - Input filters (injection heuristics, DLP, malware)
 - Orchestrator policy (allowed tools, budgets)
 - Claude (system policy + Skills)

- Tool gateway (allowlist, arg validation, server authZ)
- Output filters (PII, toxicity, brand, secrets)
- HITL queues (by risk tier)
- Audit + abuse monitoring + IR

Every exam scenario that proposes “just add a stronger system prompt” is incomplete unless other layers are considered.

Q2. Risk-tier → control baseline (copy/adapt)

Tier	Example	Auton- omy	Logging	HITL	Eval bar
T0 Informa- tional	Public FAQ para- phrase	High	Basic	Spot check	Light
T1 Assistive draft	Email draft to human	High draft / low send	Full prompts redacted	Human sends	Medium
T2 Bounded action	Refund ≤ \$50	Auto within caps	Full + tool I/O	Sampled + anoma- lies	High
T3 High impact	Medical, legal, large money, prod changes	Default deny auto	Tamper- evident	Manda- tory	Highest + red team

Q3. Guardrail trade-off table

Control	Catches	Misses / cost	Overuse failure
System prompt policy	Soft behavioral norms	Determined injection	False sense of security
Input classifier	Many jailbreaks/PII	Novel paraphrases; latency	Blocks legitimate work
Output DLP	Secrets/PII egress	Encoded exfil	Over-redaction UX

Control	Catches	Misses / cost	Overuse failure
Tool allowlist	Over-privilege	Logic bugs inside allowed tools	Shadow IT bypass
Server authZ	Confused deputy	Misconfigured ACLs	— (do not skip)
HITL	High-impact errors	Slow; reviewer fatigue	Rubber-stamping
Monitoring	Drift/abuse over time	Needs analysts	Alert fatigue

Q4. HITL design patterns

1. **Approve/reject gate** before side effect.
2. **Edit-then-send** (copilot).
3. **Review sampling** (X% or uncertainty-triggered).
4. **Four-eyes** for dual control (finance).
5. **Async specialist queue** (legal/medical).

Anti-patterns: HITL on every trivial FAQ (kills ROI); HITL with no SLA (queue death); HITL without UI showing sources/tool args (blind approval).

Q5. Prompt injection failure-mode analysis

Variant	Channel	Example	Primary mitigations
Direct	User chat	“Ignore previous...”	Policy + classifiers + refuse
Indirect	Retrieved doc/email/web	Hidden instruction in PDF	Treat content as data; cite; no raw tool trust
Tool-result injection	MCP/API payloads	Error message contains instructions	Schema validate; sanitize; allowlist
Cross-user	Shared memory/stores	Poisoned shared Skill	Tenancy; review; signing
Multimodal	Image/OCR text	Text in screenshot	Same as indirect

Never rely on: “The model is aligned so we’re fine.” Alignment reduces risk; application controls remain mandatory.

Q6. Compliance overlay mapping (scenario language, not legal advice)

Constraint cue in stem	Architecture implications
GDPR / personal data	Minimization, retention, DSR erasure including indexes/logs, DPIA-minded design, transfer review
HIPAA / health	BAA/vendor posture, PHI minimization, access controls, audit, careful on third-party tools/web
FedRAMP / gov	Authorized deployment path, strict boundaries, logging, change control
PCI hints	Never put PAN into prompts; tokenize; segment
Children’s data	Heightened restrictions; often avoid training/retention; parental/policy gates

Exam answers should **name controls**, not recite statute numbers.

Q7. Fairness & transparency practical checklist

- ☐ Who is affected if the model is wrong?
- ☐ Disparate error rates checked across relevant groups where lawful/appropriate?
- ☐ User told they interact with AI?
- ☐ Limitations and escalation path documented?
- ☐ Appeals/override for consequential decisions?
- ☐ Training/eval data governance noted?

Q8. Incident response extras for LLM apps

Add to classic IR: - Prompt/tool trace preservation (legal hold) - Ability to disable tools/models/features via flag - Injection indicator hunting -

Part R — Worked decision scenarios (Safety)

R1. “Auto-send email fixes from mailbox summary”

Reject auto-send. Summarize + draft only; human sends; treat email bodies as untrusted (indirect injection); DLP on drafts; audit.

R2. DevOps agent with cloud credentials

Read-only recon tools first; writes via change tickets + HITL; short-lived creds; no org-admin keys in MCP; network egress allowlists; break-glass separate.

R3. HR chatbot asked for medical leave legal advice

Scope to benefits docs via RAG + cite; refuse personalized legal/medical advice; escalate to HR human; log sensitive topics carefully.

R4. Marketing generator producing competitor-disparaging claims

Brand/policy output filters; require sources for factual claims; human publish gate; eval for prohibited topics.

R5. Stakeholder asks to “turn off safety filters for speed”

Architect response: safety filters are risk controls tied to tier; propose measured latency opts (caching, routing) instead; document residual risk if any control is narrowed with approval.

Part S — Production safety checklist

S1. Design review

- ☐ Threat model includes injection, leakage, over-autonomy, abuse
- ☐ Risk tier assigned with control baseline
- ☐ Tool allowlist + server authZ proven with negative tests
- ☐ HITL triggers and SLAs defined

- ☐ Data classes allowed in prompts documented
- ☐ Retention for logs/evals lawful and minimized
- ☐ Abuse monitoring signals listed
- ☐ Red-team plan scheduled pre-GA

S2. Pre-GA safety gate

- ☐ Injection suite pass rate meets bar
- ☐ DLP false-positive UX accepted by product
- ☐ High-impact paths cannot execute without HITL/policy engine
- ☐ Vendor/MCP questionnaire complete
- ☐ Incident runbook rehearsed
- ☐ Transparency/notice copy approved

S3. Steady-state

- ☐ Quarterly access review for agent credentials
- ☐ Eval drift review includes safety metrics
- ☐ Plugin/MCP updates reviewed like dependencies
- ☐ Post-incident learnings → control updates

Part T — Trade-offs: autonomy vs control

Push for more autonomy when...	Keep humans when...
Actions reversible & low blast radius	Irreversible money/health/safety/legal
Strong evals + monitoring	Sparse data / novel intents
Clear policy engines encode rules	Rules require judgment calls
Exception rate within staffing	Reviewers already overloaded

Part U — Extended Q&A (57–65)

Q57. Strongest control against unauthorized data access via tools? **A.** **Server-side authorization** on the data plane using verified identity.

Q58. Indirect injection best single description? **A.** Malicious instructions embedded in **content the model reads** (docs, web, emails), not only in the user chat box.

Q59. Select TWO for Tier-3 payout agent: mandatory HITL, open web server tool for all users, full audit traces, temperature=1.5 creative mode.
A. Mandatory HITL and full audit traces.

Q60. Constitutional AI means your app is compliant with HIPAA? **A. No**—base alignment ≠ application compliance; you still design controls and vendor posture.

Q61. Reviewer rubber-stamping—what to change? **A.** Reduce volume (better auto for T0/T1), improve UI with sources/tool args, rotate reviewers, sample quality audits.

Q62. GDPR erasure request—what systems to touch? **A.** Primary DB **and** vector indexes, caches, eval sets, logs (per policy), backups per legal schedule.

Q63. Best response if product wants model to execute shell from output? **A. Sandbox / deny by default**; never pipe unsandboxed LLM text to privileged shells.

Q64. Dual-use request for detailed biological harm—in product policy terms? **A.** Refuse / safe completion per acceptable use; escalate abuse patterns; do not provide actionable harmful detail.

Q65. Select THREE layers in defense in depth: tool allowlist, output DLP, server authZ, “trust the vendor alone”, disable all logging for privacy theater. **A.** Allowlist, output DLP, server authZ.

Part V — Rapid review (Governance 14%)

- Design controls, don’t slogan.
- Defense in depth; prompts are one layer.
- Injection: direct + indirect + tool results.
- HITL by impact tier; avoid rubber stamps.
- AuthZ in code; least privilege tools.
- Compliance = controls + evidence, not vibes.
- Fairness, notices, appeals for consequential AI.
- IR needs traces, kill switches, clear owners.

Pair with Integration authZ in 02 and lifecycle risk talk-tracks in 04.

Part W — Abuse monitoring, model cards, and exam stems

W1. Abuse monitoring signal examples

- Sudden spike in tool calls per user
- Repeated jailbreak phrasings
- DLP hit clusters
- Anomalous exfil-sized outputs
- New MCP server reaching odd egress

Wire to SOC/IR playbooks; don't collect sensitive content beyond policy.

W2. Model card vs system card (architect speak)

Model card (provider): capabilities, limitations, eval highlights of the base model. **System card / system documentation** (you): how *your* application uses the model—data flows, controls, residual risks. Exam cares that you know **application** accountability remains yours.

W3. Quick stems

Q66. Product removes output filters to “sound more helpful.” Risk? **A.** Increased leakage/policy violations; violates defense in depth; need risk acceptance formally—not silent removal.

Q67. Best evidence for auditors that HITL works? **A.** Ticketed approvals with reviewer identity, timestamps, and linked model/tool traces—not a policy PDF alone.

Q68. FedRAMP-like constraint in stem—what changes first? **A.** Deployment/boundary choices and logging/change control—not only a nicer prompt.

File 03 primary-study target band: 6000–9000 words.

04 — Stakeholder Engagement, Lifecycle & GTM-Adjacent Communication — Simplified Technical English

CCAR-P condensed domain 4 Official domain mapped here: Stakeholder Communication & Lifecycle Management (14%) **Note:** Official **Developer Productivity & Operational Enablement (7%)** is in the dedicated file **05-team-enablement-operational-productivity.md**.

Approx. questions from this domain: about 9 of 63 (plus overlap with enablement scenarios)

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names Claude, MCP, ADR, GTM, GA stay as written. These notes are original study notes. They are not official sales scripts. They are not affiliated with Anthropic. They are not legal advice or sales advice.

Disclaimer

These notes cover discovery, proposal framing, lifecycle and operations handoff, and value communication. This file does **not** copy official sales scripts or internal playbooks. It teaches durable architect skills: stakeholder discovery, expectation management, ADR communication, and lifecycle ownership. These notes are not affiliated with Anthropic. They are not legal advice. They are not sales advice.

Overview

More than one third of CCAR-P weight sits in communication domains when you combine Governance + Stakeholders + Enablement. Candidates who study only RAG/MCP lose exam points. This file trains you to:

1. Run structured discovery across business, technical, legal, security, and operations stakeholders
2. Translate architecture into narratives for each audience
3. Frame proposals with trade-offs, risks, and success metrics
4. Own the lifecycle from pilot → GA → improve → retire
5. Communicate value without confidential partner GTM machinery

For hands-on team rollout of Claude Code, CLAUDE.md, hooks, and training curricula, use **file 05**.

Key map

Task	Official mapping	Good exam answer traits
Discovery interviews	Stakeholder 14%	You get constraints, success metrics, and risks
Explain trade-offs	Stakeholder 14%	Non-technical clarity plus technical precision
Document architecture	Stakeholder 14%	ADRs, diagrams, and runbook pointers
Manage expectations	Stakeholder 14%	Phased autonomy and known limitations
Lifecycle handoff	Stakeholder 14%	Owners, SLOs, and feedback loops
Value communication	Stakeholder 14% (GTM-adjacent)	Outcomes and metrics. Do not use hype
Builder enablement	→ File 05 (7%)	See the dedicated domain file

Part A — Stakeholder discovery

A1. Stakeholder map

Role	What they care about	Questions to ask
Executive sponsor	Outcomes, risk, timeline, ROI	What decision will this change? What does success look like in 90 days?
Business owner	Process KPIs, UX, exceptions	Where do humans spend time? What errors hurt customers?
End users	Trust, speed, control	When would you reject AI help? What must never auto-send?

Role	What they care about	Questions to ask
Engineering	APIs, SLOs, maintainability	What systems are the source of truth? What are the rate limits?
Data	Quality, tenancy, pipelines	What are the freshness SLOs? What are the PII classes?
Security	Threats, identity, logging	What is the authorization boundary?
Legal/Privacy	Basis, notices, retention	Is there regulated data? What are the transfer limits?
Compliance/Risk Ops/SRE	Controls evidence Pages, runbooks, capacity	What audits apply? Who is on-call on day 1?
Finance	Unit economics	What is the cost ceiling per transaction?
Sales/CS (GTM-adjacent)	Promise vs delivery	What has already been committed externally?

A2. Discovery workshop agenda (90 minutes)

1. Problem framing (15) — jobs to be done, quantified pain
2. Constraints wall (15) — data, latency, compliance, budget
3. Success metrics (15) — leading and lagging KPIs
4. Journey mapping (20) — human+AI swimlane
5. Risk storming (15) — injection, hallucination, over-automation
6. Decision log (10) — open questions and owners

Output: a one-page brief that feeds architecture.

A3. Requirements taxonomy

- **Functional:** intents, tools, languages
- **Non-functional:** latency, availability, cost, residency
- **Safety:** disallowed actions, HITL triggers
- **Operability:** logging, ownership, support hours

- **Change:** model upgrade policy

Record **non-goals** in explicit form (example: “not a fully autonomous negotiator in v1”).

A4. Anti-patterns in discovery

- You talk only to the sponsor
- You accept “ChatGPT but on our data” as a requirement
- You skip security until late
- You ignore promises that GTM motions already sold
- You have no quantitative baseline (you cannot prove value later)

Part B — Communicating architecture

B1. Altitude switching

Audience	Artifacts	Language
Exec	One-slide value + risk + ask	Outcomes, timeline, investment
Business	Journey + exception paths	Before/after KPIs
Engineers	Sequence diagrams, schemas, ADRs	Interfaces, failure modes
Security	Data flow + threat model	Trust boundaries, controls
Legal	Processing description	Purposes, retention, rights

The system stays the same. You show different views. Exam traps: you give executives too much token-graph detail. You give engineers only vague security statements.

B2. Architecture Decision Records (ADRs)

Template:

- Context
- Decision

- Alternatives considered
- Consequences (positive/negative)
- Status & date
- Related eval evidence

Example decision: “Workflow over agent for quote validation because auditability > flexibility.”

B3. Trade-off narratives

Use **option A/B/C** tables:

Option	Quality	Latency	Cost	Risk	Notes
A Monolith prompt	Med	Best	Low	Audit weak	Reject for compli- ance
B Workflow	High	Med	Med	Low	Recom- mended
C Full agent	High?	Worst	High	Higher	Keep for phase 2

Recommend **one** option. Do not show the table without a selection.

B4. Expectation management scripts (original, non-confidential)

- “Phase 1 suggests. Humans send. Phase 2 auto-sends only for template class T with metrics M.”
- “Model will be wrong ~X% on slice S. We route those to experts.”
- “We will not promise identical wording—only process SLAs and quality rates.”

Honesty builds trust. Overclaiming creates disappointment at incident scale.

Part C — Proposal framing (GTM-adjacent, partner limits flagged)

C1. What you *can* write originally

Architects who support pre-sales or internal funding structure proposals around:

- 1. **Problem & baseline metrics**
- 2. **Target outcomes** (efficiency, quality, risk reduction)
- 3. **Solution sketch** (pattern, integrations, controls)
- 4. **Phased roadmap** (pilot → GA)
- 5. **Investment** (people, infra, licenses—at high level)
- 6. **Risks & mitigations**
- 7. **Success review date**

C2. What you should *not* invent

- Confidential partner discount schedules
- Internal vendor battle cards
- Guaranteed win rates or unpublished benchmarks as marketing claims
- Customer logos and case details that you do not have rights to use

Note: Employer-provided playbooks can add firm-specific motions. This pack stays skill-oriented.

C3. Value communication patterns

Value story	Evidence to attach
Time saved	Time-motion baseline vs pilot
Quality lift	Error rate / edit distance
Risk reduction	Fewer policy breaches. Audit readiness
Revenue assist	Conversion lift with experiment design
Developer velocity	See file 05 metrics

Avoid “AI unspecified mechanism” slides. Prefer “instrumented learning.”

C4. Handling oversold expectations

If GTM already promised full autonomy:

- 1. Acknowledge the gap. Do not run a public blame session
- 2. Show the risk of the current promise
- 3. Propose phased delivery that still lands value early
- 4. Align the sponsor on rewritten success criteria in writing

Architects protect long-term trust.

Part D — Lifecycle management

D1. Lifecycle stages

Discover → Design → Build → Pilot → GA → Operate → Improve → Retire

Each stage needs **exit criteria**.

Stage	Exit criteria examples
Discover	Brief + metrics + risk tier signed
Design	ADR + threat model + eval plan
Build	Tests green. Tools contract-tested
Pilot	N users. Metrics vs baseline. Safety probes pass
GA	On-call live. Docs. Feature flags. SLOs
Operate	Weekly quality review. Cost within envelope
Improve	Prioritized backlog from feedback/evals
Retire	Traffic drained. Data deleted per policy. Postmortem

D2. Handoff package (build → ops)

- Architecture overview + diagrams
- Model/prompt/index pins
- Runbooks (latency, bad outputs, injection suspicion, cost spike)
- Dashboards & alert thresholds
- Known failure modes
- Support escalation matrix
- Change process for prompts/tools

File 05 deepens developer and operations enablement content.

D3. Feedback loops

Sources: user thumbs, edit distance, tickets, sales objections, security findings, eval regressions. Triage into: prompt fix, retrieval fix, product UX, training, refuse-to-automate.

D4. Operating cadence

- Daily: error/cost/safety alerts
- Weekly: quality slice review
- Monthly: roadmap & risk tier revisit
- Quarterly: red team / access review / vendor review

D5. Deprecation & model end-of-life

Provider model deprecations are lifecycle events. Maintain:

- Inventory of pins
 - Migration owners
 - Compat eval suite
 - Comms plan to stakeholders
-

Part E — Change & conflict management

E1. Common conflicts

Conflict	Resolution approach
Speed vs safety	Phased autonomy + metrics gates
Cost vs quality	Routing & caching before quality cuts
Sales promise vs eng reality	Written scope reset with sponsor
Central platform vs domain teams	Clear paved road + escape hatches
Privacy vs analytics	Aggregation/redaction design

E2. RACI for lifecycle (example)

Activity	Sponsor	Architect	Eng	Security	Ops	GTM/CS
Success metrics	A	R	C	I	I	C
Threat model	I	R	C	A	C	I
GA go/no-go	A	R	R	A	A	C
Customer messaging	C	C	I	C	I	R/A

Part F — Documentation standards

Minimum viable docs:

1. README for service
2. ADR log
3. Data flow (happy + abuse)
4. Eval report template
5. User-facing limitations page

6. On-call runbooks

Prefer living docs near code. Do not use slide decks as the only source of truth.

Decision trees

Audience pitch picker

Is audience accountable for budget/risk?

YES → Lead with outcomes, risk, ask, timeline

Is audience implementing?

YES → Lead with interfaces, failure modes, ADRs

Is audience signing legal?

YES → Lead with data flows, retention, subprocessors

Pilot vs GA gate

Safety probes + tenancy tests green?

Owners & on-call named?

SLOs & dashboards live?

Support macros ready?

ALL YES → GA candidate

ELSE → Extend pilot with explicit gaps list

Exam traps

1. You jump to tech without discovery metrics.
 2. You use one communication style for all stakeholders.
 3. You have no non-goals / infinite scope.
 4. You go to GA without an operations handoff.
 5. You invent partner-confidential GTM claims.
 6. You ignore customer promises that are already committed.
 7. You track only vanity metrics (page views of a chatbot).
 8. You have no retirement/EOL plan for models.
 9. You skip legal/security until after build.
 10. You say “we will monitor” without named owners and cadence.
-

Practice Q&A (26)

Q1. Name five stakeholder groups for an enterprise Claude app. **A.** Examples: sponsor, business owner, engineering, security, and legal. Also include operations, data, finance, and users.

Q2. What belongs in discovery non-goals? **A.** Capabilities that are explicitly out of scope for the phase (example: no autonomous refunds in v1).

Q3. How should you present options to an exec? **A.** Use a short outcome/risk/ask. You can add one trade-off table with a clear recommendation.

Q4. What is an ADR consequence section for? **A.** You record follow-on costs and risks of the decision for future readers.

Q5. Pilot exit criteria example? **A.** Target users complete tasks with metric lift, and safety probes pass.

Q6. What do you do if sales promised full autonomy unsafely? **A.** Realign the sponsor on phased delivery. Rewrite success criteria in writing.

Q7. Why capture baseline metrics before build? **A.** Without a baseline you cannot prove value or defend ROI.

Q8. Name three handoff artifacts to ops. **A.** Runbooks, dashboards/alerts, and pin inventory (also escalation matrix and known failures).

Q9. Vanity metric example to avoid? **A.** Raw chatbot message count without task success or CSAT/quality linkage.

Q10. Who is typically accountable for GA go/no-go safety signoff? **A.** Security (and the sponsor overall)—per RACI. The architect helps with evidence.

Q11. What is altitude switching? **A.** You adjust artifact depth and language to the audience. You do not change the facts.

Q12. Why schedule model EOL as a lifecycle event? **A.** Deprecations break pins. You need migration owners and evals.

Q13. GTM-adjacent content you should not invent in study claims? **A.** Confidential partner discounts, battle cards, and unpublished guarantee scripts.

Q14. What makes a good success review meeting? **A.** Pre-agreed metrics, open risks, and a decision to scale, pivot, or stop.

- Q15.** How do feedback loops connect to engineering? **A.** You triage into prompt, retrieval, UX, training, or refuse-to-automate backlog items.
- Q16.** Conflict speed vs safety—architect move? **A.** Phase autonomy. Ship suggest-mode value early. Gate auto-send on metrics.
- Q17.** What is a limitations page for? **A.** Transparent user and admin expectations. This reduces misuse and support load.
- Q18.** Why include CS/Sales in discovery lightly? **A.** You learn external commitments and objection patterns early.
- Q19.** Minimum legal-oriented artifact? **A.** A processing and data-flow description with purposes and retention.
- Q20.** What does “instrumented learning” mean in value stories? **A.** Claims backed by measured pilot evidence, not slogans.
- Q21.** Who owns weekly quality review? **A.** A named role (often architect + engineering + business). The owner must be explicit.
- Q22.** Escape hatch in platform paved road? **A.** A documented way for teams to deviate with extra review. Silent forks are not allowed.
- Q23.** Why write non-functional requirements early? **A.** Latency, residency, and cost reshape architecture. Late discovery causes rework.
- Q24.** Retire stage must include what data step? **A.** Drain traffic and delete or retain data per policy (indexes and logs included).
- Q25.** Best response to “just make it like ChatGPT”? **A.** Reframe into jobs-to-be-done, constraints, and measurable outcomes.
- Q26.** Where are Claude Code rollout details? **A.** File 05–team–enablement–operational–productivity.md.
-

Checklist

- ☐ I can run a 90-minute discovery workshop structure
- ☐ I can switch altitude across exec/eng/security/legal
- ☐ I can write an ADR with alternatives and consequences
- ☐ I can frame a phased proposal without partner-confidential extra claims
- ☐ I can list lifecycle exit criteria including retire
- ☐ I can build an ops handoff package outline

- ☐ I can reset oversold autonomy expectations in a constructive way
 - ☐ I can define operating cadences and owners
 - ☐ I know GTM partner-content limits for this pack
 - ☐ I know enablement deep content is in file 05
-

Glossary

Term	Meaning
Discovery	Structured collection of requirements and constraints
ADR	Architecture Decision Record
Non-goals	Explicit out-of-scope items
Baseline	Pre-AI metric for comparison
Pilot	Limited release to learn under controls
GA	General availability
Handoff package	Artifacts transferring build → operate
Paved road	Supported standard platform path
Escape hatch	Approved deviation process
Value story	Outcome narrative backed by evidence
RACI	Responsibility assignment matrix
EOL	End of life (e.g., model deprecation)
GTM	Go-to-market
Expectation management	Aligning promises to feasible delivery
Instrumented learning	Measurement-driven iteration

Part G — Meeting patterns & facilitation

G1. Working backward from the press release (internal)

Draft a one-paragraph internal “launch note” early: who benefits, what changes, and what will not change. If stakeholders cannot agree on that paragraph, architecture work will change without control.

G2. Decision hygiene

- Single decision owner
- Deadline
- Inputs required

- Reversible vs irreversible classification
- Document in ADR within 48 hours

G3. Stakeholder resistance types

Resistance	Underlying need	Move
Fear of job loss	Role clarity	Redesign the job to oversight and exception handling
Fear of liability	Accountability	HITL + audit + clear owners
Skepticism from past AI hype	Evidence	Small pilot with strict metrics
Security as a late veto	Control	Invite security as co-designer early
Shadow IT AI use	Speed	Paved road that is easier than shadow tools

G4. Comms plan for incidents involving AI

Prewrite: who speaks, what you disclose, how you pause the feature flag, and how you update customers. Pair this with file 03 IR playbooks.

G5. Training stakeholders (light)

Business users need “how to ask / how to verify / when to escalate.” They do not need model theory. Deep builder training is in file 05.

G6. Extended scenarios

Scenario: Procurement wants AI to auto-email vendors and negotiate price. **Stakeholder move:** Convene legal, security, and procurement. Propose suggest-mode with approval. Define a savings metric. Refuse unbounded autonomy in v1.

Scenario: Board asks for an “AI strategy KPI.” **Move:** Use a portfolio view. Percent of use cases with eval gates, incident count, value realized vs target, and risk tier coverage. Do not use a single vanity chatbot score.

G7. Q&A 27–35

Q27. Why draft an internal launch paragraph early? **A.** It forces agreement on who benefits and what will not change, before build work changes without control.

Q28. Reversible vs irreversible decision—example of irreversible? **A.** Autonomously emailing legally binding commitments. Migrating production data without rollback.

Q29. How do you respond to security-as-late-gate? **A.** Bring security into discovery and design. Share threat model drafts early.

Q30. Shadow IT AI—platform response? **A.** Make the paved road faster and safer than unsanctioned tools. Educate. Monitor.

Q31. Board-level AI KPI examples? **A.** Eval-gated use-case percent, realized value vs target, AI incident rate, and risk-tier coverage.

Q32. What training do business users need first? **A.** Ask, verify, and escalate behaviors, plus limitations. Not transformer internals.

Q33. Who speaks in a public AI content incident? **A.** A pre-assigned comms owner with legal and security input. Not an ad-hoc engineer tweet.

Q34. Why classify decisions by reversibility? **A.** Irreversible ones need more evidence, HITL, and higher altitude approval.

Q35. Procurement auto-negotiate email—first workshop output? **A.** A written non-goal of autonomy, plus an approval workflow, plus a savings metric.

Part H — Narrative kits (original, reusable)

H1. Executive one-pager skeleton

1. **Problem:** quantified pain
2. **Proposal:** pattern in one sentence
3. **Value:** 2–3 metrics with targets
4. **Risk:** top 3 with mitigations
5. **Ask:** funding/people/decision
6. **Checkpoint:** date for go/no-go

H2. Security review talking points

- Data classes in prompts/logs/indexes
- Tool privilege diagram
- Injection storyboard
- Eval gates for safety
- Incident severity mapping
- Vendor/MCP list

H3. Customer success enablement (non-confidential)

CS needs: limitation cards, escalation matrix, “how to verify answers,” and when to pull engineers. Do not invent partner discount motions.

H4. Roadmap communication

Show **capability unlocks** tied to controls: “Citations+eval gate unlocks external FAQ”. “HITL refunds unlock finance pilot.” Avoid a roadmap of model brand names alone.

H5. Q&A 36–42

Q36. What are the six parts of an exec one-pager? **A.** Problem, proposal, value, risk, ask, checkpoint.

Q37. Why tie roadmap unlocks to controls? **A.** This makes safety and eval investment visible as value enablers, not blockers.

Q38. What does CS need besides feature lists? **A.** Limitation cards, verification guidance, and escalation paths.

Q39. Injection storyboard purpose in security reviews? **A.** Make indirect injection concrete with a step-by-step abuse path.

Q40. What should not appear in this pack’s GTM claims? **A.** Confidential partner playbooks, unpublished guarantees, and unauthorized case studies.

Q41. Capability unlock example? **A.** External FAQ only after the citation precision gate passes.

Q42. Why include a checkpoint date in the ask? **A.** It forces decision hygiene and prevents a pilot that never ends.

Part I — Lifecycle artifacts library (expand)

I1. Pilot charter fields

- Hypothesis
- Population & exclusions
- Duration
- Metrics & thresholds to scale
- Safety stop conditions
- Roles (sponsor, DRI, security contact)
- Communication plan

I2. GA readiness scorecard (example weights)

Area	Weight	Evidence
Quality evals	25%	Report $\geq$ thresholds
Safety probes	25%	Red-team closed or accepted
Ops readiness	20%	Runbooks + on-call drill
Privacy/legal	15%	Signoffs
Support enablement	10%	Macros + training
Cost envelope	5%	Forecast vs budget

Fail any critical safety or privacy item → no GA, regardless of average.

I3. Stakeholder update email (original template)

Subject: [Pilot] Week N outcomes Body: metric table vs baseline. Top incidents. Decision needed. Next experiment. Keep the email to one screen.

I4. Workshop facilitation tips

- Parking lot for out-of-scope items
- Timebox architects who explain transformers
- Capture verbatim user “never auto-...” quotes—they become HITL requirements
- End with owners and dates, not vague feelings

I5. Cross-cultural / multi-region stakeholders

Clarify residency early. Use local champions. Use language eval slices. Do not assume US-only legal frames for global rollouts.

I6. Q&A 43–50

Q43. Name four pilot charter fields. **A.** Hypothesis, metrics/thresholds, safety stops, roles (and related fields).

Q44. Can high quality scores override failed safety probes for GA? **A.** No—critical safety or privacy fails block GA.

Q45. What belongs in a weekly pilot email? **A.** Metrics vs baseline, incidents, decision ask, and next experiment.

Q46. Why capture “never auto” user quotes? **A.** They translate directly into HITL and product rules.

Q47. Multi-region discovery must ask what early? **A.** Data residency and local regulatory constraints.

Q48. Parking lot technique prevents what? **A.** Scope explosion that derails the workshop agenda.

Q49. Who is DRI in a pilot charter? **A.** The directly responsible individual for day-to-day delivery.

Q50. Why include support enablement in GA scorecard? **A.** Without macros and training, production creates chaos and shadow process.

Enablement, Claude Code org rollout, runbooks-as-training → 05.

Part J — Extended lifecycle playbooks

J1. From discovery notes to backlog

Convert workshop outputs into epics:

- Epic: Retrieval tenancy hardening
- Epic: Citation UX
- Epic: HITL queue v1
- Epic: Eval harness CI
- Epic: On-call runbooks

Each epic links to a metric and a risk tier control. This prevents “random prompt tweaks” that pretend to be delivery.

J2. Steering committee pack

Monthly 30 minutes: KPI dashboard, top risks, decision requests, budget burn. Pre-read only. The meeting is for decisions. Architects who speak for 25 minutes do not pass the committee.

J3. Internal FAQ for skeptical leaders

Q: Will this replace my team? **A:** Design targets toil reduction and exception-centered roles. Staffing decisions belong to leadership. Architecture must make oversight workable.

Q: Can we trust answers? **A:** You measure trust with citation rates, eval slices, and HITL on critical paths. Do not assume trust.

Q: Why not ship the demo? **A:** Demos lack tenancy tests, on-call, and abuse cases. The GA scorecard exists for a reason.

J4. Partner-limit reminder box

This pack stops at universal architect skills: discovery, framing, lifecycle, and honest value communication. Do not treat anything here as official sales doctrine.

J5. Post-GA value review (day 60)

Return to baseline metrics. Publish a short “value realized” memo. Decide invest, maintain, or sunset. Without this, unused systems stay in production.

J6. Additional Q&A 51–58

Q51. How do workshop notes become engineering work? **A.** Epics linked to metrics and controls—not orphan prompt tweaks.

Q52. What is a steering pack for? **A.** Decision-making with pre-read KPIs and risks. Not architecture lectures.

Q53. Best answer to “ship the demo”? **A.** Demo is not GA. Cite missing gates on the scorecard.

Q54. Day-60 review purpose? **A.** Measure realized value. Invest, maintain, or sunset with intent.

Q55. How to answer replacement fears? **A.** Focus on toil and exceptions. Avoid speculative HR promises.

Q56. Why link epics to risk tier controls? **A.** This keeps safety work visible in the same backlog as features.

Q57. What is an abandoned AI app? **A.** A running system without owners, metrics, or value review.

Q58. Where is developer enablement detailed? **A.** File 05. Do not mix stakeholder answers with IDE settings unless the question asks.

Part K — Communication worksheets

K1. Constraint wall prompts (use in workshops)

- Hard deadline? What happens if we miss?
- Data that must never leave region X?
- Actions that always need a human name on them?
- Maximum acceptable wrong-answer rate for slice S?
- Support hours and language coverage?
- Existing contractual SLAs with customers?

K2. Trade-off sentence stems (practice saying these)

- “We recommend B because auditability outweighs flexibility given constraint C.”
- “We can hit latency L only if we route easy traffic to model M and reserve N for hard cases.”
- “Phase 1 will not auto-send. That unlocks after metric $X \geq T$ for two weeks.”
- “Cost target requires cache hit rate $\geq H$ and top-k $\leq K$.”

K3. Lifecycle RACI quiz yourself

Select a random activity—prompt change, index rebuild, customer apology, MCP add—and assign RACI. Do not look at a key first. Then check against your organization’s real names. Exam scenarios expect you to know *which function* is accountable even if names differ.

K4. End-card

Stakeholder domain = discovery quality + altitude switching + phased honesty + lifecycle ownership. GTM-adjacent work stays non-confidential:

value, risks, phases, metrics. Enablement mechanics live in file 05.

Part L — Full discovery questionnaire bank (condensed)

Business: What breaks if the AI is wrong? What is the cost of delay today? Which step is pure toil? **Users:** When would you ignore the suggestion? What must remain human-branded? **Eng:** Rate limits? Idempotency already in APIs? Sync vs async constraints? **Data:** Freshness? Label quality? Known poison documents? **Security:** Highest-value systems in scope? Existing DLP? **Legal:** Automated decisioning rules? Retention schedule? **Ops:** Who pages at 2am? What is the rollback procedure that people already know? **Finance:** Unit-cost ceiling? CapEx vs OpEx preferences? **GTM/CS:** What was already promised in writing? Top objections?

If you run even half of these, you prevent the common “solution looking for a problem” failure. CCAR-P scenarios punish that failure.

Part M — Handoff email template (original)

Subject: Handoff — to operations Attach: diagrams, ADR index, pin list, runbooks, dashboard links, known issues, escalation matrix. Body checklist: SLO summary. Feature flags. How to roll back model, prompt, and index. Where eval reports live. Next quality review date. DRI during hypercare (usually two weeks).

Hypercare means the build team stays in the support path while operations gains practice. Then the build team steps back per agreement. If you skip hypercare, lifecycle work fails without a clear signal.

Closing card for file 04

You now have discovery banks, altitude switching, proposal framing within partner-limit honesty, lifecycle gates, and handoff discipline. Pair this with file 05 when questions mention Claude Code, CLAUDE.md, hooks, or org-wide developer tooling.

One more worked stem

Stem example: “Sponsor wants GA next month after an impressive demo. Security has not reviewed. No on-call.” Strong answer: present GA scorecard gaps. Propose a dated pilot extension. Set security review

and a runbook drill as exit criteria. Reset external promises with the sponsor. Do not ship in silence.

Expectation-reset paragraph (memorize structure)

“We can deliver measurable value by date D in suggest-mode with controls C. Full autonomy remains a phase-2 unlock after metrics M hold for period P. If you ship autonomy now, you accept risk R without compensating controls. Please confirm the revised success criteria in writing so engineering, security, and GTM share one plan.”

That structure—value now, unlock later, risk named, written alignment—answers most “pressure to over-automate” stakeholder questions. You do not need partner-specific playbooks.

(File continues — deep-dive Part N follows below.)

Hypercare without a named DRI is not real. Name the person and the end date when you hand off.

Keep a living decision log. When stakeholders revisit settled choices, point to the ADR date and evidence. Do not restart the debate from memory.

Part N — Primary-study deep dive: Discovery that still works in real conditions

Scope note (read first): This pack teaches **original** discovery, lifecycle, change management, and value framing skills only. It does not invent pricing, confidential win themes, or official lesson text. If an exam question looks like a “sales motion,” answer with durable architect behaviors: clarify commitments, align SLAs, document trade-offs, and phase delivery.

N1. Discovery depth model (five passes)

1. **Problem pass** — jobs-to-be-done, quantified pain, current workaround.
2. **Constraint pass** — data, latency, residency, budget, unions/works councils, brand.
3. **Success pass** — leading and lagging metrics, decision rights, time box.

4. **Risk pass** — safety, compliance, operational blast radius, reputational.
5. **Change pass** — who loses power or status, training load, process redesign.

If you skip the change pass, technically complete pilots still fail.

N2. Stakeholder interview bank (condensed, original)

Executive: What decision improves if this works? What would make you stop the program? **Business owner:** Which step burns minutes? What error is unacceptable? **Users:** When do you distrust AI? What must stay manual? **Security:** What is the identity boundary? What is your nightmare ticket? **Legal/Privacy:** Data classes? Transfer? Retention? Notices? **Ops:** Who pages at 2am? What is the degrade mode? **Finance:** Fully loaded cost ceiling per transaction? **Sales/CS (GTM-adjacent):** What was already promised externally?

N3. Requirement quality bar

A requirement is exam-ready when it has: owner, metric, constraint, priority, and test idea. “Make it smart” is not a requirement. “p95 < 3s for FAQ path; HITL for refunds > \$50. GDPR erasure within policy SLO” is.

N4. Anti-patterns (discovery)

Anti-pattern	Why it fails	Fix
Single stakeholder proxy	Hidden vetoes later	Map power + impacted users
Solutioneering first	Misses real job	Problem framing before pattern
Ignoring shadow process	Users bypass system	Observe real work
No non-goals	Scope grows without control	Explicit out-of-scope list
Verbal-only decisions	People forget the decision	Decision log / ADR

Part O – Value framing without partner playbooks

O1. Value hypothesis template (original)

For [user segment] who [problem],
the Claude solution will [capability]
so that [outcome metric] moves from [baseline] to [target]
within [time], under [constraints],
with residual risks [list] accepted by [owner].

O2. Value pillars → evidence

Pillar	Claim example	Evidence you must plan
Efficiency	Handle time –30%	Time-motion + production A/B
Quality	Error rate –40%	Golden set + human audit
Transformation	New self-serve channel	Adoption + containment rate
Cost	\$ / ticket –25%	Unit economics dashboard
Performance	p95 SLA met	Latency SLO burn alerts

O3. Expectation management (durable scripts)

- “Pilot proves learning rate, not final accuracy.”
- “Autonomy expands with eval evidence, not calendar dates alone.”
- “Model upgrades are changes—they need regression gates.”
- “Citations reduce hallucination risk. They do not eliminate it.”

O4. Oversell recovery

If Sales promised “fully autonomous legal advice”: 1. Document the external commitment as a fact. 2. Reframe to compliant scope (draft + attorney HITL). 3. Offer a phased roadmap with exit criteria. 4. Escalate the sponsor decision on residual risk. Do not ship unsafe scope in silence.

Part P — Lifecycle & change management deep dive

P1. Stage exit criteria (expand)

Stage	Exit when...
Discovery	Brief + metrics + constraints signed
Design	ADR + threat notes + eval plan reviewed
Build	Contracts tested. Tracing on
Pilot	Success criteria measured. Risks logged
GA	Scorecard green. On-call ready. Docs handed off
Operate	SLO reviews running. Feedback loop owned
Iterate	Backlog prioritized from traces + users
Retire	Migration + model deprecation complete

P2. Change management levers

- **Comms:** why change, what stays human, how to get help
- **Training:** role-based (users ≠ reviewers ≠ engineers)
- **Incentives:** do not reward raw automation rate if quality collapses
- **Champions:** early adopters coach peers
- **Feedback:** visible “fix this answer” with responses

P3. Conflict patterns & resolutions

Conflict	Typical resolution
Security vs speed	Paved road + time-boxed exceptions
Product vs legal	Scoped MVP. Counsel on notices
Ops vs constant new features	Error budgets limit releases
Users vs automation	Copilot first. Expand autonomy after evidence
Sponsor vs reality	Data-backed reset. Options A/B/C

P4. Handoff package (must-not-forget)

Include architecture diagrams, ADRs, tool inventory, and the authZ model. Include eval dashboards, runbooks, the HITL staffing model, and the data retention map. Include the model pin/upgrade policy, known limitations, RACI, and the support escalation matrix.

P5. Monitoring & iteration cadence

- Daily: SLO burns that page on-call
 - Weekly: eval slice + top failure themes
 - Monthly: cost unit economics + roadmap
 - Quarterly: risk tier revisit + access review
-

Part Q — Architecture communication drills

Q1. Altitude switching exercise

Same decision (“workflow not agent”): - **Exec:** “We chose a predictable staged process so auditors can see each check—faster to trust, slightly less flexible.” - **Engineer:** “Deterministic DAG with schema validators between extract→policy→recommend. Agent only for open research subtask.” - **Risk:** “Side effects gated. HITL above threshold. Full tool audit.”

Q2. ADR minimum fields

Context, decision, alternatives rejected, consequences, status, date, owners, review trigger (e.g., model deprecation).

Q3. SLA negotiation worksheet

Path	Latency	Quality	Autonomy	Support hours
FAQ auto				
Complex assist				
High-impact				

Never accept one global SLA for mixed paths.

Part R — GTM-adjacent boundaries (exam hygiene)

In scope (original skills): discovery, value hypotheses, phased proposals, risk talk-tracks, lifecycle ownership, stakeholder updates. **Out of scope for this pack:** confidential discounting, internal battle cards, non-public packaging, fabricated partnership tiers. If a question asks how

to “position Claude vs competitor X using partner kit,” select answers about customer outcomes, constraints, and proof via evals. Do not invent competitive intel.

Part S — Production checklists (lifecycle)

S1. Discovery complete?

- ☐ Stakeholder map with veto powers
- ☐ Metrics with baselines
- ☐ Constraints wall photographed/documentated
- ☐ Non-goals listed
- ☐ Risk storm captured
- ☐ Decision log started

S2. Ready to propose?

- ☐ Value hypothesis filled
- ☐ Pattern + model rationale
- ☐ Phased autonomy plan
- ☐ Cost envelope
- ☐ Residual risks + owners
- ☐ Partner-limit: no confidential GTM claims invented

S3. Ready to hand off?

- ☐ Handoff package complete
 - ☐ On-call named
 - ☐ Feedback loop tooled
 - ☐ Training delivered
 - ☐ GA scorecard archived
-

Part T — Failure modes in stakeholder/lifecycle work

Failure	Symptom	Fix
Pilot with no exit	No GA criteria	Scorecard + date + owner
Shadow commitments	Security surprised	Early risk stakeholders
Metrics for show	Vanity dashboards	Tie to decisions/money/risk

Failure	Symptom	Fix
Doc drift	Ops cannot page	Runbook tests in drills
Users reject the change	Users bypass AI	Copilot UX + champions
Model EOL with no plan	Break in prod	Pin + upgrade playbook

Part U — Extended Q&A (59–68)

Q59. First workshop output that feeds architecture? **A.** A **one-pager brief** with problem, constraints, metrics, and risks—not a model choice debate.

Q60. Exec asks “when will it be fully autonomous?” Best answer shape? **A.** Autonomy **gated by eval evidence and risk tier**, with phased milestones. Not a blind calendar promise.

Q61. Select TWO handoff must-haves: eval dashboards, on-call RACI, partner discount sheet, informal chat channel. **A.** Eval dashboards and on-call RACI.

Q62. Sales promised unsafe scope. Architect should... **A.** Document, re-frame to compliant phased scope, escalate sponsor—**not** ship in silence.

Q63. Why include ops in discovery? **A.** Degrade modes, paging, and capacity—and to prevent unsupportable GA.

Q64. ADR alternative rejected field matters because... **A.** Exam and real reviews test whether you consider agents vs workflows and similar options.

Q65. Change management missing—likely outcome? **A.** Users bypass the system. ROI never materializes.

Q66. No official GTM playbook available. What still works on CCAR-P? **A.** Structured discovery, trade-off communication, lifecycle artifacts, and value metrics.

Q67. GA scorecard red on HITL staffing—ship anyway? **A. No**—staff first, or reduce autonomy scope first.

Q68. Select THREE discovery anti-patterns: solutioneering first, single proxy stakeholder, no non-goals, writing ADRs, measuring baselines. **A.** Solutioneering first, single proxy, no non-goals.

Part V — Rapid review (Stakeholder 14%)

- Discovery before design. Change pass included.
- Requirements need metrics and owners.
- Altitude-switch communications. Keep ADRs.
- Value = hypothesis + evidence plan.
- Lifecycle exit criteria are better than vague feelings.
- Flag partner-only GTM limits. Stay original.
- Handoff = docs + owners + evals + runbooks.
- Reset oversells. Do not ship unsafe promises.

Enablement mechanics live in 05. Safety talk-tracks pair with 03.

04 — Stakeholder Engagement, Lifecycle & GTM-Adjacent Communication

CCAR-P condensed domain 4 Official domain mapped here: Stakeholder Communication & Lifecycle Management (14%) **Note:** Official **Developer Productivity & Operational Enablement (7%)** is covered in dedicated file **05-team-enablement-operational-productivity.md**. **Approx. questions from this domain:** ~9 of 63 (plus overlap with enablement scenarios)

Disclaimer

Original notes on discovery, proposal framing, lifecycle/ops handoff, and value communication. This file does **not** reproduce any official sales scripts or internal playbooks. It teaches durable architect skills: stakeholder discovery, expectation management, ADR communication, and lifecycle ownership. Not affiliated with Anthropic; not legal/sales advice.

Overview

More than a third of CCAR-P weight sits in “soft” domains when you combine Governance + Stakeholders + Enablement. Candidates who

only grind RAG/MCP leave points on the table. This file trains you to:

- 1. Run structured discovery across business, technical, legal, security, and ops stakeholders
- 2. Translate architecture into audience-specific narratives
- 3. Frame proposals with trade-offs, risks, and success metrics
- 4. Own the lifecycle from pilot → GA → improve → retire
- 5. Communicate value without confidential partner GTM machinery

Hands-on team rollout of Claude Code, CLAUDE.md, hooks, and training curricula → **file 05**.

Key map

Task	Official mapping	Good exam answer traits
Discovery interviews	Stakeholder 14%	Constraints, success metrics, risks elicited
Explain trade-offs	Stakeholder 14%	Non-technical clarity + technical precision
Document architecture	Stakeholder 14%	ADRs, diagrams, runbook pointers
Manage expectations	Stakeholder 14%	Phased autonomy, known limitations
Lifecycle handoff	Stakeholder 14%	Owners, SLOs, feedback loops
Value communication	Stakeholder 14% (GTM-adjacent)	Outcomes & metrics—not hype
Builder enablement	→ File 05 (7%)	See dedicated domain file

Part A — Stakeholder discovery

A1. Stakeholder map

Role	What they care about	Questions to ask
Executive sponsor	Outcomes, risk, timeline, ROI	What decision will this change? What does success look like in 90 days?
Business owner	Process KPIs, UX, exceptions	Where do humans spend time? What errors hurt customers?
End users	Trust, speed, control	When would you reject AI help? What must never auto-send?
Engineering	APIs, SLOs, maintainability	What systems are source of truth? Rate limits?
Data	Quality, tenancy, pipelines	Freshness SLOs? PII classes?
Security	Threats, identity, logging	What's the authorization boundary?
Legal/Privacy	Basis, notices, retention	Any regulated data? Transfer limits?
Compliance/Risk Ops/SRE	Controls evidence Pages, runbooks, capacity	What audits apply? Who is on-call day-1?
Finance	Unit economics	Cost ceiling per transaction?
Sales/CS (GTM-adjacent)	Promise vs delivery	What has already been committed externally?

A2. Discovery workshop agenda (90 minutes)

1. Problem framing (15) — jobs to be done, pain quant
2. Constraints wall (15) — data, latency, compliance, budget
3. Success metrics (15) — leading/lagging KPIs
4. Journey mapping (20) — human+AI swimlane
5. Risk storming (15) — injection, hallucination, over-automation
6. Decision log (10) — open questions & owners

Output: one-pager brief that feeds architecture.

A3. Requirements taxonomy

- **Functional:** intents, tools, languages
- **Non-functional:** latency, availability, cost, residency
- **Safety:** disallowed actions, HITL triggers
- **Operability:** logging, ownership, support hours
- **Change:** model upgrade policy

Capture **non-goals** explicitly (“not a fully autonomous negotiator in v1”).

A4. Anti-patterns in discovery

- Only talking to the sponsor
- Accepting “ChatGPT but on our data” as a requirement
- Skipping security until late
- Ignoring already-sold promises from GTM motions
- No quantitative baseline (can’t prove value later)

Part B — Communicating architecture

B1. Altitude switching

Audience	Artifacts	Language
Exec	One-slide value + risk + ask	Outcomes, timeline, investment
Business	Journey + exception paths	Before/after KPIs
Engineers	Sequence diagrams, schemas, ADRs	Interfaces, failure modes
Security	Data flow + threat model	Trust boundaries, controls
Legal	Processing description	Purposes, retention, rights

Same system, different cuts. Exam traps: drowning execs in token graphs; hand-waving security to engineers.

B2. Architecture Decision Records (ADRs)

Template:

- Context
- Decision
- Alternatives considered
- Consequences (positive/negative)
- Status & date
- Related eval evidence

Example decision: “Workflow over agent for quote validation because auditability > flexibility.”

B3. Trade-off narratives

Use **option A/B/C** tables:

Option	Quality	Latency	Cost	Risk	Notes
A Monolith prompt	Med	Best	Low	Audit weak	Reject for compli- ance
B Workflow	High	Med	Med	Low	Recom- mended
C Full agent	High?	Worst	High	Higher	Park for phase 2

Recommend **one**, don’t dump the table without a pick.

B4. Expectation management scripts (original, non-confidential)

- “Phase 1 suggests; humans send. Phase 2 auto-sends only for template class T with metrics M.”
- “Model will be wrong ~X% on slice S; we route those to experts.”
- “We will not promise identical wording—only process SLAs and quality rates.”

Honesty builds trust; overclaiming creates incident-class disappointment.

Part C — Proposal framing (GTM-adjacent, partner limits flagged)

C1. What you *can* write originally

Architects supporting pre-sales or internal funding should structure proposals around:

- 1. **Problem & baseline metrics**
- 2. **Target outcomes** (efficiency, quality, risk reduction)
- 3. **Solution sketch** (pattern, integrations, controls)
- 4. **Phased roadmap** (pilot → GA)
- 5. **Investment** (people, infra, licenses—at high level)
- 6. **Risks & mitigations**
- 7. **Success review date**

C2. What you should *not* invent

- Confidential partner discount schedules
- Internal vendor battle cards
- Guaranteed win rates or unpublished benchmarks as marketing claims
- Customer logos/case details you don’t have rights to

Note: Employer-provided playbooks may add firm-specific motions; this pack stays skill-oriented.

C3. Value communication patterns

Value story	Evidence to attach
Time saved	Time-motion baseline vs pilot
Quality lift	Error rate / edit distance
Risk reduction	Fewer policy breaches; audit readiness
Revenue assist	Conversion lift with experiment design
Developer velocity	See file 05 metrics

Avoid “AI magic” slides. Prefer “instrumented learning.”

C4. Handling oversold expectations

If GTM already promised full autonomy:

- 1. Acknowledge the gap without blame theater
- 2. Show risk of current promise
- 3. Propose phased delivery that still lands value early
- 4. Align sponsor on rewritten success criteria in writing

Architects protect long-term trust.

Part D — Lifecycle management

D1. Lifecycle stages

Discover → Design → Build → Pilot → GA → Operate → Improve → Retire

Each stage needs **exit criteria**.

Stage	Exit criteria examples
Discover	Brief + metrics + risk tier signed
Design	ADR + threat model + eval plan
Build	Tests green; tools contract-tested
Pilot	N users; metrics vs baseline; safety probes pass
GA	On-call live; docs; feature flags; SLOs
Operate	Weekly quality review; cost within envelope
Improve	Prioritized backlog from feedback/evals
Retire	Traffic drained; data deleted per policy; postmortem

D2. Handoff package (build → ops)

- Architecture overview + diagrams
- Model/prompt/index pins
- Runbooks (latency, bad outputs, injection suspicion, cost spike)
- Dashboards & alert thresholds
- Known failure modes
- Support escalation matrix
- Change process for prompts/tools

File 05 deepens developer/ops enablement content.

D3. Feedback loops

Sources: user thumbs, edit distance, tickets, sales objections, security findings, eval regressions. Triage into: prompt fix, retrieval fix, product

UX, training, refuse-to-automate.

D4. Operating cadence

- Daily: error/cost/safety alerts
- Weekly: quality slice review
- Monthly: roadmap & risk tier revisit
- Quarterly: red team / access review / vendor review

D5. Deprecation & model end-of-life

Provider model deprecations are lifecycle events. Maintain:

- Inventory of pins
- Migration owners
- Compat eval suite
- Comms plan to stakeholders

Part E — Change & conflict management

E1. Common conflicts

Conflict	Resolution approach
Speed vs safety	Phased autonomy + metrics gates
Cost vs quality	Routing & caching before quality cuts
Sales promise vs eng reality	Written scope reset with sponsor
Central platform vs domain teams	Clear paved road + escape hatches
Privacy vs analytics	Aggregation/redaction design

E2. RACI for lifecycle (example)

Activity	Sponsor	Architect	Eng	Security	Ops	GTM/CS
Success metrics	A	R	C	I	I	C
Threat model	I	R	C	A	C	I
GA go/no-go	A	R	R	A	A	C
Customer messaging	C	C	I	C	I	R/A

Part F — Documentation standards

Minimum viable docs:

1. README for service
2. ADR log
3. Data flow (happy + abuse)
4. Eval report template
5. User-facing limitations page
6. On-call runbooks

Prefer living docs near code over slide decks as sole source of truth.

Decision trees

Audience pitch picker

Is audience accountable for budget/risk?

YES → Lead with outcomes, risk, ask, timeline

Is audience implementing?

YES → Lead with interfaces, failure modes, ADRs

Is audience signing legal?

YES → Lead with data flows, retention, subprocessors

Pilot vs GA gate

Safety probes + tenancy tests green?

Owners & on-call named?

SLOs & dashboards live?

Support macros ready?

ALL YES → GA candidate

ELSE → Extend pilot with explicit gaps list

Exam traps

1. Jumping to tech without discovery metrics.
2. One-size communication for all stakeholders.
3. No non-goals / infinite scope.
4. GA without ops handoff.
5. Inventing partner-confidential GTM claims.

6. Ignoring already-committed customer promises.
 7. Metrics only vanity (page views of chatbot).
 8. No retirement/EOL plan for models.
 9. Skipping legal/security until after build.
 10. “We’ll monitor” without named owners/cadence.
-

Practice Q&A (26)

Q1. Name five stakeholder groups for an enterprise Claude app. **A.** e.g., sponsor, business owner, eng, security, legal (also ops, data, finance, users).

Q2. What belongs in discovery non-goals? **A.** Explicitly out-of-scope capabilities for the phase (e.g., no autonomous refunds in v1).

Q3. How should you present options to an exec? **A.** Short outcome/risk/ask; optional one trade-off table with a clear recommendation.

Q4. What is an ADR consequence section for? **A.** Recording follow-on costs/risks of the decision for future readers.

Q5. Pilot exit criteria example? **A.** Target users completed tasks with metric lift and safety probes passing.

Q6. What do you do if sales promised full autonomy unsafely? **A.** Realign sponsor on phased delivery and rewrite success criteria in writing.

Q7. Why capture baseline metrics before build? **A.** Without baseline you cannot prove value or defend ROI.

Q8. Name three handoff artifacts to ops. **A.** Runbooks, dashboards/alerts, pin inventory (also escalation matrix, known failures).

Q9. Vanity metric example to avoid? **A.** Raw chatbot message count without task success or CSAT/quality linkage.

Q10. Who is typically accountable for GA go/no-go safety signoff? **A.** Security (and sponsor overall)—per RACI; architect facilitates evidence.

Q11. What is altitude switching? **A.** Adjusting artifact depth/language to audience without changing facts.

Q12. Why schedule model EOL as a lifecycle event? **A.** Deprecations break pins; need migration owners and evals.

- Q13.** GTM-adjacent content you should not invent in study claims? **A.** Confidential partner discounts, battle cards, unpublished guarantee scripts.
- Q14.** What makes a good success review meeting? **A.** Pre-agreed metrics, open risks, decision to scale/pivot/stop.
- Q15.** How do feedback loops connect to engineering? **A.** Triage into prompt/retrieval/UX/training/refuse-to-automate backlog items.
- Q16.** Conflict speed vs safety—architect move? **A.** Phase autonomy; ship suggest-mode value early; gate auto on metrics.
- Q17.** What is a limitations page for? **A.** Transparent user/admin expectations; reduces misuse and support load.
- Q18.** Why include CS/Sales in discovery lightly? **A.** Learn external commitments and objection patterns early.
- Q19.** Minimum legal-oriented artifact? **A.** Processing/data-flow description with purposes and retention.
- Q20.** What does “instrumented learning” mean in value stories? **A.** Claims backed by measured pilot evidence, not slogans.
- Q21.** Who owns weekly quality review? **A.** Named role (often architect + eng + business)—must be explicit.
- Q22.** Escape hatch in platform paved road? **A.** Documented way for teams to deviate with extra review—not silent forks.
- Q23.** Why write non-functional requirements early? **A.** Latency/residency/cost reshape architecture; late discovery causes rework.
- Q24.** Retire stage must include what data step? **A.** Drain traffic and delete/retain data per policy (indexes/logs included).
- Q25.** Best response to “just make it like ChatGPT”? **A.** Reframe into jobs-to-be-done, constraints, and measurable outcomes.
- Q26.** Where are Claude Code rollout details? **A.** File 05-team-enablement-operational-productivity.md.
-

Checklist

- ☐ I can run a 90-minute discovery workshop structure

- ☐ I can switch altitude across exec/eng/security/legal
- ☐ I can write an ADR with alternatives and consequences
- ☐ I can frame a phased proposal without partner-confidential fluff
- ☐ I can list lifecycle exit criteria including retire
- ☐ I can build an ops handoff package outline
- ☐ I can reset oversold autonomy expectations constructively
- ☐ I can define operating cadences and owners
- ☐ I know GTM partner-content limits for this pack
- ☐ I know enablement deep content is in file 05

Glossary

Term	Meaning
Discovery	Structured requirements & constraint elicitation
ADR	Architecture Decision Record
Non-goals	Explicit out-of-scope items
Baseline	Pre-AI metric for comparison
Pilot	Limited release to learn under controls
GA	General availability
Handoff package	Artifacts transferring build → operate
Paved road	Supported standard platform path
Escape hatch	Approved deviation process
Value story	Outcome narrative backed by evidence
RACI	Responsibility assignment matrix
EOL	End of life (e.g., model deprecation)
GTM	Go-to-market
Expectation management	Aligning promises to feasible delivery
Instrumented learning	Measurement-driven iteration

Part G — Meeting patterns & facilitation

G1. Working backward from the press release (internal)

Draft a one-paragraph internal “launch note” early: who benefits, what changes, what won’t. If stakeholders cannot agree on that paragraph, architecture will thrash.

G2. Decision hygiene

- Single decision owner
- Deadline
- Inputs required
- Reversible vs irreversible classification
- Document in ADR within 48 hours

G3. Stakeholder resistance types

Resistance	Underlying need	Move
Fear of job loss	Role clarity	Redesign job to oversight/exception handling
Fear of liability	Accountability	HITL + audit + clear owners
Skepticism from past AI hype	Evidence	Small pilot with ruthless metrics
Security veto culture	Control	Invite security as co-designer early
Shadow IT AI use	Speed	Paved road that's easier than shadow tools

G4. Comms plan for incidents involving AI

Prewrite: who speaks, what we disclose, how we pause the feature flag, how we update customers. Pair with file 03 IR playbooks.

G5. Training stakeholders (light)

Business users need “how to ask / how to verify / when to escalate,” not model theory. Deep builder training → file 05.

G6. Extended scenarios

Scenario: Procurement wants AI to auto-email vendors negotiating price.
Stakeholder move: Convene legal+security+procurement; propose suggest-mode with approval; define savings metric; refuse unbounded autonomy in v1.

Scenario: Board asks for “AI strategy KPI.” **Move:** Portfolio view — % use cases with eval gates, incident count, value realized vs target, risk tier coverage—not a single vanity chatbot score.

G7. Q&A 27–35

Q27. Why draft an internal launch paragraph early? **A.** Forces agreement on who benefits and what won’t change before build thrash.

Q28. Reversible vs irreversible decision—example of irreversible? **A.** Autonomously emailing legally binding commitments; migrating prod data without rollback.

Q29. How do you respond to security-as-late-gate? **A.** Bring security into discovery/design; share threat model drafts early.

Q30. Shadow IT AI—platform response? **A.** Make paved road faster/safer than unsanctioned tools; educate; monitor.

Q31. Board-level AI KPI examples? **A.** Eval-gated use-case %, realized value vs target, AI incident rate, risk-tier coverage.

Q32. What training do business users need first? **A.** Ask/verify/escalate behaviors and limitations—not transformer internals.

Q33. Who speaks in a public AI content incident? **A.** Pre-assigned comms owner with legal/security input—not an ad-hoc engineer tweet.

Q34. Why classify decisions by reversibility? **A.** Irreversible ones need more evidence, HITL, and higher altitude approval.

Q35. Procurement auto-negotiate email—first workshop output? **A.** Written non-goal of autonomy + approval workflow + savings metric.

Part H — Narrative kits (original, reusable)

H1. Executive one-pager skeleton

- 1. **Problem:** quantified pain
- 2. **Proposal:** pattern in one sentence
- 3. **Value:** 2–3 metrics with targets
- 4. **Risk:** top 3 with mitigations
- 5. **Ask:** funding/people/decision

6. **Checkpoint:** date for go/no-go

H2. Security review talking points

- Data classes in prompts/logs/indexes
- Tool privilege diagram
- Injection storyboard
- Eval gates for safety
- Incident severity mapping
- Vendor/MCP list

H3. Customer success enablement (non-confidential)

CS needs: limitation cards, escalation matrix, “how to verify answers,” when to pull engineers. Do not invent partner discount motions.

H4. Roadmap communication

Show **capability unlocks** tied to controls: “Citations+eval gate unlocks external FAQ”; “HITL refunds unlock finance pilot.” Avoid roadmap of model brand names alone.

H5. Q&A 36–42

Q36. What are the six parts of an exec one-pager? **A.** Problem, proposal, value, risk, ask, checkpoint.

Q37. Why tie roadmap unlocks to controls? **A.** Makes safety/eval investment visible as value enablers, not blockers.

Q38. What does CS need besides feature lists? **A.** Limitation cards, verification guidance, escalation paths.

Q39. Injection storyboard purpose in security reviews? **A.** Make indirect injection concrete with a step-by-step abuse path.

Q40. What should not appear in this pack’s GTM claims? **A.** Confidential partner playbooks, unpublished guarantees, unauthorized case studies.

Q41. Capability unlock example? **A.** External FAQ only after citation precision gate passes.

Q42. Why include a checkpoint date in the ask? **A.** Forces decision hygiene and prevents endless pilot limbo.

Part I — Lifecycle artifacts library (expand)

I1. Pilot charter fields

- Hypothesis
- Population & exclusions
- Duration
- Metrics & thresholds to scale
- Safety stop conditions
- Roles (sponsor, DRI, security contact)
- Communication plan

I2. GA readiness scorecard (example weights)

Area	Weight	Evidence
Quality evals	25%	Report $\geq$ thresholds
Safety probes	25%	Red-team closed or accepted
Ops readiness	20%	Runbooks + on-call drill
Privacy/legal	15%	Signoffs
Support enablement	10%	Macros + training
Cost envelope	5%	Forecast vs budget

Fail any critical safety/privacy item → no GA regardless of average.

I3. Stakeholder update email (original template)

Subject: [Pilot] Week N outcomes Body: metric table vs baseline; top incidents; decision needed; next experiment. Keep to one screen.

I4. Workshop facilitation tips

- Parking lot for out-of-scope
- Timebox architects explaining transformers
- Capture verbatim user “never auto-...” quotes—they become HITL requirements
- End with owners + dates, not vibes

15. Cross-cultural / multi-region stakeholders

Clarify residency early; local champions; language eval slices; don't assume US-only legal frames for global rollouts.

16. Q&A 43–50

Q43. Name four pilot charter fields. **A.** Hypothesis, metrics/thresholds, safety stops, roles (etc.).

Q44. Can high quality scores override failed safety probes for GA? **A.** No—critical safety/privacy fails block GA.

Q45. What belongs in a weekly pilot email? **A.** Metrics vs baseline, incidents, decision ask, next experiment.

Q46. Why capture “never auto” user quotes? **A.** They translate directly into HITL/product rules.

Q47. Multi-region discovery must ask what early? **A.** Data residency and local regulatory constraints.

Q48. Parking lot technique prevents what? **A.** Scope explosion derailing the workshop agenda.

Q49. Who is DRI in a pilot charter? **A.** Directly responsible individual for day-to-day delivery.

Q50. Why include support enablement in GA scorecard? **A.** Without macros/training, production creates chaos and shadow process.

Enablement, Claude Code org rollout, runbooks-as-training → 05.

Part J — Extended lifecycle playbooks

J1. From discovery notes to backlog

Convert workshop outputs into epics:

- Epic: Retrieval tenancy hardening
- Epic: Citation UX
- Epic: HITL queue v1
- Epic: Eval harness CI
- Epic: On-call runbooks

Each epic links to a metric and a risk tier control. This prevents “random prompt tweaks” masquerading as delivery.

J2. Steering committee pack

Monthly 30 minutes: KPI dashboard, top risks, decision requests, budget burn. Pre-read only; meeting is for decisions. Architects who narrate for 25 minutes fail the committee.

J3. Internal FAQ for skeptical leaders

Q: Will this replace my team? **A:** Design targets toil reduction and exception-centered roles; staffing decisions are leadership's, but architecture should make oversight workable.

Q: Can we trust answers? **A:** Trust is measured—citation rates, eval slices, HITL on critical paths—not assumed.

Q: Why not ship the demo? **A:** Demos lack tenancy tests, on-call, and abuse cases; GA scorecard exists for a reason.

J4. Partner-limit reminder box

This pack deliberately stops at universal architect skills: discovery, framing, lifecycle, and honest value communication. Do not treat anything here as official sales doctrine.

J5. Post-GA value review (day 60)

Return to baseline metrics; publish a short “value realized” memo; decide invest/maintain/sunset. Without this, zombies accumulate.

J6. Additional Q&A 51–58

Q51. How do workshop notes become engineering work? **A.** Epics linked to metrics and controls—not orphan prompt tweaks.

Q52. What is a steering pack for? **A.** Decision-making with pre-read KPIs/risks—not architecture lectures.

Q53. Best answer to “ship the demo”? **A.** Demo $\neq$ GA; cite missing gates on the scorecard.

Q54. Day-60 review purpose? **A.** Measure realized value; invest/maintain/sunset deliberately.

Q55. How to answer replacement fears? **A.** Focus on toil/exceptions; avoid speculative HR promises.

Q56. Why link epics to risk tier controls? **A.** Keeps safety work visible in the same backlog as features.

Q57. What is a zombie AI app? **A.** Running system without owners, metrics, or value review.

Q58. Where is developer enablement detailed? **A.** File 05—do not dilute stakeholder answers with IDE settings unless asked.

Part K — Communication worksheets

K1. Constraint wall prompts (use in workshops)

- Hard deadline? What happens if we miss?
- Data that must never leave region X?
- Actions that always need a human name on them?
- Maximum acceptable wrong-answer rate for slice S?
- Support hours and language coverage?
- Existing contractual SLAs with customers?

K2. Trade-off sentence stems (practice saying these)

- “We recommend B because auditability outweighs flexibility given constraint C.”
- “We can hit latency L only if we route easy traffic to model M and reserve N for hard cases.”
- “Phase 1 will not auto-send; that unlocks after metric $X \geq T$ for two weeks.”
- “Cost target requires cache hit rate $\geq H$ and top-k $\leq K$.”

K3. Lifecycle RACI quiz yourself

Pick a random activity—prompt change, index rebuild, customer apology, MCP add—and assign RACI without looking. Then check against your organization’s real names. Exam scenarios expect you to know *which function* is accountable even if names differ.

K4. End-card

Stakeholder domain = discovery quality + altitude switching + phased honesty + lifecycle ownership. GTM-adjacent work stays non-confidential: value, risks, phases, metrics. Enablement mechanics live in file 05.

Part L — Full discovery questionnaire bank (condensed)

Business: What breaks if the AI is wrong? What is the cost of delay today? Which step is pure toil? **Users:** When would you ignore the suggestion? What must remain human-branded? **Eng:** Rate limits? Idempotency already in APIs? Sync vs async constraints? **Data:** Freshness? Label quality? Known poison documents? **Security:** Crown-jewel systems in scope? Existing DLP? **Legal:** Automated decisioning rules? Retention schedule? **Ops:** Who pages at 2am? What's the rollback muscle memory? **Finance:** Unit-cost ceiling? CapEx vs OpEx preferences? **GTM/CS:** What was already promised in writing? Top objections?

Running even half of these prevents the classic “solution looking for a problem” failure mode that CCAR-P scenarios punish.

Part M — Handoff email template (original)

Subject: Handoff — to operations Attach: diagrams, ADR index, pin list, runbooks, dashboard links, known issues, escalation matrix. Body checklist: SLO summary; feature flags; how to roll back model/prompt/index; where eval reports live; next quality review date; DRI during hypercare (usually two weeks).

Hypercare means the build team stays in the support path while ops gains reps—then steps back per agreement. Skipping hypercare is how “lifecycle” fails silently.

Closing card for file 04

You now have discovery banks, altitude switching, proposal framing within partner-limit honesty, lifecycle gates, and handoff discipline. Pair with file 05 when stems mention Claude Code, CLAUDE.md, hooks, or org-wide developer tooling.

One more worked stem

Stem flavor: “Sponsor wants GA next month after a flashy demo; security has not reviewed; no on-call.” Strong answer: present GA scorecard gaps; propose dated pilot extension; secure review + runbook drill as exit criteria; reset external promises with sponsor—not silent shipping.

Expectation-reset paragraph (memorize structure)

“We can deliver measurable value by date D in suggest-mode with controls C. Full autonomy remains a phase-2 unlock after metrics M hold for period P. Shipping autonomy now would accept risk R without compensating controls. Please confirm the revised success criteria in writing so engineering, security, and GTM share one plan.”

That structure—value now, unlock later, risk named, written alignment—answers most “pressure to over-automate” stakeholder stems without partner-specific playbooks.

(File continues — deep-dive Part N follows below.)

Hypercare without a named DRI is theater; name the person and the end date when you hand off.

Keep a living decision log; when stakeholders revisit settled choices, point to the ADR date and evidence rather than re-litigating from memory.

Part N — Primary-study deep dive: Discovery that survives contact with reality

Scope note (read first): This pack teaches **original** discovery, lifecycle, change management, and value framing skills only — no invented pricing, confidential win themes, or official lesson text. If an exam stem smells like a “sales motion,” answer with durable architect behaviors: clarify commitments, align SLAs, document trade-offs, phase delivery.

N1. Discovery depth model (five passes)

1. **Problem pass** — jobs-to-be-done, pain quant, current workaround.
2. **Constraint pass** — data, latency, residency, budget, unions/works councils, brand.
3. **Success pass** — leading/lagging metrics, decision rights, time box.
4. **Risk pass** — safety, compliance, operational blast radius, reputational.
5. **Change pass** — who loses power/status, training load, process redesign.

Skipping the change pass is why “technically perfect” pilots die.

N2. Stakeholder interview bank (condensed, original)

Executive: What decision improves if this works? What would make you stop the program? **Business owner:** Which step burns minutes? What error is unacceptable? **Users:** When do you distrust AI? What must stay manual? **Security:** What’s the identity boundary? What’s your nightmare ticket? **Legal/Privacy:** Data classes? Transfer? Retention? Notices? **Ops:** Who pages at 2am? What’s the degrade mode? **Finance:** Fully loaded cost ceiling per transaction? **Sales/CS (GTM-adjacent):** What was already promised externally?

N3. Requirement quality bar

A requirement is exam-ready when it has: owner, metric, constraint, priority, and test idea. “Make it smart” is not a requirement. “p95 < 3s for FAQ path; HITL for refunds > \$50; GDPR erasure within policy SLO” is.

N4. Anti-patterns (discovery)

Anti-pattern	Why it fails	Fix
Single stakeholder proxy	Hidden vetoes later	Map power + impacted users
Solutioneering first	Misses real job	Problem framing before pattern
Ignoring shadow process	Users bypass system	Observe real work
No non-goals	Scope cream	Explicit out-of-scope list
Verbal-only decisions	Amnesia	Decision log / ADR

Part O — Value framing without partner playbooks

O1. Value hypothesis template (original)

For [user segment] who [problem],
the Claude solution will [capability]
so that [outcome metric] moves from [baseline] to [target]
within [time], under [constraints],
with residual risks [list] accepted by [owner].

O2. Value pillars → evidence

Pillar	Claim example	Evidence you must plan
Efficiency	Handle time –30%	Time-motion + production A/B
Quality	Error rate –40%	Golden set + human audit
Transformation	New self-serve channel	Adoption + containment rate
Cost	\$ / ticket –25%	Unit economics dashboard
Performance	p95 SLA met	Latency SLO burn alerts

O3. Expectation management (durable scripts)

- “Pilot proves learning rate, not final accuracy.”
- “Autonomy expands with eval evidence, not calendar dates alone.”
- “Model upgrades are changes—they need regression gates.”
- “Citations reduce hallucination risk; they do not eliminate it.”

O4. Oversell recovery

If Sales promised “fully autonomous legal advice”: 1. Document the external commitment factually. 2. Reframe to compliant scope (draft + attorney HITL). 3. Offer phased roadmap with exit criteria. 4. Escalate sponsor decision on residual risk—do not quietly ship unsafe scope.

Part P – Lifecycle & change management deep dive

P1. Stage exit criteria (expand)

Stage	Exit when...
Discovery	Brief + metrics + constraints signed
Design	ADR + threat notes + eval plan reviewed
Build	Contracts tested; tracing on
Pilot	Success criteria measured; risks logged

Stage	Exit when...
GA	Scorecard green; on-call ready; docs handed off
Operate	SLO reviews running; feedback loop owned
Iterate	Backlog prioritized from traces + users
Retire	Migration + model deprecation complete

P2. Change management levers

- **Comms:** why change, what stays human, how to get help
- **Training:** role-based (users ≠ reviewers ≠ engineers)
- **Incentives:** don't reward raw automation rate if quality collapses
- **Champions:** early adopters coach peers
- **Feedback:** visible "fix this answer" with responses

P3. Conflict patterns & resolutions

Conflict	Typical resolution
Security vs speed	Paved road + time-boxed exceptions
Product vs legal	Scoped MVP; counsel on notices
Ops vs feature factory	Error budgets bind releases
Users vs automation	Copilot first; earn autonomy
Sponsor vs reality	Data-backed reset; options A/B/C

P4. Handoff package (must-not-forget)

Architecture diagrams, ADRs, tool inventory + authZ model, eval dashboards, runbooks, HITL staffing model, data retention map, model pin/upgrade policy, known limitations, RACI, support escalation matrix.

P5. Monitoring & iteration cadence

- Daily: page-worthy SLO burns
- Weekly: eval slice + top failure themes
- Monthly: cost unit economics + roadmap
- Quarterly: risk tier revisit + access review

Part Q — Architecture communication drills

Q1. Altitude switching exercise

Same decision (“workflow not agent”): - **Exec:** “We chose a predictable staged process so auditors can see each check—faster to trust, slightly less flexible.” - **Engineer:** “Deterministic DAG with schema validators between extract→policy→recommend; agent only for open research sub-task.” - **Risk:** “Side effects gated; HITL above threshold; full tool audit.”

Q2. ADR minimum fields

Context, decision, alternatives rejected, consequences, status, date, owners, review trigger (e.g., model deprecation).

Q3. SLA negotiation worksheet

Path	Latency	Quality	Autonomy	Support hours
FAQ auto				
Complex assist				
High-impact				

Never accept one global SLA for mixed paths.

Part R — GTM-adjacent boundaries (exam hygiene)

In scope (original skills): discovery, value hypotheses, phased proposals, risk talk-tracks, lifecycle ownership, stakeholder updates. **Out of scope for this pack:** confidential discounting, internal battle cards, non-public packaging, fabricated partnership tiers. If a question asks how to “position Claude vs competitor X using partner kit,” choose answers about **customer outcomes, constraints, and proof via evals**—not invented competitive intel.

Part S — Production checklists (lifecycle)

S1. Discovery complete?

- ☐ Stakeholder map with veto powers
- ☐ Metrics with baselines
- ☐ Constraints wall photographed/documentated
- ☐ Non-goals listed
- ☐ Risk storm captured
- ☐ Decision log started

S2. Ready to propose?

- ☐ Value hypothesis filled
- ☐ Pattern + model rationale
- ☐ Phased autonomy plan
- ☐ Cost envelope
- ☐ Residual risks + owners
- ☐ Partner-limit: no confidential GTM claims invented

S3. Ready to hand off?

- ☐ Handoff package complete
- ☐ On-call named
- ☐ Feedback loop tooled
- ☐ Training delivered
- ☐ GA scorecard archived

Part T — Failure modes in stakeholder/lifecycle work

Failure	Symptom	Fix
Pilot purgatory	No GA criteria	Scorecard + date + owner
Shadow commitments	Security surprised	Early risk stakeholders
Metric theater	Vanity dashboards	Tie to decisions/money/risk
Doc drift	Ops can't page	Runbook tests in drills
Change revolt	Users bypass AI	Copilot UX + champions
Model EOL panic	Break in prod	Pin + upgrade playbook

Part U — Extended Q&A (59–68)

Q59. First workshop output that feeds architecture? **A.** A **one-pager brief** with problem, constraints, metrics, risks—not a model choice debate.

Q60. Exec asks “when will it be fully autonomous?” Best answer shape? **A.** Autonomy **gated by eval evidence and risk tier**, with phased milestones—not a blind calendar promise.

Q61. Select TWO handoff must-haves: eval dashboards, on-call RACI, partner discount sheet, meme channel. **A.** Eval dashboards and on-call RACI.

Q62. Sales promised unsafe scope. Architect should... **A.** Document, reframe to compliant phased scope, escalate sponsor—**not** silently ship.

Q63. Why include ops in discovery? **A.** Degrade modes, paging, capacity—and to prevent unsupportable GA.

Q64. ADR alternative rejected field matters because... **A.** Exam and real reviews test whether you **considered** agents vs workflows etc.

Q65. Change management missing—likely outcome? **A.** Users bypass the system; ROI never materializes.

Q66. No official GTM playbook available. What still works on CCAR-P? **A.** Structured discovery, trade-off communication, lifecycle artifacts, value metrics.

Q67. GA scorecard red on HITL staffing—ship anyway? **A.** **No**—staff or reduce autonomy scope first.

Q68. Select THREE discovery anti-patterns: solutioneering first, single proxy stakeholder, no non-goals, writing ADRs, measuring baselines. **A.** Solutioneering first, single proxy, no non-goals.

Part V — Rapid review (Stakeholder 14%)

- Discovery before design; change pass included.
- Requirements need metrics and owners.
- Altitude-switch communications; keep ADRs.
- Value = hypothesis + evidence plan.
- Lifecycle exit criteria beat vibes.
- Flag partner-only GTM limits; stay original.
- Handoff = docs + owners + evals + runbooks.

- Reset oversells; don't ship unsafe promises.

Enablement mechanics live in 05. Safety talk-tracks pair with 03.

05 — Team Enablement & Operational Productivity — Simplified Technical English

CCAR-P condensed domain 5 (dedicated) Official domain mapped here: Developer Productivity & Operational Enablement (7%) **Approx. questions:** ~4 of 63. There are fewer items. The scenarios are concrete (org rollout, settings hierarchy, secure defaults). **Cross-links:** Architecture patterns in 01. MCP/tools in 02. Safety baselines in 03. Lifecycle/comms in 04.

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names Claude Code, MCP, CLAUDE.md, Skills, and hooks stay as written. These notes are original enablement notes. They use themes from **public** Claude Code and Anthropic product documentation (project instructions, MCP, plugins/marketplaces, settings hierarchies, hooks). They also use standard enterprise developer-productivity practice. They are **not** official course content. Product UI names and setting keys change. Check current public Claude Code docs before a production rollout. This pack is independent. It is not affiliated with Anthropic.

Disclaimer

These are original enablement notes. They use themes from **public** Claude Code and Anthropic product documentation. Those themes include project instructions, MCP, plugins/marketplaces, settings hierarchies, and hooks. They also use standard enterprise developer-productivity practice. They are **not** official course content. Product UI names and setting keys change. Check current public Claude Code docs before a production rollout. This pack is independent. It is not affiliated with Anthropic.

Overview

The 7% domain has more impact than the percentage shows. Wrong answers are obvious if you know the **control plane** for teams. You roll out Claude Code (or similar builder tooling) with **enforced security, shared skills, standardized workflows**, and **operational runbooks**. You must not let every developer become an unsupervised admin of production credentials.

This file covers:

- 1. Team rollout strategy for Claude Code / AI coding assistants
- 2. Org instruction files (CLAUDE.md and related), Skills, hooks
- 3. Managed settings vs recommended settings vs local overrides
- 4. Plugin/marketplace distribution of approved tools
- 5. Productivity workflows (code, test, docs, debug)
- 6. Operational support patterns & runbooks
- 7. Training and onboarding curricula for builders

Key map

Enablement task	Exam signal
Enforce org security settings	Server-managed / highest-precedence controls that users cannot override
Distribute approved Skills/MCP/plugins	Managed marketplace / catalog—not “install anything”
Standardize repo guidance	CLAUDE.md / project instructions versioned in git
Raise velocity safely	AI for code/test/docs with review gates
Ops debugging with AI	Guardrailed assistants + runbooks—not unrestricted production keys
Train builders	Role-based onboarding + paved road

Part A — Rollout strategy

A1. Why architects own enablement design

If platform and security teams do not make a paved road, teams adopt consumer AI. Those teams then paste secrets. Architects design:

- Identity (SSO) into approved tooling
- Data handling rules (what code/secrets you may send)
- Approved MCP/plugin set
- Secrets via vault—not chat
- Metrics for adoption **and** incidents

A2. Phased rollout

Phase	Population	Goal
0 Design	Platform + security	Settings, marketplace, logging
1 Pilot	1–2 volunteer teams	Friction log. Productivity signals
2 Expand	All eng with training	Default secure config
3 Optimize	Org-wide	Skills library. Inner source
4 Continuous	—	Version upgrades. Access reviews

Exit criteria per phase: you close security findings. The support queue stays manageable. You measure useful adoption (not seat count alone).

A3. Success metrics for enablement

Metric	Intent
Time-to-first-merged-PR with assistant	Onboarding efficacy
PR cycle time / review iterations	Velocity (check other causes)
Secret-scan incidents involving AI tools	Safety
% repos with CLAUDE.md	Standardization
% MCP from approved marketplace only	Supply-chain hygiene
Support tickets per 100 seats	Enablement quality
Developer satisfaction (survey)	UX of paved road

Part B — CLAUDE.md, Skills, and project instructions

B1. What org instruction files are for

Public Claude Code docs describe project guidance files (commonly CLAUDE.md or equivalent project instructions). These files help the coding agent know:

- Repo structure & conventions
- Build/test commands
- Code style & review expectations
- Forbidden actions (e.g., do not touch prod/ without a ticket)
- How to run linters/security scanners

Architect guidance: Treat these files as **versioned product code**. Review them in PRs. Keep secrets out. Link to deeper docs. Do not paste long documents (context budget).

B2. Layered instructions

Layer	Example	Scope
Org default	Company coding standards	Skill All repos
Repo CLAUDE.md	Service-specific architecture	One repo
Directory hints	/payments stricter rules	Subtree
Personal	User preferences	Local only—must not we

Security-critical rules belong in **enforced settings**. Do not put them only in markdown. A model can under-weight markdown.

B3. Skills for teams

Skills package reusable instructions/resources for recurring tasks: “add OpenTelemetry span,” “write ADR,” “migrate endpoint.” Benefits:

- Consistency across humans
- Smaller per-turn prompts
- Reviewable artifacts

Governance: Skill review owners. Semver. Deprecate. Security scan for exfil instructions.

B4. Hooks

Hooks are automation around agent lifecycle events (formatters, secret scanners, policy checks). Hooks enforce **deterministic** controls where prompts are soft:

- Pre-commit / pre-push style checks triggered in agent loops
- Block commits containing secrets
- Auto-run formatters
- Require tests for certain paths

Exam theme: combine **soft guidance** (CLAUDE.md) with **hard hooks/settings**.

Part C — Settings hierarchy & enforcement

C1. Precedence mental model

Public Claude Code enterprise features describe a hierarchy. **Server-managed / organization-managed settings** sit at the highest precedence. Users **cannot override** them with project, user, local, or CLI flags. Lower layers give defaults and team conventions.

Layer (conceptual)	Override by users?	Use for
Server / org-managed	No	Security policies, denied tools, env constraints
Marketplace restrictions	No/limited	Approved plugin sources
Shared project recommendations	Yes (unless managed)	Conventions
User/local	Yes	Personal DX

Common exam error: “Publish a recommended settings JSON in a repo. Also, ask teams to copy it.” This is not enough when the requirement is **enforce**. Prefer managed settings + managed marketplace.

C2. What to enforce org-wide

- Disallowed MCP sources / unchecked plugin installs

- Telemetry/logging baselines required by security
- Environment variable policies
- Command allow/deny lists for agent shell
- Network egress constraints where supported
- Requirement to use SSO-backed installs

C3. What to leave flexible

- Editor keybindings
- Non-security model preference within approved set
- Personal prompt snippets that do not grant tools

C4. Managed plugin marketplace

Use a central catalog to distribute approved:

- Skills
- Agents
- Hooks
- MCP servers

The catalog supports versioning, discovery, updates, and **source control**. This is better than each developer using curl on random GitHub. Pair this with policy: only marketplace installs on corp devices.

Part D — MCP & credentials for builders

D1. Separation from production runtime

Builder environments must **not** reuse production write credentials. Give:

- Dev/stage credentials with narrow scopes
- Read-only prod replicas where justified
- Break-glass prod access via existing privileged access management—not Claude config files

D2. Approved connector set

Start small: issue tracker, code host, internal docs search, CI status. Add SaaS MCPs only after vendor review (file 03). Document the data classes that each connector may send.

D3. Secret hygiene training (mandatory)

- Never paste API keys into prompts
- Use vault + short-lived tokens
- Assume the system may retain transcripts per policy
- Redact in screenshots for support

Part E — Developer productivity workflows

E1. High-ROI assistant use cases

Workflow	How AI helps	Guardrail
Codebase onboarding	Explain modules, find examples	Cite files. Verify
Feature scaffolding	Boilerplate + tests	Human review. Architecture fit
Bug investigation	Trace logs ↔ code	No prod writes
Test generation	Edge cases	Still run in CI
PR description / changelog	Summarize diffs	Author accountable
Doc drift fixes	Update READMEs	Review for accuracy
Refactors	Mechanical transforms	Tests required

E2. Anti-patterns

- Accept large AI diffs without a read
- Skip tests because “the model said it passes”
- Let an agent force-push / rewrite git history unsafely
- Paste customer PII into prompts for debugging
- Generate crypto/auth code without expert review

E3. Review culture upgrades

AI raises the **volume** of diffs. Raise review standards:

- Require tests for AI-touched auth/payment paths
- Ownership annotations
- Smaller PRs
- Threat-model checklist on tool/MCP changes

E4. Measuring productivity without false metrics

Do not use “lines of code” vanity metrics. Prefer:

- Lead time for changes
- Change fail rate
- Perceived toil reduction surveys
- Onboarding time to first meaningful PR

Check team maturity and concurrent process changes.

Part F — Operational enablement & runbooks

F1. AI-assisted ops (safe pattern)

Use assistants to:

- Summarize alert storms
- Correlate logs (within policy)
- Draft incident timelines
- Suggest dashboard queries
- Draft postmortem outlines

Do **not** give unrestricted cloud admin to chat agents at the start.

F2. Runbook template for AI features

For each production Claude feature (product runtime—not only IDE):

1. Symptoms
2. Dashboards
3. Feature flag / pin rollback
4. Distinguishing model vs retrieval vs tool failure
5. Safety incident fork → security IR
6. Customer comms pointer
7. Escalation graph

Enablement means on-call staff can execute. They do not page the original architect every time.

F3. Support patterns for builder tooling

Tier	Handler	Examples
L1	IT/helpdesk	Install, SSO, license
L2	Platform AI enablement	Settings, MCP allowlist requests
L3	Security + architect	Suspected exfil, malicious Skill

Knowledge base: golden CLAUDE.md examples, FAQ, known failure modes, request form for new MCP.

F4. Debugging productivity loops

Teach builders a standard loop:

1. Reproduce
2. Ask assistant with **minimal** sensitive context
3. Verify against tests
4. Document Skill if pattern recurs

Part G — Training & onboarding curriculum

G1. Role-based paths

Role	Curriculum focus
IC developer	CLAUDE.md, prompt hygiene, PR review with AI, secrets
Tech lead	Skills authoring, hooks, metrics, team norms
SRE/ops	Runbooks, safe ops assistants, rollback
Security champion	MCP threat model, settings enforcement, audits
PM/EM	Lifecycle metrics, expectation management (file 04)

G2. Onboarding week outline

- Day 1: Account, SSO, policy acknowledgment
- Day 2: Tour paved road Skills. Complete lab in sandbox repo
- Day 3: Pair on real task with review checklist
- Day 4: Secret hygiene + injection awareness mini-module
- Day 5: Office hours. Feedback form

G3. Labs (original ideas)

1. Add a Skill for “generate ADR from template”
2. Configure a deny-list hook for `.env` commits
3. Use MCP docs search to answer architecture Q with citations
4. Deliberately break tenancy in a toy RAG and fix filters (with 02)
5. Practice rollback of a prompt pin in staging

G4. Champions network

Volunteer champions per org unit reduce L2 load. They meet biweekly to share Skills and incidents. Architects start the first Skill library.

Part H — Governance of enablement itself

- Quarterly access review of who can publish marketplace items
 - Changelog for org-managed settings
 - Track exceptions (teams with temporary elevated tools)
 - Align with SDLC policies in file 03
 - Retire unused Skills to reduce attack surface
-

Decision trees

Enforce vs recommend

Could a developer override cause data leak or prod damage?

YES → Org-managed enforced setting / hook / deny tool

NO → Recommended project settings / CLAUDE.md guidance

New MCP request

Vendor review + least scopes + stage credentials + Skill docs +

ALL YES → Marketplace publish

ELSE → Reject or sandbox spike only

Exam traps

1. Recommended repo settings as a substitute for org-enforced policy.

2. Allowing arbitrary plugin installs then “audit later.”
 3. Production admin credentials in IDE MCP configs.
 4. Long CLAUDE.md files full of secrets.
 5. Measuring only seat adoption, not safety or outcomes.
 6. AI ops bot with unrestricted cloud IAM.
 7. No support tier / knowledge base—architect as permanent L1.
 8. Skills without owners or review.
 9. Training only on prompts, never on hooks/settings.
 10. Agents amplify a force-push culture.
-

Practice Q&A (28)

Q1. How do you enforce settings developers cannot override? **A.** Use organization/server-managed settings at highest precedence.

Q2. Why is “put recommended settings in git” insufficient for mandatory security? **A.** Teams can ignore or change them. Managed settings cannot be overridden.

Q3. What should a managed marketplace distribute? **A.** Approved Skills, hooks, agents, MCP servers (versioned).

Q4. Where do repo-specific build commands belong? **A.** Versioned project instructions (e.g., CLAUDE.md), reviewed in PRs.

Q5. Name two hard controls beyond markdown guidance. **A.** Hooks (secret scan/format) and org-managed deny lists / tool policies.

Q6. Why separate builder credentials from prod write keys? **A.** Limit blast radius of prompt injection or mistaken agent actions.

Q7. Good enablement metric beyond seats? **A.** Time-to-first-merged-PR, secret incidents, % approved MCP only, CSAT.

Q8. What is a Skill useful for in teams? **A.** Packaging recurring workflows for consistency and reuse.

Q9. Who should publish to the org marketplace? **A.** Controlled publishers with review—not every developer by default.

Q10. AI-generated auth code requires what extra step? **A.** Expert human review (+ tests). Do not trust the output without a check.

Q11. L1 vs L2 support for Claude Code? **A.** L1 install/SSO. L2 platform settings/MCP allowlist requests.

- Q12.** What belongs in an AI feature on-call runbook? **A.** Rollback pins/flags, dashboards, failure differentiation, escalation.
- Q13.** Why teach injection awareness to developers using MCP? **A.** Untrusted content can steer agents that hold tools.
- Q14.** Vanity productivity metric to avoid? **A.** Raw lines of code generated.
- Q15.** What is a champions network for? **A.** Federated enablement, Skill sharing, reduced central ticket load.
- Q16.** Can personal settings weaken org security policy? **A.** They must not. If you manage settings correctly, org policy wins.
- Q17.** First step when a team requests a new SaaS MCP? **A.** Security/vendor review + scope minimization (not instant install).
- Q18.** How do hooks complement CLAUDE.md? **A.** Hooks enforce deterministic checks. Markdown is advisory context.
- Q19.** What training do tech leads need beyond ICs? **A.** Skills authoring, hooks, team norms, metrics.
- Q20.** Why redact customer PII before you debug with AI? **A.** Privacy/policy compliance. You reduce retention and leak risk.
- Q21.** Break-glass prod access should go through what? **A.** Existing PAM/privileged workflows—not long-lived keys in agent config.
- Q22.** What does % repos with CLAUDE.md show? **A.** Standardization of agent guidance (leading enablement indicator).
- Q23.** Risk of unowned Skills? **A.** Drift, abandonment, potential malicious stale instructions.
- Q24.** How should AI affect PR size norms? **A.** Keep PRs reviewable. Do not merge huge unread AI changes.
- Q25.** Office hours purpose in onboarding? **A.** Unblock pilots. Gather friction for paved-road fixes.
- Q26.** What changelog should platform keep? **A.** Org-managed settings and marketplace item versions.
- Q27.** Is this domain only about IDE tooling? **A.** Primarily builder productivity + ops enablement. Product runtime ops runbooks also matter.
- Q28.** Select TWO for org-wide enforcement of approved tools: managed marketplace + server-managed settings—why both? **A.** Marketplace

distributes capabilities. Managed settings enforce policy/source restrictions.

Checklist

- ☐ I can explain enforced vs recommended settings
 - ☐ I can design a phased Claude Code rollout with exit criteria
 - ☐ I can outline CLAUDE.md + Skills + hooks together
 - ☐ I can specify marketplace governance
 - ☐ I can separate builder vs prod credentials
 - ☐ I can name enablement metrics beyond adoption seats
 - ☐ I can draft L1/L2/L3 support boundaries
 - ☐ I can write a rollback-oriented runbook section
 - ☐ I can sketch a 5-day onboarding plan
 - ☐ I can answer “Select TWO” enforcement questions confidently
-

Glossary

Term	Meaning
CLAUDE.md	Project instruction file for Claude Code-style agents
Skill	Reusable instruction/resource pack for recurring tasks
Hook	Deterministic automation around agent/dev lifecycle events
Managed settings	Org-level enforced configuration (highest precedence)
Marketplace	Catalog for distributing approved plugins/Skills/MCP
Paved road	Supported standard path for teams
PAM	Privileged access management
Enablement metrics	Measures of safe productivity adoption
Champions network	Federated volunteer experts per team
Break-glass	Emergency privileged access with audit

Term	Meaning
Inner source	Internal open-source style sharing of Skills/tools
Change fail rate	DORA-style metric of deployment failures
Lead time	Time from commit to production
Telemetry baseline	Required logging/analytics for security/ops
Sandbox repo	Safe training environment without prod data

Part I — Deep dive: designing the paved road

I1. Reference architecture for corp Claude Code

IdP (SSO) → Claude Code enterprise → Org-managed settings

↓

Managed marketplace
(Skills, hooks, MCP)

↓

Dev/stage credentials (vault)

↓

Optional: AI gateway for API use

Logging sinks to SIEM per security baseline. You track exceptions in ticketing.

I2. Sample org policies (illustrative)

1. Only marketplace MCP servers on managed devices.
2. No production write scopes in developer agents.
3. All repos owning services must include CLAUDE.md with test commands.
4. Skills that touch auth/payment require security co-review.
5. Agent shell deny: terraform apply in prod dirs without ticket ID env.

I3. Migration from shadow ChatGPT usage

- Provide equivalent convenience (docs MCP, issue MCP)

- Clear data-handling comparison sheet
- Amnesty window to rotate pasted secrets
- Monitor outbound to unsanctioned AI domains if DLP supports

14. Multi-repo monorepo notes

Monorepos need directory-scoped guidance. Hooks must fire only on touched paths for performance. Shared Skills for language ecosystems (Go/Java/TS) reduce duplication.

15. Compliance evidence from enablement

Auditors may ask how you control AI coding tools. Keep: settings screenshots/exports, marketplace ACL lists, training completion records, exception register, incident tickets.

16. Extended scenarios

Scenario: Org must prevent developers from overriding security settings. The org must also distribute approved workflows. **Answer shape:** Server-managed settings + managed plugin marketplace (common Select TWO). Not periodic audits of local installs alone. Not repo-only recommendations.

Scenario: SRE wants Claude to auto-remediate kube pods in prod. **Answer shape:** Start with read-only diagnose + suggested kubectl. Use human approval. Later, use narrow automations with PAM. Never put a broad admin kubeconfig in the IDE.

Scenario: New hire floods support with “how do I connect Jira MCP.” **Answer shape:** Improve onboarding lab + L1 KB article + champions. Do not custom-configure each laptop manually as a permanent process.

17. Q&A 29–40

Q29. Draw the control plane pieces for corp Claude Code. **A.** SSO, org-managed settings, marketplace, vault credentials, logging/SIEM.

Q30. Why directory-scoped hooks in monorepos? **A.** Performance and relevance—do not run unrelated policy on every edit.

Q31. Amnesty window means? **A.** Period for rotating secrets previously pasted into unsanctioned tools. Do not punish people in this period.

- Q32.** Select TWO common enforcement actions? **A.** Managed settings + managed marketplace (per common practice-exam pattern).
- Q33.** Why co-review Skills for payments? **A.** Higher abuse impact. Instructions could coax unsafe code patterns.
- Q34.** What evidence shows training worked? **A.** Lab completion, fewer L1 tickets, fewer secret incidents, survey confidence.
- Q35.** Auto-remediate prod—first architecture? **A.** Suggest-only with human approve. PAM. Narrow later.
- Q36.** What belongs in an exception register? **A.** Team, elevated tool, expiry, approver, compensating controls.
- Q37.** How does an AI gateway help builders calling Messages API? **A.** Central keys, quotas, redaction, attribution—aligned with file 02.
- Q38.** Why not measure success only by plugin install counts? **A.** Installs ≠ safe or valuable use. Track outcomes and incidents.
- Q39.** What is inner source of Skills? **A.** Teams contribute reusable Skills via review to a shared catalog.
- Q40.** Who is accountable for org-managed settings changes? **A.** Platform/security owners with change tickets. Do not use ad-hoc admin clicks that you do not document.

Extended checklist

- ☐ I can sketch corp Claude Code control plane
 - ☐ I can write five illustrative org AI-coding policies
 - ☐ I can plan shadow-IT migration with amnesty
 - ☐ I can answer Select-TWO enforcement questions
 - ☐ I can design suggest-only SRE automation first
 - ☐ I can list auditor evidence for AI coding controls
-

Glossary additions

Term	Meaning
Control plane	Settings/marketplace/IAM governing assistants

Term	Meaning
Exception register	Tracked temporary policy deviations
Amnesty window	Coordinated secret rotation period
Directory-scoped	Rules applying to repo subtrees
Suggest-only	Assistant proposes. Human executes
SIEM sink	Security log destination
Co-review	Dual ownership review (e.g., eng+security)
DORA metrics	Elite engineering performance metrics set
KB	Knowledge base for support
Federated enablement	Central standards + local champions

Part J — Enablement exam drill & labs detail

J1. “Select TWO” pattern family

Whenever the stem says *enforce + distribute approved tools*, look for:

- Org/server-managed settings
- Managed marketplace / approved catalog

Distractors: periodic audits, repo-only recommendations, project-level optional configs, unconstrained installs.

J2. Lab write-up: secrets hook

Goal: agent cannot commit .env. Steps: add a hook that scans staged files. Deny on pattern. Document in CLAUDE.md as a soft warning too. Prove with an attempted commit in sandbox. Record an evidence screenshot for the training KB.

J3. Lab write-up: Skill for ADR

Skill provides ADR template + “ask for context/decision/alternatives/consequences”. It outputs markdown path /docs/adr/XXXX.md. CI checks ADR filename pattern on certain PRs.

J4. Productivity appearance vs real gains

If cycle time drops but change-fail rate spikes, AI may accelerate bad changes. Tighten review on sensitive paths. Enablement owners watch **joint** metrics.

J5. Q&A 41–48

Q41. Common distractor for enforcement questions? **A.** “Audit local configs periodically” but you do not prevent bad installs.

Q42. What twin controls appear in Select TWO org rollout stems? **A.** Managed settings + managed marketplace.

Q43. Why track change-fail rate with AI adoption? **A.** Velocity without quality may increase incidents.

Q44. ADR Skill should produce what artifact property? **A.** Versioned markdown under a known docs path with required sections.

Q45. Soft warning in CLAUDE.md vs hook—difference? **A.** Warning is advisory. A hook can block deterministically.

Q46. Who maintains the training KB articles for Jira MCP? **A.** Enablement/platform L2—not each squad reinventing.

Q47. Sensitive path review policy example? **A.** Extra human review (+ tests) for auth/payments touched by AI diffs.

Q48. What evidence closes a secrets-hook lab? **A.** Documented failed commit attempt + hook config reviewed into the paved road.

This completes the five-file user domain set. Return to README for the official-7 mapping.

Part K — Rapid review (Enablement 7%)

Even with ~4 questions, missed points still cost you. Drill:

1. **Enforce** → org/server-managed settings (not git recommendations alone).
2. **Distribute approved tools** → managed marketplace.
3. **CLAUDE.md** → versioned guidance. **hooks** → hard checks.
4. **No prod write keys** in builder MCP.
5. **Metrics** beyond seats. Watch fail rate with velocity.
6. **Support tiers** + KB + champions.

7. **Training** role-based with labs and secret hygiene.
8. **Select TWO** stems often pair managed settings + marketplace.

If a scenario says “developers must not override,” remove any answer that relies on goodwill or periodic audits as the primary control.

Part L — Sample paved-road checklist (copy/adapt)

- ☐ SSO enforced for Claude Code seats
- ☐ Org-managed settings documented & change-controlled
- ☐ Marketplace ACL: publishers listed. Consumers read-only install
- ☐ Default deny for non-marketplace MCP
- ☐ Vault pattern doc for dev credentials
- ☐ Golden CLAUDE.md template in platform repo
- ☐ Secret-scan hook enabled by default
- ☐ Onboarding lab in sandbox repo
- ☐ L1/L2 runbooks published
- ☐ Quarterly access review scheduled
- ☐ Exception register exists with expiry
- ☐ Metrics dashboard: adoption, incidents, cycle time, fail rate

This completed list is a stronger “enablement architecture” than any single prompt tip. When you are not sure on CCAR-P, select the answer that builds this control plane.

(File continues — deep-dive Part M follows below.)

Part M — Primary-study deep dive: Paved-road enablement

Domain weight is only **7%**, but items are concrete: settings precedence, secure defaults, CLAUDE.md/Skills/hooks, approved MCP/plugins, and ops debugging patterns. Wrong answers usually violate **least privilege**. They also confuse **builder tooling** with **production runtime**.

M1. Control-plane mental model

Org managed settings (non-overridable security)

- Team / enterprise defaults
- Repo CLAUDE.md + Skills (versioned)
- User local settings (where allowed)
- Session prompts

Exam rule: Security-relevant controls belong at the **highest precedence that users cannot bypass**. You can recommend productivity niceties as defaults.

M2. What to enforce vs recommend (expanded)

Control	Enforce org-wide	Recommend
Secret exfiltration guards / hooks	Yes	—
Disallow unapproved MCP/marketplaces	Yes	—
Block pasting .env / keys patterns	Yes	—
Require SSO	Yes	—
Default model for interactive	Often	Allow power users if policy OK
Commit message style Skill	—	Yes
Aggressive auto-run of shell	Deny by default	Enable per trusted repo carefully
Telemetry / audit of tool use	Yes where lawful	—

M3. CLAUDE.md design guide

Good org/repo instruction files: - Architecture map and service boundaries - Test commands and “definition of done” - Security never-dos (prod creds, raw PII) - How to propose MCP/Skill additions - Links to runbooks—not secrets

Bad files: - Entire style guides pasted twice - Credentials - Contradictions vs org managed policy - Unmaintained guidance from 18 months ago

Layering: Org baseline → repo specifics → personal preferences (if allowed). Test precedence with a deliberate conflict in staging.

M4. Skills for teams

Skill type	Example	Governance
ADR writer	Structured decision record	Reviewed like docs

Skill type	Example	Governance
Test generator	Framework-specific	No prod network tools
Threat-model stub	STRIDE prompts	Security co-owned
Migrate-langX	Internal frameworks	Versioned. Owners

Skills that fetch remote content inherit **prompt-injection** risk. Treat them like MCP.

M5. Hooks that give real value

High-ROI hooks (illustrative patterns from public product themes): - Pre-tool: block commands matching secret/exfil patterns - Pre-commit assistant: remind tests - Stop: summarize session for handoff notes

Do not add many hooks that slow every keystroke without security value.

M6. MCP for builders ≠ MCP for production agents

Dimension	Builder (Claude Code)	Production agent
Creds	Dev/sandbox roles	Scoped service roles
Blast radius	Repo / dev accounts	Customer data / money
Change speed	Fast inner loop	Change-controlled
Approval	Marketplace allowlist	Architecture + security review

Never reuse unrestricted production credentials in developer MCP “for convenience.”

Part N — Productivity workflows that pass review

N1. High-ROI loops (with review gates)

- 1. **Explain** → **propose** → **patch** → **test** → **PR summary** with human review.
- 2. **Failing test** → **root cause hypotheses** → **minimal fix**.
- 3. **Legacy module** → **characterization tests before refactor**.
- 4. **Incident stub** → **timeline draft from logs (redacted)** → **human edit**.

N2. Anti-patterns

Anti-pattern	Why bad	Replace with
Accept all diffs without a read	Ships vulns/secrets	Mandatory review culture
Paste prod dumps into chat	Data leakage	Redacted fixtures
Metric = lines generated	False signal	Cycle time and change fail rate
Untracked Skills with tools	Supply chain	Managed marketplace
One shared API key on laptop	Attribution/abuse	SSO + per-user

N3. Measuring productivity with true metrics

Prefer: PR cycle time, review iterations, escaped defects, secret-scan incidents, developer satisfaction, time-to-first-productive-PR for new hires. Distrust: raw acceptance rate, token spend as “value,” lines of AI code.

N4. Debugging & operational enablement

Safe pattern: AI assists on **read-only** diagnostics. Remediations go through existing change systems. Provide runbooks the assistant can retrieve. Do not grant cloud-admin MCP to every engineer’s laptop agent.

Runbook sections: symptoms → gather (commands) → hypothesize → mitigate → verify → escalate → postmortem links.

Part O — Training curriculum (original outline)

O1. Role paths

Role	Must learn
IC engineer	CLAUDE.md, safe MCP, PR review with AI diffs
Tech lead	Skills governance, eval mindset for assistants
SRE/ops	Guardrailed debug loops, no unrestricted production keys

Role	Must learn
Security	Hook policies, marketplace review, DLP
PM/adjacent	What AI can/cannot promise. How to file good bugs

O2. Onboarding week (example)

Day 1: SSO + policy + secret hygiene lab. Day 2: Repo CLAUDE.md tour + first assisted PR. Day 3: MCP approved set only. Request process. Day 4: Incident tabletop with AI assist (read-only). Day 5: Office hours. Feedback into paved road backlog.

O3. Champions network

Volunteers per org unit. Office hours. Inner-source Skills. Escalate friction to platform—not shadow SaaS.

Part P — Architecture decision scenarios (Enablement)

P1. Shadow consumer ChatGPT usage discovered

Response: Do not only ban. **Make a paved road:** SSO Claude tooling, clear data rules, migrate use cases, measure adoption, security exceptions time-boxed.

P2. Team wants “install any MCP from the internet”

Response: Managed marketplace / allowlist. Threat review. Network egress controls. Owners. Versions pinned.

P3. Conflict: repo CLAUDE.md says auto-run shell. Org policy forbids

Response: Org managed settings win. Fix the repo file. Educate. Alert on violations.

P4. Vanity-metrics dashboard

Response: Replace lines-generated with quality/velocity/safety metrics. Use qualitative interviews.

Part Q — Failure-mode analysis (enablement)

Failure	Signal	Mitigate
Settings bypass	Local overrides weaken DLP	Enforce managed non-overridable controls
Secret in PR via AI	Scan hits	Hooks + training + vault
Skill supply-chain	Unexpected network tool	Signing/review. Marketplace only
Support overload	Tickets per seat ↑	Better docs. Champions. UX fixes
Repo instruction drift	Contradictory guidance	Owners + CI lint for CLAUDE.md
Prod creds in dev MCP	Audit findings	Separate IdP apps / roles

Part R — Production checklists

R1. Before org pilot

- ☐ Managed security settings defined
- ☐ SSO live
- ☐ Marketplace policy live
- ☐ Secret hygiene training mandatory
- ☐ Support channel + SLA
- ☐ Success metrics agreed (incl. safety)

R2. Before org-wide expand

- ☐ Pilot friction log closed or accepted
- ☐ Reference CLAUDE.md templates published
- ☐ Hooks validated (false positive rate OK)
- ☐ Access review process scheduled
- ☐ Incident tabletop done

R3. Quarterly

- ☐ Plugin/MCP inventory review
 - ☐ Settings drift audit
 - ☐ Curriculum update for new model/tooling
 - ☐ Productivity + safety metrics review
-

Part S — Trade-off tables

S1. Speed of enablement vs control

Approach	Speed	Control	When
Uncontrolled local tools	Fast	Poor	Never for enterprise
Recommended defaults only	Medium	Medium	Low-risk orgs
Managed enforce + paved Skills	Medium	High	Default CCAR-P answer
Lockdown with no paved road	Slow adoption	High on paper	Creates shadow IT

S2. Central platform vs federated team Skills

Centralize security and core Skills. Federate domain Skills with review. Pure central work becomes a bottleneck. Pure federated work becomes chaos. Hybrid wins.

Part T — Extended Q&A (49–60)

Q49. Highest-precedence place for “deny unapproved MCP”? **A. Org managed / non-overridable** settings—not a hopeful CLAUDE.md line alone.

Q50. Select TWO enablement success metrics: secret-scan incidents, seat count only, time-to-first-merged-PR, raw tokens burned. **A.** Secret-scan incidents and time-to-first-merged-PR.

Q51. Builder MCP with prod database admin—acceptable? **A. No**—use sandbox/read-only roles. Prod changes go via controlled runtime.

Q52. Hooks vs Skills—security blocking of exfil commands? **A. Prefer hooks/enforced controls.** Skills are instructions, not hard enforcement.

Q53. Shadow AI usage—best architect move? **A. Paved road + policy + migrate, not ban-only.**

Q54. CLAUDE.md contains API keys—action? **A. Rotate keys, purge history, teach vault. Add secret scanning.**

Q55. Select THREE rollout phases: design controls, pilot, expand, skip security until GA, email passwords to team. **A. Design, pilot, expand.**

Q56. Why separate production agent creds from Claude Code MCP? **A. Different blast radius, change control, and attribution needs.**

Q57. Productivity ineffective activity example? **A. You celebrate lines of AI-generated code while change-fail rate rises.**

Q58. Managed plugin marketplace purpose? **A. Distribute approved tools/Skills. Reduce supply-chain risk.**

Q59. Ops wants AI to “just fix prod.” Safe pattern? **A. Read-only diagnosis + human/change-managed remediation.**

Q60. Repo instruction conflicts with org deny-list—who wins? **A. Org enforced settings. Fix the repo file.**

Part U — Rapid review (Enablement 7%)

- Enforce security high in the settings hierarchy.
- CLAUDE.md/Skills versioned. No secrets.
- Managed marketplace for MCP/plugins.
- Builder tooling ≠ production runtime creds.
- Measure velocity + safety, not vanity.
- Hooks for hard stops. Skills for guidance.
- Paved road beats shadow AI.
- Train by role. Champions sustain adoption.

Cross-link: MCP deep dive 02. Safety 03. Rollout comms 04.

05 — Team Enablement & Operational Productivity

CCAR-P condensed domain 5 (dedicated) Official domain mapped here: Developer Productivity & Operational Enablement (7%) **Approx. questions:** ~4 of 63 — fewer items, but scenarios are concrete (org rollout, settings hierarchy, secure defaults). **Cross-links:** Architecture patterns in 01; MCP/tools in 02; safety baselines in 03; lifecycle/comms in 04.

Disclaimer

Original enablement notes grounded in **public** Claude Code / Anthropic product documentation themes (project instructions, MCP, plugins/marketplaces, settings hierarchies, hooks) and standard enterprise developer-productivity practice. **Not** official course content. Product UI names and setting keys evolve—verify against current public Claude Code docs before production rollout. Independent pack; not affiliated with Anthropic.

Overview

The 7% domain punches above its weight because wrong answers are obvious if you know the **control plane** for teams: how to roll out Claude Code (or similar builder tooling) with **enforced security**, **shared skills**, **standardized workflows**, and **operational runbooks**—without turning every developer into an unsupervised admin of production credentials.

This file covers:

1. Team rollout strategy for Claude Code / AI coding assistants
 2. Org instruction files (CLAUDE.md and related), Skills, hooks
 3. Managed settings vs recommended settings vs local overrides
 4. Plugin/marketplace distribution of approved tools
 5. Productivity workflows (code, test, docs, debug)
 6. Operational support patterns & runbooks
 7. Training and onboarding curricula for builders
-

Key map

Enablement task	Exam signal
Enforce org security settings	Server-managed / highest-precedence controls that users cannot override
Distribute approved Skills/MCP/plugins	Managed marketplace / catalog—not “install anything”
Standardize repo guidance	CLAUDE.md / project instructions versioned in git
Raise velocity safely	AI for code/test/docs with review gates
Ops debugging with AI	Guardrailed assistants + runbooks—not prod god keys
Train builders	Role-based onboarding + paved road

Part A — Rollout strategy

A1. Why architects own enablement design

If platform/security doesn’t pave a road, teams adopt consumer AI with paste-secrets habits. Architects design:

- Identity (SSO) into approved tooling
- Data handling rules (what code/secrets may be sent)
- Approved MCP/plugin set
- Secrets via vault—not chat
- Metrics for adoption **and** incidents

A2. Phased rollout

Phase	Population	Goal
0 Design	Platform + security	Settings, marketplace, logging
1 Pilot	1–2 volunteer teams	Friction log; productivity signals
2 Expand	All eng with training	Default secure config
3 Optimize	Org-wide	Skills library; inner source
4 Continuous	—	Version upgrades; access reviews

Exit criteria per phase: security findings closed; support queue manageable; measured useful adoption (not seat count alone).

A3. Success metrics for enablement

Metric	Intent
Time-to-first-merged-PR with assistant	Onboarding efficacy
PR cycle time / review iterations	Velocity (careful with confounders)
Secret-scan incidents involving AI tools	Safety
% repos with CLAUDE.md	Standardization
% MCP from approved marketplace only	Supply-chain hygiene
Support tickets per 100 seats	Enablement quality
Developer satisfaction (survey)	UX of paved road

Part B — CLAUDE.md, Skills, and project instructions

B1. What org instruction files are for

Public Claude Code docs describe project guidance files (commonly CLAUDE.md or equivalent project instructions) that orient the coding agent to:

- Repo structure & conventions
- Build/test commands
- Code style & review expectations
- Forbidden actions (e.g., don't touch prod/ without ticket)
- How to run linters/security scanners

Architect guidance: Treat these as **versioned product code**. Review in PRs. Keep secrets out. Link to deeper docs rather than pasting novels (context budget).

B2. Layered instructions

Layer	Example	Scope
Org default	Company coding standards	Skill All repos
Repo CLAUDE.md	Service-specific architecture	One repo
Directory hints	/payments stricter rules	Subtree
Personal	User preferences	Local only—must not we

Security-critical rules belong in **enforced settings**, not only markdown that a model might under-weight.

B3. Skills for teams

Skills package reusable instructions/resources for recurring tasks: “add OpenTelemetry span,” “write ADR,” “migrate endpoint.” Benefits:

- Consistency across humans
- Smaller per-turn prompts
- Reviewable artifacts

Governance: Skill review owners; semver; deprecate; security scan for exfil instructions.

B4. Hooks

Hooks (automation around agent lifecycle events—formatters, secret scanners, policy checks) enforce **deterministic** controls where prompts are soft:

- Pre-commit / pre-push style checks triggered in agent loops
- Block commits containing secrets
- Auto-run formatters
- Require tests for certain paths

Exam theme: combine **soft guidance** (CLAUDE.md) with **hard hooks/settings**.

Part C — Settings hierarchy & enforcement

C1. Precedence mental model

Public Claude Code enterprise features describe a hierarchy where **server-managed / organization-managed settings** sit at the highest

precedence and **cannot be overridden** by project, user, local, or CLI flags. Lower layers provide defaults and team conventions.

Layer (conceptual)	Override by users?	Use for
Server / org-managed	No	Security policies, denied tools, env constraints
Marketplace restrictions	No/limited	Approved plugin sources
Shared project recommendations	Yes (unless managed)	Conventions
User/local	Yes	Personal DX

Exam trap: “Publish a recommended settings JSON in a repo and ask teams to copy it” — insufficient when the requirement is **enforce**. Prefer managed settings + managed marketplace.

C2. What to enforce org-wide

- Disallowed MCP sources / unchecked plugin installs
- Telemetry/logging baselines required by security
- Environment variable policies
- Command allow/deny lists for agent shell
- Network egress constraints where supported
- Requirement to use SSO-backed installs

C3. What to leave flexible

- Editor keybindings
- Non-security model preference within approved set
- Personal prompt snippets that don’t grant tools

C4. Managed plugin marketplace

Central catalog to distribute approved:

- Skills
- Agents
- Hooks
- MCP servers

Supports versioning, discovery, updates, and **source control**—vs each developer curling random GitHub. Pair with policy: only marketplace installs in corp devices.

Part D — MCP & credentials for builders

D1. Separation from production runtime

Builder environments must **not** reuse production write credentials. Provide:

- Dev/stage credentials with narrow scopes
- Read-only prod replicas where justified
- Break-glass prod access via existing privileged access management—not Claude config files

D2. Approved connector set

Start small: issue tracker, code host, internal docs search, CI status. Add SaaS MCPs only after vendor review (file 03). Document data classes allowed through each connector.

D3. Secret hygiene training (mandatory)

- Never paste API keys into prompts
 - Use vault + short-lived tokens
 - Assume transcripts may be retained per policy
 - Redact in screenshots for support
-

Part E — Developer productivity workflows

E1. High-ROI assistant use cases

Workflow	How AI helps	Guardrail
Codebase onboarding	Explain modules, find examples	Cite files; verify
Feature scaffolding	Boilerplate + tests	Human review; architecture fit

Workflow	How AI helps	Guardrail
Bug investigation	Trace logs ↔ code	No prod writes
Test generation	Edge cases	Still run in CI
PR description / changelog	Summarize diffs	Author accountable
Doc drift fixes	Update READMEs	Review for accuracy
Refactors	Mechanical transforms	Tests required

E2. Anti-patterns

- Accepting large AI diffs without reading
- Skipping tests because “the model said it passes”
- Letting agent force-push / rewrite git history unsafely
- Pasting customer PII into prompts for debugging
- Generating crypto/auth code without expert review

E3. Review culture upgrades

AI increases **volume** of diffs → raise review standards:

- Require tests for AI-touched auth/payment paths
- Ownership annotations
- Smaller PRs
- Threat-model checklist on tool/MCP changes

E4. Measuring productivity without lying

Be careful with “lines of code” vanity. Prefer:

- Lead time for changes
- Change fail rate
- Perceived toil reduction surveys
- Onboarding time to first meaningful PR

Control for team maturity and concurrent process changes.

Part F — Operational enablement & runbooks

F1. AI-assisted ops (safe pattern)

Use assistants to:

- Summarize alert storms
- Correlate logs (within policy)
- Draft incident timelines
- Suggest dashboard queries
- Draft postmortem outlines

Do **not** give unrestricted cloud admin to chat agents on day one.

F2. Runbook template for AI features

For each production Claude feature (product runtime—not only IDE):

1. Symptoms
2. Dashboards
3. Feature flag / pin rollback
4. Distinguishing model vs retrieval vs tool failure
5. Safety incident fork → security IR
6. Customer comms pointer
7. Escalation graph

Enablement means on-call can execute without paging the original architect every time.

F3. Support patterns for builder tooling

Tier	Handler	Examples
L1	IT/helpdesk	Install, SSO, license
L2	Platform AI enablement	Settings, MCP allowlist requests
L3	Security + architect	Suspected exfil, malicious Skill

Knowledge base: golden CLAUDE.md examples, FAQ, known failure modes, request form for new MCP.

F4. Debugging productivity loops

Teach builders a standard loop:

- 1. Reproduce
- 2. Ask assistant with **minimal** sensitive context
- 3. Verify against tests
- 4. Document Skill if pattern recurs

Part G — Training & onboarding curriculum

G1. Role-based paths

Role	Curriculum focus
IC developer	CLAUDE.md, prompt hygiene, PR review with AI, secrets
Tech lead	Skills authoring, hooks, metrics, team norms
SRE/ops	Runbooks, safe ops assistants, rollback
Security champion	MCP threat model, settings enforcement, audits
PM/EM	Lifecycle metrics, expectation management (file 04)

G2. Onboarding week outline

- Day 1: Account, SSO, policy acknowledgment
- Day 2: Tour paved road Skills; complete lab in sandbox repo
- Day 3: Pair on real task with review checklist
- Day 4: Secret hygiene + injection awareness mini-module
- Day 5: Office hours; feedback form

G3. Labs (original ideas)

- 1. Add a Skill for “generate ADR from template”
- 2. Configure a deny-list hook for `.env` commits
- 3. Use MCP docs search to answer architecture Q with citations
- 4. Deliberately break tenancy in a toy RAG and fix filters (with 02)
- 5. Practice rollback of a prompt pin in staging

G4. Champions network

Volunteer champions per org unit reduce L2 load; they meet biweekly to share Skills and incidents. Architects seed the first Skill library.

Part H — Governance of enablement itself

- Quarterly access review of who can publish marketplace items
 - Changelog for org-managed settings
 - Track exceptions (teams with temporary elevated tools)
 - Align with SDLC policies in file 03
 - Retire unused Skills to reduce attack surface
-

Decision trees

Enforce vs recommend

Could a developer override cause data leak or prod damage?

YES → Org-managed enforced setting / hook / deny tool

NO → Recommended project settings / CLAUDE.md guidance

New MCP request

Vendor review + least scopes + stage credentials + Skill docs +

ALL YES → Marketplace publish

ELSE → Reject or sandbox spike only

Exam traps

1. Recommended repo settings as a substitute for org-enforced policy.
 2. Allowing arbitrary plugin installs then “audit later.”
 3. Production admin credentials in IDE MCP configs.
 4. CLAUDE.md novels full of secrets.
 5. Measuring only seat adoption, not safety or outcomes.
 6. AI ops bot with unrestricted cloud IAM.
 7. No support tier / knowledge base—architect as forever L1.
 8. Skills without owners or review.
 9. Training only on prompts, never on hooks/settings.
 10. Force-push culture amplified by agents.
-

Practice Q&A (28)

Q1. How do you enforce settings developers cannot override? **A.** Use organization/server-managed settings at highest precedence.

Q2. Why is “put recommended settings in git” insufficient for mandatory security? **A.** Teams can ignore or change them; managed settings cannot be overridden.

Q3. What should a managed marketplace distribute? **A.** Approved Skills, hooks, agents, MCP servers (versioned).

Q4. Where do repo-specific build commands belong? **A.** Versioned project instructions (e.g., `CLAUDE.md`), reviewed in PRs.

Q5. Name two hard controls beyond markdown guidance. **A.** Hooks (secret scan/format) and org-managed deny lists / tool policies.

Q6. Why separate builder credentials from prod write keys? **A.** Limit blast radius of prompt injection or mistaken agent actions.

Q7. Good enablement metric beyond seats? **A.** Time-to-first-merged-PR, secret incidents, % approved MCP only, CSAT.

Q8. What is a Skill useful for in teams? **A.** Packaging recurring workflows for consistency and reuse.

Q9. Who should publish to the org marketplace? **A.** Controlled publishers with review—not every developer by default.

Q10. AI-generated auth code requires what extra step? **A.** Expert human review (+ tests); don’t trust blindly.

Q11. L1 vs L2 support for Claude Code? **A.** L1 install/SSO; L2 platform settings/MCP allowlist requests.

Q12. What belongs in an AI feature on-call runbook? **A.** Rollback pins/flags, dashboards, failure differentiation, escalation.

Q13. Why teach injection awareness to developers using MCP? **A.** Untrusted content can steer agents that hold tools.

Q14. Vanity productivity metric to avoid? **A.** Raw lines of code generated.

Q15. What is a champions network for? **A.** Federated enablement, Skill sharing, reduced central ticket load.

Q16. Can personal settings weaken org security policy? **A.** They must not—if managed correctly, org policy wins.

Q17. First step when a team requests a new SaaS MCP? **A.** Security/vendor review + scope minimization (not instant install).

Q18. How do hooks complement CLAUDE.md? **A.** Hooks enforce deterministic checks; markdown is advisory context.

Q19. What training do tech leads need beyond ICs? **A.** Skills authoring, hooks, team norms, metrics.

Q20. Why redact customer PII before debugging with AI? **A.** Privacy/policy compliance; reduce retention and leak risk.

Q21. Break-glass prod access should go through what? **A.** Existing PAM/privileged workflows—not long-lived keys in agent config.

Q22. What does % repos with CLAUDE.md indicate? **A.** Standardization of agent guidance (leading enablement indicator).

Q23. Risk of unowned Skills? **A.** Drift, abandonment, potential malicious stale instructions.

Q24. How should AI affect PR size norms? **A.** Keep PRs reviewable; don't merge huge unread AI dumps.

Q25. Office hours purpose in onboarding? **A.** Unblock pilots; gather friction for paved-road fixes.

Q26. What changelog should platform keep? **A.** Org-managed settings and marketplace item versions.

Q27. Is this domain only about IDE tooling? **A.** Primarily builder productivity + ops enablement; product runtime ops runbooks also matter.

Q28. Select TWO for org-wide enforcement of approved tools: managed marketplace + server-managed settings—why both? **A.** Marketplace distributes capabilities; managed settings enforce policy/source restrictions.

Checklist

- ☐ I can explain enforced vs recommended settings
- ☐ I can design a phased Claude Code rollout with exit criteria
- ☐ I can outline CLAUDE.md + Skills + hooks together
- ☐ I can specify marketplace governance
- ☐ I can separate builder vs prod credentials
- ☐ I can name enablement metrics beyond adoption seats

- ☐ I can draft L1/L2/L3 support boundaries
 - ☐ I can write a rollback-oriented runbook section
 - ☐ I can sketch a 5-day onboarding plan
 - ☐ I can answer “Select TWO” enforcement questions confidently
-

Glossary

Term	Meaning
CLAUDE.md	Project instruction file for Claude
Skill	Code-style agents
Hook	Reusable instruction/resource pack for recurring tasks
Managed settings	Deterministic automation around agent/dev lifecycle events
Marketplace	Org-level enforced configuration (highest precedence)
Paved road	Catalog for distributing approved plugins/Skills/MCP
PAM	Supported standard path for teams
Enablement metrics	Privileged access management
Champions network	Measures of safe productivity adoption
Break-glass	Federated volunteer experts per team
Inner source	Emergency privileged access with audit
Change fail rate	Internal open-source style sharing of Skills/tools
Lead time	DORA-style metric of deployment failures
Telemetry baseline	Time from commit to production
Sandbox repo	Required logging/analytics for security/ops
	Safe training environment without prod data

Part I — Deep dive: designing the paved road

I1. Reference architecture for corp Claude Code

IdP (SSO) → Claude Code enterprise → Org-managed settings

↓

Managed marketplace
(Skills, hooks, MCP)

↓

Dev/stage credentials (vault)

↓

Optional: AI gateway for API use

Logging sinks to SIEM per security baseline. Exceptions tracked in ticketing.

I2. Sample org policies (illustrative)

1. Only marketplace MCP servers on managed devices.
2. No production write scopes in developer agents.
3. All repos owning services must include `CLAUDE.md` with test commands.
4. Skills that touch auth/payment require security co-review.
5. Agent shell deny: `terraform apply` in prod dirs without ticket ID env.

I3. Migration from shadow ChatGPT usage

- Provide equivalent convenience (docs MCP, issue MCP)
- Clear data-handling comparison sheet
- Amnesty window to rotate pasted secrets
- Monitor outbound to unsanctioned AI domains if DLP supports

I4. Multi-repo monorepo notes

Monorepos need directory-scoped guidance and hooks that only fire on touched paths for performance. Shared Skills for language ecosystems (Go/Java/TS) reduce duplication.

I5. Compliance evidence from enablement

Auditors may ask how you control AI coding tools. Keep: settings screenshots/exports, marketplace ACL lists, training completion records, ex-

ception register, incident tickets.

16. Extended scenarios

Scenario: Org must prevent developers from overriding security settings while distributing approved workflows. **Answer shape:** Server-managed settings + managed plugin marketplace (classic Select TWO). Not periodic audits of local installs alone; not repo-only recommendations.

Scenario: SRE wants Claude to auto-remediate kube pods in prod. **Answer shape:** Start with read-only diagnose + suggested kubectl; human approval; eventual narrow automations with PAM; never broad admin kubeconfig in IDE.

Scenario: New hire floods support with “how do I connect Jira MCP.” **Answer shape:** Improve onboarding lab + L1 KB article + champions; don’t custom-configure each laptop manually forever.

17. Q&A 29–40

Q29. Draw the control plane pieces for corp Claude Code. **A.** SSO, org-managed settings, marketplace, vault credentials, logging/SIEM.

Q30. Why directory-scoped hooks in monorepos? **A.** Performance and relevance—don’t run unrelated policy on every edit.

Q31. Amnesty window means? **A.** Period for rotating secrets previously pasted into unsanctioned tools without punishment theater.

Q32. Select TWO classic enforcement actions? **A.** Managed settings + managed marketplace (per common practice-exam pattern).

Q33. Why co-review Skills for payments? **A.** Higher abuse impact; instructions could coax unsafe code patterns.

Q34. What evidence shows training worked? **A.** Lab completion, fewer L1 tickets, fewer secret incidents, survey confidence.

Q35. Auto-remediate prod—first architecture? **A.** Suggest-only with human approve; PAM; narrow later.

Q36. What belongs in an exception register? **A.** Team, elevated tool, expiry, approver, compensating controls.

Q37. How does an AI gateway help builders calling Messages API? **A.** Central keys, quotas, redaction, attribution—aligned with file 02.

- Q38.** Why not measure success only by plugin install counts? **A.** Installs ≠ safe or valuable use; track outcomes and incidents.
- Q39.** What is inner source of Skills? **A.** Teams contribute reusable Skills via review to a shared catalog.
- Q40.** Who is accountable for org-managed settings changes? **A.** Platform/security owners with change tickets—not ad-hoc admin clicks undocumented.
-

Extended checklist

- ☐ I can sketch corp Claude Code control plane
 - ☐ I can write five illustrative org AI-coding policies
 - ☐ I can plan shadow-IT migration with amnesty
 - ☐ I can answer Select-TWO enforcement questions
 - ☐ I can design suggest-only SRE automation first
 - ☐ I can list auditor evidence for AI coding controls
-

Glossary additions

Term	Meaning
Control plane	Settings/marketplace/IAM governing assistants
Exception register	Tracked temporary policy deviations
Amnesty window	Coordinated secret rotation period
Directory-scoped	Rules applying to repo subtrees
Suggest-only	Assistant proposes; human executes
SIEM sink	Security log destination
Co-review	Dual ownership review (e.g., eng+security)
DORA metrics	Elite engineering performance metrics set
KB	Knowledge base for support
Federated enablement	Central standards + local champions

Part J — Enablement exam drill & labs detail

J1. “Select TWO” pattern family

Whenever the stem says *enforce + distribute approved tools*, look for:

- Org/server-managed settings
- Managed marketplace / approved catalog

Distractors: periodic audits, repo-only recommendations, project-level optional configs, unconstrained installs.

J2. Lab write-up: secrets hook

Goal: agent cannot commit `.env`. Steps: add hook scanning staged files; deny on pattern; document in `CLAUDE.md` as soft warning too; prove with attempted commit in sandbox; record evidence screenshot for training KB.

J3. Lab write-up: Skill for ADR

Skill provides ADR template + “ask for context/decision/alternatives/consequences”; outputs markdown path `/docs/adr/XXXX.md`; CI checks ADR filename pattern on certain PRs.

J4. Productivity theater vs real gains

If cycle time drops but change-fail rate spikes, AI may be accelerating bad changes—tighten review on sensitive paths. Enablement owners watch **joint** metrics.

J5. Q&A 41–48

Q41. Classic distractor for enforcement questions? **A.** “Audit local configs periodically” without preventing bad installs.

Q42. What twin controls appear in Select TWO org rollout stems? **A.** Managed settings + managed marketplace.

Q43. Why track change-fail rate with AI adoption? **A.** Velocity without quality may increase incidents.

Q44. ADR Skill should produce what artifact property? **A.** Versioned markdown under a known docs path with required sections.

Q45. Soft warning in CLAUDE.md vs hook—difference? **A.** Warning is advisory; hook can block deterministically.

Q46. Who maintains the training KB articles for Jira MCP? **A.** Enablement/platform L2—not each squad reinventing.

Q47. Sensitive path review policy example? **A.** Extra human review (+ tests) for auth/payments touched by AI diffs.

Q48. What evidence closes a secrets-hook lab? **A.** Documented failed commit attempt + hook config reviewed into the paved road.

This completes the five-file user domain set. Return to README for the official-7 mapping.

Part K — Rapid review (Enablement 7%)

Even with ~4 questions, missable points hurt. Drill:

1. **Enforce** → org/server-managed settings (not git recommendations alone).
2. **Distribute approved tools** → managed marketplace.
3. **CLAUDE.md** → versioned guidance; **hooks** → hard checks.
4. **No prod write keys** in builder MCP.
5. **Metrics** beyond seats; watch fail rate with velocity.
6. **Support tiers** + KB + champions.
7. **Training** role-based with labs and secret hygiene.
8. **Select TWO** stems often pair managed settings + marketplace.

If a scenario mentions “developers must not override,” eliminate any answer relying on goodwill or periodic audits as the primary control.

Part L — Sample paved-road checklist (copy/adapt)

- ☐ SSO enforced for Claude Code seats
- ☐ Org-managed settings documented & change-controlled
- ☐ Marketplace ACL: publishers listed; consumers read-only install
- ☐ Default deny for non-marketplace MCP
- ☐ Vault pattern doc for dev credentials
- ☐ Golden CLAUDE.md template in platform repo
- ☐ Secret-scan hook enabled by default
- ☐ Onboarding lab in sandbox repo
- ☐ L1/L2 runbooks published
- ☐ Quarterly access review scheduled

- ☐ Exception register exists with expiry
- ☐ Metrics dashboard: adoption, incidents, cycle time, fail rate

Completing this list is a stronger “enablement architecture” than any single prompt tip. On CCAR-P, when in doubt, pick the answer that builds this control plane.

(File continues — deep-dive Part M follows below.)

Part M — Primary-study deep dive: Paved-road enablement

Domain weight is only **7%**, but items are concrete: settings precedence, secure defaults, CLAUDE.md/Skills/hooks, approved MCP/plugins, and ops debugging patterns. Wrong answers usually violate **least privilege** or confuse **builder tooling** with **production runtime**.

M1. Control-plane mental model

Org managed settings (non-overridable security)

- Team / enterprise defaults
- Repo CLAUDE.md + Skills (versioned)
- User local settings (where allowed)
- Session prompts

Exam rule: Security-relevant controls belong at the **highest precedence that users cannot bypass**. Productivity niceties can be recommended defaults.

M2. What to enforce vs recommend (expanded)

Control	Enforce org-wide	Recommend
Secret exfiltration guards / hooks	Yes	—
Disallow unapproved MCP/marketplaces	Yes	—
Block pasting .env / keys patterns	Yes	—
Require SSO	Yes	—
Default model for interactive	Often	Allow power users if policy OK

Control	Enforce org-wide	Recommend
Commit message style Skill	—	Yes
Aggressive auto-run of shell	Deny by default	Enable per trusted repo carefully
Telemetry / audit of tool use	Yes where lawful	—

M3. CLAUDE.md design guide

Good org/repo instruction files: - Architecture map and service boundaries - Test commands and “definition of done” - Security never-dos (prod creds, raw PII) - How to propose MCP/Skill additions - Links to runbooks—not secrets

Bad files: - Entire style guides pasted twice - Credentials - Contradictions vs org managed policy - Unmaintained lore from 18 months ago

Layering: Org baseline → repo specifics → personal preferences (if allowed). Test precedence with a deliberate conflict in staging.

M4. Skills for teams

Skill type	Example	Governance
ADR writer	Structured decision record	Reviewed like docs
Test generator	Framework-specific	No prod network tools
Threat-model stub	STRIDE prompts	Security co-owned
Migrate-langX	Internal frameworks	Versioned; owners

Skills that fetch remote content inherit **prompt-injection** risk—treat like MCP.

M5. Hooks that earn their keep

High-ROI hooks (illustrative patterns from public product themes): - Pre-tool: block commands matching secret/exfil patterns - Pre-commit assistant: remind tests - Stop: summarize session for handoff notes

Avoid hook sprawl that slows every keystroke without security value.

M6. MCP for builders ≠ MCP for production agents

Dimension	Builder (Claude Code)	Production agent
Creds	Dev/sandbox roles	Scoped service roles
Blast radius	Repo / dev accounts	Customer data / money
Change speed	Fast inner loop	Change-controlled
Approval	Marketplace allowlist	Architecture + security review

Never reuse prod god credentials in developer MCP “for convenience.”

Part N — Productivity workflows that survive scrutiny

N1. High-ROI loops (with review gates)

- 1. **Explain** → **propose** → **patch** → **test** → **PR summary** with human review.
- 2. **Failing test** → **root cause hypotheses** → **minimal fix**.
- 3. **Legacy module** → **characterization tests before refactor**.
- 4. **Incident stub** → **timeline draft from logs (redacted)** → **human edit**.

N2. Anti-patterns

Anti-pattern	Why bad	Replace with
Blind accept all diffs	Ships vulns/secrets	Mandatory review culture
Paste prod dumps into chat	Data leakage	Redacted fixtures
Metric = lines generated	Theater	Cycle time + change fail rate
Untracked Skills with tools	Supply chain	Managed marketplace
One shared API key on laptop	Attribution/abuse	SSO + per-user

N3. Measuring productivity honestly

Prefer: PR cycle time, review iterations, escaped defects, secret-scan incidents, developer satisfaction, time-to-first-productive-PR for new hires. Distrust: raw acceptance rate, token spend as “value,” lines of AI code.

N4. Debugging & operational enablement

Safe pattern: AI assists on **read-only** diagnostics; remediations go through existing change systems. Provide runbooks the assistant can retrieve; do not grant cloud-admin MCP to every engineer’s laptop agent.

Runbook sections: symptoms → gather (commands) → hypothesize → mitigate → verify → escalate → postmortem links.

Part O — Training curriculum (original outline)

O1. Role paths

Role	Must learn
IC engineer	CLAUDE.md, safe MCP, PR review with AI diffs
Tech lead	Skills governance, eval mindset for assistants
SRE/ops	Guardrailed debug loops, no prod god keys
Security	Hook policies, marketplace review, DLP
PM/adjacent	What AI can/can’t promise; how to file good bugs

O2. Onboarding week (example)

Day 1: SSO + policy + secret hygiene lab. Day 2: Repo CLAUDE.md tour + first assisted PR. Day 3: MCP approved set only; request process. Day 4: Incident tabletop with AI assist (read-only). Day 5: Office hours; feedback into paved road backlog.

O3. Champions network

Volunteers per org unit; office hours; inner-source Skills; escalate friction to platform—not shadow SaaS.

Part P — Architecture decision scenarios (Enablement)

P1. Shadow consumer ChatGPT usage discovered

Response: Don't only ban—**pave the road**: SSO Claude tooling, clear data rules, migrate use cases, measure adoption, security exceptions time-boxed.

P2. Team wants “install any MCP from the internet”

Response: Managed marketplace / allowlist; threat review; network egress controls; owners; versions pinned.

P3. Conflict: repo CLAUDE.md says auto-run shell; org policy forbids

Response: Org managed settings win; fix repo file; educate; alert on violations.

P4. Productivity theater dashboard

Response: Replace lines-generated with quality/velocity/safety metrics; qualitative interviews.

Part Q — Failure-mode analysis (enablement)

Failure	Signal	Mitigate
Settings bypass	Local overrides weaken DLP	Enforce managed non-overridable controls
Secret in PR via AI	Scan hits	Hooks + training + vault
Skill supply-chain	Unexpected network tool	Signing/review; marketplace only
Support overload	Tickets per seat ↑	Better docs; champions; UX fixes
Repo instruction drift	Contradictory guidance	Owners + CI lint for CLAUDE.md
Prod creds in dev MCP	Audit findings	Separate IdP apps / roles

Part R — Production checklists

R1. Before org pilot

- ☐ Managed security settings defined
- ☐ SSO live
- ☐ Marketplace policy live
- ☐ Secret hygiene training mandatory
- ☐ Support channel + SLA
- ☐ Success metrics agreed (incl. safety)

R2. Before org-wide expand

- ☐ Pilot friction log closed or accepted
- ☐ Reference CLAUDE.md templates published
- ☐ Hooks validated (false positive rate OK)
- ☐ Access review process scheduled
- ☐ Incident tabletop done

R3. Quarterly

- ☐ Plugin/MCP inventory review
 - ☐ Settings drift audit
 - ☐ Curriculum update for new model/tooling
 - ☐ Productivity + safety metrics review
-

Part S — Trade-off tables

S1. Speed of enablement vs control

Approach	Speed	Control	When
Wild-west local tools	Fast	Poor	Never for enterprise
Recommended defaults only	Medium	Medium	Low-risk orgs
Managed enforce + paved Skills	Medium	High	Default CCAR-P answer

Approach	Speed	Control	When
Lockdown with no paved road	Slow adoption	High on paper	Creates shadow IT

S2. Central platform vs federated team Skills

Centralize security and core Skills; federate domain Skills with review. Pure central bottleneck vs pure federated chaos—hybrid wins.

Part T — Extended Q&A (49–60)

Q49. Highest-precedence place for “deny unapproved MCP”? **A. Org managed / non-overridable** settings—not a hopeful CLAUDE.md line alone.

Q50. Select TWO enablement success metrics: secret-scan incidents, seat count only, time-to-first-merged-PR, raw tokens burned. **A.** Secret-scan incidents and time-to-first-merged-PR.

Q51. Builder MCP with prod database admin—acceptable? **A. No**—use sandbox/read-only roles; prod changes via controlled runtime.

Q52. Hooks vs Skills—security blocking of exfil commands? **A.** Prefer **hooks/enforced controls**; Skills are instructions, not hard enforcement.

Q53. Shadow AI usage—best architect move? **A.** Paved road + policy + migrate, not ban-only.

Q54. CLAUDE.md contains API keys—action? **A.** Rotate keys, purge history, teach vault; add secret scanning.

Q55. Select THREE rollout phases: design controls, pilot, expand, skip security until GA, email passwords to team. **A.** Design, pilot, expand.

Q56. Why separate production agent creds from Claude Code MCP? **A.** Different blast radius, change control, and attribution needs.

Q57. Productivity theater example? **A.** Celebrating lines of AI-generated code while change-fail rate rises.

Q58. Managed plugin marketplace purpose? **A.** Distribute **approved** tools/Skills; reduce supply-chain risk.

Q59. Ops wants AI to “just fix prod.” Safe pattern? **A.** Read-only diagnosis + human/change-managed remediation.

Q60. Repo instruction conflicts with org deny-list—who wins? **A.** **Org enforced settings**; fix the repo file.

Part U — Rapid review (Enablement 7%)

- Enforce security high in the settings hierarchy.
- CLAUDE.md/Skills versioned; no secrets.
- Managed marketplace for MCP/plugins.
- Builder tooling ≠ production runtime creds.
- Measure velocity + safety, not vanity.
- Hooks for hard stops; Skills for guidance.
- Paved road beats shadow AI.
- Train by role; champions sustain adoption.

Cross-link: MCP deep dive 02; safety 03; rollout comms 04.

CCAR-P Pro Tips — how to take this exam well

Companion to files 01–05; distilled 2026-08-23 from the official Exam Guide v1.0 and this pack. Exam craft only.

What this exam actually rewards: disciplined *elimination under multiple constraints*. Professional-level stems stack 3–5 requirements (latency + residency + audit + cost + an org quirk); every option that violates even one named constraint is dead, and what survives is usually the least exciting architecture on the list. The pack’s leftmost-pattern rule is the exam in one sentence: **prefer the simplest pattern that meets the requirement, and move right only when evidence says the left fails.**

The numbers that drive strategy

- **63 items / 120 min → ~1.9 min each** — the tightest pacing of the four exams, against the longest stems. Non-negotiable technique: read the question sentence first, then harvest constraints from the stem as a list. If you’re 30 seconds in and still “absorbing the scenario,” you’re reading it wrong.

- **Triage:** Integration 19% + Solution Design 17% + Evaluation 16% = 52%. But the soft domains (Governance 14 + Stakeholder 14 + Enablement 7 = 35%) are cheap points with one shared template — **mechanism + metric + owner**. An answer that names all three beats an answer with better adjectives.
- **720 scaled, no guessing penalty, multiple-response all-or-nothing** (and this exam loves Select-TWO with three defensible-sounding options — the discriminator is almost always a violated constraint, not quality).

The constraint-elimination drill (use it on every long stem)

1. Question sentence first.
2. List the constraints: SLA numbers, residency/compliance words, volumes, budget signals, “auditors”, “regulated”, “multi-tenant”, who owns what.
3. Cross off options violating any single constraint.
4. Among survivors: leftmost pattern, smallest model that passes evals, least privilege, fewest moving parts.

If the stem says X → think Y

Stem signal	Reflex
“Auditors need evidence per step”	Workflow with stage logs + validators — not an autonomous agent
“Steps unknown / open-ended investigation”	Agent + tool allowlist + budgets + HITL on writes
Agent has too many tools / does everything	Least privilege: remove tools first (official sample Q1)
Cache hit rate low / cost spike on stable prompts	Stable prefix first, volatile last; stop reshuffling tool defs (sample Q2)
RAG answers wrong despite good model	Diagnose retrieval before touching the model (sample Q3); citations or refuse
Choosing Direct API vs Bedrock vs Vertex vs Foundry	Constraint elimination on residency/procurement/identity — never brand vibes (file 02 D5)

Stem signal	Reflex
“How do we know it’s good?”	Gold + adversarial + regression evals; code judges before model judges; freeze thresholds before looking at experiment data (file 02 M7)
Refunds/payments/writes by an agent	AuthZ in the tool server + schema validation in code + caps enforced by code, not model judgment
Multi-agent proposal	Real role/permission boundaries or it’s theater — one workflow may do
“Stakeholders want it fully autonomous”	Value now + unlock later + risk named + written alignment; HITL stays on high impact
Handoff/hypercare/lifecycle	Named DRI + end date + runbook + ADR log — ownership answers score
Model upgrade in prod	Pin → shadow/replay → canary + kill switch → flip; prompt and model are one release artifact
Business value pillars	The guide’s five: efficiency, transformation, productivity , cost, performance SLAs

Distractor archetypes to expect

- **The impressive architecture:** multi-agent, biggest model, every tool — violating the simplest-that-works rule. Sophistication is the bait.
- **The single-constraint answer:** perfect on latency, silently violates residency (or vice versa). This is *the* Select-TWO killer — check every survivor against every constraint.
- **The prompt-as-control:** governance “solved” by prompt text. Prompts are never the whole control story — pair with authZ, HITL, monitoring, evidence.
- **The trust-the-model:** letting Claude decide refund eligibility, tenancy, or thresholds that code must enforce.
- **The eval afterthought:** shipping on demo vibes; peeking at data before fixing thresholds; crowning an A/B winner on 20 chats.

Freshness watchlist (re-verify the week of your exam)

This pack deliberately teaches decision logic over SKUs, so little rote. Worth 10 minutes anyway: current model tiers + adaptive thinking/effort (thinking budgets and sampling params are legacy — the pack is already corrected), platform availability differences (Bedrock/Vertex/Foundry), Claude Code org-managed settings behavior.

The 48-hour plan

1. Re-read file 02 Parts K–M (Integration + Evals deep dives — 35% of the form) and file 01's Part J scenarios with the scoring keys covered.
2. Re-do files 03–05's traps + the review-routing matrix (03 D4) — the soft 35% compresses into mechanism/metric/owner + named-DRI answers.
3. Write two ADRs cold from the scenario banks (pattern, model, rejected alternatives) — the exam's long stems are ADRs in disguise.
4. Timed 63Q/120min set at full pace; practice the constraint-list drill until it's automatic, not the content.

Day-of (online proctored)

ID, clean desk, room scan, stable connection. Pace ruthlessly: ~1.9 min/item means any item at 3+ minutes gets your best surviving option and a flag. The soft-domain items are your recovery time — bank it. On the flagged-item pass, re-check constraints, not preferences. Answer all 63.